CS 6290 — Georgia Tech OMSCS

High Performance Computer Architecture

講義ノート

kinginu

Fall 2026

目次

1 イントロダクション **1**
- 1.1 コンピュータアーキテクチャとは何か . . . 1
- 1.2 技術トレンド . . . 1
- 1.3 速度・電力・コスト . . . 2
- 1.4 消費電力 . . . 2
- 1.5 製造コスト . . . 3

2 性能指標と評価 **4**
- 2.1 性能 . . . 4
- 2.2 性能の比較 . . . 4
- 2.3 性能の測定 . . . 4
- 2.4 性能の要約 . . . 5
- 2.5 性能の鉄則 . . . 5
- 2.6 Amdahl の法則 . . . 6

3 パイプライン **8**
- 3.1 パイプライン化の考え方 . . . 8
- 3.2 パイプライン CPI . . . 8
- 3.3 ストールとフラッシュ . . . 9
- 3.4 制御依存 . . . 9
- 3.5 データ依存 . . . 10
- 3.6 依存とハザード . . . 10
- 3.7 ハザードの処理 . . . 10
- 3.8 ステージ数はいくつにすべきか . . . 11

4 分岐 **12**
- 4.1 パイプラインにおける分岐 . . . 12
- 4.2 予測精度と性能 . . . 13
- 4.3 不成立予測 . . . 13
- 4.4 よりよい予測の必要性 . . . 14
- 4.5 分岐先バッファ . . . 15
- 4.6 方向予測 . . . 15
- 4.7 2 ビット予測器 . . . 16
- 4.8 履歴ベース予測器 . . . 18
- 4.9 共有カウンタ: PShare と GShare . . . 19
- 4.10 トーナメント予測器と階層型予測器 . . . 20
- 4.11 リターンアドレススタック . . . 22

5 プレディケーション **24**
- 5.1 予測とプレディケーション . . . 24
- 5.2 if 変換 . . . 25
- 5.3 条件付きムーブ命令 . . . 25
- 5.4 if 変換の性能 . . . 27
- 5.5 条件付きムーブの利点とコスト . . . 28
- 5.6 完全プレディケーション . . . 28

6 ILP **30**
- 6.1 すべての命令を同一サイクルで . . . 30
- 6.2 同一サイクル実行における依存 . . . 30
- 6.3 偽の依存の除去 . . . 31
- 6.4 レジスタリネーミング . . . 31
- 6.5 リネーミング後の依存 . . . 32
- 6.6 命令レベル並列性 . . . 33
- 6.7 構造依存と制御依存 . . . 34
- 6.8 ILP と IPC . . . 35

7 命令スケジューリング 37
7.1 IPC の向上 . . . 37
7.2 Tomasulo のアルゴリズム . . . 37
7.3 構成 . . . 38
7.4 発行 . . . 39
7.5 ディスパッチ . . . 40
7.6 結果書き込み . . . 40
7.7 1 サイクル内のステップ . . . 42
7.8 ロード命令とストア命令 . . . 42
7.9 完全な例 . . . 43
7.10 構造を追跡しないタイミングの算出 . . . 44

8 リオーダバッファ 47
8.1 アウトオブオーダ実行における例外 . . . 47
8.2 分岐予測ミス . . . 47
8.3 正しいアウトオブオーダ実行 . . . 48
8.4 リオーダバッファ . . . 49
8.5 分岐予測ミス回復 . . . 50
8.6 リオーダバッファのもとでの例外 . . . 52
8.7 コミット時の RAT 更新 . . . 52
8.8 リオーダバッファの例 . . . 52
8.9 リオーダバッファのもとでのタイミング . . . 54
8.10 統合リザベーションステーション . . . 55
8.11 スーパースカラプロセッサ . . . 56
8.12 用語 . . . 56
8.13 何がアウトオブオーダなのか . . . 56

9 メモリオーダリング 58
9.1 メモリを介した依存 . . . 58
9.2 メモリが書かれるとき . . . 58
9.3 ロードストアキュー . . . 59
9.4 ロードストアのアウトオブオーダ実行とインオーダ実行 . . . 60
9.5 ストアロードフォワーディング . . . 62
9.6 ロードストアキューの例 . . . 62
9.7 LSQ と ROB とリザベーションステーション . . . 63

10 コンパイラによる ILP 65
10.1 コンパイラはどのように IPC を高められるか . . . 65
10.2 木の高さ削減 . . . 66
10.3 命令スケジューリング . . . 67
10.4 スケジューリングと if 変換 . . . 68
10.5 ループ展開 . . . 69
10.6 インライン展開 . . . 71
10.7 その他のコンパイラ技法 . . . 73

11 VLIW 74
11.1 スーパースカラプロセッサと VLIW プロセッサ . . . 74
11.2 利点と欠点 . . . 75
11.3 VLIW 命令 . . . 76
11.4 実際の VLIW プロセッサ . . . 77

12 キャッシュの復習 79
12.1 局所性の原理 . . . 79
12.2 キャッシュ . . . 80
12.3 キャッシュの性能 . . . 80
12.4 ブロックとライン . . . 81
12.5 ブロック番号，タグ，有効ビット . . . 82
12.6 ブロックを置ける場所 . . . 83
12.7 ダイレクトマップキャッシュ . . . 83

12.8 セットアソシアティブキャッシュ . . . 84
12.9 フルアソシアティブキャッシュ . . . 85
12.10 キャッシュの置換 . . . 86
12.11 書き込み方針 . . . 87
12.12 全体のまとめ . . . 87

13 仮想記憶 **90**
13.1 なぜ仮想記憶が必要か . . . 90
13.2 物理メモリと仮想アドレス空間 . . . 90
13.3 ページとページテーブル . . . 91
13.4 ページフォールト . . . 92
13.5 アドレス変換 . . . 93
13.6 単一レベルページテーブルの大きさ . . . 94
13.7 多段ページテーブル . . . 94
13.8 ページサイズの選択 . . . 96
13.9 変換を伴うメモリアクセス時間 . . . 97
13.10 TLB . . . 97
13.11 TLB の構成と性能 . . . 99

14 高度なキャッシュ **100**
14.1 キャッシュ性能の改善 . . . 100
14.2 パイプライン化キャッシュ . . . 100
14.3 アドレス変換とヒット時間 . . . 101
14.4 エイリアシング . . . 103
14.5 ウェイ予測 . . . 104
14.6 置換アルゴリズムとヒット時間 . . . 105
14.7 ミスの原因 . . . 106
14.8 キャッシュブロックの大型化 . . . 107
14.9 プリフェッチ . . . 108
14.10 ループ交換 . . . 109
14.11 ミスの重なり . . . 110
14.12 キャッシュ階層 . . . 111
14.13 局所ミス率と大域ミス率 . . . 112
14.14 包含性 . . . 112

15 メモリ **114**
15.1 メモリ技術 . . . 114
15.2 SRAM セル . . . 114
15.3 DRAM セル . . . 115
15.4 メモリチップの構成 . . . 116
15.5 リフレッシュ . . . 118
15.6 アドレスの多重化 . . . 118
15.7 ファストページモード . . . 118
15.8 DRAM とプロセッサの接続 . . . 119

16 ストレージ **121**
16.1 ストレージ . . . 121
16.2 磁気ディスク . . . 121
16.3 ディスクのアクセス時間 . . . 122
16.4 磁気ディスクの動向 . . . 123
16.5 光ディスク . . . 124
16.6 磁気テープ . . . 124
16.7 SSD . . . 124
16.8 磁気ディスクとフラッシュのハイブリッド . . . 125
16.9 I/O デバイスの接続 . . . 126

17 フォールトトレランス **128**
17.1 ディペンダビリティ . . . 128
17.2 フォールト・エラー・故障 . . . 128

17.3 信頼性と可用性 . . . 129
17.4 フォールトの種類 . . . 130
17.5 信頼性と可用性の向上 . . . 130
17.6 フォールトトレランスの技法 . . . 131
17.7 メモリとストレージのための符号 . . . 132
17.8 独立ディスク冗長アレイ . . . 132
17.9 RAID 0: ストライピング . . . 133
17.10 RAID 1: ミラーリング . . . 134
17.11 RAID 4: ブロックインターリーブパリティ . . . 135
17.12 RAID 5: 分散パリティ . . . 136
17.13 RAID 6: 2 つのチェックブロック . . . 137

18 マルチプロセッシング **139**
18.1 Flynn の分類 . . . 139
18.2 なぜマルチプロセッサか . . . 140
18.3 マルチプロセッサには並列プログラムが必要である . . . 140
18.4 集中共有メモリ . . . 141
18.5 分散共有メモリ . . . 141
18.6 メッセージパッシングと共有メモリ . . . 142
18.7 共有メモリのハードウェア . . . 144
18.8 マルチスレッディングの性能 . . . 145
18.9 SMT のためのハードウェアの変更 . . . 146
18.10 SMT とデータキャッシュ・TLB . . . 147
18.11 SMT とキャッシュ性能 . . . 147

19 キャッシュコヒーレンス **149**
19.1 コヒーレンスの問題 . . . 149
19.2 コヒーレンス . . . 150
19.3 コヒーレンスの実現 . . . 150
19.4 書き込み時更新方式 . . . 151
19.5 書き込み時無効化方式 . . . 152
19.6 MSI プロトコル . . . 153
19.7 MOSI プロトコル . . . 154
19.8 排他状態 . . . 155
19.9 ディレクトリ方式 . . . 157
19.10 キャッシュミスとコヒーレンス . . . 158

20 同期 **160**
20.1 同期の必要性 . . . 160
20.2 ロック . . . 161
20.3 ロックの実装 . . . 162
20.4 アトミックな読み書き命令 . . . 162
20.5 LL/SC . . . 163
20.6 ロックと性能 . . . 165
20.7 バリア同期 . . . 166

21 メモリー貫性 **169**
21.1 コヒーレンスと一貫性 . . . 169
21.2 一貫性が問題になる理由 . . . 169
21.3 逐次一貫性 . . . 171
21.4 逐次一貫性の実現 . . . 172
21.5 緩和一貫性 . . . 173
21.6 データ競合と一貫性 . . . 175
21.7 一貫性モデル . . . 176

22 メニーコア **178**
22.1 コヒーレンストラフィック . . . 178
22.2 ネットワークオンチップ . . . 178
22.3 オフチップトラフィックと分散 LLC . . . 180

22.4 オンチップディレクトリ 181
22.5 電力バジェットの共有 182
22.6 ターボモード 183
22.7 SMT，コア，チップ 183
22.8 課題と解決策 184

参考文献 **185**

索引 **188**

English index **192**

第 1 章

イントロダクション

講義動画	Lesson 1，動画 1–19
教科書	H&P [1] §1.1–1.6
キーワード	コンピュータアーキテクチャ，ムーアの法則，メモリウォール，動的電力，静的電力，製造コスト，歩留まり

この章では，コンピュータアーキテクチャとは何かを定義し，それが独立した学問分野として必要とされる理由を述べ，以降のすべての設計判断を制約する技術トレンドと 3 つの量—性能・電力・コスト—を導入する．

1.1 コンピュータアーキテクチャとは何か

コンピュータアーキテクチャ とは，利用可能な製造技術を用いて，目的に適したコンピュータを設計する学問分野である．この定義には強調すべき点が 2 つある．

▷ computer architecture

出典 Lesson 1，動画 2–5；H&P §1.1–1.3．

- **目的に適していること**．携帯電話，ノート PC，データセンタのサーバはいずれもコンピュータだが，最適化の対象が異なる．一方では電池寿命・大きさ・重さ，他方では生の性能やコスト当たりのスループットである．したがってアーキテクチャとは唯一の最良設計を求めることではなく，**トレードオフ**を扱うことである．
- **利用可能な技術**．アーキテクトはトランジスタそのものを設計しないが，現在のプロセスで何が実現できるか—1 チップに載るトランジスタ数，スイッチング速度，消費電力—を知っていなければならない．それらが達成可能な範囲を定めるからである．

1.1.1 なぜ必要か

コンピュータアーキテクチャの目標は 2 つある．

1. **性能の向上**（広い意味で）: 実行速度，電池寿命，大きさ，重さ，エネルギー効率．
2. **能力の向上**: これまでできなかったことを可能にする．例えば 3D グラフィックス，デバッグのハードウェア支援，セキュリティ機能．

規模の目安: Intel 4004（1971 年）のトランジスタ数は 2300，NVIDIA H100（2022 年）は 800 億である．ベンダ公表値．

もう 1 つの理由はタイミングである．今日設計するプロセッサが製造されるのは数年後なので，アーキテクトは現在のパラメータではなく，その時点で技術がどこにあるかを**予測**して設計しなければならない．

要約すれば「性能の向上」と「能力の向上」である．

ヒント 後の章で過剰設計に見える判断があったら，それがどの技術トレンドを見越したものかを考えるとよい．

1.2 技術トレンド

1.2.1 ムーアの法則

ムーアの法則とは，一定面積のチップに載せられるトランジスタ数が約 18–24 か月ごとに倍増するという経験則である．物理法則ではなくトレンドだが，数十年にわたって成り立ち，業界全体の期待を形作ってきた．同じ 18–24 か月の倍増周期で次の帰結が導かれる．

▷ Moore’s law

出典 [2]；Lesson 1，動画 6；H&P §1.4．

- プロセッサの**速度**が倍になる（トランジスタが増えれば並列性を高められ，ゲートは小さく速くなる）．
- **演算当たりのエネルギー**が半分になる．
- **メモリ容量**が倍になる．

注意 ムーア自身は 1965 年に 1 年で倍増と述べ，1975 年に 2 年へ改めた．よく引かれる「18 か月」は David House の数字であり，ムーアのものではない．

例 1.1. 速度が 2 年ごとに倍になるなら，10 年で $2^5 = 32$ 倍の向上が期待される．逆に言えば，4 年かけて従来品の 2 倍しか速くないプロセッサを作った設計チームは，トレンドに遅れている．同じ期間にトレンドが予測するのは $2^2 = 4$ 倍だからである．

1.2.2 メモリウォール

すべての構成要素が同じ曲線に従うわけではない．プロセッサ性能が 18–24 か月で倍増する一方，メモリの**レイテンシ**は同じ期間に約 1.1 倍しか改善しなかった．その結果，プロセッサがデータを消費できる速さとメモリがそれを供給できる速さの差は世代ごとに広がる（図 1.1）．この差を**メモリウォール**と呼び，キャッシュ（第 12–14 章）が重要である理由はここにある．

出典 [3]；Lesson 1，動画 8；H&P §2.1．

▷ memory wall

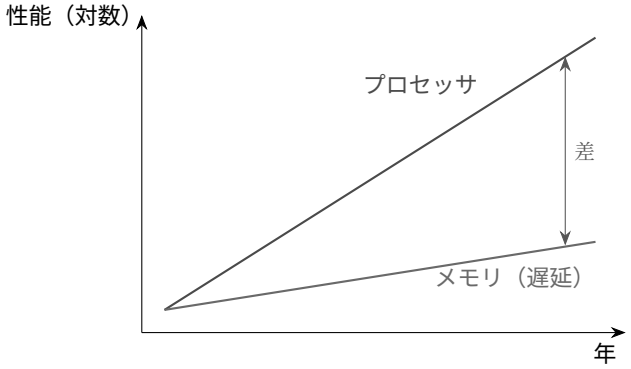


図 1.1: プロセッサ性能とメモリレイテンシは異なる速さで改善する．その差がメモリウォールである．

> **要点**
> メモリ容量はムーアの法則に追随するが，メモリの**レイテンシ**は追随しない．本書で扱うメモリシステム設計の大半は，このレイテンシを隠すために存在する．

Skylake では L1 ヒットが 4 サイクル，L2 ヒットが 12 サイクル，DRAM アクセスは 60–90 ns，すなわち 3 GHz では 180–270 サイクルに相当する．Intel 最適化マニュアル；7-cpu.com．

1.3 速度・電力・コスト

長い間「より良い」とは単に「より速い」を意味した．今日のアーキテクトは 3 つの量のバランスを取らなければならない．

性能 想定するワークロードをどれだけ速く実行できるか．
電力 どれだけのエネルギーを消費するか．携帯機器では電池寿命を，サーバでは冷却コストを決める．
コスト 製造にいくらかかるか．

どれが支配的かは製品による．携帯電話なら重さと電池寿命，組込みコントローラならコスト，スーパーコンピュータなら性能である．

出典 Lesson 1，動画 9–10．

電力予算: 携帯 SoC 2–5 W，デスクトップ CPU 65–125 W，サーバ CPU 最大 500 W（EPYC 9005，Xeon 6900P）．ベンダ公表値．

注意 ワークロードと電力・コスト予算を伴わない「より速く」は，意味のある設計目標ではない．

1.4 消費電力

プロセッサが消費する電力は 2 つの成分からなる．

出典 Lesson 1，動画 11–13；H&P §1.5．

1.4.1 動的電力（アクティブ電力）

動的電力は，信号が切り替わるたびに配線やゲートの容量を充放電することで消費される．

▷ dynamic power

$$P_{\text{dyn}} = \frac{1}{2} C V^2 f \alpha \tag{1.1}$$

ここで C はスイッチングされる総容量，V は電源電圧，f はクロック周波数，α は**活性化率**，すなわち各サイクルで実際に切り替わる容量の割合である．

C はチップ面積とトランジスタ数とともに増え，α はワークロードと，どれだけの論理が遊んでいるかに依存する．H&P は $f\alpha$ の積を「スイッチする周波数」と書く．

式 (1.1) から直ちに 2 つのことが分かる．

- 電力は電圧の *2* **乗**に比例するので，V を下げるのが最も効果的な手段である．ただし回路が維持できる最大周波数も V とともに下がるため，実際には V と f を同時に下げることになり，電力はおよそ V^3 に比例して減る．

- 電圧一定で周波数を倍にすると電力は倍になり，トランジスタ数を倍にする（並列ハードウェアを追加する）場合も C が倍になるので電力は倍になる．違いは，周波数を上げるには通常電圧も上げる必要があり電力が 2 乗で増えるのに対し，並列ハードウェアは同じかより低い V と f で動かせることである．[1]

例 1.2. C, f, α を固定したまま電源電圧を 15 % 下げると，動的電力は元の $0.85^2 \approx 0.72$ 倍，すなわち約 28 % 減る．

1.4.2 静的電力

静的電力（リーク電力）は，信号が切り替わらなくても消費される．トランジスタは完全なスイッチではなく，オフでも電流が漏れるからである．リーク電流は，電源電圧–それに伴ってトランジスタのしきい値電圧–を下げるほど大きくなる．したがってトレードオフがある．V を下げると動的電力は減るが静的電力は増え，合計 $P = P_{\text{dyn}} + P_{\text{static}}$ はある中間の電圧で最小になる（図 1.2）．

> **要点 — 電力**
> 動的電力 $\propto CV^2 f\alpha$ は低電圧を好み，静的電力は低電圧を罰する．現代の設計は両者の和の最小点付近にあり，さらなる改善は V よりも C, α, f の削減から得ている．

[1] 2005 年ごろまでは，新しいプロセスごとにトランジスタとともに V も縮小され，同じ電力のままチップを高速化できた．リーク電流により V の縮小は 1 V 付近で止まり，クロック周波数は 3–4 GHz で頭打ちとなって，増えたトランジスタはコア数の増加（第 18 章）に向けられた．

▷ static power

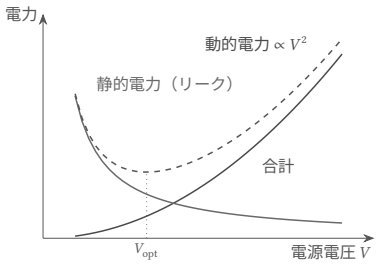


図 1.2: 電源電圧を変えると，動的電力と静的電力は逆方向に動く．

1.5 製造コスト

チップは円形のシリコン**ウェハ**上に製造され，各ウェハは多数の同一の長方形の**ダイ**に切り分けられる．ウェハ 1 枚を処理するコストはほぼ一定なので，ダイ 1 個のコストは次式で与えられる．

$$\text{ダイ当たりコスト} = \frac{\text{ウェハ当たりコスト}}{\text{ウェハ当たりダイ数} \times \text{歩留まり}} \tag{1.2}$$

▷ wafer

▷ die

出典 Lesson 1，動画 15–17；H&P §1.6．

1.5.1 歩留まり

すべてのダイが動作するわけではない．ウェハ上には欠陥が散在し，欠陥を含むダイは通常廃棄される．**歩留まり**とは，機能するダイの割合である．欠陥はランダムな位置に生じるため，大きなダイほど欠陥を含みやすく，歩留まりはダイ面積が増えるにつれて低下する．

▷ yield

H&P §1.6 には経験的モデル 歩留まり $\approx 1/(1 + \text{欠陥密度} \times \text{面積})^N$ が示されている．成熟したプロセスの欠陥密度は cm^2 当たり 0.1 程度である．

1.5.2 帰結

式 (1.2) と歩留まりの振る舞いを合わせると，この節の中心的な結論が得られる．ダイ面積を半分にすると，ダイ当たりコストは半分以下になる．ウェハ当たりのダイ数が倍になり，かつ歩留まりも上がるからである．ムーアの法則と合わせて読むと，アーキテクトには技術世代ごとに 2 つの選択肢がある．

- ダイ面積を一定に保ち，2 倍のトランジスタを詰め込む–ほぼ同じコストでより高機能なチップ．
- 設計を一定に保ち，面積を半分に縮小する–同じチップを半分よりずっと安いコストで．

例 1.3. ウェハが \$5000 で，ある設計のダイが 100 個取れ，歩留まりが 50 % だとする．動作するダイ 1 個のコストは $5000/(100 \times 0.5) = \$100$ である．縮小によりウェハ当たり 200 個取れ，歩留まりが 70 % に上がると，コストは $5000/(200 \times 0.7) \approx \$36$ と半分以下になる．

ヒント ウェハ当たりダイ数 $\approx \pi(d/2)^2/A - \pi d/\sqrt{2A}$．第 2 項はウェハ端の欠けたダイを除く分である．H&P §1.6．

300 mm ウェハの面積は約 $707\ \text{cm}^2$，リソグラフィの 1 フィールドで描ける最大のダイは $26 \times 33\ \text{mm} = 858\ \text{mm}^2$ であり，NVIDIA H100 はその $814\ \text{mm}^2$ を使う．

> **要点 — 章のまとめ**
> アーキテクチャとは，技術トレンド（ムーアの法則）とその例外（メモリウォール）に駆動されながら，性能・電力・コストという 3 つの制約の下でトレードオフを設計することである．次章では「性能」を厳密に定義する．

第 2 章

性能指標と評価

講義動画	Lesson 2，動画 1–25
教科書	H&P [1] §1.8–1.9
キーワード	レイテンシ，スループット，高速化率，ベンチマーク，幾何平均，性能の鉄則，CPI，Amdahl の法則，収穫逓減

性能を改善するには，まず性能を測定し，2 つのマシンを比較し，変更の効果を予測できなければならない．この章では用語（レイテンシ，スループット，高速化率），方法論（ベンチマークとその要約法），そして本講義を通じて繰り返し現れる 2 つの解析ツール—性能の鉄則と Amdahl の法則—を導入する．

2.1 性能

「性能」は，何を測るのかを言わない限り曖昧である．

出典 Lesson 2，動画 2–3；H&P §1.8．

定義 2.1 (レイテンシ). タスクの**レイテンシ**とは，開始から完了までの時間である．

▷ latency

定義 2.2 (スループット). システムの**スループット**とは，単位時間当たりに完了するタスク数である．

▷ throughput

スループットは単純にレイテンシの逆数**ではない**．タスクが重なって（パイプライン化，第 3 章）あるいは並列に（マルチプロセッサ）実行される場合，個々のタスクに長い時間がかかっても，システムは毎秒多数のタスクを完了できる．

例 2.3. 2 つのプログラムを順に実行し，それぞれに 10 s かかるプロセッサは，レイテンシがプログラム当たり 10 s，スループットが 0.1 programs/s である．プログラム当たり 15 s かかるが 2 つを同時に実行できる別のプロセッサは，レイテンシは劣るがスループットは $2/15 \approx 0.13$ programs/s となる．

注意 スループット = 1/レイテンシ が成り立つのは，タスクを一度に 1 つずつ，厳密に順番に処理する場合だけである．

DDR4-3200 は読み出しから 14 ns でデータを返すが（CL22），アクセスの重なりで 25.6 GB/s を持続する（第 15 章）．JEDEC JESD79-4．

2.2 性能の比較

マシン X がマシン Y より「N 倍速い」と言うために**高速化率**を用いる．

$$N = \frac{\text{speed}(X)}{\text{speed}(Y)} = \frac{\text{latency}(Y)}{\text{latency}(X)} = \frac{\text{throughput}(X)}{\text{throughput}(Y)} \tag{2.1}$$

▷ speedup

出典 Lesson 2，動画 4–7；H&P §1.8．

レイテンシは**逆向き**に現れることに注意する．時間が短いほど速いからである．高速化率が 1 より大きければ X が優れ，1 未満なら劣る（「高速化率 0.8」は減速である）．高速化率は乗法的に合成される．X が Y の 2 倍速く，Y が Z の 3 倍速ければ，X は Z の 6 倍速い．

注意 百分率は高速化率ではない．時間が 30 % 短いことは $N \approx 1.43$ であって 1.3 ではない．

ヒント 比は常に 旧時間/新時間 の形で保つこと．符号の誤りの大半は，この比の逆転が原因である．

2.3 性能の測定

理想的な測定は，ユーザの実際のワークロードを実際のマシンで走らせることである．これが可能なことは稀で—マシンはまだ存在しないかもしれず，ワークロードは通常不明である—そこで**ベンチマーク**に頼る．ベンチマークとは，実ワークロードの代理としてコミュニティが合意したプログラムと入力データの組である．そのようなプログラムの集まりを**ベンチマークスイート**と呼ぶ．

▷ benchmark

出典 Lesson 2，動画 8–10；H&P §1.8．

2.3.1 ベンチマークの種類

代表性の高い順に:

実アプリケーション 現実的な入力を伴う完全なプログラム．最も代表的だが，準備が最も難しく，シミュレータや未完成のプロトタイプでは実行できない．

カーネル 実アプリケーションの最も時間を消費する部分（例えば行列積）を切り出し，単独で実行できるようにしたもの．実行しやすく，実際の処理を反映している．

合成ベンチマーク 実アプリケーションと同様に振る舞う（命令やメモリアクセスの構成が似ている）ように書かれた，より実行の簡単なプログラム．設計の初期段階で有用．[1]

ピーク性能 ハードウェアが理論上維持できる最大レート（例えば全ユニットが常に稼働した場合の毎秒命令数）．実プログラムが到達することのないマーケティング用の数値．

注 2.4. ピーク性能はベンチマーク結果ですらない．実プログラムがどれだけそれに近づけるかを何も語らない上限値である．

2.3.2 ベンチマークの標準

ベンダは自社製品に有利なベンチマークを選びうるため，独立した組織が標準スイートを定めている．SPEC（Standard Performance Evaluation Corporation）は CPU，グラフィックス，サーバのスイートを公開し，TPC はトランザクション処理ワークロードに，EEMBC は組込みシステムに対して同様のことを行う．これらの組織の会員は，プログラム，入力，報告規則について合意する．

2.4 性能の要約

スイートからはプログラムごとに 1 つの数値が得られるが，通常はマシンごとに 1 つの数値が欲しい．

- **実行時間**は通常の算術平均で合成してよい．平均時間は一意に定まり，2 台のマシンの平均時間の比は総実行時間の高速化率に等しい．
- **高速化率**（比）を算術平均してはならない．比の算術平均はどちらのマシンを基準にするかに依存し，平均時間の比とも一致しない．**幾何平均**を用いる．

$$\bar{S} = \left(\prod_{i=1}^{n} S_i\right)^{1/n} \tag{2.2}$$

例 2.5. プログラム A はマシン X で 10 s，Y で 20 s かかり，プログラム B は X で 20 s，Y で 10 s かかる．X の Y に対するプログラムごとの高速化率は 2 と 0.5 である．算術平均 1.25 は X が速いことを示唆するが，Y を基準にして計算した算術平均も 1.25 となり，Y が速いことを示唆する—矛盾である．幾何平均はどちら向きでも $\sqrt{2 \times 0.5} = 1$ となり，このスイートでは両マシンが同等であることを正しく報告する．

> **要点**
> **時間**は算術平均，**高速化率**は幾何平均で平均する．高速化率の幾何平均は，どちらのマシンを基準にしても，実行時間の幾何平均の比に等しい．

2.5 性能の鉄則

性能の鉄則は，CPU 時間を，計算機システムの異なる部分に対応する 3 つの因子に分解する．

$$\text{CPU 時間} = \underbrace{\text{命令数}}_{\text{IC}} \times \underbrace{\frac{\text{サイクル数}}{\text{命令}}}_{\text{CPI}} \times \underbrace{\frac{\text{秒}}{\text{サイクル}}}_{T_{\text{clk}}} \tag{2.3}$$

命令数（IC）アルゴリズム，コンパイラ，命令セット（ISA）で決まる．

命令当たりサイクル数（CPI）ISA とプロセッサのマイクロアーキテクチャ（パイプライン化，アウトオブオーダ実行，キャッシュなど）で決まる．本書の大半は CPI を下げることに関するものである．

gem5 のようなサイクルレベルシミュレータは，模擬する実機より 3–5 桁遅い．

[1] 古典的な例は Whetstone（1972 年，浮動小数点）と Dhrystone（Weicker，1984 年，整数）である．いずれも，結果が使われないコードを最適化コンパイラが削除するようになって信頼を失った．合成プログラムがもはや最終的な測定として認められないのはこのためである．

出典 Lesson 2，動画 12；[4]．

SPEC CPU2017 は 43 本のプログラムを 4 つのスイート（整数と浮動小数点，それぞれ speed 版と rate 版）に分けて含む．`spec.org`．

出典 Lesson 2，動画 13–14；H&P §1.8．

▷ geometric mean

ヒント $\exp\left(\frac{1}{n}\sum_i \ln S_i\right)$ として計算するとよい．多数の比の積はオーバーフローするが，対数の和はしない．

注意 幾何平均は一貫しているが予測的ではない．スイート全体の所要時間は分からない．それを知るには時間を足し合わせる．

▷ iron law

出典 Lesson 2，動画 15–18；H&P §1.9．

クロックサイクル時間（T_{clk}）マイクロアーキテクチャ（2 つのラッチの間にどれだけの論理があるか），回路設計，製造技術で決まる．

この法則の価値は，トレードオフを露わにすることにある．1 つの因子を減らす変更は，しばしば別の因子を増やす．深いパイプラインは T_{clk} を下げるが CPI を上げる．高機能な ISA は IC を下げるが CPI や T_{clk} を上げうる．

Pentium 4（Prescott）は 31 段・3.8 GHz，Core 2（Conroe，2006 年）は 14 段・2.93 GHz だが，より速い．Intel 公表値．

注意 変更を単一の因子で評価してはならない．「クロック周波数が高い」が「速い」になるのは，IC×CPI の増加率が周波数の増加率より小さい場合だけである．

注意 ベンダは IPC = 1/CPI で表すことが多い．スイート全体で IPC を平均することには，高速化率を平均するのと同じ欠陥がある．

2.5.1 命令ごとに時間が異なる場合

命令クラスごとに必要なサイクル数が異なる場合，CPI は重み付き平均になる．n 個のクラスについて，

$$\text{CPU 時間} \;=\; \Big(\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i\Big) \times T_{\text{clk}} \tag{2.4}$$

ここで IC_i，CPI_i はクラス i の命令数と命令当たりサイクル数である．

例 2.6. あるプログラムが 2 GHz のプロセッサで 10^9 命令を実行する．60 % が ALU 演算（1 サイクル），30 % がロード/ストア（2 サイクル），10 % が分岐（3 サイクル）である．平均 CPI は $0.6 \cdot 1 + 0.3 \cdot 2 + 0.1 \cdot 3 = 1.5$ なので，CPU 時間は $10^9 \times 1.5 \times 0.5\,\text{ns} = 0.75\,\text{s}$ である．

現実的な桁としては，SPEC の整数プログラムはロードが約 25 %，ストアが 10 %，分岐が 15 % である．H&P 付録 A．

2.6 Amdahl の法則

Amdahl の法則は，実行の**一部**だけを高速化したときの全体の高速化率を予測する．

▷ Amdahl's law

出典 [5]；Lesson 2，動画 19–22；H&P §1.9．

定義 2.7 (Amdahl の法則). ある改善が**元の**実行時間のうち割合 f_{enh} を s_{enh} 倍速くするとき，全体の高速化率は

$$S_{\text{overall}} \;=\; \frac{1}{(1 - f_{\text{enh}}) + \dfrac{f_{\text{enh}}}{s_{\text{enh}}}} \tag{2.5}$$

である．

注意 f_{enh} は元の時間に対する割合であって，変更後の時間に対する割合でも，命令数に対する割合でもない．

例 2.8. 浮動小数点命令が実行時間の 40 % を占める．新しい FP ユニットによりそれらが 4 倍速くなった．全体の高速化率は $1/(0.6 + 0.4/4) = 1/0.7 \approx 1.43$ である．FP ユニットを無限に速くしても $1/0.6 \approx 1.67$ にしか届かない．[2]

[2] Amdahl の 3 ページの論文（AFIPS 1967）は，この式を用いずに文章で大規模並列機に反論した．時間の 95 % が並列でも，コア数によらず 20× を超えない．第 18 章でコアに適用する．

出典 Lesson 2，動画 21–24．

2.6.1 含意

式 (2.5) で $s_{\text{enh}} \to \infty$ とすると，上限 $S_{\text{overall}} \le 1/(1 - f_{\text{enh}})$ が得られる．ここから 2 つの設計原則が導かれる．

> **要点 — 頻出ケースを速くせよ**
> 時間の大きな部分をそこそこ速くする方が，小さな部分を劇的に速くするより効く．プログラムが実際に時間を費やしている部分を最適化すること．

その逆命題は冗談めかして **Lhadma の法則**と呼ばれる．頻出ケースを速くする際に，**稀なケースを犠牲にしすぎてはならない**．ある最適化で手を付けなかった 10 % の部分が副作用で 10 倍遅くなれば，その部分だけで元のプログラム全体と同じ時間がかかり，全体としては減速になる．

注意「Lhadma」は「Amdahl」を逆から綴ったもので，講義独自の呼び名であり標準的な用語ではない．

2.6.2 収穫逓減

実行の同じ部分を繰り返し改善すると**収穫逓減**が生じる．その部分が縮んでしまえば，もはや頻出ケースではなく，残りの時間はそれ以外のすべてに支配される（図 2.1）．改善のたびにアーキテクトは再測定し，**今**の頻出ケースは何かを問い直すべきである．

▷ diminishing returns

例 2.9. 前の例（FP = 時間の 40 %）から始めると，最初の FP 2 倍高速化で $S = 1/(0.6 + 0.2) = 1.25$ が得られる．新しいマシンでは FP は時間の $0.2/0.8 = 25\,\%$ しか占めないので，2 回目の FP 2 倍高速化はわずか $1/(0.75 + 0.125) \approx 1.14$ にしかならない．

要点 — 章のまとめ
状況に応じてレイテンシまたはスループットで測り，高速化率で比較し，幾何平均で要約する．性能の鉄則（式 (2.3)）は時間を IC・CPI・クロック周期に帰着させ，Amdahl の法則（式 (2.5)）は部分的改善で達成できる上限を定める．

Amdahl の法則は問題の規模が固定であることを仮定している．マシンとともに問題が大きくなれば，直列部分の割合は小さくなる（Gustafson 1988）．

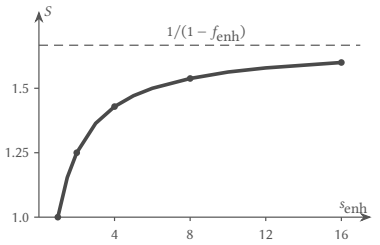


図 2.1: $f_{\text{enh}} = 0.4$ のときの全体の高速化率 S．改善部分が速くなるほど，同じ部分をさらに改善して得られる効果は小さくなる．

第 3 章

パイプライン

講義動画	Lesson 3，動画 1–18
教科書	H&P [1] 付録 C.1–C.3
キーワード	パイプライン，ステージ，パイプライン CPI，ストール，フラッシュ，制御依存，データ依存，RAW，WAR，WAW，ハザード，フォワーディング，パイプライン段数

パイプライン化はスループットを高めるための最初にして最も基本的な技法である．1 つの命令を終えてから次を始めるのではなく，プロセッサは複数の命令の実行を，それぞれ異なる進行段階で重ね合わせる．この章では，パイプライン化されたプロセッサの動作を復習し，理想的な性能を定量化した後，理想の達成を妨げる命令間の**依存**を調べる．

▷ pipelining

3.1 パイプライン化の考え方

命令の実行は，異なるハードウェアを使う一連のステップからなる．ステップを**ラッチ**で区切られた**ステージ**として配置すれば，命令 I_1 がステージ 1 を離れてステージ 2 に進んだ瞬間に，命令 I_2 がステージ 1 に入れる．k ステージなら，最大 k 個の命令が同時に処理中となる．

▷ stage

出典 Lesson 3，動画 2–3；H&P §C.1．

第 2 章の 2 つの指標に対する帰結は異なる．

- **スループット**は最大 k 倍に向上する．定常状態では，k ステージ時間ごとではなく 1 ステージ時間ごとに 1 命令が完了する．
- 個々の命令の**レイテンシ**は改善しない．総作業量は同じであり，ステージ間のラッチがわずかなオーバーヘッドを加えるため，命令当たりのレイテンシはむしろ少し増える．

3.1.1 プロセッサにおけるパイプライン

古典的な **5 段パイプライン**（図 3.1）を，本講義を通じて基準となる設計として用いる．

▷ five-stage pipeline

現在の x86 コアは約 14–20 段である（Agner Fog，マイクロアーキテクチャマニュアル）が，5 段モデルでここで重要な問題はすべて捉えられる．



図 3.1: 5 段の命令パイプライン．

IF 命令フェッチ: **プログラムカウンタ**（PC）が指す命令を命令メモリから読み，PC を進める．
ID 命令デコードとレジスタ読み出し: 演算の種類を決定し，ソースオペランドをレジスタファイルから読む．
EX 実行: ALU 演算を行う，あるいはロード/ストアのメモリアドレスを計算する，あるいは分岐条件を評価する．
MEM メモリ: ロード/ストアのためにデータメモリにアクセスする．
WB 書き戻し: 結果をレジスタファイルに書く．

図 3.2 は独立な 5 命令がパイプラインを流れる様子を示す．最初の 4 サイクルでパイプラインが満たされた後は，毎サイクル 1 命令が完了する．

各ステージはメモリアクセスも含めてちょうど 1 サイクルと仮定する．この仮定は第 12–15 章で覆される．

注意 ここでは命令メモリとデータメモリが別ポートであると仮定している．1 ポートなら構造ハザードが加わる（H&P 付録 C）．

3.2 パイプライン CPI

図 3.2 の理想的な場合，パイプラインは毎サイクル 1 命令を完了するので，性能の鉄則（式 (2.3)）の CPI は 1 である．これを 1 より大きくする効果が 2 つある．

出典 Lesson 3，動画 6；H&P §C.1．

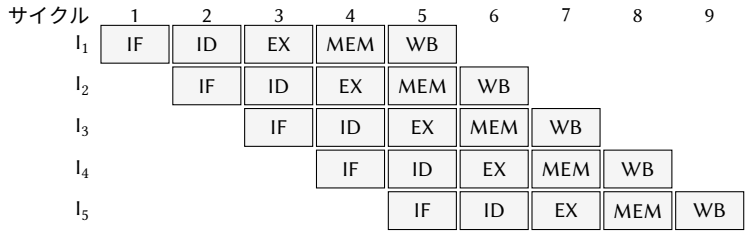


図 3.2: 5 段パイプラインの命令タイミング．サイクル 5 以降は毎サイクル 1 命令が完了する．

- **充填と排出**．最初の命令が完了するまでに k サイクルかかる．n 命令のプログラムでは合計 $k + (n-1)$ サイクル，すなわち $\mathrm{CPI} = (k+n-1)/n \to 1$（$n$ が大きいとき）となる．実プログラム（n は数十億）では無視できる．
- **ストール**．パイプラインが命令を次のステージへ進められないとき，命令が完了しないままサイクルが過ぎる．パイプライン化されたプロセッサの CPI を実際に決めるのはこのサイクルである．

$$\mathrm{CPI} \;=\; 1 + \frac{\text{ストールサイクル数}}{\text{命令}} \tag{3.1}$$

3.3 ストールとフラッシュ

パイプラインでサイクルを失う機構は 2 つある．

出典 Lesson 3，動画 7–8；H&P §C.2．

定義 3.1 (ストール). **ストール**とは，先行する命令は進み続ける一方で，ある命令（とその後続すべて）を現在のステージに 1 サイクル以上留めることである．その代わりに，**バブル**—空きスロット—がパイプラインを流れていく．

▷ stall

▷ bubble

定義 3.2 (フラッシュ). **フラッシュ**とは，すでにパイプラインにフェッチされた命令を，そもそも実行すべきでなかったという理由で破棄することである．典型的には，結果を誤って予測した分岐の後続命令が対象となる．

▷ flush

c サイクルのストールは c サイクルを，m 命令のフラッシュは m サイクルを失わせる．フラッシュされた各命令は，有用な命令が完了できたはずのスロットを占めていたからである．

3.4 制御依存

制御依存とは，ある命令がそもそも実行されるかどうかが，先行する分岐の結果に依存することである．5 段パイプラインでは，分岐の条件と飛び先は EX の終わりまで分からない．[1] その時点で後続の 2 命令はすでにフェッチされている（図 3.3）．分岐が成立していれば，それらをフラッシュし，飛び先からフェッチをやり直さなければならない．

▷ control dependence

出典 Lesson 3，動画 9–10；H&P §C.2．

[1] 初期の RISC パイプラインは，無駄になるスロットをフラッシュせずに露出させた．MIPS と SPARC では分岐の直後の命令（遅延スロット）が実行され，コンパイラがそれを埋める．処理中の命令が多くなるとスロット 1 つの効果は小さく，MIPS Release 6（2014）は遅延スロットのないコンパクト分岐を追加し，RISC-V は最初から遅延スロットを持たない．

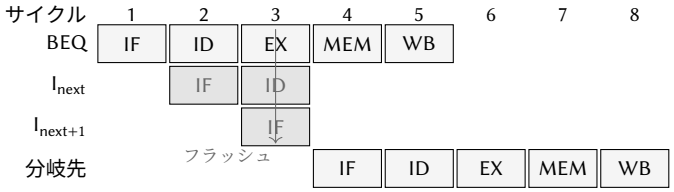


図 3.3: EX で解決される成立分岐は，2 つのフェッチスロットを無駄にする．

これが CPI に与えるコストは，どれだけ頻繁に起こるかに依存する．典型的な数値は，命令の 20 % が分岐で，分岐の 60 % が成立するというものである．常に次の命令をフェッチする単純な方針（「不成立予測」）では，成立した分岐だけがフラッシュを引き起こす．

下の 20 %/60 % は説明用に丸めた数字である．整数コードは実際におよそ 5 命令に 1 回分岐する（H&P 第 3 章）．

$$\mathrm{CPI} = 1 + 0.2 \times 0.6 \times 2 = 1.24 \tag{3.2}$$

例 3.3. 分岐がステージ 10 で解決されるより深いパイプラインでは，成立分岐ごとに 9 命令がフラッシュされる．同じ分岐統計で CPI は $1 + 0.2 \times 0.6 \times 9 = 2.08$ となり，プロセッサは 2 サイクルに 1 命令も完了できない．パイプラインが深いほど分岐予測（第 4 章）が重要になるのはこのためである．

ヒント 制御依存で失うサイクル =（分岐命令の割合）×（予測を外す割合）×（予測ミス当たりのフラッシュ命令数）．最後の因子は（分岐が解決するステージ）− 1 である．

3.5 データ依存

データ依存は，2 つの命令が同じレジスタまたはメモリ位置にアクセスし，少なくとも一方がそれを書き込むときに存在する．プログラム順でのアクセスの順序にちなんで名付けられた 3 種類がある．

▷ data dependence

RAW（read after write）後の命令が，先の命令が**書く**値を**読む**．**真の依存**あるいは**フロー依存**とも呼ばれる．データが実際に一方から他方へ流れなければならない．

WAR（write after read）後の命令が，先の命令が**読む**位置に**書く**．**逆依存**．

WAW（write after write）両命令が同じ位置に**書く**．後の書き込みが残らなければならない．**出力依存**．

出典 Lesson 3，動画 11–12；H&P §C.2．

WAR と WAW は**偽の依存**あるいは**名前依存**と呼ばれる．命令間でデータは移動せず，単に同じレジスタ名を再利用しているだけだからである．これらはリネーミング（第 6 章）で除去できる．RAW 依存は除去できない．

▷ false dependence

RAR—同じレジスタの 2 回の読み出し—は依存ではない．順序は問題にならない．

例 3.4.

```
I1:  ADD R1, R2, R3      ; R1 <- R2 + R3
I2:  SUB R4, R1, R5      ; I1 と R1 について RAW
I3:  MUL R1, R6, R7      ; I1 と R1 について WAW，I2 と WAR
```

3.6 依存とハザード

依存は**プログラム**の性質であり，それが問題を起こすかどうかは**パイプライン**に依存する．

出典 Lesson 3，動画 13–14；H&P §C.2．

定義 3.5 (ハザード). **ハザード**とは，特定のパイプラインがそのまま進行すると依存に違反してしまう状況—依存する命令が古い値を読んだり，実行すべきでないときに実行されたりする状況—である．

▷ hazard

データ依存によるハザードを**データハザード**，制御依存によるものを**制御ハザード**という．

▷ data hazard

▷ control hazard

5 段パイプラインでは:

- **RAW** 依存は，命令が十分近いときだけハザードになる．レジスタは WB で書かれ ID で読まれるので，ある命令は直前の命令とその前の命令に対してハザードを持つが，3 つ以上後ろの命令は書き込み後にレジスタを読む．
- **WAR** と **WAW** 依存はここではハザードにならない．命令は**プログラム順**に ID でレジスタを読み WB で書くので，後の命令が先の命令を追い越すことはないからである．これらはアウトオブオーダのパイプライン（第 6 章）でハザードになる．
- **制御**依存は常にハザードである．パイプラインは分岐が解決する前に後続命令をフェッチするからである．

これはレジスタファイルがサイクルの前半に書き込み，後半に読み出すことを仮定している（H&P §C.2）．そうでなければ 3 つ後ろの命令もハザードになる．

> **要点**
> 依存はプログラムが決め，ハザードはパイプラインが決める．同じプログラムでも，より深い，あるいはアウトオブオーダのパイプラインではハザードが増える．

3.7 ハザードの処理

ハザードを検出したら，パイプラインには 3 つの選択肢がある．

出典 Lesson 3，動画 15–16；H&P §C.2．

1. 依存する命令を**フラッシュ**する–誤った命令がフェッチされた制御ハザードの場合.
2. 必要な値が書き込まれるまで依存する命令を**ストール**する–常に正しいが，サイクルを失う.
3. **フォワーディング**（バイパス）で値を**補う**: 結果をレジスタファイルに到達するのを待たずに，生成したステージから必要とするステージへ直接送る（図 3.4）.

▷ forwarding

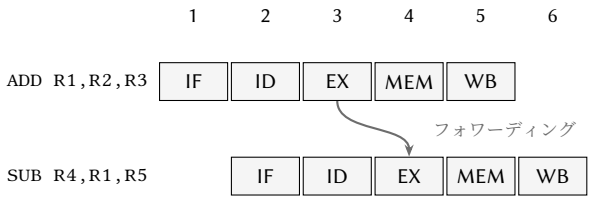


図 3.4: フォワーディング: ADD の ALU 結果を SUB の EX ステージへ直接送り，ストールを回避する.

フォワーディングは RAW ストールの大半を除去するが，すべてではない．値を**時間を遡って**送ることはできないからである．ロードの結果は MEM の終わりにならないと得られないが，次の命令はそれを自身の EX の開始時–同じサイクル–に必要とする．したがって**ロード-使用**の組では，完全なフォワーディングがあっても 1 サイクルのストールが避けられない.

第 9 章ではストアロードフォワーディングが加わる．ロードストアキューの中で，待機中のストアが後続のロードに値を渡す.

ヒント ストールサイクル数 =（値を使える最初のサイクル）−（値が必要なサイクル）．≤ 0 ならフォワーディングだけで足りる.

例 3.6.

```
LW   R1, 0(R2)    ; R1 は MEM の終わり（サイクル 4）に確定
ADD  R3, R1, R4   ; EX（サイクル 4）で R1 が必要: 1 ストール
```

フォワーディングありでストールなしだと ADD は古い R1 を使ってしまう．フォワーディングなしなら 2 サイクルストールする.

Skylake のコアでは L1 ヒットに 4 サイクルかかるため，ロード-使用の組のストールは 1 ではなく約 3 サイクルになる（Intel 最適化マニュアル）.

3.8 ステージ数はいくつにすべきか

k 段にパイプライン化するとスループットが最大 k 倍になるなら，なぜ数百段にしないのか．**パイプライン段数**を増やすと 3 つの効果が異なる方向に働く.

- **クロック周期が短くなる**（ステージ当たりの論理が減る）．これは性能の鉄則の T_{clk} を下げる，意図された利点である．しかしステージ当たりのラッチのオーバーヘッドは縮まないので，利得は頭打ちになる.
- **CPI が増える**: ステージが増えると，分岐とその解決の間に処理中の命令が増え，予測ミス当たりのフラッシュ命令数が増える．また RAW ハザードにストールが必要な距離も伸びる.
- **電力が増える**: 毎サイクル切り替わるラッチが増え，周波数の上昇が動的電力（式 (1.1)）を押し上げる.

性能のみ（CPI $\times T_{\text{clk}}$ の積）を考えると最適は深く，30–40 段程度である．電力を考慮に入れると最適はおよそ 10–15 段に移り，現在の設計もこれに近い.[2]

出典 Lesson 3，動画 17；H&P §C.1，§1.5.

▷ pipeline depth

注意 パイプラインを深くしても必ず速くなるとは限らない．決めるのは積 CPI $\times T_{\text{clk}}$ である（第 2 章）.

分岐予測ミス 1 回のコストは Skylake 級のコアで約 16–20 サイクルである（Agner Fog，マイクロアーキテクチャマニュアル）.

> **要点 — 章のまとめ**
> パイプライン化はスループットを改善するが，レイテンシは改善しない．理想 CPI は 1 で，すべてのストールとフラッシュがそれに加算される（式 (3.1)）．制御依存と RAW 依存はサイクルを失わせるハザードとなる．フォワーディングは RAW ストールの大半を除去し，分岐予測（次章）が制御ハザードに対処する．パイプラインの深さはクロック周期と CPI・電力とのトレードオフである.

[2] x86 で出荷された最も深いパイプラインは 2004 年の Pentium 4「Prescott」で，31 段，最高 3.8 GHz であり，それ以上の深化は電力に阻まれた．後継の Core マイクロアーキテクチャ（2006）は 14 段に戻り，以後の深さは 15–20 段前後にとどまっている.

第 4 章

分岐

講義動画	Lesson 4，動画 1–48
教科書	H&P [1] §3.3，§3.9，§C.2
キーワード	分岐予測，予測ミスペナルティ，不成立予測，分岐先バッファ，分岐履歴テーブル，2 ビット予測器，履歴ベース予測器，PShare，GShare，トーナメント予測器，階層型予測器，リターンアドレススタック，プリデコード

第 3 章では，制御依存がどのパイプラインでもハザードになることを見た．プロセッサは，分岐の行き先が分かるよりずっと前に分岐の後続命令をフェッチしなければならず，選択を誤るたびに作業がフラッシュされる．この章では，フェッチステージがその選択をどのように行うかを調べる．最も単純な方針—常に次の命令に進む—から出発して，実際のプロセッサで使われる構造を順に導入する．すなわち，分岐先バッファ，飽和カウンタ，分岐履歴からパターンを学習する予測器，複数の予測器の組合せ，そして関数からのリターン専用の予測器である．その過程で，パイプラインが深いほど予測精度が重要になる理由も示す．

4.1 パイプラインにおける分岐

出典 Lesson 4，動画 1–3；H&P §C.2．

第 3 章の 5 段パイプラインで，条件分岐 `BEQ R1,R2,L` を考える．R1 と R2 が等しければ，実行はラベル L から続く．このとき分岐は**成立分岐**であり，L のアドレスがその**分岐先アドレス**である．

▷ taken branch

▷ branch target address

そうでなければ，実行はメモリ上の次の命令，すなわち PC+4 から続き，分岐は**不成立分岐**となる．どちらの場合であるかをパイプラインが知るのは，分岐が 3 つのステージを通過した後である．IF が分岐をフェッチし，ID が R1 と R2 を読み，EX がそれらを比較して分岐先を計算する．分岐は EX の終わりに解決される（図 4.1）．

▷ not-taken branch

フェッチステージはそれを待てない．分岐が ID にあるサイクル 2 にも，続くサイクル 3 にも，フェッチステージは何らかの命令を読み込まなければならない．分岐が解決されるとき，その後ろにはすでに 2 命令がパイプラインに入っている．それらが本当に分岐の後に続く命令であれば何も失われないが，そうでなければフラッシュされ，2 サイクルが無駄になる．この間に何もフェッチしなければ同じ 2 サイクルが無条件に無駄になるので，正しい可能性が最も高い命令をフェッチすることが常に得策である．

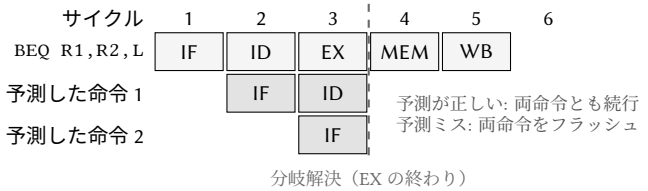


図 4.1: 5 段パイプラインにおける分岐．分岐が解決される前にフェッチされる 2 命令は予測であり，予測が誤っていればフラッシュされる．

定義 4.1 (分岐予測)．現在の命令がデコードも実行もされないうちに，次の命令をフェッチするアドレスをフェッチステージで選ぶことを**分岐予測**という．

▷ branch prediction

定義 4.2 (予測ミス)．**分岐予測ミス**は，解決された分岐が予測とは別の場所へ進むと判明したときに生じる．誤ったアドレスからフェッチされた命令はフラッシュされ，正しいアドレスからフェッチがやり直される．

▷ branch misprediction

4.1.1 予測器が知っていること

フェッチの時点で得られる情報は，フェッチ中の命令の PC だけである．命令そのものはまだデコードすらされていない．このアドレスだけから，予測器は 3 つの問いに答えなければならない．

1. その命令はそもそも分岐か．
2. 分岐であれば，成立するか．
3. 成立するならば，分岐先アドレスはどこか．

答えは次の PC という 1 つの数である．分岐でない命令と不成立分岐では PC+4，成立分岐では分岐先アドレスとなる．

無条件ジャンプ，コール，リターンは常に成立する分岐であり，これらに関係するのは問い 1 と 3 だけである．

4.2 予測精度と性能

予測器があれば，制御ハザードで失われるサイクルは，予測ミスした経路で費やされたサイクルだけになる．式 (3.1) を分岐に特化すると次式を得る．

出典 Lesson 4，動画 4–5；H&P §C.2．

$$\mathrm{CPI} \;=\; 1 \;+\; \frac{\text{予測ミス数}}{\text{命令}} \times \frac{\text{ペナルティ}}{\text{予測ミス}} \tag{4.1}$$

定義 4.3 (精度とペナルティ)．**分岐予測精度**とは，結果が正しく予測される分岐の割合である．したがって

▷ branch prediction accuracy

$$\frac{\text{予測ミス数}}{\text{命令}} = \frac{\text{分岐数}}{\text{命令}} \times (1 - \text{精度})$$

となる．ここで，分岐でない命令は次の PC を PC+4 として常に正しく予測されると仮定している．**予測ミスペナルティ**とは，予測ミス 1 回当たりに失われるサイクル数である．1 ステージに 1 命令を保持するパイプラインでは，フラッシュされる命令数に等しい．ステージ r で解決される分岐のペナルティは $r-1$ サイクルである．

▷ misprediction penalty

式 (4.1) の 2 つの因子は由来が異なる．第 1 の因子は予測器（およびプログラムが実行する分岐の数）で決まり，第 2 の因子はパイプライン，すなわち分岐がどれだけ遅く解決されるかで決まる．よりよい予測器は第 1 の因子を小さくし，より深いパイプラインは第 2 の因子を大きくする．そのため同じ予測器でも，深いパイプラインほどコストが大きくなる．完全な予測器であれば，深さによらず理想の CPI 1 に到達する．

例 4.4. あるプログラムでは命令の 15 % が分岐であり，予測器の精度は 92 % である．命令当たりの予測ミス数は $0.15 \times 0.08 = 0.012$ である．分岐が EX（ステージ 3，ペナルティ 2）で解決されるなら $\mathrm{CPI} = 1 + 0.012 \times 2 = 1.024$ となる．分岐をステージ 11（ペナルティ 10）で解決するパイプラインでは，同じ予測器で $\mathrm{CPI} = 1 + 0.012 \times 10 = 1.12$ となる．

4.3 不成立予測

予測に代わる方法は，予測を拒むことである．フェッチステージは次の PC が分かるまで単に待つ．命令が分岐かどうかさえ ID の後にならないと分からないので，この待ちはすべての命令に影響する．分岐でない命令の次のフェッチはその命令自身のフェッチの 2 サイクル後になり，分岐の次のフェッチは，成立・不成立にかかわらず 3 サイクル後になる．

出典 Lesson 4，動画 6–8；H&P §C.2．

表 4.1: 5 段パイプラインにおいて，ある命令のフェッチから正しい次の命令がフェッチされるまでのサイクル数．

命令	予測を拒む	不成立予測
成立分岐	3	3
不成立分岐	3	1
分岐以外	2	1

最も単純な実用的予測器は**不成立予測**，すなわち常に PC+4 からフェッチすることである．フェッチステージはどのみち PC をインクリメントするので，これにはメモリも，フェッチステージがすでに持つもの以外のハードウェアも必要ない．分岐でない命令と不成立分岐に対しては常に正しく，誤るのは成立分岐だけであり，そのコストは予測を拒む場合と同じく 3 サイクルである．表 4.1 は 2 つの方針を比較している．不成立予測が劣ることは決してない．

▷ predict not taken

しかし，その精度は低い．大半の分岐は成立する—ループの分岐はたいてい後方へジャンプし，無条件ジャンプ，コール，リターンは常に成立する—ので，不成立予測は分岐の過半数を予測ミスする．第 3 章で用いた典型的な統計では，全命令の 12 % で予測ミスが起こり，式 (3.2) の CPI となる．

注 4.5 (処理中の複数の予測)**.** 深いパイプラインでは，IF と解決ステージの間に複数の分岐が同時に存在しうる．それぞれの分岐は，フェッチ時に行われた予測を伴っている．ある分岐が予測ミスと判明すると，その後にフェッチされたものは，後続の分岐とその予測も含めてすべてフラッシュされる．したがって，誤った経路でフェッチされた分岐がそれ自身のフラッシュを引き起こすことはない．サイクルを失わせるのは，正しい経路上にある分岐の予測ミスだけである．

4.4 よりよい予測の必要性

出典 Lesson 4，動画 9–11；H&P §3.3．

よりよい予測器にどれだけの価値があるかは，パイプラインに依存する．1 ステージ当たり w 命令を扱うプロセッサは，同じ深さで幅が 1 命令のパイプラインと比べて，予測ミス 1 回当たり w 倍の命令をフラッシュする．したがって，完全な予測に対する相対実行時間は

$$\frac{T}{T_{\text{perfect}}} = 1 + \frac{\text{予測ミス数}}{\text{命令}} \times (\text{予測ミス当たりのフラッシュ命令数}) \qquad (4.2)$$

であり，$w = 1$ のときはちょうど式 (4.1) の CPI になる．

実際の値：Skylake 級のコアは予測ミス 1 回につき約 16–20 サイクルを失い（Agner Fog，マイクロアーキテクチャマニュアル），224 エントリのリオーダバッファを埋め直す（Intel 最適化マニュアル）．

例 4.6. 命令の 15 % が分岐で，そのうち 70 % が成立するとする．このとき不成立予測の精度は 30 % である．表 4.2 は，3 種類の予測器と 3 種類のパイプラインについて式 (4.2) を評価したものである．パイプラインは，5 段パイプライン（フラッシュ 2 命令），分岐をステージ 13 で解決するパイプライン（フラッシュ 12 命令），および同じ深さで 1 ステージ当たり 3 命令を扱うパイプライン（フラッシュ 36 命令）である．

注意 予測ミスは分岐単位ではなく命令単位で数える．1 − 精度 に，命令のうち分岐が占める割合を掛ける．

表 4.2: 例 4.6 の分岐統計における，完全な予測に対する相対実行時間．

予測器	予測ミス/命令	ステージ 3	ステージ 13	ステージ 13，幅 3
不成立（30 %）	0.105	1.21	2.26	4.78
精度 90 %	0.015	1.03	1.18	1.54
精度 99 %	0.0015	1.003	1.018	1.054

ここから 3 つのことが分かる. 浅いパイプラインでは, 不成立予測でさえコストは 21 % にすぎず, 精度を 90 % から 99 % に改善しても利得は 3 % に満たない. 深いパイプラインでは, 不成立予測は実行時間を 2 倍以上にし, 同じ改善は $1.18/1.018 \approx 1.16$ の高速化に相当する．深く幅も広いパイプラインでは $1.54/1.054 \approx 1.46$ に相当する．現代のプロセッサは深く，かつ幅も広い．だからこそ分岐予測に多大な投資をしているのである．

ヒント（命令当たりの予測ミス数）×（フラッシュ命令数）がそのまま性能低下率になる．$0.015 \times 12 = 0.18$，すなわち中段の 18 % である．

> **要点**
> 重要なのは精度ではなく**予測ミス**率である．精度 90 % から 99 % への向上は小さな一歩に見えるが，予測ミスの 90 % を取り除く．そしてパイプラインが深く幅広いほど，残った予測ミス 1 回当たりのコストは大きくなる．

4.5 分岐先バッファ

不成立予測は，フェッチ中の命令に関する情報を何も使わない．よりよい予測器はその**履歴**，すなわちこの PC にある命令が過去に実行されたときに何をしたかを用いる．次の PC は，PC とこの履歴の関数として予測される．あるアドレスに格納された命令は変化しないので，それが分岐かどうか，直接分岐がどこへジャンプするかは毎回同じである．また，ほとんどの分岐はたいてい同じ方向に進む．

出典 Lesson 4，動画 12–15；H&P §3.9；[6]．

定義 4.7 (分岐先バッファ). **分岐先バッファ**（BTB）とは，フェッチ中の命令の PC でインデックスされ，その命令の予測された次の PC を格納するテーブルである．

▷ branch target buffer

IF では現在の PC で BTB を読み，その出力を次の PC として用いる．分岐が解決されると，実際の次の PC を予測と比較する．一致しなければ，誤ってフェッチされた命令をフラッシュし，実際の分岐先を BTB のエントリに書き込む．こうして，同じ命令の次の実行は新しい分岐先で予測される．

4.5.1 現実的な BTB

BTB は，あり得るすべての命令アドレスに 1 つずつエントリを持つことはできない．BTB はフェッチそのものに加えて同じフェッチサイクル内で読まなければならないので，小さなテーブル—数百から数千エントリ—に限られる．そこで PC は単純なハッシュ，すなわちその下位の数ビットによってエントリに対応付けられる．ループの命令のように時間的に近接して実行される命令は，メモリ上でも近接している．それらは上位ビットを共有するが下位ビットが異なるので，別々のエントリに分散される．2 つの命令が衝突するのはコード上で遠く離れている場合だけであり，そのような命令が同時に活発に実行される可能性は低い．

実際の容量：Intel Skylake の BTB は約 4096 エントリ（一般に報告されている値），AMD の Zen 2 は 16，512，7168 エントリの 3 段構成である（Agner Fog，マイクロアーキテクチャマニュアル）．

ワード境界に整列した 4 バイト固定長命令では，すべての PC の下位 2 ビットはゼロであり情報を持たない．そのため 2^k エントリの BTB はビット $k+1$ から 2 でインデックスされる（図 4.2）．可変長命令（x86）では，インデックスはビット 0 から始まる．

ヒント エントリ番号 = (PC ≫ 2) & (エントリ数 − 1)．常にゼロの 2 ビットを捨て，下位 k ビットを残す．

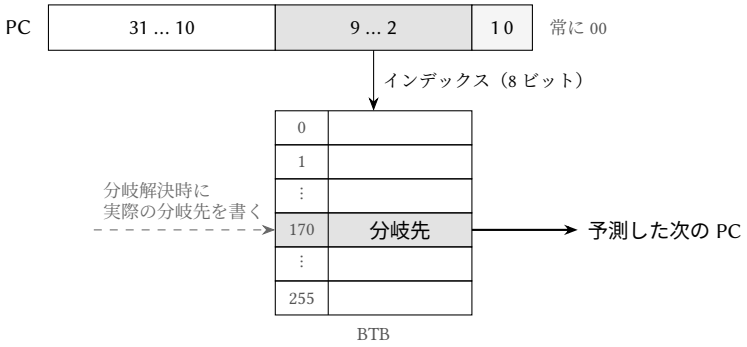


図 4.2: PC のビット 9–2 でインデックスされる 256 エントリの BTB．エントリは IF で読まれ，分岐が解決されると実際の分岐先で書き込まれる．

例 4.8. 256 エントリの BTB では，0x004012A8 にある命令は PC のビット 9–2 を用いる．(0x12A8 ≫ 2) & 0xFF = 0x4AA & 0xFF = 0xAA，すなわちエントリ 170 である．次の命令（0x004012AC）はエントリ 171 を用い，256 命令先の 0x004016A8 にある命令はエントリ 170 を共有する．

4.6 方向予測

BTB だけでも，方向の問いには暗黙のうちに答えている．PC+4 を保持するエントリは「不成立」を意味する．これでは，分岐でない命令と不成立分岐のすべてに分岐先アドレス 1 つ分の記憶を費やすことになる．予測を分割する方が安価である．**方向予測器**が，その命令が成立分岐であるかどうかを判断し，成立分岐と判断したときにだけ BTB を参照する．

出典 Lesson 4，動画 16–21；H&P §C.2．

▷ direction predictor

定義 4.9 (分岐履歴テーブル). **分岐履歴テーブル**（BHT）とは，PC の下位ビットでインデックスされるテーブルの形をとる方向予測器である．最も単純な形では，各エントリは分岐の前回の実行結果を保持する 1 ビットであり，0 なら次の PC として PC+4 を，1 なら BTB の分岐先を選ぶ．これが **1 ビット予測器**（1BP）である．

▷ branch history table

▷ one-bit predictor

分岐が解決されると，その BHT エントリには実際の結果が設定され，分岐が成立した場合には分岐先が BTB に書き込まれる（図 4.3）．この構成には 3 つの利点がある．分岐でない命令の BHT エントリは書き込まれることがなく 0 のままなので，PC+4 を正しく予測する．BTB に必要なのは成立分岐のためのエントリだけであり，成立分岐は全命令よりはるかに少ない．そして BHT のエントリは 1 ビットなので，同じコストで BHT は BTB よりはるかに多くのエントリを持つことができ，異なる分岐がエントリを共有することはまれになる．

注意 どちらのテーブルにもタグがない．成立分岐が同じエントリに割り当てられると，分岐でない命令も成立と予測され，誤りはデコード時にしか分からない．

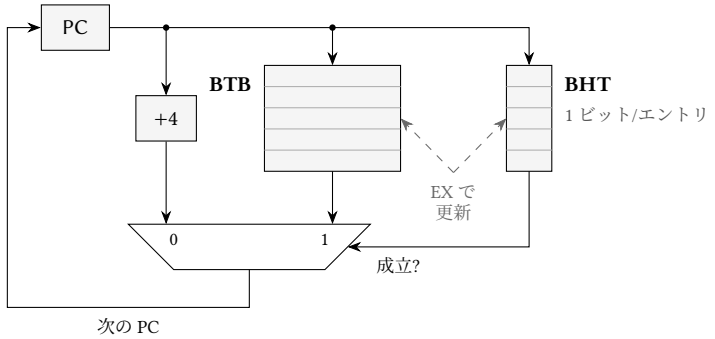


図 4.3: BHT と BTB による次の PC の予測．BHT のビットが PC+4 と BTB の分岐先のどちらかを選ぶ．分岐が解決されると両方のテーブルが更新される．

例 4.10. ある内側ループは，入るたびに配列の 3 要素を処理する．

```
0x0400  Inner: LW   R2, 0(R3)      ; 要素をロード
0x0404         ADD  R5, R5, R2     ; 合計に加算
0x0408         ADDI R3, R3, 4      ; 次の要素
0x040C         ADDI R1, R1, -1     ; カウントを減らす
0x0410         BNE  R1, R0, Inner  ; 1 回の実行で T, T, N
```

64 エントリの BHT（インデックスはビット 7–2）では，5 つの命令は BHT エントリ 0 から 4 を用い，BNE はエントリ 4 を用いる．8 エントリの BTB（ビット 4–2）では，BNE は BTB エントリ 4 を用い，これを 8 命令前の 0x03F0 にある命令と共有する．その命令が分岐でなければ，その BHT エントリ（エントリ 60）は 0 のままなので，BTB エントリを使うのはこの分岐だけである．表 4.3 は，すべてのテーブルがクリアされた状態から始めて，ループ 2 回の実行にわたり BNE を追跡したものである．

表 4.3: 例 4.10 の BNE（BHT エントリ 4）の，ループ 2 回の実行にわたる 1 ビット予測．

実行	1	2	3	4	5	6
実行前の BHT[4]	0	1	1	0	1	1
予測した次の PC	0x414	0x400	0x400	0x414	0x400	0x400
結果	T	T	N	T	T	N
正しいか	いいえ	はい	いいえ	いいえ	はい	いいえ

最初の実行では，BTB エントリ 4 に 0x400 も書き込まれる．それ以降，分岐先は常に正しいが，方向は 6 回の実行のうち 4 回で誤っている．

4.7 2 ビット予測器

1 ビット予測器は常に同じ方向に進む分岐にはよく機能するが，2 つの弱点がある．

出典 Lesson 4，動画 22–26；H&P §C.2；[7]．

- ほとんど常に一方向に進む分岐は，例外 1 回につき *2* **回**の予測ミスを被る．1 回は例外そのもので，もう 1 回はその次の通常の結果である．例外によってビットが反転してしまうからである．
- 方向が頻繁に変わる分岐はうまく予測できない．交互の分岐（T, N, T, N, ...）は毎回予測ミスし，表 4.3 の短いループは 3 回に 2 回予測ミスする．

定義 4.11 (2 ビット予測器). **2 ビット予測器**（2BP）は，2 ビットカウンタ（2BC）とも呼ばれ，各 BHT エントリに，分岐が成立すると増加し，不成立だと減少する 2 ビットカウンタを格納する．

▷ two-bit predictor

▷ saturating counter

このカウンタは**飽和カウンタ**である．すなわち，11 で増加させても 11 に留まり，00 で減少させても 00 に留まる．上位ビットは**予測ビット**であり，1 なら成立を予測する．下位ビットは**ヒステリシスビット**である．下位ビットは予測ビットと合わせて，カウンタが自らの予測をどれだけ強く信じているかを記録する．2 ビットが等しいときカウンタは強い状態に，異なるときは弱い状態にあるので，4 つの状態は強い不成立（00），弱い不成立（01），弱い成立（10），強い成立（11）である（図 4.4）．強い状態から予測を変えるには，予測に反する結果が 2 回続く必要があるので，単発の例外による予測ミスは 1 回で済む．

ヒント 2 ビットを 0–3 の数とみなすとよい．T で 1 増え，N で 1 減り，両端で飽和する．値が 2 以上なら成立と予測する．

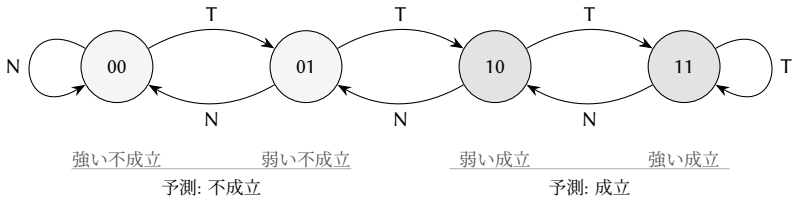


図 4.4: 2 ビット飽和カウンタの状態遷移図．T（成立）で右へ，N（不成立）で左へ移る．上位ビットが予測である．

表 4.4: 例 4.10 のループ分岐に対する，00 から始まる 2 ビットカウンタによる予測（ループ 3 回の実行）．予測ミスを × で示す．

実行	1	2	3	4	5	6	7	8	9
結果	T	T	N	T	T	N	T	T	N
実行前のカウンタ	00	01	10	01	10	11	10	11	11
予測	N	N	T	N	T	T	T	T	T
2BP	×	×	×	×		×			×
1BP（比較用）	×		×	×		×	×		×

表 4.4 は短いループに対する効果を示している．カウンタがいったん強い成立（11，実行 6 の前）に達すると，1 回のループ脱出ではカウンタは弱い成立（10）に下がるだけで，次の成立の結果で元に戻る．そのため予測ミスするのは脱出だけであり，予測ミスはループ 1 回の実行につき 2 回ではなく 1 回になる．

注意 弱い状態からは，予測に反する結果 1 回で予測が変わる．2 ビットカウンタが予測を変えるのに，常に 2 回の予測ミスが必要なわけではない．

4.7.1 初期化とビット数の増加

強い状態に初期化されたカウンタは，分岐が実際には逆方向に進む場合に 2 回の予測ミスを招く．弱い状態に初期化されたカウンタなら 1 回で済む．しかし弱い状態は，カウンタが最も脆弱な状態でもある．交互の分岐 T, N, T, N, ... は，カウンタが 01 から始まると 01 と 10 の間を行き来して毎回予測ミスするが，00 から始まれば予測ミスは 1 回おきで済む．どちらの選択もすべての分岐にとって最善ではなく，どの初期状態にも，それが不利になる結果の系列が存在する．そこでカウンタは，単に最も単純な状態である 00 にリセットされる．

どの予測器も万能ではない．どんな予測器に対しても，常にその時点の予測に反する結果の系列が存在する．

3 ビット以上のカウンタはヒステリシスを増やす．すなわち，予測を変えるのにより多くの例外が連続する必要がある．これが役立つのは例外がまとまって生じる分岐だけであり，振る舞いを本当に変えた分岐への追従を遅くし，ビット数も増える．1 ビットから 2 ビットへの変更は大きな改善だが，それ以上ビットを増やしても得るものは少ない．2 ビットカウンタが標準的な構成要素であるのはこのためである．

4.8 履歴ベース予測器

分岐の中には，カウンタでは捉えられない規則的なパターンに従うものがある．カウンタはどちらの方向が多いかを追跡するだけだからである．ループインデックスが偶数かどうかを判定する分岐は T, N, T, N と交互に変わり，例 4.10 のループ分岐は T, T, N を繰り返す．どちらも直近の結果が分かれば完全に予測できる．

出典 Lesson 4，動画 27–33；H&P §3.3；[8]．

定義 4.12 (履歴ベース予測器). **履歴ベース予測器**は，分岐の直近の結果（その履歴）を記録し，それを用いて複数のカウンタから 1 つを選ぶ．各カウンタは，ある特定の履歴の後に分岐が何をするかを学習する．

▷ history-based predictor

4.8.1 1 ビット履歴

最も単純な構成では，各 BHT エントリに 1 ビットの履歴（分岐の前回の結果）と 2 つの 2 ビットカウンタを格納する．一方は前回の結果が N のときに，他方は T のときに使われる．予測では，履歴ビットがカウンタを選び，そのカウンタの上位ビットが予測となる．更新では，選ばれたカウンタを実際の結果に従って増加または減少させ，その結果を新しい履歴ビットとする．エントリには $1+2\times2=5$ ビットが必要である．

交互に変わる分岐では，N の後に選ばれるカウンタは T だけを，T の後に選ばれるカウンタは N だけを見る．それぞれの学習が済めば，分岐は毎回正しく予測される．パターン T, T, N にはこれでは足りない．T の後で分岐は成立することもしないこともあるので，T の後に選ばれるカウンタは予測ミスを繰り返す．

4.8.2 2 ビット履歴と N ビット履歴

履歴 2 ビットとカウンタ 4 つ（エントリ当たり 10 ビット，図 4.5）では，可能な 4 つの履歴 NN，NT，TN，TT のそれぞれが専用のカウンタを持つ．T, T, N では，履歴 TT の後は常に N であり，TN と NT の後は常に T である．表 4.5 はループ分岐を追跡したものである．カウンタの学習が済むと，すべての実行が正しく予測される．NN のカウンタは二度と使われない．

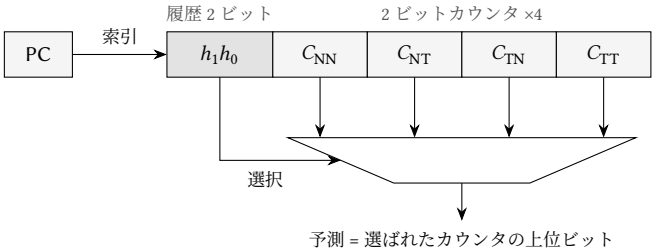


図 4.5: 2 ビット履歴予測器のエントリ．履歴ビットが 4 つの 2 ビットカウンタから 1 つを選ぶ．分岐が解決されると，選ばれたカウンタが更新され，結果が履歴にシフトインされる．

表 4.5: 例 4.10 のループ分岐に対する 2 ビット履歴予測器．履歴 NN，全カウンタ 00 から始める．履歴は古い結果から順に書き，予測ミスを × で示す．

実行	1	2	3	4	5	6	7	8	9	10	11	12
履歴	NN	NT	TT	TN	NT	TT	TN	NT	TT	TN	NT	TT
使うカウンタ	00	00	00	00	01	00	01	10	00	10	11	00
予測	N	N	N	N	N	N	N	T	N	T	T	N
結果	T	T	N	T	T	N	T	T	N	T	T	N
予測ミス	×	×		×	×		×					

定義 4.13 (N ビット履歴予測器). **N ビット履歴予測器**は，各分岐の直近 N 回の結果と，エントリ当たり 2^N 個の 2 ビットカウンタ（可能な履歴ごとに 1 つ）を保持する．1 エントリのコストは次のとおりである．

▷ N-bit history predictor

$$N + 2 \cdot 2^N \text{ ビット} \tag{4.3}$$

学習後，N ビット履歴予測器は，周期が $N+1$ 以下のあらゆる繰り返しパターンを正しく予測できる．そのようなパターンでは，直近 N 回の結果が次の結果を決めるからである．コストは N に対して指数的に増大し，その大半は無駄になる．周期 p の分岐は，2^N 個のカウンタのうち高々 p 個しか使わない．また，使われるカウンタはそれぞれ個別に学習しなければならないので，履歴が長いほど学習にも時間がかかる．

例 4.14. 常に 10 回反復するループの分岐は，周期 10 のパターン TTTTTTTTTN を持つ．これを完全に予測するには $N = 9$，すなわち BHT エントリ当たり 9+2·512 = 1033 ビットが必要であり，分岐が使うのは 512 個のカウンタのうち 10 個だけである．そのようなエントリ 1024 個からなる BHT は 1 メガビットを超える．

> **要点**
> 履歴により，予測器は偏りだけでなくパターンを学習できる．しかし，分岐ごとに専用の 2^N 個のカウンタを与えると，長い履歴は法外に高価になり，しかもそれらのカウンタのほとんどは一度も使われない．

4.9 共有カウンタ: PShare と GShare

カウンタを分岐専用にしなければ，この無駄はなくなる．BHT はエントリごとに N ビットの履歴だけを保持し，すべての分岐が 2 ビットカウンタからなる 1 つの大きなテーブルを共有する．

出典 Lesson 4，動画 34–39；H&P §3.3；[9]．

定義 4.15 (パターン履歴テーブル). **パターン履歴テーブル**（PHT）とは，すべての分岐が共有する 2 ビットカウンタのテーブルであり，分岐の履歴と PC の組合せ，典型的にはそれらのビットごとの XOR でインデックスされる．

▷ pattern history table

分岐が占めるカウンタは，その分岐が持つ異なる履歴の数だけになる．常に成立する分岐の履歴は TT…T であり，カウンタを 1 つしか使わない．交互に変わる分岐は 2 つ，例 4.14 の 10 回反復ループの分岐は 10 個を使う．PC のビットを混ぜることで，同じ履歴を持つ異なる分岐が異なるカウンタに送られる．衝突の可能性は残るが，PHT が大きければまれである．

例 4.16. 12 ビットの履歴を保持する 1024 エントリの BHT は 12 288 ビットを，$2^{12} = 4096$ 個のカウンタからなる PHT は 8192 ビットを占め，合計 20 480 ビットとなる．12 ビット履歴と専用カウンタを持つ 1024 エントリの予測器では $1024 \times (12 + 2 \cdot 4096) = 8\,400\,896$ ビットが必要であり，これは約 410 倍である．

4.9.1 ローカル履歴とグローバル履歴

PHT をインデックスする履歴の選び方は 2 通りある（図 4.6）．

定義 4.17 (PShare). 分岐の**ローカル分岐履歴**とは，その分岐自身の直近の結果の系列であり，BHT エントリに保持される．**PShare 予測器**は，ローカル履歴と PC の XOR で共有 PHT をインデックスする．

▷ private branch history
▷ PShare predictor

定義 4.18 (GShare). **グローバル分岐履歴**とは，すべての分岐の直近の結果の系列であり，解決された各分岐が結果をシフトインする単一のシフトレジスタに保持される．**GShare 予測器**は，グローバル履歴と PC の XOR で共有 PHT をインデックスする．

▷ global branch history
▷ GShare predictor

PShare は，独自のパターンを持つ分岐，すなわち交互に変わる分岐や短いループに適している．GShare は，直前に実行された他の分岐の結果に依存する**相関のある**分岐に適している．

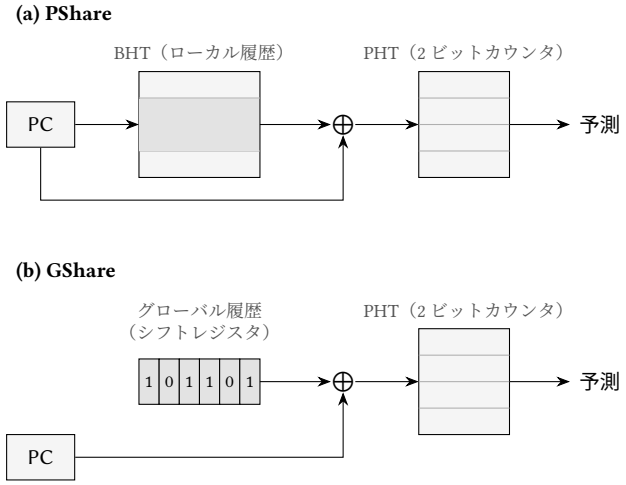


図 4.6: 共有カウンタを用いる予測器．(a) PShare は分岐自身の履歴を PC と組み合わせ，(b) GShare は全分岐のグローバル履歴を PC と組み合わせる．

例 4.19. 次の断片では，2 番目の分岐は，1 番目の分岐が不成立のときに成立し，成立のときに不成立となる．

```
        BNE  R1, R0, Skip1    ; R1 == 0 でなければスキップ
        ADDI R2, R2, 1        ; ゼロの個数を数える
Skip1:  BEQ  R1, R0, Skip2    ; R1 == 0 ならスキップ
        ADD  R3, R3, R1       ; 非ゼロの値を合計
Skip2:  ...
```

R1 の値がデータに依存するなら，どちらの分岐のローカル履歴もランダムに見え，PShare はうまく予測できない．2 番目の分岐から見たグローバル履歴には 1 番目の分岐の結果が含まれるので，これらの履歴に対応する PHT カウンタの学習が済めば，PHT でのエイリアシングがない限り，GShare は 2 番目の分岐を完全に予測する．

結果がデータに依存する分岐は，予測するより取り除く方が安上がりなことがある．第 5 章はそのような分岐をプレディケーションで置き換える．

どちらもすべての分岐に対して優れているわけではない．GShare も原理的には分岐自身のパターンを学習できるが，その間に実行された他の分岐の結果がグローバル履歴に混ざり込むので，同じパターンに PShare よりかなり長い履歴—そしてより大きな PHT—が必要になる．プログラムには両方の種類の分岐が含まれるので，自然な次の一歩は両方の予測器を使うことである．

注意 標準的な名称は GShare（McFarling）だけである．講義が PShare と呼ぶものは，文献ではローカル履歴予測器（Yeh と Patt の PAs 方式）と呼ばれる．

4.10 トーナメント予測器と階層型予測器

分岐によって最適な予測器が異なるので，プロセッサは複数の予測器を組み合わせ，分岐ごとにそれに適したものを使うことができる．組合せ方には対称なものと階層的なものがある．

出典 Lesson 4，動画 40–43；H&P §3.3；[9]．

4.10.1 トーナメント予測器

トーナメント予測器は，2 つの予測器，例えば GShare と PShare を並行して動作させ，どちらを信頼するかを分岐ごとに個別に決める．

▷ tournament predictor

定義 4.20 (メタ予測器)**.** トーナメント予測器の**メタ予測器**とは，PC でインデックスされる 2 ビットカウンタのテーブルであり，ある分岐に対応するカウンタが，2 つの構成予測器のどちらが正しいかを予測する（図 4.7）．

▷ meta-predictor

両方の構成予測器は，通常どおりすべての分岐で学習する．メタ予測器のカウンタが変化するのは構成予測器の正誤が分かれたときだけであり，そのときは正しかった方へ向かって動く．両方とも正しいとき，または両方とも誤ったときは変化しない．

Alpha 21264 は，10 ビットのローカル履歴 1024 個と 3 ビットのローカルカウンタ 1024 個，12 ビットのグローバル履歴で引く 4096 エントリの表，4096 エントリの選択器を組み合わせ，合計約 29 キロビットであった（H&P §3.3）．

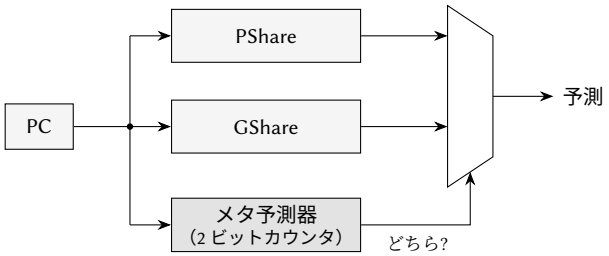


図 4.7: トーナメント予測器．メタ予測器が，どの構成予測器の予測を使うかを分岐ごとに選ぶ．

4.10.2 階層型予測器

トーナメント予測器は，よい–したがって高価な–予測器を 2 つ組み合わせ，すべての分岐で両方を更新する．しかし，ほとんどの分岐は予測が容易である．

定義 4.21 (階層型予測器). **階層型予測器**は，すべての分岐を扱う安価な予測器と，安価な予測器がうまく扱えない分岐に対してだけ使用・学習される，より優れた高価な 1 つ以上の予測器とを組み合わせる．

▷ hierarchical predictor

高価な予測器が学習すべき分岐ははるかに少ないので，同じ精度ならトーナメント予測器の場合よりずっと小さくでき，同じサイズなら精度をより高くできる．

典型的な構成は 3 レベルからなる（図 4.8）．PC でインデックスされる 2 ビットカウンタの大きなテーブルが，すべての分岐を予測する．その上に，小さなローカル履歴予測器と小さなグローバル履歴予測器があり，選ばれた分岐のエントリだけを保持する．それらのエントリには PC のビットからなる**タグ**が付けられており，参照時にその分岐のエントリがあるかどうかが分かる．予測は，一致するエントリを持つ最も上位のレベルから得る．カウンタテーブルは常に更新され，分岐に一致するローカルまたはグローバルのエントリも同様に更新される．カウンタテーブルが予測ミスした分岐にはローカル予測器のエントリが与えられ，ローカル予測器が予測ミスした分岐にはグローバル予測器のエントリが与えられる．カウンタでよく予測できる分岐が高価なレベルのエントリを占めることはまれなので，それらの小さなテーブルは，エントリを必要とする分岐のために残される．[1]

AMD の Zen 2 は速度で階層化する．高速なパーセプトロンが予測し，低速な TAGE 予測器がそれを上書きする．予測ミスは Zen 1 より約 30 % 少ない（AMD）．

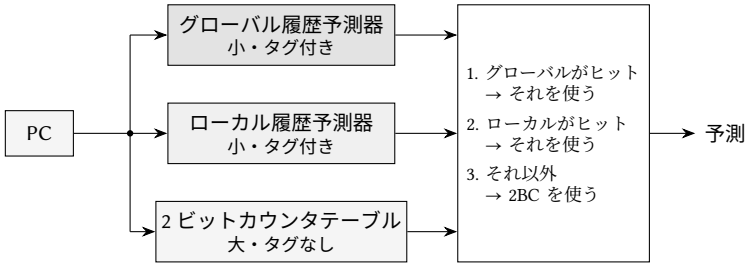


図 4.8: 階層型予測器．タグ付きの上位レベルは，保持している分岐について 2 ビットカウンタテーブルの予測を置き換える．

[1] Intel の Pentium M（2003）が典型例である．全分岐を対象とする 2 ビットカウンタの表を，タグ付きのローカル履歴予測器とグローバル履歴予測器，ループ分岐の反復回数を学習するループ検出器，そして間接分岐専用の予測器が補う（Gochman ら，*Intel Technology Journal* 7(2)，2003）．

表 4.6 は，どの種類の分岐をどの種類の予測器が扱うかをまとめたものである．

表 4.6: 分岐の振る舞いと，それをうまく扱える最も単純な予測器．

分岐の振る舞い	予測器
ほとんど常に同じ方向	2 ビットカウンタ
独自の繰り返しパターン（交互，短いループ）	ローカル履歴（PShare）
先行する分岐と相関がある	グローバル履歴（GShare）
1 つのプログラム内で上記が混在	トーナメントまたは階層型
関数からのリターン	リターンアドレススタック（次節）

4.11 リターンアドレススタック

関数からのリターン（RET）は無条件分岐なので，方向の予測は自明であり，常に成立と予測すればよい．しかし，その分岐先は関数がどこから呼ばれたかに依存する．BTB は最新の分岐先しか覚えていないので，関数が異なる場所から代わる代わる呼ばれると，リターンは毎回予測ミスする．

出典 Lesson 4，動画 44–47；H&P §3.9；[10]．

例 4.22. ある関数が 2 か所から呼ばれる．

```
0x1000          CALL Func     ; 戻りアドレス 0x1004
  ...
0x1040          CALL Func     ; 戻りアドレス 0x1044
  ...
0x2000  Func:   ...
0x2030          RET           ; まず 0x1004 へ，次に 0x1044 へ
```

1 回目のリターンが解決されると，RET の BTB エントリには 0x1004 が入る．このリターン自体は，エントリがまだ空だったので予測ミスする．0x1044 へ戻る 2 回目のリターンは予測ミスし，エントリを上書きする．2 か所からの呼び出しが交互に起こると，リターンは毎回予測ミスする．

定義 4.23 (リターンアドレススタック). **リターンアドレススタック**（RAS）とは，フェッチステージにある小さなハードウェアスタックである．コールがフェッチされると，その戻りアドレス（PC+4）をプッシュする．リターンがフェッチされると，先頭のエントリをポップし，予測した分岐先として用いる．

▷ return address stack

コールとリターンは入れ子になるので，RAS はリターンをほぼ完全に予測する（図 4.9）．RAS は小さいので，コールが深く入れ子になると満杯になりうる．RAS が満杯のときにコールがフェッチされると，プッシュはラップアラウンドして最も古いエントリを上書きする．リターンはコールの逆順に起こるので，最も早く必要になるのは最新のエントリであり，またほとんどのコールとリターンはコールツリーの深い位置で起こる．最も古いエントリを上書きしても，失われるのは最も外側の関数からのリターンの予測だけであり，それはまれである．一方，新しい戻りアドレスを捨てれば，最も深いレベルで頻繁に起こるコールの予測が失われる．

実際のリターンアドレススタックは，Intel のコアで 16 エントリ，AMD の Zen 2 で 32 エントリである（Agner Fog，マイクロアーキテクチャマニュアル）．

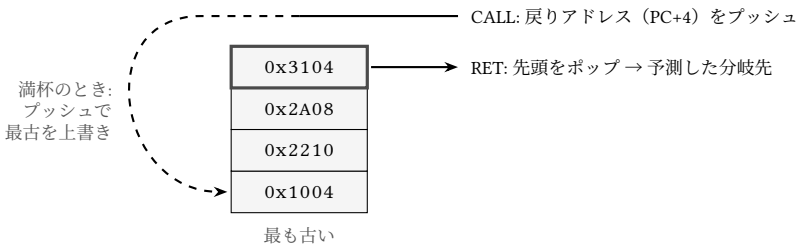


図 4.9: 4 エントリのリターンアドレススタック．コールでプッシュし，リターンでポップする．スタックが満杯のときは，プッシュがラップアラウンドして最も古いエントリを上書きする．

4.11.1 リターンの識別

RAS は，RET がデコードされる前，すなわちフェッチされている最中に使わなければならない．命令がリターンであることを知る方法は 2 つある．

- **予測する．** 予測器のテーブルに，PC ごとにその命令がリターンかどうかを格納する．エントリは，その命令が初めてデコードされたときに設定される．あるアドレスにある命令は変化しないので，この予測はほぼ常に正しい．

- **プリデコードする．プリデコード**では，プロセッサは各命令をメインメモリからプロセッサのキャッシュに取り込む際に部分的にデコードし，例えばそれが分岐かリターンかといった数ビットの追加情報を，キャッシュされた命令とともに格納する．フェッチステージはこれらのビットを命令と一緒に読むので，命令の種類がただちに分かる．

▷ predecoding

要点 — 章のまとめ
すべてのフェッチは予測である．予測ミスは，それがフラッシュするすべての命令の作業を無駄にし（式 (4.2)），フラッシュされる命令数はパイプラインの深さと幅とともに増える．不成立予測はコストがかからないが不正確である．BTB は分岐先を，BHT は方向を予測し，BHT は 2 ビット飽和カウンタを用いると最もよく機能する．履歴ベース予測器はパターンを学習する．そのカウンタを PHT で共有すれば長い履歴も現実的なコストで実現でき，自己反復する分岐にはローカル履歴（PShare），相関のある分岐にはグローバル履歴（GShare）を用いる．トーナメント予測器と階層型予測器は複数の予測器を組み合わせ，リターンアドレススタックは関数からのリターンを予測する．

第 5 章

プレディケーション

講義動画	Lesson 5，動画 1–12
教科書	H&P [1] §A.6，§C.2，§H.4，§H.6
キーワード	プレディケーション，予測困難な分岐，if 変換，条件付きムーブ，MOVZ，MOVN，CMOVcc，条件コード，損益分岐予測ミス率，完全プレディケーション，Itanium，述語レジスタ

第 4 章の予測器はほとんどの分岐を正しく予測するが，どの予測器にとっても予測が困難な分岐が残る．典型的には，分岐の方向が処理対象のデータに依存するためである．また深いパイプラインでは，予測ミスのたびに数十命令がフラッシュされる．この章では，そのような分岐に対する代替手段である**プレディケーション**を調べる．プレディケーションは分岐の両方向の処理を実行し，正しい方向の結果だけを残すので，予測ミスの対象となるものが何も残らない．どの分岐がその恩恵を受けるか，コンパイラが if 変換によって分岐をどのように除去するか，それが前提とする条件付きムーブ命令はどのようなものか，この変換が有利になるのはどのような場合か，そしてすべての命令を条件付きにできる命令セットはどのようなものかを，順に調べる．

5.1 予測とプレディケーション

出典 Lesson 5，動画 1–2；H&P §C.2．

分岐予測器は，分岐がどこへ進むかをフェッチステージで推測し，プロセッサは推測した経路に沿ってフェッチを続ける．推測が正しければコストはかからない．推測が誤っていれば，分岐の後にフェッチされたすべての命令がフラッシュされる．幅が数命令で段数も多い現代のプロセッサでは，これは数十命令に達する（例 4.6）．したがって分岐のコストは，ほとんどの場合ゼロであり，ときどき大きくなる．

定義 5.1 (プレディケーション). 分岐の**両方**の方向の処理を実行し，誤りと判明した方向の結果を捨てることを**プレディケーション**という．

▷ predication

プレディケーションは何も予測しないので予測ミスペナルティがないが，不要な方向の処理を常に無駄にする．両方向の命令数が等しければ，処理の半分が捨てられる．分岐の実行 1 回当たり，プレディケーションは不要な方向の命令を無駄にし，予測は予測ミスペナルティに予測ミスの確率を掛けた分を無駄にする．どちらが小さいかは分岐の種類に依存する（表 5.1）．プレディケーションがよりよい選択となりうるのは，各方向の命令数が予測ミスでフラッシュされる命令数よりはるかに少ない，小さな if-then-else だけである．

注意 何も推測されず，取り消しも起こらない．両方向は意図的に実行され，不要な結果はコード自身が捨てる．

小さな if-then-else では，どちらを選ぶかは分岐がどれだけよく予測されるかに依存する．予測ミスが 40 命令をフラッシュし，不要な方向が 4 命令からなるなら，実行の 10 回に 1 回より多く予測ミスが起こる時点で，プレディケーションの無駄は予測の無駄より少なくなる．5.4 節では，コンパイラが実際に生成す

実際のペナルティはこれより大きい．Skylake 級のコアでは予測ミス 1 回が約 17 サイクル（第 4 章）であり，1 サイクル当たり 4 命令なら約 70 命令分のスロットに当たる．

表 5.1: 分岐の種類ごとの予測とプレディケーション．

分岐	予測	プレディケーション	有利な方
ループ	正確	反復のたびにループの後のコードも実行する	予測
コール，リターン	常に成立．BTB と RAS が分岐先を与える	もう一方の経路の処理が常に無駄になる	予測
大きな if-then-else	ときどき数十命令をフラッシュする	大きな方向を毎回無駄にする	予測
小さな if-then-else	予測ミスごとに数十命令をフラッシュする	数命令を毎回無駄にする	予測困難ならプレディケーション

るコードについてこの比較を厳密に行う．

> **要点**
> 予測は，正しければコストがかからず，誤れば数十命令のコストがかかる．プレディケーションは不要な方向を常に無駄にする．そのため分岐は原則として予測され，プレディケーションを検討する価値があるのは，予測が困難な小さな if-then-else の分岐だけである．

5.2 if 変換

プレディケーションには，両方向の処理が，間に分岐を挟まずに次々と実行される命令の形になっている必要がある．そのようなコードを生成するのはコンパイラの役割である．次の if-then-else を考える．これはしきい値未満の値としきい値以上の値を別々に合計し，しきい値未満の値の個数を数える．v の値が t のどちら側になるかに規則性がなければ，この分岐は予測が困難である．

出典 Lesson 5，動画 3；H&P §H.4；[11]．

```
if (v < t) {
  lo = lo + v;
  n  = n + 1;
} else {
  hi = hi + v;
}
```

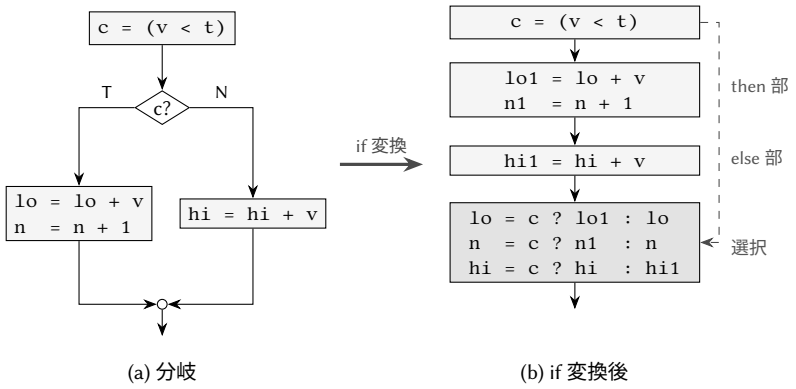
定義 5.2 (if 変換)．両方向の文の制御依存（第 3 章）を分岐条件に対するデータ依存に変換することで，そのような分岐を除去するコンパイラの変換を **if 変換** という．

▷ if-conversion

通常の命令を用いる場合，変換後のコード（図 5.1）は両方向の文を無条件に実行してその結果を一時変数に書き込み，次に条件 c をデータとして用いて，いずれかの方向で代入されるすべての変数の新しい値を選択する．両方向が変数 x に代入する場合，選択は 2 つの一時変数のどちらかを選ぶ（x = c ? x1 : x2）．この例の各変数のように一方の方向だけが代入する場合，もう一方の選択肢は元の値である．変換後のコードには分岐がない．

注意 不要な方向も実行されるので，その方向がメモリにストアしたり，例えば無効なポインタを介したロードでフォールトを起こしたりしてはならない．

分岐の除去は一続きの直線的なコードを残すことにもなり，コンパイラはその全体をまとめてスケジューリングできる（第 10 章）．



(a) 分岐　　(b) if 変換後

図 5.1: 小さな if-then-else の if 変換．(a) 元のコードは c で分岐する．(b) 変換後のコードは両方向を一時変数に計算し，c をデータとして用いて結果を選択する．

この変換で何かが得られるのは，選択を分岐なしで実行できる場合だけである．通常の分岐命令に翻訳すると，各選択はそれ自体が c による if-then-else となり，除去した分岐と同じく予測が困難である．そのため変換後のコードは依然として予測ミスを起こし，そのうえ両方向の処理も実行することになる．if 変換が安価になるのは，命令の効果を条件付きにできる命令セットの場合だけである．

通常の算術演算でも，両候補を AND と OR でマスクすれば分岐なしに選択できるが，数命令を要する．

5.3 条件付きムーブ命令

そのような命令のうち最も単純なものは，コピーだけを条件付きにする．

出典 Lesson 5，動画 4–5；H&P §H.4，§A.6．

定義 5.3 (条件付きムーブ). **条件付きムーブ**とは，条件が成り立てば値を宛先にコピーし，成り立たなければ宛先を変更しない命令である．

▷ conditional move

この命令はどちらの場合も実行され，書き込むかどうかだけが条件に依存する．制御の流れを変えないので，予測すべきものは何もない．

MIPS の命令セットには，レジスタの内容を条件とする条件付きムーブが 2 つある．命令 **MOVZ**（move if zero）はレジスタ Rt がゼロを保持していればレジスタ Rs をレジスタ Rd にコピーし，命令 **MOVN**（move if not zero）は Rt がゼロでなければコピーする．

▷ MOVZ
▷ MOVN

```
MOVZ Rd, Rs, Rt      ; if (Rt == 0) Rd <- Rs
MOVN Rd, Rs, Rt      ; if (Rt != 0) Rd <- Rs
```

選択 x = c ? x1 : x2 は x のレジスタへの 2 つのムーブとなり，そのうちちょうど 1 つが書き込む．c が R3，x1 と x2 が R1 と R2，x が R4 にあるとすると次のようになる．

```
MOVN R4, R1, R3      ; if c != 0: x <- x1
MOVZ R4, R2, R3      ; if c == 0: x <- x2
```

条件を保持するレジスタは通常，SLT のような比較によって設定される．SLT は条件が成り立てば 1 を，成り立たなければ 0 を書き込む．

ヒント SLT の後，条件レジスタは条件が成り立つとき，かつそのときに限り非ゼロである．したがって MOVN が then 部を，MOVZ が else 部を担う．

例 5.4. v が R1，t が R2，lo，n，hi が R4，R5，R6 にあるとすると，5.2 節のコードは分岐を使うと次のようにコンパイルされ，

```
      SLT  R3, R1, R2     ; R3 <- (v < t)
      BEQ  R3, R0, Skip   ; v >= t なら else 部へ
      ADD  R4, R4, R1     ; lo <- lo + v
      ADDI R5, R5, 1      ; n  <- n + 1
      J    Cont           ; else 部を飛ばす
Skip: ADD  R6, R6, R1     ; hi <- hi + v
Cont: ...
```

if 変換すると次のようになる．

```
      SLT  R3, R1, R2     ; R3 <- (v < t)
      ADD  R7, R4, R1     ; then 部を R7, R8 に計算
      ADDI R8, R5, 1
      ADD  R9, R6, R1     ; else 部を R9 に計算
      MOVN R4, R7, R3     ; lo <- R7 if v < t
      MOVN R5, R8, R3     ; n  <- R8 if v < t
      MOVZ R6, R9, R3     ; hi <- R9 if v >= t
```

一時変数 R7–R9 が必要なのは，どれを変更するかをムーブが決めるまで，R4–R6 が元の値を保持しなければならないからである．ここでは各変数が一方の方向でしか代入されないので，変数当たり 1 つのムーブで足りる．

他の命令セットでは，条件付きムーブの条件はレジスタの内容ではなくフラグに基づく．

定義 5.5 (条件コード). **条件コード**（フラグ）とは，算術命令や比較命令が副作用として設定し，後続の命令が検査する 1 ビットの状態値である．例えばゼロ，負，キャリー，オーバーフローがある．

▷ condition code

x86 の命令セットには条件付きムーブの一群 **CMOVcc** があり，cc は条件コードに対する条件を表す．CMOVZ はゼロフラグがセットされていればムーブし，CMOVNZ はクリアされていればムーブする．CMOVG は直前の比較の結果が「より大きい」ならば，CMOVL は「より小さい」ならばムーブする，といった具合である．よく使われるパターンは，一方の選択肢を無条件に代入し，1 つの条件付きムーブでそれをもう一方の選択肢で上書きするものである．$m = \max(a, b)$ の次の計算がその例である．

▷ CMOVcc

```
MOV   EAX, EBX       ; m <- a
CMP   EBX, ECX       ; a - b でフラグを設定
CMOVL EAX, ECX       ; if a < b: m <- b
```

5.4 if 変換の性能

if 変換にはコストがかかる．変換後のコードは，毎回の実行で両方向の処理と条件付きムーブを実行する．コード断片の 2 つの版を比較するため，第 3 章と同様にパイプラインのスロットを数える．実行される命令はそれぞれ 1 スロットを占め，予測ミスはそれがフラッシュする命令数と同じだけのスロットを無駄にする．分岐版は実行 1 回当たり平均 n_b 命令を実行し，1 回当たり m 回の予測ミスを起こし，予測ミス 1 回のペナルティは P 命令であるとする．また変換版は n_c 命令を実行するとする．if 変換が有利になるのは次の場合である．

$$n_c \;<\; n_b + mP \qquad\Longleftrightarrow\qquad m \;>\; \frac{n_c - n_b}{P}. \tag{5.1}$$

右辺が**損益分岐予測ミス率**であり，変換後のコードで増える命令数を，予測ミスペナルティを単位として測ったものである．5.1 節の大まかな比較と比べると，増える命令には条件付きムーブが含まれ，変換で除去される分岐命令とジャンプ命令の分が差し引かれる．

例 5.6. 例 5.4 の分岐版は，then 部が実行されるときは 5 命令（SLT，BEQ，ADD，ADDI，J），そうでないときは 3 命令（SLT，BEQ，ADD）を実行する．両方の部分が同じ頻度で実行されるなら $n_b = 4$ であり，変換版は常に $n_c = 7$ 命令を実行する．この分岐に対する予測器の精度が 75 % で，$P = 20$ であるとする．分岐版のコストは実行 1 回当たり $4 + 0.25 \times 20 = 9$ スロット，変換版は 7 スロットであり，この断片の高速化率は $9/7 \approx 1.29$ である．式 (5.1) より損益分岐率は $m = 3/20 = 0.15$ であり，分岐版が速いのは予測器の精度が 85 % を超える場合だけである．$P = 40$ なら損益分岐率は半分の 0.075，精度にして 92.5 % となる．

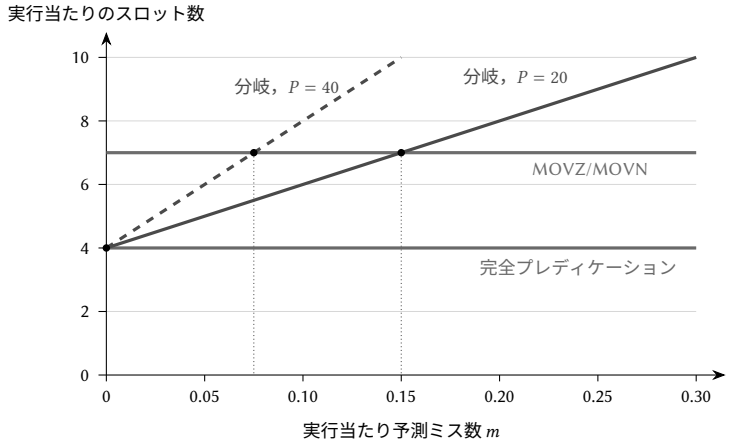


図 5.2: 例 5.6 の断片の実行当たりスロット数を，予測ミス率の関数として示す．点は損益分岐率を表す．完全プレディケーション版については 5.6 節で述べる．

図 5.2 は，この比較を m の関数として示す．分岐版のコストは予測ミス率に対して線形に増え，その傾きはペナルティに等しい．一方，変換版のコストはどの m でも同じである．パイプラインが深く，あるいは幅広くなるとペナルティが増えるので if 変換が有利になり，then 部と else 部が大きくなると $n_c - n_b$ が増えるので分岐が有利になる．

> **要点**
> if 変換は，ときどき生じる大きなペナルティを一定のオーバーヘッドに置き換える．実行当たりの予測ミス数とペナルティの積が，変換後のコードで増える命令数を上回るとき，if 変換は有利になる（式 (5.1)）．

出典 Lesson 5，動画 6–7；H&P §C.2．

注意 m はこの 1 つの分岐の率，すなわち断片の実行 1 回当たりの予測ミス数であり，第 4 章の命令当たりの率ではない．P はサイクルではなく命令で数える．

ヒント 変換後のコードの命令が 1 つ増えるごとに，損益分岐率は $1/P$ 上がる．$P = 20$ なら 1 命令につき予測精度 5 ポイントである．

5.5 条件付きムーブの利点とコスト

出典 Lesson 5，動画 8；H&P §H.4.

条件付きムーブで分岐を除去することには，1 つの利点といくつかのコストがある．

- **利点．**予測困難な分岐が消え，それとともにその予測ミスもなくなる．
- **コンパイラのサポート**が必要である．プロセッサは与えられたコードを実行するだけであり，分岐が除去されるのは，コンパイラが適切な if-then-else を認識して変換した場合だけである．
- **より多くのレジスタ**が必要である．すべての結果はまず一時変数に計算される．
- **より多くの命令**が実行される．分岐が正しく予測されたはずの実行でも両方向の処理が実行され，さらに結果を選択する条件付きムーブも実行される．

このうち，コンパイラのサポート，分岐の除去，両方向の実行はプレディケーションに本質的なものだが，追加のレジスタとムーブはそうではない．これらが生じるのは，条件付きなのがムーブだけだからである．結果を残すかどうかを条件付きムーブが決められるように，すべての結果を一時変数に計算しておかなければならない．結果を計算する命令そのものを条件付きにできれば，一時変数もムーブも不要になる．

5.6 完全プレディケーション

出典 Lesson 5，動画 9–12；H&P §H.4，§H.6；[12]．

条件付きムーブは独立したオペコードであり，それが条件付きにする操作はコピーだけである．その代わりとなるのは，すべての命令を条件付きにする機能を命令セットでサポートすることである．

定義 5.7 (完全プレディケーション)．すべての命令に条件ビットが付加され，各命令はその条件が成り立つときにだけ効果を持ち，成り立たなければ何の効果も持たないとき，その命令セットは**完全プレディケーション**を提供するという．

▷ full predication

Intel の Itanium プロセッサの IA-64 命令セットがその例である（図 5.3）．IA-64 のほとんどの命令では，41 ビットの命令の下位 6 ビットが**限定述語**（qualifying predicate）を保持する．これは 64 個ある 1 ビットのレジスタのうちの 1 つの番号であり，それぞれのレジスタを**述語レジスタ**と呼ぶ．命令はどちらの場合もフェッチされてパイプラインのスロットを占めるが，効果を持つのは選択された述語レジスタが 1 を保持するときだけである．そうでなければ命令は無効化され，結果を書き込まず，例外も発生させない．述語レジスタ 0 は常に 1 を保持するので，限定述語が 0 の命令は無条件に実行される．述語レジスタは比較命令によって設定される．比較命令は，条件に対するものとその否定に対するものという，互いに相補的な値を持つ述語の対を書き込む．

▷ predicate register

128 ビットのバンドルは 41 ビット命令 3 つと 5 ビットのテンプレートを収める．IA-64 には整数レジスタと浮動小数点レジスタも 128 本ずつある（第 11 章；H&P §H.6）．

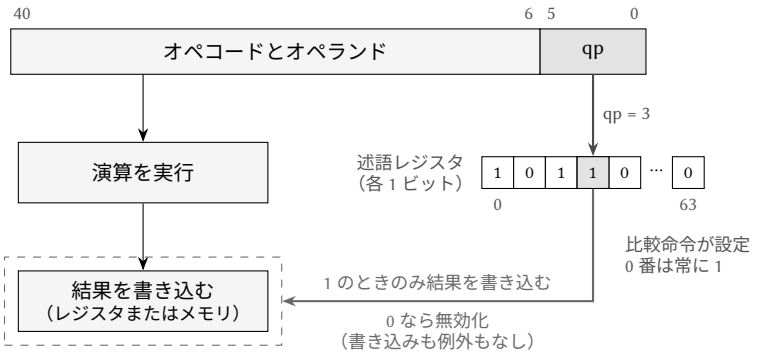


図 5.3: IA-64 における完全プレディケーション．命令の限定述語（qp）が，64 個の 1 ビット述語レジスタのうち 1 つを選択する．命令はどちらの場合もパイプラインを通過するが，結果が書き込まれるのはそのレジスタが 1 を保持するときだけである．そうでなければ命令は無効化され，書き込みも例外も起こらない．

例 5.8. 完全プレディケーションがあれば，if 変換は両方向の各文を条件またはその否定によって直接条件付けし，5.2 節のコードは次のようになる．

```
      CMPLT P1, P2, R1, R2 ; P1 <- (v < t), P2 <- !P1
(P1)  ADD   R4, R4, R1     ; lo <- lo + v  if v < t
(P1)  ADDI  R5, R5, 1      ; n  <- n + 1   if v < t
(P2)  ADD   R6, R6, R1     ; hi <- hi + v  if v >= t
```

ここで限定述語は命令の前の括弧内に書かれており，CMPLT は，R1 が R2 より小さければ P1 を 1，P2 を 0 に設定し，そうでなければその逆に設定する比較命令を表す．加算そのものが条件付きなので，結果は変数に直接書き込まれ，比較命令が SLT と分岐の両方を置き換える．表 5.2 は，例 5.6 の数値を用いて 3 つの版を比較したものである．

表 5.2: 精度 75 % の予測器と $P = 20$ における，例の断片の 3 つの版．

版	命令数	一時変数	ムーブ	予測ミス	スロット
分岐	5 または 3	0	0	0.25	9
MOVZ/MOVN	7	3	3	0	7
完全プレディケーション	4	0	0	0	4

完全プレディケーション版が実行するのは，比較命令と両方向の命令だけである．[1] 分岐版と比べると，不要な方向を実行する代わりに分岐もジャンプも実行しない．ここでは両者が釣り合い，その $n_c = 4$ は分岐版の平均 n_b に等しい．式 (5.1) による損益分岐率は $m = (4 - 4)/20 = 0$ である．すなわち，完全な予測を用いても分岐版は速くならず，予測ミスが起こるたびに遅くなる（図 5.2）．一般に，完全プレディケーション版は条件付きムーブ 1 つ当たり MOVZ/MOVN 版より 1 命令少なく実行するので，ムーブが M 個なら損益分岐率は M/P だけ低い．条件付きムーブが条件をすでに保持しているレジスタを検査する場合には，条件付きムーブ版は比較を必要としないのに対し，完全プレディケーションは依然として比較を必要とするので，その差は $(M - 1)/P$ となる．損益分岐率を下げることで，完全プレディケーションは，条件付きムーブでは引き合わないほどよく予測される分岐に対しても if 変換を有利にする．

[1] IA-64 は 1994 年から HP と Intel が共同で開発したものであり，完全プレディケーションは，並列性をハードウェアではなくコンパイル時に引き出すという構想の一部であった．最初の Itanium は 2001 年に出荷されたが，x86-64 に取って代わることはなく，Intel は 2021 年 7 月に最後の 1 台を出荷した（第 11 章；Intel のプレスリリース）．

ARM の 32 ビット命令 A32 は 4 ビットの条件フィールドを持ち，ほぼすべての命令を条件付きで実行できる．AArch64（2011 年）はこれを廃し，条件付きセレクト（CSEL）を残した（Arm アーキテクチャリファレンスマニュアル）．

要点 — 章のまとめ

分岐予測は，正しければコストがかからず，誤れば数十命令のコストがかかる．プレディケーションは分岐の両方向を実行して予測ミスを起こさないが，その代償として不要な処理を常に行う．そのため分岐は原則として予測され，コンパイラがプレディケーションを使うのは小さく予測困難な if-then-else の分岐に対してだけであり，ループ，コール，リターン，大きな if-then-else の分岐には使わない．if 変換は，制御依存を条件に対するデータ依存に変換することでそのような分岐を除去する．通常の命令を用いる場合，if 変換は両方向を一時変数に計算し，MIPS の MOVZ と MOVN や，x86 の条件コードに基づく CMOVcc といった条件付きムーブで結果を選択する．実行当たりの予測ミス数とペナルティの積が増える命令数を上回るとき，if 変換は有利になり（式 (5.1)），パイプラインが深いほど有利になる．IA-64 のようにすべての命令に条件を付ける完全プレディケーションは，一時変数とムーブを不要にし，損益分岐予測ミス率を下げる．

第 6 章

ILP

講義動画	Lesson 6，動画 1–20
教科書	H&P [1] §3.1，§3.4；[13, 14]
キーワード	RAW 依存，WAW 依存，偽の依存，アーキテクチャレジスタ，物理レジスタ，レジスタリネーミング，レジスタエイリアステーブル，命令レベル並列性，理想プロセッサ，サイクル当たり命令数，プロセッサ幅，インオーダ実行，アウトオブオーダ実行

分岐予測（第 4 章）とプレディケーション（第 5 章）は，制御依存によってパイプラインが失うサイクルの大半を取り除き，CPI を理想値の 1 に近づける．1 を下回るには複数の命令を同じサイクルで実行しなければならず，それを妨げるのが命令間のデータ依存である．この章では，本質的な依存は RAW 依存だけであること，それ以外の依存はレジスタリネーミングで除去されること，そして残った依存がプログラムの命令レベル並列性–実際のプロセッサのサイクル当たり命令数を測る基準となる上限–を定めることを示す．

6.1 すべての命令を同一サイクルで

ストールもフラッシュもしないパイプラインは毎サイクル 1 命令を完了するので，その CPI は 1 である（式 (3.1)）．CPI がそれより低いということは，1 サイクルに複数の命令が完了するということである．極端な場合，プログラムの N 個の命令すべてが同じサイクルでフェッチされ，次のサイクルで一斉にデコードされ，その次のサイクルで一斉に実行され，以下同様に進む（図 6.1）．5 段プロセッサはプログラム全体を 5 サイクルで完了し，CPI は $5/N$ となる．これは長いプログラムでは 0 に近づく．

実際のプログラムはこのようには実行できない．命令が他の命令の結果を使うからである．この章の前半を通じて用いる例として，次の 5 命令を考える．

```
I1:  ADD R3, R1, R2      ; R3 <- R1 + R2
I2:  SUB R5, R3, R4      ; R5 <- R3 - R4
I3:  AND R6, R1, R4      ; R6 <- R1 AND R4
I4:  OR  R4, R2, R7      ; R4 <- R2 OR R7
I5:  XOR R5, R1, R7      ; R5 <- R1 XOR R7
```

5 命令すべてが同じサイクルで実行ステージにあるとき，I2 は，I1 がまだ R3 を計算している間に R3 – R4 を計算するので，R3 の古い値を使ってしまう．フォワーディング（図 3.4）ではこれを正せない．フォワーディングは，あるサイクルで計算された結果をより後のサイクルで実行される命令に届けるものであるが，ここでは結果とその使用が同じサイクルに属するからである．したがって，プロセッサがどこまで理想に近づけるかは，プログラムのデータ依存によって決まる．

出典 Lesson 6，動画 1–3；H&P §3.1．

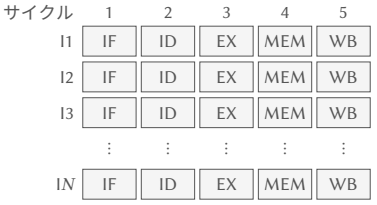


図 6.1: 理想的な場合．N 個の命令すべてが同じ 5 サイクルで各ステージを通過する．

6.2 同一サイクル実行における依存

I1 に対する I2 の RAW 依存は守らなければならない．I2 を実行できるのは早くても I1 の 1 サイクル後，すなわち I1 の結果をフォワーディングで受け取れるときである．I3，I4，I5 は，先行する命令が書き込むレジスタを読まないので，依然として I1 と同じサイクルで実行できる．図 6.2 はその結果のタイミングを示す．I2 は 1 サイクル待ち，I3 から I5 は，プログラム上では I2 の後にあるにもかかわらず I2 より先に実行される．したがって RAW 依存は，幅に制限がなくても CPI をどこまで下げられるかを制約する．すべての命令が直前の命令の結果を読むなら，毎サイクル 1 命令しか完了せず，CPI は 1 のままである．

先行する命令を追い越して命令を実行すると，I2 と I5 の間の WAW 依存が破られる．両者はともに R5 に書き込み，プログラム順では I5 が最後に書くので，これらの命令の後で R5 は I5 の結果を保持していなければならない．R5 を読む後続の命令はすべてその値を前提としている．しかし図 6.2 では，I5 がサイクル 5

出典 Lesson 6，動画 4–6；H&P §3.1．

ヒント 名前の後半は先行する命令の動作を表す．write-after-read なら先行命令が読む側である．

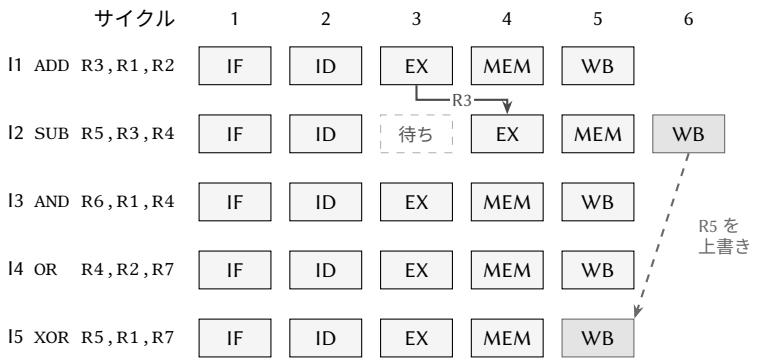


図 6.2: 例の 5 命令の同一サイクル実行．R3 に関する RAW 依存が I2 を 1 サイクル遅らせる．その結果 I2 は I5 より後に R5 に書き込み，両者の WAW 依存に違反する．

で R5 に書き込み，遅延した I2 がサイクル 6 でそれを上書きする．正しい値を保つには，I5 の計算は I2 から何も必要としないにもかかわらず，I5 も I2 の書き込みの後まで遅らせなければならない．

WAR 依存も同じように危うくなる．I4 は I2 が読む R4 に書き込み，プログラム順では I2 は R4 の古い値を使わなければならない．I2 が遅延すると，I2 の実行時には I4 が生成した新しい値がすでに存在しており，I2 がそれを使わないようにしなければならない．このように，1 つの RAW 遅延は波及する．遅延した命令が読み書きするレジスタに書き込む後続の命令は，すべて同様に遅らせなければならない．そうしてはじめて，各命令はプログラム中の自分の位置に対応するレジスタ値を見ることができる．

6.3 偽の依存の除去

出典 Lesson 6，動画 7–8；H&P §3.1．

第 3 章の 3 種類のデータ依存は性質が異なる．RAW 依存では，後の命令が先の命令の計算した値を使う．データは実際に一方の命令から他方へ流れるのであり，どのようなハードウェアでも，読む側の命令を書く側の命令より先に実行させることはできない．WAW 依存と WAR 依存は偽の依存である．2 つの命令の間で値は受け渡されない．これらの依存は，プログラムが異なる値を同じレジスタに格納するというだけの理由で存在する．それは，レジスタの数に限りがあること，そして繰り返し実行される命令–例えばループの各反復で実行される命令–が常に同じレジスタに書き込むことの帰結である．この例では，I2 と I5 の結果は，たまたま R5 という名前が付いた無関係な値である．

注意 H&P はこれらを名前依存と呼ぶ．WAR は逆依存，WAW は出力依存である．

したがって，異なる値を異なる場所に保持すれば偽の依存は消える．その方法の 1 つは，レジスタの複数のバージョンを保持することである．I2 と I5 がともに R5 に書き込むとき，プロセッサは一方で他方を上書きするのではなく両方の結果を格納し，R5 を読む各命令はプログラム順によって選ばれるバージョンを使う．I2 と I5 の間にある命令は I2 の結果を，I5 の後にある命令は I5 の結果を使う．2 つの書き込みが実際に起こる順序はもはや問題にならず，I5 は I2 が存在しない場合と同じだけ早く実行できる．同様に，I2 は，I4 が新しいバージョンをいつ生成するかにかかわらず，R4 の古いバージョンを読む．

難しいのはその管理である．読み出しのたびに，プロセッサは正しいバージョン，すなわちプログラム順で最も近い先行する書き込み命令が生成したバージョンを見つけなければならず，またバージョンが不要になったことを認識しなければならない．レジスタリネーミングはこの管理を系統的に行う．

6.4 レジスタリネーミング

リネーミングは，プログラムが名前で指定するレジスタと，プロセッサがその値を保持する記憶領域とを分離する．

出典 Lesson 6，動画 9–11；H&P §3.1，§3.4；[13]．

定義 6.1 (アーキテクチャレジスタと物理レジスタ). 命令セットで定義され，したがってプログラム中で名前で指定されるレジスタ（例えば R0 から R31）を**アーキテクチャレジスタ**という．プロセッサがレジスタの値を実際に保持する記憶位

▷ architectural register

置を**物理レジスタ**という．リネーミングを行うプロセッサは，アーキテクチャレジスタよりはるかに多くの物理レジスタを持つ．

▷ physical register

定義 6.2 (レジスタリネーミング)．各命令をデコード時に書き換え，アーキテクチャレジスタの代わりに物理レジスタを読み書きさせ，結果ごとに新たに割り当てた物理レジスタを用いるようにすることを**レジスタリネーミング**という．

▷ register renaming

定義 6.3 (レジスタエイリアステーブル)．**レジスタエイリアステーブル**（RAT）は，アーキテクチャレジスタごとに 1 つのエントリを持つ．各エントリは，それまでにリネーミングされた命令の中でプログラム順に最も新しい，そのアーキテクチャレジスタの値を保持する物理レジスタを示す．

▷ register alias table

各命令は 3 つのステップでリネーミングされる（図 6.3）．

1. 各ソースレジスタについて，その RAT エントリを読む．リネーミング後の命令はその物理レジスタを読む．
2. 結果のために，空いている物理レジスタ，すなわちまだ必要な値を保持していない物理レジスタを割り当てる．リネーミング後の命令はそこに書き込む．
3. 宛先レジスタの RAT エントリを新しい物理レジスタで上書きする．これにより，このアーキテクチャレジスタを読む後続の命令は新しい結果を使う．

講義では RAT を *register allocation table* の略としている．どちらの名前も同じテーブルを指す．

ソースの参照は，宛先のエントリを上書きする前に行わなければならない．そうしないと，ADD R1，R1，R2 のような命令が R1 の以前の値ではなく自分自身の結果を読んでしまう．

実際の設計でリネーミングがアウトオブオーダ実行とどのように組み合わされるかは，第 7 章と第 8 章で示す．

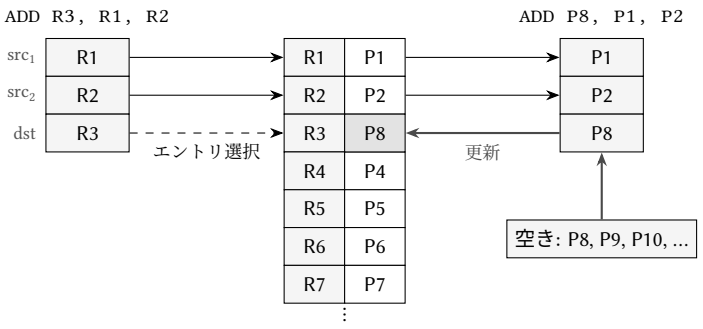


図 6.3: ADD R3，R1，R2 のリネーミング．R1 と R2 を RAT で参照し，結果には空いている物理レジスタ P8 を与え，P3 を保持していた R3 の RAT エントリを P8 で上書きする．

物理レジスタは，6.3 節で述べたバージョンにほかならない．書き込みのたびに新しいバージョンが作られ，RAT はプログラム順でどのバージョンが現在のものかを記録するので，読み出す命令は自動的に，最も近い先行する書き込み命令が生成したバージョンを使う．[1]

例 6.4．例の 5 命令をリネーミングする．I1 の前には，RAT は例で使うレジスタ R1 から R7 を P1 から P7 に，その他のアーキテクチャレジスタを例で使わない物理レジスタに対応付けている．P8，P9，... は空いており，番号の小さい順に割り当てられる．表 6.1 は，リネーミング後の各命令と，その命令の後の RAT エントリを示す．I2 が I1 の結果である P8 を読むのは，I1 がすでに R3 のエントリを更新しているからである．I3 は，I4 が R4 を P11 に対応付ける前に，古い R4 である P4 を読む．R5 への 2 つの書き込みは P9 と P12 に向かい，最後に RAT は R5 を P12 に対応付けるので，後続の命令はプログラム順が要求するとおり I5 の結果を読む．

[1] リネーミングは命令そのものを肩代わりすることもある．Ivy Bridge 以降の Intel のコアは，レジスタ間のムーブの多くを RAT だけで処理できる．このとき宛先のエントリをソースの物理レジスタに向ける．このムーブは実行ユニットを使わず，レイテンシもない（Agner Fog，マイクロアーキテクチャマニュアル）．

実際の規模：Skylake は物理レジスタを整数用に 180 個，ベクタ用に 168 個持つ．アーキテクチャレジスタはそれぞれ 16 個である（Intel，WikiChip）．

6.5 リネーミング後の依存

リネーミングされたプログラムでは，すべての結果がそれぞれ専用の物理レジスタに書き込まれる．2 つの命令が同じ位置に書き込むことはなく，先行する命令が読む位置に書き込む命令もないので，レジスタアクセス間の WAW 依存と

出典 Lesson 6，動画 12；H&P §3.4．

表 6.1: 例の 5 命令のリネーミング．命令によって変更されたエントリを太字で示す．R1，R2，R7 は終始 P1，P2，P7 のままである．

	命令	リネーミング後	R3	R4	R5	R6
	（初期状態）		P3	P4	P5	P6
I1	ADD R3, R1, R2	ADD P8, P1, P2	**P8**	P4	P5	P6
I2	SUB R5, R3, R4	SUB P9, P8, P4	P8	P4	**P9**	P6
I3	AND R6, R1, R4	AND P10, P1, P4	P8	P4	P9	**P10**
I4	OR R4, R2, R7	OR P11, P2, P7	P8	**P11**	P9	P10
I5	XOR R5, R1, R7	XOR P12, P1, P7	P8	P11	**P12**	P10

WAR 依存はもはや存在しない．一方，すべての読み出しは，元のプログラムでその値を生成した命令の結果を依然として参照するので，RAW 依存はすべて保たれる．表 6.1 で残る依存は，P8 を介した I1 に対する I2 の依存だけである．

メモリ位置を介した依存はレジスタリネーミングの影響を受けない．それは第 9 章で扱う．

同一サイクル実行が守らなければならないのは，今やこの 1 つの依存だけである．I1，I3，I4，I5 は最初のサイクルで，I2 は 2 番目のサイクルで実行され，I1 の結果は I2 にフォワーディングされる．I2 と I5 はそれぞれ P9 と P12 に書き込むので書き込みの順序は無関係であり，I2 が読む P4 に I4 は触れない．5 命令は 2 サイクルで実行され，後続の読み出し命令はすべて RAT を通じて正しい値を見つける．

> **要点**
> RAW 依存は命令間の唯一の真の依存である．レジスタリネーミングはレジスタに関するすべての WAW 依存と WAR 依存を除去するので，命令の並列実行に残る制約はデータの流れそのものだけである．

6.6 命令レベル並列性

偽の依存を除去すれば，プログラムの実行に必要なサイクル数はその RAW 依存だけで決まる．この数はプログラム自体の性質であり，特定のプロセッサの性質ではない．これは仮想的な**理想プロセッサ**上で測定される．理想プロセッサは次の性質を持つ．

出典 Lesson 6，動画 13–15；H&P 第 3 章；[14]．

▷ ideal processor

- レジスタをリネーミングするので，偽の依存は存在しない．
- 各命令を，それが読む値がすべて得られる最初のサイクルで実行する．それは，先行する命令に依存しなければ最初のサイクル，依存すれば依存先の命令のうち最も遅いものの次のサイクルである．
- 1 サイクルに実行する命令数に制限がない．
- ハードウェアに制限がないので，使用中の実行ユニットを待つ命令はない．すなわち構造依存は存在しない．
- 各命令を，演算の種類によらずちょうど 1 サイクルで実行する．

定義 6.5 (命令レベル並列性). N 命令を実行するプログラムの**命令レベル並列性**（ILP）を次式で定義する．

▷ instruction-level parallelism

$$\mathrm{ILP} \;=\; \frac{N}{C}, \tag{6.1}$$

ここで C は，理想プロセッサがそれらの命令を実行するサイクル数である．

理想プロセッサが命令 I を実行するサイクル $c(I)$ は，RAW 依存から直接求まる．

$$c(I) \;=\; 1 + \max_{J \in D(I)} c(J), \qquad C = \max_{I} c(I), \tag{6.2}$$

ここで $D(I)$ は I が結果を読む命令の集合であり，空集合に対する最大値は 0 とする．したがって C は，プログラム中の RAW 依存の最長の連鎖の長さである．どのプロセッサも，命令が必要とする値が存在する前にその命令を実行することはできないので，ILP は，任意のプロセッサがこのプログラムを実行できる速度（サイクル当たりの命令数）の上限である（6.8 節）．

ヒント 紙の上でリネーミングを行う必要はない．各レジスタの読み出しについて，最も近い先行する書き込み命令を探せばよい．後の書き込みは関係しない．

例 6.6. 次の 8 命令の ILP を求める．

```
I1:  ADD R1, R2, R3
I2:  MUL R4, R1, R5     ; R1 は I1 から，R5 は後で書かれる
I3:  SUB R6, R2, R7     ; R2，R7 は後でしか書かれない
I4:  ADD R1, R6, R8     ; R6 は I3 から
I5:  AND R9, R1, R8     ; R1 は I1 からではなく I4 から
I6:  OR  R2, R8, R3
I7:  XOR R7, R4, R2     ; R4 は I2 から，R2 は I6 から
I8:  ADD R5, R3, R8
```

I1，I3，I6，I8 は，それより前に書き込まれていないレジスタだけを読むので，サイクル 1 で実行される．I2 と I4 はサイクル 1 の命令に依存し，サイクル 2 で実行される．I5 と I7 はそれぞれサイクル 2 の命令に依存し，サイクル 3 で実行される（図 6.4）．よって $C = 3$，ILP $= 8/3 \approx 2.67$ である．[2] ここには 2 つの落とし穴が見られる．R1 には I1 と I4 の両方が書き込み，I5 が読むのは I4 の値である．I1 を生成元とみなすと，I5 を誤ってサイクル 2 に置いてしまう．また I3 は R2 と R7 を読むが，I6 と I7 がそれらに書き込むのは後になってからである．これらは WAR 依存であり，リネーミングで除去されるので，I3 を遅らせない．

[2] 実際のプログラムでも，これより大きいとはいえ大差はない．Wall は，現実的なものから，完全ではないが実現不可能なほど優れた分岐予測，リネーミング，エイリアス解析までのモデルでシミュレーションを行い，平均の並列度が 7 を超えることはまれで，5 程度が多いことを示した．完全な技法を仮定した場合にのみ，並列度ははるかに大きくなる [14]．

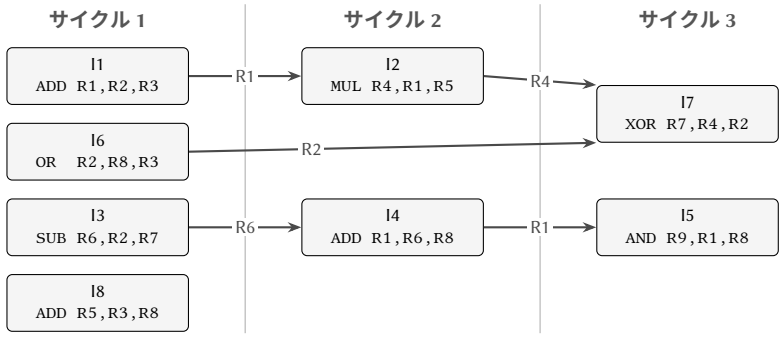


図 6.4: 例 6.6 の RAW 依存．矢印には値を運ぶレジスタを付した．各命令は，理想プロセッサがそれを実行するサイクルに置かれている．最長の連鎖の長さは 3 である．

6.7 構造依存と制御依存

構造依存，すなわちある命令が別の命令の使用しているハードウェアを待つ依存は，準備のできたすべての命令に十分なハードウェアを持つ理想プロセッサでは生じない．しかし，分岐は制御依存を加える．分岐の後の命令が実行されるかどうかは，分岐の結果に依存する．ILP を定義する理想プロセッサは，すべての分岐を完全に予測するとも仮定する．したがって，どの命令が実行されるかを最初から知っており，分岐の後の命令は分岐の実行を待たない．分岐自体は他の命令と同じく 1 つの命令であり，N に数えられ，そのオペランドを生成する命令の 1 サイクル後に実行される．分岐の結果によって飛ばされる命令は実行されず，数えられない．

出典 Lesson 6，動画 16；H&P §3.1．

注意 H&P はこれを構造ハザードと呼ぶ．プログラムではなくハードウェアに由来するので，本来は依存ではない．

プレディケーション（第 5 章）は制御依存そのものを取り除く．ただし両方の経路を実行する代償を伴う．

例 6.7. 次の断片では分岐が成立するので，I4 は実行されない．

```
I1:          ADD R1, R2, R3      ; サイクル 1
I2:          SUB R4, R1, R5      ; サイクル 2: R1 は I1 から
I3:          BEQ R4, R6, Skip    ; サイクル 3: R4 は I2 から
I4:          ADD R7, R8, R9      ; 実行されない
I5:    Skip: XOR R8, R2, R6      ; サイクル 1
I6:          AND R9, R8, R1      ; サイクル 2: R8 は I5 から
```

分岐を完全に予測すれば，分岐がサイクル 3 になってようやく実行されるにもかかわらず，I5 はサイクル 1 で実行される．5 命令が 3 サイクルで実行されるので，ILP $= 5/3 \approx 1.67$ である．I5 と I6 が分岐を待たなければならないとすると，それらはサイクル 4 と 5 で実行され，ILP は 1 に下がる．

6.8 ILP と IPC

ILP は理想プロセッサ上でのプログラムを特徴づける量であり，実際のプロセッサを特徴づけるのは，それが実際に達成する速度である．

出典 Lesson 6，動画 17–20；H&P 第 3 章．

定義 6.8 (サイクル当たり命令数). プロセッサが 1 サイクル当たりに完了する命令の平均数を，そのプロセッサの**サイクル当たり命令数**（IPC）という．IPC は性能の鉄則（式 (2.3)）の CPI の逆数であり，IPC = 1/CPI である．

▷ instructions per cycle

注意 プログラム全体の IPC は総命令数を総サイクル数で割った値であり，各区間の IPC の平均ではない．

すべてのプロセッサとプログラムについて IPC ≤ ILP が成り立つ．理想プロセッサは，各命令を依存が許す限り早く実行するからである．実際のプロセッサは，いくつかの理由で理想に届かない．

幅 1 サイクルに実行できる命令数は有限の w に限られる．**幅の狭い**プロセッサは 1 つか 2 つ，**幅の広い**プロセッサは 4 つ以上の命令を実行する．プログラムによらず IPC ≤ w である．

実際の幅：Skylake は 1 クロック 4 命令，Ice Lake は 5 命令を狙った設計である（Agner Fog，マイクロアーキテクチャマニュアル）．

実行ユニット 加算器，乗算器などの実行ユニットの数は限られており，乗算のような演算には複数サイクルかかることがある．必要な種類のユニットが使用中である命令は，入力が揃っていても待つ．これは構造依存であり，理想プロセッサにはないものである．

実行順序 **インオーダ実行**を行うプロセッサは，プログラム上で先行する命令より先に命令を実行することは決してない．連続する独立な命令は同じサイクルで実行できるが，入力を待たなければならない命令は，その後ろにあるすべての命令を引き止める．**アウトオブオーダ**実行を行うプロセッサは，入力が揃った任意の命令を実行できる（第 7 章）．

▷ in-order execution

アウトオブオーダプロセッサがそうするにはリネーミングが必要である．リネーミングがなければ，6.2 節の WAW 依存と WAR 依存も守らなければならない．インオーダプロセッサはレジスタをプログラム順に読み，すべての命令の実行時間が等しければ書き込みもプログラム順に行うので，リネーミングを必要としない．

例 6.9. 例 6.6 のプログラムを 4 つのプロセッサで実行する．いずれのプロセッサも各命令を 1 サイクルで実行し，あらゆる種類の実行ユニットを十分に持つ．アウトオブオーダのプロセッサはレジスタをリネーミングし，準備のできた命令が幅を超えるときは古い命令から実行する．表 6.2 は各サイクルで実行される命令を示す．インオーダのプロセッサはサイクル 1 で I1 しか実行しない．I2 が R1 を待たなければならず，I3 から I8 をせき止めるからである．同じせき止めが I4（R6 待ち），I5（I4 からの R1 待ち），I7（R2 待ち）でも繰り返されるので，これらのプロセッサは幅 2 でも幅 4 でも 5 サイクルを要する．幅 2 のアウトオブオーダプロセッサは毎サイクル準備のできた 2 命令を実行し，4 サイクルを要する．ILP に到達するのは，幅 4 のアウトオブオーダプロセッサだけである．

表 6.2: 例 6.6 の命令のうち，各サイクルで実行されるもの．理想プロセッサのスケジュールは，幅 4 のアウトオブオーダプロセッサのものと等しい．

	サイクル					
プロセッサ	1	2	3	4	5	IPC
インオーダ，幅 2	1	2, 3	4	5, 6	7, 8	1.60
インオーダ，幅 4	1	2, 3	4	5, 6	7, 8	1.60
アウトオブオーダ，幅 2	1, 3	2, 4	5, 6	7, 8		2.00
アウトオブオーダ，幅 4	1, 3, 6, 8	2, 4	5, 7			2.67

この例では，2 つのインオーダプロセッサはいずれも順序に制約されており，幅を 2 から 4 に広げても何も得られない．アウトオブオーダ実行は，幅 2 のプロセッサをその幅まで（1.60 から $w = 2$ へ）しか引き上げないが，幅 4 のプロセッサは ILP まで（1.60 から 2.67 へ）引き上げる．一般には（表 6.3），幅の狭いプロセッサを主に制約するのは幅である（$w = 1$ ならどちらの順序でも IPC ≤ 1）ので，アウトオブオーダ実行による IPC の向上は限定的である．幅の広いインオーダプロセッサでは，待っている命令が後ろの命令をせき止め，幅の大半が使われないままになる．したがって，幅の広いプロセッサはその幅を活かすためにアウトオブオーダ実行を必要とし，アウトオブオーダ実行は幅の広いプロセッサでは

じめて十分に効果を発揮する．

表 6.3: プログラムの ILP に対して，プロセッサの IPC を制限するもの．

	インオーダ	アウトオブオーダ
幅が狭い（幅 1–2）	主に幅	主に幅
幅が広い（幅 4 以上）	主に順序（幅は使われない）	わずか（IPC は ILP に近づく）

要点 — 章のまとめ
多くの命令を同じサイクルで実行すれば CPI は 1 を大きく下回るはずだが，データ依存がそれを妨げる．真の依存は RAW 依存だけであり，WAW 依存と WAR 依存はレジスタ名の再利用から生じる．レジスタエイリアステーブルに基づくレジスタリネーミングは，アーキテクチャレジスタを新たに割り当てた物理レジスタに対応付け，レジスタアクセス間の偽の依存をすべて除去する．プログラムの ILP（式 (6.1)）は，リネーミングを行い，幅に制限がなく，構造依存がなく，分岐を完全に予測する理想プロセッサのサイクル当たり命令数，すなわち命令数を RAW 依存の最長の連鎖の長さで割った値である．これはプログラムの性質であり，あらゆるプロセッサの IPC の上限となる．実際のプロセッサは，幅，実行ユニット，インオーダ実行に制約されてこれを下回る．ILP に近づくには幅の広いアウトオブオーダプロセッサが必要である．次章では，リネーミングを伴って命令をアウトオブオーダに実行する Tomasulo のアルゴリズムを紹介する．

どちらも同じスマートフォンに載る．Arm の Cortex-A55 は幅 2 のインオーダ，Cortex-A78 は幅 4 のアウトオブオーダである（Arm の製品資料）．

第 7 章

命令スケジューリング

講義動画	Lesson 7，動画 1–30
教科書	H&P [1] §3.4–3.5；[15]
キーワード	アウトオブオーダ実行，Tomasulo のアルゴリズム，命令キュー，リザベーションステーション，レジスタエイリアステーブル，タグ，共通データバス，発行，ディスパッチ，結果書き込み，古くなった結果，最古優先

第 6 章では，プログラムにはインオーダプロセッサが利用できるよりはるかに多くの命令レベル並列性が含まれていること，そしてそれに近づくには，レジスタをリネーミングし命令をアウトオブオーダに実行するプロセッサが必要であることを示した．この章では，そのようなプロセッサが，どの命令を実行するかをサイクルごとにどのように決めるかを述べる．その方法が Tomasulo のアルゴリズムである．これはレジスタリネーミングと，オペランドの揃った命令の検出とを組み合わせたもので，今日のアウトオブオーダプロセッサでも依然として基礎となっている．命令がたどる 3 つのステップを追い，複数の命令や結果が同じハードウェアを奪い合うときに生じる選択を調べ，完全な例をトレースし，最後に各命令がいつ実行されるかを求める簡便法を導く．

7.1 IPC の向上

実際のプログラムの ILP は 1 を大きく上回るので，IPC をそれに近づけようとするプロセッサは，命令間の 4 種類の依存に対処しなければならない．それぞれに対策がある．

出典 Lesson 7，動画 1–2；H&P §3.1．

制御依存 分岐に続く命令が分岐の実行前にパイプラインに入れるのは，分岐の結果を推測する場合だけである．分岐予測（第 4 章）と，予測の難しい分岐に対するプレディケーション（第 5 章）がこれに対処する．

偽のデータ依存 WAW 依存と WAR 依存は，命令がレジスタ名を再利用するというだけの理由で存在する．レジスタリネーミング（第 6 章）がこれらを除去する．

真のデータ依存 RAW 依存は除去できない．命令は，自分が読む値が計算されるまで待たなければならない．避けられるのは，その命令に依存しない命令までが一緒に待つことである．

注意 依存はプログラムの性質であり，ハザードはそれを解消しなければならないパイプラインの性質である．H&P §3.1．

構造依存 命令はハードウェアも奪い合う．リザベーションステーション，実行ユニット，そして後述するバスである．対策は，これらの資源をより多く備えた幅の広いプロセッサ（第 6 章）であり，それによって同じサイクルにより多くの命令を発行し実行できる．

定義 7.1 (アウトオブオーダ実行)．プログラム順で先行する命令がまだ待っている間でも，各命令を，それが読む値が得られ次第実行することを**アウトオブオーダ実行**という．

▷ out-of-order execution

アウトオブオーダ実行はリネーミングと組み合わせて用いられる．リネーミングがなければ，命令が追い越せるのは WAW 依存も WAR 依存も持たない先行命令だけとなり，ILP の多くが失われる．残るのはスケジューリングの問題である．プロセッサは毎サイクル，どの命令のオペランドがすべて揃っているかを判定し，そのうちどれを実行するかを決めなければならない．

7.2 Tomasulo のアルゴリズム

アウトオブオーダプロセッサで用いられるスケジューリング手法は，IBM System/360 Model 91 で導入された．

出典 Lesson 7，動画 3；H&P §3.4；[15]．

定義 7.2 (Tomasulo のアルゴリズム)．Tomasulo が 1967 年に発表したハードウェアアルゴリズムで，レジスタをリネーミングし，オペランドが揃った命令を判定

することによって，各命令をそのオペランドが計算され次第実行させるものを**Tomasulo のアルゴリズム**という．

▷ Tomasulo's algorithm

現在のプロセッサも本質的に同じ方法で命令をスケジューリングする．相違点を表 7.1 に示す．最初の 2 つはアルゴリズムの適用範囲に関わるだけだが，3 つ目はその機構を変える．Tomasulo のアルゴリズムには例外への備えがない．命令がアウトオブオーダに実行された後では，レジスタの内容は，例外を受け付けられるプログラム中のどの時点にも対応しているとは限らない．例外をサポートするには追加の構造が必要であり，それが第 8 章の主題である．[1]

現代の規模：Skylake はスケジューラに 97 命令，リオーダバッファ（第 8 章）に 224 命令を保持する．Intel 最適化マニュアル．

表 7.1: Tomasulo のアルゴリズムと現代のアウトオブオーダプロセッサ．

	Tomasulo のアルゴリズム	現代のプロセッサ
スケジューリングする命令	浮動小数点演算	すべての命令
同時に処理中の命令	数個	数百個
例外	サポートしない	サポートする（第 8 章）

[1] Model 91（1967）のクロックは 60 ns で，このアルゴリズムを適用したのは浮動小数点演算だけであった．加算は 2 サイクル，乗算は 3 サイクルであった．その割り込みが不正確であったのは，まさに例外への備えがないという理由による．この考え方はその後ほとんど使われず，1990 年代半ばに Pentium Pro と PowerPC 604 で復活した．

7.3 構成

図 7.1 は，Tomasulo のアルゴリズムを実行するハードウェアを示す．フェッチステージは，フェッチした各命令を**命令キュー**に追加する．命令キューは先入れ先出しのバッファであり，フェッチされた命令を，リザベーションステーションへ移せるようになるまでプログラム順に保持する．

出典 Lesson 7，動画 4；H&P §3.4．

▷ instruction queue

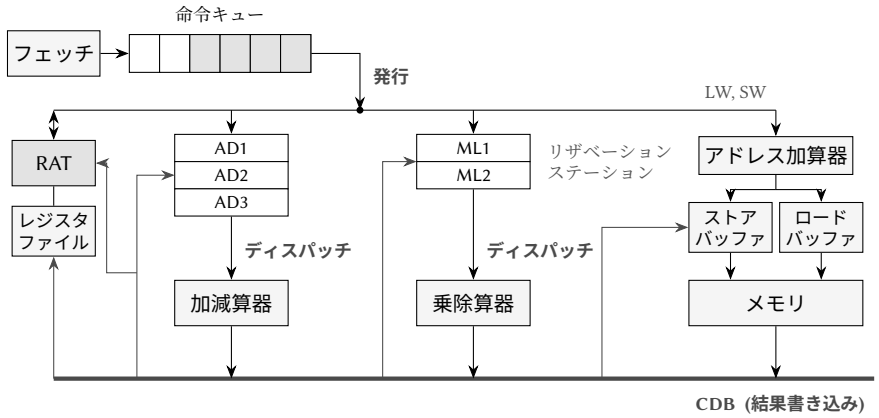


図 7.1: Tomasulo のアルゴリズムを用いるプロセッサの構成．発行は，RAT とレジスタファイルを用いて，キューの先頭の命令をリザベーションステーションに移す．ディスパッチは，準備のできた命令をその実行ユニットに送る．結果は共通データバスを通って，リザベーションステーション，RAT，レジスタファイル，ストアバッファに戻る．ロードとストアは，アドレス加算器とロードバッファ・ストアバッファを経由する経路をとる．

定義 7.3 (リザベーションステーション). **リザベーションステーション**（RS）は，1 つの命令を，それが命令キューを離れた時点から結果がブロードキャストされるまで保持する．RS は，使用中かどうか，演算の種類，各オペランドについてその値あるいは値がまだ計算されていなければその値のタグ，そして命令がすでにディスパッチされたかどうかを記録する．

▷ reservation station

リザベーションステーションは，それを担当する実行ユニットごとにまとめられる．この章の例では，1 つの加減算器が担当する加算・減算用の 3 つのステーション AD1，AD2，AD3 と，1 つの乗除算器が担当する乗算・除算用の 2 つのステーション ML1，ML2 を用いる．ステーションの名前は，その命令の結果がブロードキャストされるときの**タグ**である．

360/91 はまさにこの構成であった．浮動小数点加算器に 3 つ，乗除算器に 2 つのリザベーションステーションを備えていた [15]．

第 6 章のレジスタエイリアステーブルは，レジスタごとに 1 つのエントリを持つ．エントリが空であれば，レジスタファイルがそのレジスタの現在の値を保持

している．そうでなければ，エントリは，その値を生成する命令が入っているステーションのタグを保持する．したがってタグは，第 6 章の物理レジスタの役割を果たす．すべての結果が新しい名前を受け取り，レジスタを読む命令は，最も近い先行する書き込み命令の結果を参照する．

定義 7.4 (共通データバス). **共通データバス**（CDB）は，実行ユニットが計算した各結果を，そのタグとともに，すべてのリザベーションステーション，RAT，レジスタファイルに運ぶ．

▷ common data bus

ロードとストアは別の経路を用いる．加算器がメモリアドレスを計算した後，ロードはアドレスを保持するロードバッファで，ストアはアドレスと格納する値（CDB から受け取ることもある）を保持するストアバッファで待つ．ロードが読んだ値は，他の結果と同様に CDB に載せられる（7.8 節）．

すべての命令は，図 7.1 に示した 3 つのステップを経る．

発行 命令キューの先頭の命令を，RAT を通じてオペランドと宛先を解決したうえで，空いているリザベーションステーションに置く．このステップを**発行**という．

▷ issue

ディスパッチ オペランドがすべて揃った命令を，そのリザベーションステーションから実行ユニットに送る．これを命令の**ディスパッチ**という．

▷ dispatch

結果書き込み 結果を CDB に載せ，そこから待機中のリザベーションステーション，RAT，レジスタファイルに届ける．このステップを**結果書き込み**，あるいはブロードキャストという．

▷ write result

7.4 発行

出典 Lesson 7，動画 5–7；H&P §3.5．

発行は，命令キューの先頭の命令を取り出し，次のステップを実行する．この章の例では 1 サイクルに 1 命令を発行する．幅の広いプロセッサは 1 サイクルに複数の命令を，やはりプログラム順に発行する．

1. プログラム順で次の命令を，キューの先頭から取り出す．
2. 各オペランドの出所を決める．オペランドのレジスタの RAT エントリが空であれば，値はレジスタファイルにある．そうでなければ，エントリがその値のタグを示す．
3. その演算用の空いているリザベーションステーションを探す．その種類のステーションがすべて使用中であれば，命令はこのサイクルには発行されない．命令はキューの先頭に留まり，その後ろの命令も待つ．
4. 命令をステーションに置く．演算の種類と，各オペランドについてレジスタファイルの値あるいは RAT のタグを格納する．
5. 宛先レジスタの RAT エントリを，そのステーションの名前に設定する．

命令は厳密にプログラム順に発行される．そのため RAT は，発行中の命令の位置から見えるレジスタ値を表し，各オペランドは最も近い先行する書き込み命令を参照する．リネーミングと同様に，オペランドの参照は宛先のエントリを上書きする前に行う．

注意 発行はオペランドを待つことはなく，空いているリザベーションステーションだけを待つ．

例 7.5. 次のプログラムを，この章の実行例とする．

```
I1:  MUL R1, R2, R3     ; R1 <- R2 * R3
I2:  ADD R4, R1, R5     ; R4 <- R1 + R5
I3:  SUB R6, R2, R5     ; R6 <- R2 - R5
I4:  ADD R1, R5, R3     ; R1 <- R5 + R3
I5:  DIV R2, R6, R3     ; R2 <- R6 / R3
I6:  MUL R3, R4, R1     ; R3 <- R4 * R1
I7:  SUB R5, R3, R2     ; R5 <- R3 - R2
```

初期状態では RAT は空で，レジスタファイルは R1 = 1，R2 = 14，R3 = 4，R4 = 3，R5 = 6，R6 = 7 を保持する．サイクル 1 で，I1 がオペランド値 14 と 4 とともに ML1 に発行され，R1 の RAT エントリは ML1 になる．サイクル 2 で，I2 が AD1 に発行される（図 7.2）．R1 の RAT エントリは ML1 を保持しているので，第 1 オペランドはタグ ML1 となる．R5 のエントリは空なので，第 2 オペランドは値 6 となる．R4 の RAT エントリは AD1 になる．I3 と I4 がサイクル 3 と 4 で続く．I4 は再び R1 に書き込み，R1 のエントリの ML1 を AD3 で置き換える．これ以降に発行される命令のうち R1 を読むものはすべて，I1 ではなく I4 の結果を使う．

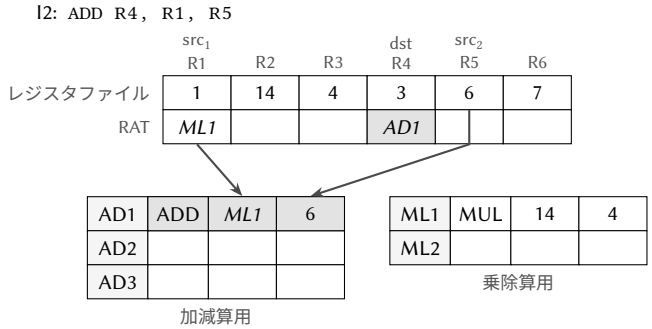


図 7.2: 例 7.5 のサイクル 2 における I2 の発行．R1 の RAT エントリからタグ ML1 が得られ（タグは斜体），R5 のエントリは空なので値 6 がレジスタファイルから取られ，R4 のエントリは AD1 に設定される．網掛けのセルがこのステップで書き込まれる．

7.5 ディスパッチ

命令は，すべてのオペランドが値になるまでリザベーションステーションで待つ．CDB に結果が現れるたびに，すべてのリザベーションステーションが自身のオペランドのタグをバス上のタグと比較し，一致したところではタグをバスの値で置き換える．これを値の**取り込み**という．オペランドがすべて値になったステーションは，**準備のできた**命令を保持している．そのステーションを担当する実行ユニットが空いていれば，準備のできた命令はディスパッチされる．すなわち，その演算の種類とオペランド値がユニットに送られ，ユニットが実行を開始する．命令の実行中もステーションは使用中のままである．ステーションの名前が，結果をブロードキャストするときのタグだからである．

出典 Lesson 7，動画 8；H&P §3.5．

注意 文献によって名前が逆になる．命令をステーションに置くことをディスパッチ，ユニットに送ることを発行と呼ぶ流儀もある．H&P のステップは発行・実行・結果書き込みである．

7.5.1 準備のできた命令が複数ある場合

実行ユニットが空いたときに複数の命令が準備できていることがあり，その中から 1 つを選ばなければならない．準備のできた命令は必要な値をすべて持っているので，どれを選んでも正しい．選択が影響するのは性能とハードウェアコストだけである．[2]

最古優先 最初に発行された命令をディスパッチする．他の条件が同じなら，古い命令ほど，処理中の命令のうちその後に発行されてその結果を待っている可能性のあるものが多い．したがってこの方針は次の方針を近似し，しかも発行の順序を記録するだけで済む．

依存命令最多優先 待機中の命令のうち最も多くがその結果を必要とする命令をディスパッチし，できるだけ多くの命令を早く準備完了にする．最良の性能が期待できるが，その命令を見つけるには，すべての候補のタグをすべてのリザベーションステーションのオペランドと比較する必要があり，遅すぎるうえにコストも大きすぎる．

ランダム 選択論理が最も単純である．多くの命令が依存する命令を，重要でない命令の後ろで待たせることがあるので，性能が低下する．

最古優先は他の 2 つの方針のよい折衷であり，この章の例で用いる規則である．実行例では，サイクル 7 で加減算器が空いたとき，I2 と I4 がともに準備できている（7.9 節）．I4 のほうが 2 サイクル長く準備完了の状態にあるが，最古優先により I2 がディスパッチされる．

出典 Lesson 7，動画 9–10．

[2] ブロードキャストされたタグを待機中の全オペランドと比較するウェイクアップと，ユニットごとに準備のできた命令を 1 つ選ぶセレクトは，あわせて 1 サイクルに収まらなければならない．そうでなければ依存する命令を連続して実行できない．このループのコストはステーション数とともに増えるので，リオーダバッファが数百命令を保持するのに対し，スケジューラが保持するのは 100 程度である [16]．

7.6 結果書き込み

実行ユニットが命令の実行を終えると，結果書き込みのステップがその結果をブロードキャストする．

1. 結果と，命令のリザベーションステーションのタグを CDB に載せる．
2. このタグを保持するすべてのオペランドが値を取り込む（7.5 節）．
3. あるレジスタの RAT エントリがそのタグを保持していれば，値をレジスタファイルのそのレジスタに書き込み，RAT エントリを空にする．これによ

出典 Lesson 7，動画 11；H&P §3.5．

り，後で発行される命令はレジスタファイルから値を読む．
4. リザベーションステーションを解放する．

ステーションは宛先レジスタを記録する必要がない．タグを保持する RAT エントリがそれを特定するからである．図 7.3 は，実行例における I2 の結果のブロードキャストを示す．

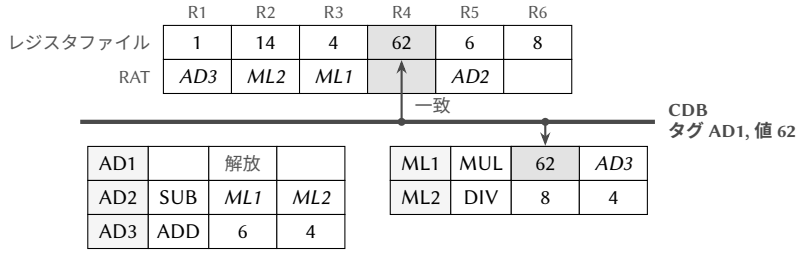


図 7.3: 実行例のサイクル 9 における I2 の結果書き込み．ML1（I6）はタグ AD1 の代わりに 62 を取り込む．R4 の RAT エントリは AD1 を保持しているので，R4 は 62 になり，エントリは空になる．AD1 は解放される．AD1 を保持するオペランドや RAT エントリは他にない．

7.6.1 1 サイクルに複数の結果がある場合

2 つの実行ユニットが同じサイクルに実行を終えることがあるが，1 本のバスは一度に 1 つの結果しか運べない．1 つの解決法は，ユニットごとに別々のバスを設けることである．その場合，すべてのリザベーションステーションのすべてのオペランドが自身のタグを各バスと比較し，一致したバスから値を選ばなければならず，レジスタファイルにはバスごとに書き込みポートが必要になる．もともと大きい比較論理が，バスの数に比例して増大する．もう 1 つの方法は，バスを 1 本のままとし，一方の結果を次のサイクルまで待たせることである．ここで用いる規則は，**遅い**ほうのユニットの結果を優先する．その命令はより前にディスパッチされたので古い命令である傾向があり，その結果を待っている命令も多い可能性が高いからである．例では，結果がバスを待っているユニットは次の命令を開始できないと仮定する．

出典 Lesson 7，動画 12．

7.6.2 古くなった結果

結果がブロードキャストされる時点で，同じレジスタに書き込む後の命令がすでに発行されていることがある．このときそのレジスタの RAT エントリは，バス上のタグではなく後の命令のタグを保持しており，ステップ 3 は何の効果も持たない．レジスタファイルも RAT も変化しない．結果はそのレジスタにとってのみ**古くなった**ものであり，そこでそれを無視するのは正しい．両方の命令の後では，レジスタは後の結果を保持していなければならず，その結果は後の命令がブロードキャストするときに届く．先の結果もなお必要だが，それを必要とするのは 2 つの書き込み命令の間に発行された命令だけである．それらはそのタグを保持しており，ステップ 2 で値を取り込む．後の書き込み命令より後に発行された命令は，後のタグを参照する．

出典 Lesson 7，動画 13；H&P §3.5．

例 7.6. 実行例では，I1 と I4 がともに R1 に書き込む．サイクル 6 で I1 がタグ ML1 のもとで 56 をブロードキャストするとき，I4 がサイクル 4 で発行されているので，R1 の RAT エントリはすでに AD3 を保持している．レジスタファイルの R1 は古い値 1 のままであり，エントリも AD3 のままである．一方，I2 を保持する AD1 は 56 を取り込む．R1 が 10 になるのは，サイクル 11 で I4 が AD3 のもとでブロードキャストするときである．

> **要点**
> ブロードキャストは，そのタグを待つすべてのオペランドに値を届けるが，レジスタファイルと RAT を更新するのは，宛先の RAT エントリがまだそのタグを保持している場合だけである．

7.7 1 サイクル内のステップ

1 つの命令から見ると，Tomasulo のアルゴリズムは，発行，リザベーションステーションでの待機と取り込み，ディスパッチ，実行，結果書き込みという一連の流れである．プロセッサ全体から見ると，毎サイクルすべてのステップが，それぞれ異なる命令に対して行われる．同じサイクルに，1 つの命令が発行され，複数のステーションがブロードキャストされた値を取り込み，準備のできた命令が空いているユニットにディスパッチされることがあり，そして 1 つの結果がブロードキャストされる．

出典 Lesson 7，動画 14–19；H&P §3.5．

同じ命令の 2 つのステップが同じサイクルに行われうるかどうかは，回路設計の問題である．最も単純な設計では，各ステップはサイクル開始時点の構造の状態を調べ，すべての更新はサイクルの終わりに反映される．このとき次のようになる．

- サイクル C で発行された命令がリザベーションステーションに入っているのはサイクル $C+1$ からであり，ディスパッチされるのは早くても $C+1$ である．
- サイクル C で最後に残っていたオペランドを取り込んだ命令は，サイクル $C+1$ から準備完了となり，ディスパッチされるのは早くても $C+1$ である．
- サイクル C のブロードキャストで解放されたステーションは，サイクル $C+1$ に新しい命令を受け入れられる．

取り込みとディスパッチの間の遅延は，追加の論理，すなわち現在のサイクルにバス上に到着するオペランドも考慮するディスパッチによって取り除くことができ，ブロードキャストで解放されたステーションも同様に同じサイクルで再利用できる．追加の論理は，それが拡張するステップと同じサイクルに置かれるので，クロックサイクルを長くする．ただし，命令が発行されたサイクルにディスパッチされることはない．発行が命令をステーションに書き込んでいる間，ディスパッチからはその命令がまだ見えないからである．

同じサイクルにおける発行と結果書き込みの間の 2 つの相互作用には注意が必要である．

- 両者が同じレジスタの RAT エントリを変更することがある．古い書き込み命令のブロードキャストはエントリを空にし，新しい書き込み命令の発行はエントリを新しいタグに設定する．発行はプログラム順で後の命令に属するので，発行が優先されなければならない．言い換えれば，結果書き込みを先に，発行を後に適用する．そうしないと，後続の命令はレジスタファイルを読み，新たに発行された命令の結果を見落としてしまう．
- ある命令が，そのオペランドの 1 つがブロードキャストされるサイクルに発行されることがある．最も単純な設計では，エントリはサイクルの終わりにならないと空にならないので，命令は RAT にタグを見つけるが，ステーションに入るのは値がバスを通過した後である．したがって発行はオペランドのタグをバスと比較し，値を直接取らなければならない．そうしないと，命令は決して来ないブロードキャストを待ち続ける．ブロードキャストによる更新が同じサイクルで発行から見えるなら，発行は単にレジスタファイルから新しい値を読む．

> **要点**
> 最も単純な設計では，命令は 1 サイクルに高々 1 ステップしか進まない．サイクル C で発行された命令がディスパッチされるのは早くても $C+1$ であり，サイクル W でブロードキャストされた結果を待つ命令がディスパッチされるのは早くても $W+1$ である．

7.8 ロード命令とストア命令

ロードとストアは，レジスタを介した依存と同様の，メモリを介した依存を生む．

出典 Lesson 7，動画 20；H&P §3.5．

```
SW  R1, 0(R2)       ; MEM[R2] <- R1
LW  R3, 8(R4)       ; R3 <- MEM[R4 + 8]
```

R4 + 8 が R2 に等しければ，ロードは直前に格納された値を読む．これはメモリ

を介した RAW 依存である．同じアドレスへの 2 つのストアは WAW 依存を持ち，先行するロードが読むアドレスへのストアは WAR 依存を持つ．RAT はこれらのうち偽の依存を除去できない．RAT がリネーミングするのはレジスタであってメモリ位置ではなく，また 2 つのメモリアクセスが衝突するかどうかは，両方のアドレスが計算されてはじめて分かるからである．

そのため Tomasulo のアルゴリズムは，ロードとストアをプログラム順に実行する．ロードとストアは他の命令と同様に，オペランドを RAT を通じて解決して発行され，アドレス加算器がアドレスを計算する．その後，ロードはロードバッファで，ストアはストアバッファで待つ（図 7.1）．格納する値がまだ計算されていないストアは，それを CDB から取り込む．メモリアクセスは，ロードとストアが発行された順に，これらのバッファから行われる．ロードは読んだ値を，演算の結果とまったく同じように，自身のタグのもとで CDB にブロードキャストする．現代のプロセッサはロードとストアもアウトオブオーダに実行するが，そのためにはプロセッサがメモリを介した依存を検出して処理する必要がある．これは第 9 章で扱う．

Skylake：処理中のロード 72，ストア 56．Intel 最適化マニュアル．

注意 プログラム順とはメモリアクセスの順序のことである．各アドレスをいつ計算するかは別の設計上の選択である（H&P §3.5 はプログラム順に計算する）．

第 9 章では，アドレスがまだ分からないストアをロードが追い越すことを許し，2 つのアドレスが重なっていたかどうかを後から検査する．

7.9 完全な例

ここで 例 7.5 のプログラムを最後まで実行する．前提は，7.7 節 の最も単純な設計に次の仮定を加えたものである．

- 1 サイクルに発行する命令は高々 1 つであり，必要な種類の空いているステーションのうち番号の最も小さいものに発行する．
- 加減算器は ADD と SUB を 2 サイクルで，乗除算器は MUL と DIV を 4 サイクルで実行する．サイクル D でディスパッチされ，実行時間が k サイクルの命令は，サイクル D から $D+k-1$ まで実行され，サイクル $D+k$ でブロードキャストする．そのサイクルには，ユニットはすでに次の命令を受け入れられる．
- CDB は 1 本であり，乗除算器が加減算器より優先される（7.6.1 節）．結果がバスを待っているユニットは新しい命令を受け入れない．準備のできた命令は最古優先でディスパッチされる．

表 7.2 は各サイクルに起こることを列挙し，表 7.3 はサイクル 5，6，7 の後のリザベーションステーションと RAT を示す．

出典 Lesson 7，動画 21–26；H&P §3.5．

参考までに：Skylake では浮動小数点の加算・乗算が 4 サイクル，倍精度除算が 13–14 サイクル，整数加算が 1 サイクルである．Agner Fog, *Instruction tables*．

ここでの各ユニットはパイプライン化されていない．サイクル D から $D+k-1$ まで占有される．実際の加算器や乗算器は毎サイクル新しい演算を受け付ける．

表 7.2: 完全な例の各サイクルの事象．取り込みは，それを引き起こすブロードキャストとともに示す．

サイクル	発行	ディスパッチ	結果書き込みと備考
1	I1 → ML1		
2	I2 → AD1	I1	
3	I3 → AD2		I2 は ML1 を待つ
4	I4 → AD3	I3	R1 の RAT エントリが AD3 になる
5	I5 → ML2		全ステーションが使用中；I4 は準備完了，加減算器は使用中
6			I1: ML1 = 56，R1 には古くなった結果；AD1 が取り込む；I3 は実行終了，バス待ち；I6 用の空きステーションなし
7	I6 → ML1	I2	I3: AD2 = 8，R6 ← 8；ML2 が取り込む；I2 と I4 が準備完了，I2 が古い
8	I7 → AD2	I5	
9		I4	I2: AD1 = 62，R4 ← 62；ML1 が取り込む
10			
11			I4: AD3 = 10，R1 ← 10；ML1 が取り込む
12		I6	I5: ML2 = 2，R2 ← 2；AD2 が取り込む
13–15			I6 を実行中
16			I6: ML1 = 620，R3 ← 620；AD2 が取り込む
17		I7	
18			
19			I7: AD2 = 618，R5 ← 618

表 7.3: サイクル 5，6，7 の終了時のリザベーションステーションと RAT．タグは斜体で示し，「実行中」はディスパッチ済みの命令を表す．空の RAT エントリはレジスタファイルの値で示す．

	サイクル 5 の後	サイクル 6 の後	サイクル 7 の後
AD1	I2: *ML1*, 6	I2: 56, 6	I2: 56, 6（実行中）
AD2	I3: 14, 6（実行中）	I3: 14, 6（実行中）	空き
AD3	I4: 6, 4	I4: 6, 4	I4: 6, 4
ML1	I1: 14, 4（実行中）	空き	I6: *AD1*, *AD3*
ML2	I5: *AD2*, 4	I5: *AD2*, 4	I5: 8, 4
R1	*AD3*	*AD3*	*AD3*
R2	*ML2*	*ML2*	*ML2*
R3	4	4	*ML1*
R4	*AD1*	*AD1*	*AD1*
R5	6	6	6
R6	*AD2*	*AD2*	8

サイクル 1–5． 1 サイクルに 1 命令ずつ発行される．I1 と I3 は発行時にすべてのオペランドが揃っており，次のサイクルで I1 は乗除算器に，I3 は加減算器にディスパッチされる．I4 はサイクル 5 から準備完了だが，加減算器は I3 を実行中である．I2 はタグ ML1 を，I5 は AD2 を待つ．サイクル 5 の後，すべてのステーションが使用中となる．

サイクル 6． 両ユニットはサイクル 5 で実行を終えている．バスは遅いほうの乗除算器の結果，すなわち ML1 のもとでの 56 を運ぶ．これは R1 にとっては古くなった結果である（例 7.6）．AD1 がそれを取り込み，ML1 は解放される．I3 の結果は加減算器で待つ．サイクルの開始時点で乗除算用のステーションが 2 つとも使用中なので，I6 は発行されない．

サイクル 7． I3 が AD2 のもとで 8 をブロードキャストする．R6 の RAT エントリは AD2 を保持しているので，R6 は 8 になり，エントリは空になる．ML2 は I5 のためにこの値を取り込み，AD2 は解放される．加減算器は新しい命令を受け入れられ，I2 と I4 がともに準備できているので，最古優先により I2 がディスパッチされる．I6 は前のサイクルで解放された ML1 に発行され，AD1 と AD3 の 2 つのタグを受け取る．

サイクル 8–19． I7 は AD2 に発行され，ML1 と ML2 を待つ．I5 はサイクル 8 に乗除算器で実行を始め，I4 は I2 が加減算器を解放するサイクル 9 に加減算器で実行を始める．I6 は I2（サイクル 9）と I4（サイクル 11）の結果，そして I5 がサイクル 12 に解放する乗除算器を必要とする．I7 は，サイクル 16 にブロードキャストされる I6 の結果を必要とする．サイクル 19 の最後のブロードキャストの後，レジスタファイルは R1 = 10，R2 = 2，R3 = 620，R4 = 62，R5 = 618，R6 = 8 を保持する．これはプログラムを順に実行して得られる値とまったく同じである．

この例の IPC は 7/19 ≈ 0.37 である．ステーションとユニットが無制限でも，I1→I2→I6→I7 の連鎖に 17 サイクルを要する．

> **要点**
> この例の命令は I1，I3，I2，I4，I5，I6，I7 の順にブロードキャストし，各命令はオペランド，空いているステーション，空いているユニットが許す限り早く実行される．すべてのオペランドは発行時にタグを通じて解決され，古くなった結果はレジスタファイルに届かないので，最終的なレジスタ値は逐次実行の場合と同じになる．

7.10 構造を追跡しないタイミングの算出

すべての構造をサイクルごとにトレースするのは手間がかかる．タイミングだけが問題であれば，各命令の発行，ディスパッチ，結果書き込みのサイクルを求めれば十分である．命令 n のこれらのサイクルを I_n，D_n，W_n で，その実行時間を k_n で表す．7.9 節 の仮定のもとで，次が成り立つ．

出典 Lesson 7，動画 27–29；H&P §3.5．

発行 I_n は，I_{n-1} より後で，必要な種類のステーションが空いている最初のサイクルである．命令 m の結果書き込みで解放されたステーションは，サイクル $W_m + 1$ から空いている．

ディスパッチ D_n は，次式を満たし，かつ実行ユニットが空いている最初のサイクルである．

$$D_n \;\geq\; 1 + \max\Big(I_n,\; \max_{p \in P(n)} W_p\Big), \tag{7.1}$$

ここで $P(n)$ は，n のオペランドを生成する命令（最も近い先行する書き込み命令）の集合である．ユニットが空いているとは，他の命令 m に占有されておらず（m はサイクル D_m から $W_m - 1$ までユニットを占有する），準備のできたより古い命令にも取られていないことである．

結果書き込み $W_n = D_n + k_n$ である．ただし，そのサイクルのバスが優先権を持つ結果に取られていれば，それより後になる．

ユニットやバスは，先に準備のできた**後の**命令に取られることがあるので，行を単純にプログラム順に埋めていくことはできない．ユニットとバスは，すべての命令にわたってサイクル順に割り当てる．各サイクルで，空いているユニットはそのサイクルに準備のできている命令のうち最も古いものを取り，バスは優先権を持つ結果を取る．まだ準備のできていない古い命令のためにユニットが空いたまま待つことはない．

ヒント $P(n)$ は，n が読む各レジスタについての，最も近い先行する書き込み命令からなる（第 6 章）．

ヒント 各ステップが待つ資源は 1 つである．発行は空きステーション，ディスパッチはオペランドと空きユニット，結果書き込みはバスを待つ．

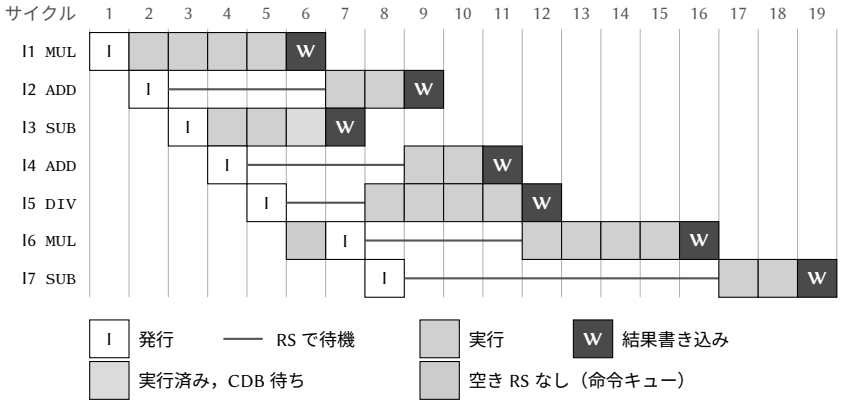


図 7.4: 実行例のタイミング．アウトオブオーダ実行により，I2 が待っている間に I3 が実行され，I4 が待っている間に I5 が実行を始める．I3（バス），I4（加減算器），I6（リザベーションステーション）の遅延は，例 7.7 で導いたものである．

例 7.7. 表 7.4 はこれらの規則を実行例に適用したものであり，図 7.4 はその結果を図示する．I3，I4，I6 にすべての規則が現れている．I3 は発行の 1 サイクル後のサイクル 4 にディスパッチされ，サイクル 5 に実行を終える．結果書き込みはサイクル 6 になるはずだが，そのときバスは I1 の結果を運ぶので，$W_3 = 7$ となる．したがって，サイクル 7 に加減算器が I2 のために空いているかどうかは，後の命令 I3 の行に依存する．I4 はサイクル 5 から準備完了だが，加減算器は，結果がバスを待っている I3 にその結果書き込みのサイクル 7 まで占有され，続いてより古い I2 にサイクル 9 まで占有されるので，$D_4 = 9$，$W_4 = 11$ となる．I6 は，$W_1 = 6$ で ML1 が解放されるまで乗除算用のステーションが 2 つとも使用中なので，$I_6 = 7$ となる．そのオペランドは I2 と I4 から来るので，式 (7.1) より $D_6 \geq 1 + \max(7, 9, 11) = 12$ であり，これは I5 が乗除算器を解放するサイクルでもあるので，$D_6 = 12$，$W_6 = 16$ となる．この表は表 7.2 の完全なトレースと一致する．

ヒント バスが 1 本なら，2 つの命令が同じサイクルにブロードキャストすることはない．W_n の列に同じ値が 2 つあれば，バスの競合を見落としている．

表 7.4: 実行例の発行，ディスパッチ，結果書き込みのサイクル．

	命令	I_n	D_n	W_n	タイミングを決める要因
I1	`MUL R1, R2, R3`	1	2	6	発行時に準備完了
I2	`ADD R4, R1, R5`	2	7	9	$W_1 = 6$；$W_3 = 7$ まで加減算器は I3 が使用
I3	`SUB R6, R2, R5`	3	4	7	発行時に準備完了；サイクル 6 はバスが使用中
I4	`ADD R1, R5, R3`	4	9	11	加減算器を I3，次いでより古い I2 が使用
I5	`DIV R2, R6, R3`	5	8	12	$W_3 = 7$
I6	`MUL R3, R4, R1`	7	12	16	$W_1 = 6$ までステーションなし；$W_4 = 11$；$W_5 = 12$ まで乗除算器が使用中
I7	`SUB R5, R3, R2`	8	17	19	$W_6 = 16$

要点 — 章のまとめ

Tomasulo のアルゴリズムは，レジスタリネーミングと，各命令をオペランドが揃い次第実行するアウトオブオーダ実行とを組み合わせる．発行は，命令をプログラム順に命令キューからリザベーションステーションへ移し，各オペランドをレジスタファイルの値または RAT のタグとして取得し，宛先の RAT エントリがそのステーションを指すようにする．ステーションは命令の準備ができるまで共通データバスから値を取り込み，その後ディスパッチが命令を空いている実行ユニットに送る．複数の命令が準備できていれば最も古いものを選ぶ．結果書き込みは値とタグをブロードキャストし，ステーションを解放し，結果が古くなっていない場合に限りレジスタファイルと RAT を更新する．1 本のバスは 1 サイクルに 1 つの結果を運び，遅いユニットが優先される．ロードとストアはプログラム順に実行される．1 サイクルに 1 ステップという前提のもとでは，各命令の発行，ディスパッチ，結果書き込みのサイクルは，そのオペランド，利用可能なステーションとユニット，そしてバスから決まる．Tomasulo のアルゴリズムは例外をサポートしない．次章のリオーダバッファがこれを加える．

発行・ディスパッチ・結果書き込みはここで述べたままである．第 8 章は，レジスタファイルをプログラム順に書き換える第 4 のステップ，コミットを加える．

第 8 章

リオーダバッファ

講義動画	Lesson 8，動画 1–40
教科書	H&P [1] §3.6，§3.8，§C.4；[17]
キーワード	例外，正確な例外，ファントム例外，リオーダバッファ，コミットポインタ，発行ポインタ，コミット，分岐予測ミス回復，レジスタエイリアステーブル，統合リザベーションステーション，スーパースカラ，アウトオブオーダ実行

第 7 章の Tomasulo のアルゴリズムは，各命令をそのオペランドが揃い次第実行し，結果がブロードキャストされた時点でそれをレジスタファイルに書き込む．リネーミングによって計算される値そのものは正しく保たれるが，実際のプロセッサがどれも扱わなければならない 2 つの事象が，この仕組みを破綻させる．例外と分岐予測ミスである．どちらの場合も，問題を起こした命令よりプログラム順で後ろにある命令がすでにレジスタを上書きしている可能性があり，その書き込みは取り消せない．この章では，命令をアウトオブオーダに実行しながらレジスタの更新は厳密にプログラム順に行う，リオーダバッファを導入する．そうして得られるプロセッサの動作を詳細に追い，最後に統合リザベーションステーション，スーパースカラ実行，そしてアウトオブオーダプロセッサのどのステップが実際に順序どおりでないのかを正確に述べる．

8.1 アウトオブオーダ実行における例外

例外とは，0 除算や不正なメモリアドレスへのアクセスのように，命令が実行時に発生させる事象である．プロセッサは例外を起こした命令のアドレスを保存し，通常はオペレーティングシステムの一部である例外ハンドラに制御を移す．ハンドラが原因に対処し終えると，保存されたアドレスに復帰することがあり，そのとき例外を起こした命令が再び実行されて，プログラムはその後を続行する．ハンドラが正しく働くため，そして復帰後にプログラムが正しく続行するためには，レジスタとメモリが，命令を 1 つずつ実行するプロセッサでの姿と同じように見えなければならない．

出典 Lesson 8，動画 1–2；H&P §C.4，§3.6．

注意 MIPS の慣習に従う文献では，命令が原因とならない事象，たとえばタイマの満了や I/O の完了を割り込みと呼んで例外と区別する．H&P §C.4 はどちらも例外と呼ぶ．本章の仕組みはそのどちらにも使える．

定義 8.1 (正確な例外)．ハンドラが開始する時点で，例外を起こした命令よりプログラム順で先行する命令がすべて完了してレジスタとメモリを更新しており，かつ例外を起こした命令もそれより後続の命令もそれらを変更していないとき，その例外を**正確な例外**という．

▷ precise exception

Tomasulo のアルゴリズムは正確な例外を提供しない．次の例を考える．

```
I1:  DIV R1, R2, R3      ; R3 = 0: 0 除算
I2:  ADD R2, R4, R5      ; I1 が読む R2 を上書きする
I3:  ADD R6, R6, R7      ; R6 <- R6 + R7
```

I2 と I3 は I1 に依存しないので，発行から数サイクルのうちに実行され，R2 と R6 に書き込む．除算は多数のサイクルを要し，ここで用いるモデルでは，除算器は演算が完了した時点で 0 除算を報告する（図 8.1）．そのときには後続の 2 つの結果はすでにレジスタファイルに入っており，3 つの不都合が生じる．

- ハンドラは，I1 で停止した逐次実行では生じえない R2 と R6 の値を見る．
- 復帰後，I1 が再び実行され，今度は I2 が書いた R2 の値を読む．プログラム順では決して見てはならない値である．
- I2 と I3 も 2 度目の実行を受ける．I2 は同じ値を計算し直すだけだが，I3 は R6 に R7 を 2 回加えてしまう．

8.2 分岐予測ミス

分岐予測ミスも同じ問題を引き起こす．次のプログラムでは，I2 が除算の結果を

出典 Lesson 8，動画 3；H&P §3.6．

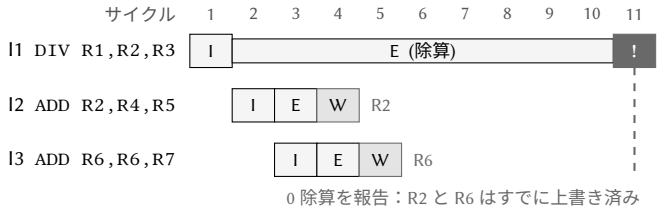


図 8.1: Tomasulo のアルゴリズムのもとでの例外（I: 発行，E: 実行，W: 結果書き込み）．I2 と I3 は，除算 I1 が 0 除算を報告するよりはるかに早く R2 と R6 に書き込む．

待っている間に，分岐予測器のおかげでプロセッサは I3 から I6 までをフェッチし発行できる．

```
I1:         DIV R1, R2, R3      ; 多数のサイクルを要する
I2:         BEQ R1, R4, Skip    ; 不成立と予測される
I3:         ADD R5, R6, R7      ; I3-I6: 分岐が不成立の
I4:         LW  R8, 0(R5)       ;    ときだけ実行される
I5:         MUL R9, R5, R5
I6:         SUB R6, R9, R7
            ...
     Skip: ...
```

I2 は R1 を必要とするので除算が完了するまで実行できないが，I3 はただちにオペランドが揃い，I4 もその 1 サイクル後に揃う．Tomasulo のアルゴリズムのもとでは，R5 と R8 は分岐が解決されるよりはるかに早く書き込まれる．分岐が成立すると判明した場合，誤った経路の命令はリザベーションステーションから取り除き，`Skip` からフェッチをやり直せるが，R5 と R8 はプログラムが決して計算しない値で上書きされており，正しい値は失われている．

第 2 の問題は，誤った経路の命令が例外を発生させるときに生じる．プログラムが R5 をアドレスとして使うのは分岐が不成立のときだけであり，成立するときには R6 + R7 が有効なアドレスである必要はない．したがってロード I4 は，I2 が実行されるよりはるかに早く例外を発生させうる．

定義 8.2 (ファントム例外). 予測を外した経路上にあり，したがってプログラムの逐次実行では決して実行されない命令が発生させる例外を**ファントム例外**という．

▷ phantom exception

ファントム例外はハンドラに届いてはならない．プログラム順では，それを発生させた命令は存在しないからである．

注意 ファントム例外は本講義の用語である．H&P は投機実行された命令が発生させる例外と呼ぶ．その例外も同様にバッファされ，命令もろとも破棄される．

8.3 正しいアウトオブオーダ実行

出典 Lesson 8，動画 4；H&P §3.6；[17]．

2 つの問題の原因は同じである．命令が結果を計算し次第レジスタを変更してしまい，先行するすべての命令が例外も予測ミスもなく完了したと判明する前に書き込みが行われるのである．対策は，結果を計算することと，それを可視にすることとを分離することである．

- 命令はこれまでどおり，オペランドが揃い次第アウトオブオーダに**実行**される．
- 結果もこれまでどおりアウトオブオーダに**ブロードキャスト**され，待機中の命令は遅れなくそれを受け取る．
- しかし結果が**レジスタに書き込まれる**のはプログラム順であり，しかもその命令がプログラムの実行に属すると確実になってからである．

書き込みが許されるまで結果を保持するには，発行された後の命令のプログラム順を覚えておく構造が必要である．リザベーションステーションにはこれができない．命令は結果がブロードキャストされ次第ステーションを離れ，またステーションは特定の順序で割り当てられるわけでもないからである．

Smith と Pleszkun [17] は同じ目的を果たす方式をほかに 3 つ挙げている．結果シフトレジスタ，ヒストリバッファ，フューチャファイルである．

要点
実行はアウトオブオーダ，ブロードキャストもアウトオブオーダ，しかしレジスタへの書き込みはプログラム順に行う．

8.4 リオーダバッファ

プログラム順を覚えておく構造は，発行済みの命令のキューである．

出典 Lesson 8，動画 5–10；H&P §3.6．

定義 8.3 (リオーダバッファ). **リオーダバッファ**（ROB）は，発行済みで，かつ結果をレジスタファイルに書き込む 定義 8.4 のコミットステップをまだ通過していない命令を，すべてプログラム順に保持する．各エントリは 3 つのフィールドを持つ．

▷ reorder buffer

Reg 命令の宛先レジスタ．
Value 計算された後の，その結果．
Done Value が結果を保持していることを示すビット．

本章の例にはこの 3 つのフィールドで足りる．実際にはエントリは，命令が例外を発生させたか，あるいは予測を外した分岐であるかと，命令のアドレスも記録しなければならない．コミットが例外を受け付けたりフェッチをやり直したりするのに，これらが必要だからである（8.5 節，8.6 節）．

ROB は 2 つのポインタで管理される循環バッファである（図 8.2）．**発行ポインタ**は次に発行される命令が受け取るエントリを示し，命令が発行されるたびに 1 エントリ進む．まだコミットしていない最も古い命令は**コミットポインタ**の位置にある．コミットポインタから発行ポインタの手前までのエントリが使用中であり，命令をプログラム順に保持する．それ以外はすべて空きである．2 つのポインタが等しい状態は空のバッファと満杯のバッファの両方を表しうるので，使用中エントリ数か満杯ビットで両者を区別する．通常の動作では両ポインタは進むだけで，最後のエントリから先頭へ巻き戻る．発行ポインタが戻るのは回復のときだけである（8.5 節）．すべてのエントリが使用中のときは，最も古い命令が ROB を離れるまで命令を発行できない．発行される命令はすべてエントリを受け取り，レジスタに書き込まない分岐も例外ではない．分岐のエントリはプログラム順における位置を保ち，後述するように予測を外したかどうかを記録する．

▷ issue pointer
▷ commit pointer

ROB は大きくなり続けてきた．Pentium Pro の 40 エントリから，Skylake は 224（第 7 章），AMD Zen 3 は 256，Intel Golden Cove は 512 である．Intel 最適化マニュアル，AMD Family 19h ソフトウェア最適化ガイド．

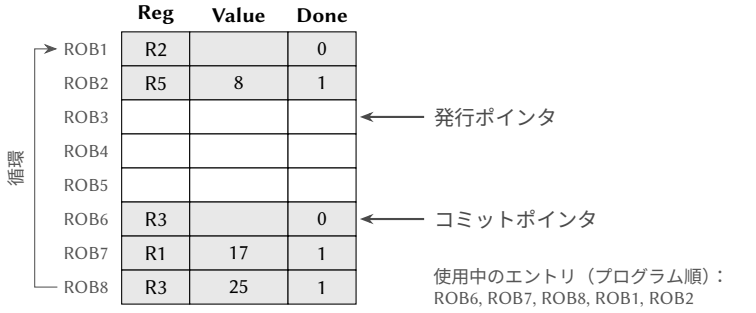


図 8.2: 8 エントリのリオーダバッファ．コミットポインタから発行ポインタの手前までの網掛けのエントリが使用中である．ここではこの範囲が ROB8 から ROB1 へ巻き戻っている．

8.4.1 発行，ディスパッチ，結果書き込み

ROB を備えると，Tomasulo のアルゴリズムの 3 つのステップは次のように変わる．

発行 命令は，空いているリザベーションステーション（RS）と，発行ポインタの位置にある空き ROB エントリを必要とする．どちらかが欠ければ発行は待つ．ROB エントリは宛先レジスタを受け取り，Done ビットはクリアされ，発行ポインタが進む．続いて各オペランドを RAT で調べる．RAT のエントリは，いまや RS ではなく ROB エントリを示す．

- RAT エントリがレジスタファイルを指していれば，値をレジスタから複製する．
- Done ビットの立った ROB エントリを示していれば，値をそのエントリから複製する．
- そうでなければ，RS はその ROB エントリをオペランドのタグとして記録し，そのタグを伴うブロードキャストを待つ．

最後に，宛先レジスタの RAT エントリを新しい ROB エントリに設定し，RS はそのエントリを自身の結果のタグとして記録する．

ディスパッチ これまでと同様に，オペランドがすべて揃った命令が実行を開始する．RS はこの時点で解放される．

結果書き込み 結果は，そのタグである ROB エントリとともに CDB にブロードキャストされる．待機中の RS はこれまでどおり値を取り込み，ROB エントリは値を格納して Done ビットを立てる．レジスタファイルも RAT も変更されない．

2 つ目の参照のケースは新しい．結果は結果書き込みからコミットまで ROB に留まり，その間 RAT はその ROB エントリを示し続ける．この区間に発行される命令は，値をそこで見つけなければならない．見つけられなければブロードキャストを待つことになるが，そのブロードキャストはすでに起きてしまっているからである．

RS をより早く解放する点も新しい．Tomasulo のアルゴリズムでは，ステーション自体が結果をブロードキャストするときのタグなので，RS は結果書き込みまで確保され続けなければならない．ROB があればタグは ROB エントリなので，演算が始まってしまえば RS は不要になり，同じ数の RS でより多くの待機中の命令を保持できる．

8.4.2 コミット

すべての命令を完了させる第 4 のステップがある．

▷ commit

定義 8.4 (コミット). **コミット**のステップでは，プロセッサはコミットポインタの位置にあるエントリを調べる．その Done ビットが立っていれば，値を宛先レジスタに書き込み，そのレジスタの RAT エントリがまだこの ROB エントリを示していれば，RAT エントリをレジスタファイルを指すように戻す．続いてコミットポインタが進み，エントリが解放される．

コミットポインタの位置にある命令が完了していなければ，後続のエントリが完了していても何もコミットされない．結果がレジスタファイルに入るのはプログラム順にかぎられる．RAT エントリを戻すのは，それがまだコミットするエントリを示している場合だけである．そうでなければ，後の命令が同じレジスタに書き込むということであり，RAT はその命令のエントリを示し続けなければならない（8.7 節）．コミットは**リタイア**とも呼ばれる．

ベンダのマニュアルではリタイアが普通の語である．Intel の INST_RETIRED.ANY カウンタはコミットを数えており，実測の IPC（第 2 章）はこれから求める．

8.4.3 ハードウェア構成

図 8.3 は，第 7 章の構成に ROB を加えたものを示す．CDB はいまや結果を RS と ROB に届け，RAT は ROB エントリを示し，レジスタファイルに書き込むのはコミットだけである．[1] 命令は，フェッチ，デコード，発行，実行（ディスパッチから演算の完了まで），結果書き込み，コミットという 6 つのステップを経る．

注 8.5. 任意の時点で，これまでに発行された命令をプログラム順に見たときの，あるアーキテクチャレジスタの最新の値は，3 つの場所のいずれかにある．RAT がレジスタファイルを指していればレジスタファイルの中，Done ビットの立った ROB エントリの中，あるいはまだ計算されておらず，Done ビットがクリアされた ROB エントリに入るのを待っている状態である．レジスタファイル自体は，常に最後にコミットした命令の時点での値を保持する．

[1] 結果を ROB に保持する本章の方式は，Intel の P6 コア（Pentium Pro，1995 年）の設計である．Sandy Bridge（2011 年）以降は，結果は第 6 章の物理レジスタファイルに直接書き込まれ，ROB エントリはその命令に与えられた物理レジスタを記録するだけである．コミットは値を複製せず対応を変え，回復は分岐より後に確保した物理レジスタを解放する．

8.5 分岐予測ミス回復

ROB があれば，予測を外した経路の命令は，コミットする前に取り除かれるかぎりレジスタファイルを変更できない．8.2 節 のプログラムでは，I2 が R1 を待っ

出典 Lesson 8，動画 11–12；H&P §3.6.

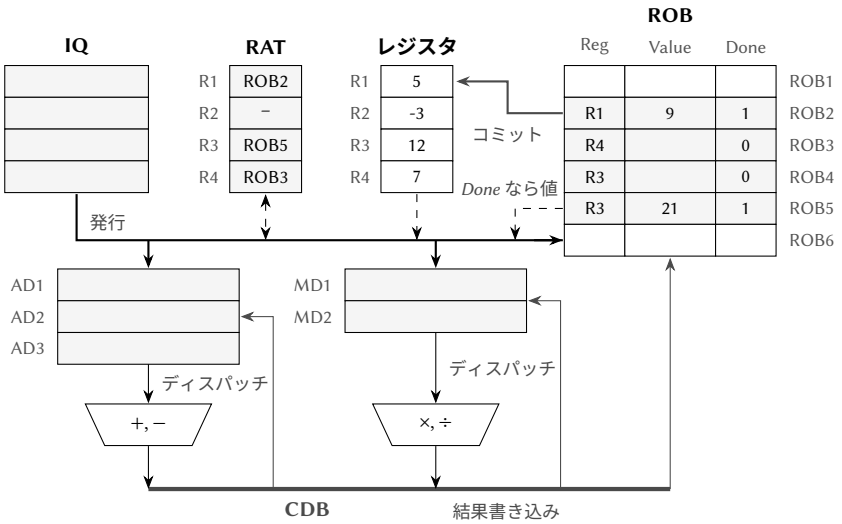


図 8.3: リオーダバッファを備えた Tomasulo の構成．発行は RS と ROB エントリを確保し，RAT を通じてオペランドをリネームする．オペランドの値はレジスタファイルから，あるいは Done の立った ROB エントリから得られる．結果は CDB から RS と ROB へ送られ，コミットはコミットポインタの位置の値をレジスタファイルに複製する．

ている間に I3 から I6 が発行され，まだ実行中の I6 を除くすべてが完了する．I3 と I5 は結果を自身の ROB エントリに書き込み，ロード I4 はファントム例外を発生させ，それが自身のエントリに記録される（8.6 節）．I2 がようやく実行され，分岐の予測が外れていたと分かると，予測ミスが I2 の ROB エントリに記録され，その時点では他に何も起きない．分岐より前の命令は影響を受けず，実行とコミットを続ける．

分岐がコミットポインタに達したとき，それより前の命令はすべてコミットしているので，レジスタファイルは，逐次実行が分岐を実行し終えた時点で持つ値をちょうど保持している．プロセッサ内に残る命令はすべて分岐より後ろにあり，誤った経路の上にある．そこでプロセッサは**分岐予測ミス回復**を行う．これは次の 4 つの動作からなる（図 8.4）．

1. 発行ポインタをコミットポインタと等しく設定してすべての ROB エントリを破棄し，ROB を空にする．
2. すべての RAT エントリを，レジスタファイルを指すように戻す．
3. すべての RS を解放し，実行ユニット内の演算を放棄する．それらの結果は，破棄された ROB エントリにしか行き場がない．
4. 分岐の正しい後続命令からフェッチをやり直す．

予測が当たった分岐は単にコミットする．コミットポインタが進むだけで，他には何も起きない．[2]

分岐が待たされる時間は長くなりうる．32 ビット整数除算だけでも Skylake では 26 サイクルかかる（Agner Fog，*Instruction tables*）．

▷ branch misprediction recovery

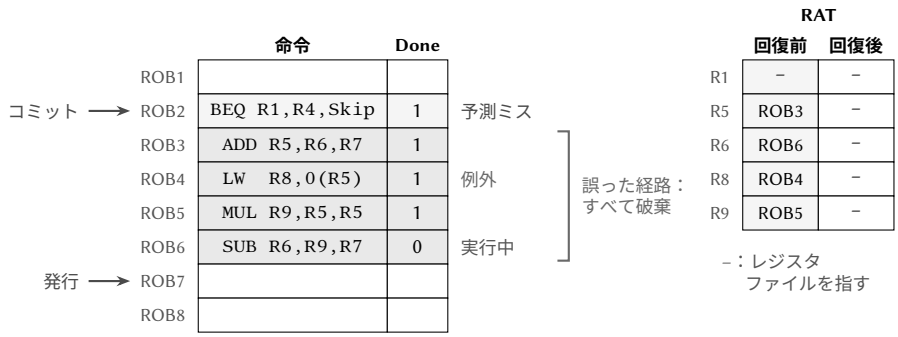


図 8.4: 予測を外した分岐 I2 がコミットポインタに達したときの回復．I1 はすでにコミットしている．I3 から I6 の ROB エントリは，I4 に記録された例外もろとも破棄され，RAT のエントリはすべてレジスタファイルに戻る．

[2] コミット時に回復するのは最も単純な方式であり，本章で用いるのもこれである．実際のプロセッサは，予測を外した分岐が実行され次第回復を開始し，分岐より後ろの ROB エントリだけを破棄して直ちにフェッチをやり直す．こうすれば，分岐がコミットを待つ間にサイクルを失わずに済む（H&P §3.6）．その場合 RAT は分岐の時点の状態に戻さなければならない．たとえば分岐の発行時に保存しておいた複製から戻す．

8.6 リオーダバッファのもとでの例外

出典 Lesson 8，動画 13–15；H&P §3.6.

例外は，それを発生させた命令のもう 1 つの結果として扱うことで処理する．命令が実行中に例外を発生させると，プロセッサはハンドラを呼び出す代わりに，その例外を命令の ROB エントリに記録する．そのエントリがコミットポインタに達したとき，例外が受け付けられる．プロセッサは，予測ミス回復とまったく同様に ROB エントリを破棄し，RAT を戻し，RS と実行ユニットを解放したうえで，例外を起こした命令のアドレスを保存し，例外ハンドラのフェッチを開始する．

これにより 8.1 節，8.2 節 の 2 つの問題がともに解消される．

正確な例外 例外を起こした命令がコミットポインタに達した時点で，先行する命令はすべてコミットしており，後続の命令はレジスタファイルに書き込んでいない．ストアも同様にコミット時にはじめてメモリに書き込む（第 9 章）ので，メモリも後続の命令によって変更されていない．ハンドラは定義 8.1 の状態をちょうど見ることになり，復帰後には後続の命令がその状態から実行される．

デマンドページングはこれに依存する．ハンドラが不足したページを取ってきた後，フォールトを起こしたロードが再び実行される（第 13 章）．

ファントム例外は起きない 予測を外した経路上の例外は，予測を外した分岐より後ろのエントリに記録される．先に分岐がコミットし，その回復がそのエントリを例外もろとも破棄するので，例外が受け付けられることはない．図 8.4 では，I2 がコミットするときに I4 の例外が消える．

8.6.1 実行の外部からの見え方

ROB は，命令がいつ実行されたのかという 2 つの捉え方を分離する．プロセッサの内部では，命令は実行ユニットがその演算を行ったときに，順序を問わず実行される．一方，プログラマ，オペレーティングシステム，例外ハンドラに見えるのはレジスタファイル，メモリ，プログラムカウンタだけであり，これらにとって命令が実行されるのはコミットしたときである．コミットはプログラム順に 1 命令ずつ行われ，その様子は，各命令を完了させてから次を始めるプロセッサとまったく変わらない．結果が ROB にあってもまだコミットしていない命令は，外から見ればまだ実行されていない．その結果が破棄されれば，まったく実行されなかったことになる．8.8 節 の例は，最後にその実例を示す．

8.7 コミット時の RAT 更新

出典 Lesson 8，動画 16；H&P §3.6.

定義 8.4 の RAT 更新に条件が付いているのは，1 つのレジスタに書き込む命令が同時に複数 ROB の中にありうるからである．ROB2 と ROB5 の命令がともに R4 に書き込み，ROB5 がまだ ROB にある状態で ROB2 がコミットするとする．R4 の RAT エントリは ROB5 を示している．そちらの命令のほうが後で発行されたからである．

- コミットは ROB2 の値を R4 に書き込む．これは ROB2 の後のアーキテクチャ上正しい値であり，ROB2 と ROB5 の間の命令が例外を発生させたときに例外ハンドラが見なければならない値でもある．
- それでも RAT エントリは ROB5 を示し続けなければならない．いま発行される命令はプログラム順で ROB5 より後ろにあり，レジスタファイルに入ったばかりの古い値ではなく，ROB5 の値を使わなければならない．

ROB5 がコミットするとき，その間にさらに後の書き込み命令が発行されていなければ，R4 の RAT エントリは依然として ROB5 を示しており，そのときにかぎってエントリが戻される．逆に，コミット済みのエントリを示したまま RAT エントリを放置することも，同じく誤りである．エントリはコミット時に解放され，まもなく無関係な命令に与えられうるからである．

ヒント あるレジスタへの最も新しい書き込み命令だけが，自分の RAT エントリを戻す．それより古い命令はレジスタファイルに書き込むだけで，RAT には触れない．

8.8 リオーダバッファの例

出典 Lesson 8，動画 17–30；H&P §3.6.

ここで，ROB を備えたプロセッサでプログラムを最後まで追う．このプロセッサは，加算・減算用に 3 つの RS（AD1–AD3），乗算・除算用に 2 つの RS（MD1, MD2），そして 8 エントリの ROB を持つ．ADD と SUB は 1 サイクル，MUL は 4

サイクル，DIV は 6 サイクルを要し，実行ユニットは十分にあって，準備のできた命令がユニットを待つことはない．各ステップは次の規約に従う．

- 1 サイクルに 1 命令を発行し，必要な種類の空いている RS のうち番号の最も小さいものに入れる．
- サイクル C のディスパッチで解放された RS と，サイクル C のコミットで解放された ROB エントリは，サイクル $C+1$ から新しい命令に与えられる．
- 命令がディスパッチされるのは，発行の次のサイクル以降であり，かつ待っている各オペランドのブロードキャストの次のサイクル以降である．
- サイクル D にディスパッチされ，実行時間が k サイクルの演算は，サイクル D から $D+k-1$ まで実行され，サイクル $D+k$ に結果を書き込む．CDB は 1 本であり，この例では 2 つの結果が同じサイクルに書き込まれることはない．
- 命令がコミットするのは早くとも結果書き込みの次のサイクルであり，1 サイクルにコミットする命令は高々 1 つである．

1 サイクル 1 コミットはこの例の仮定である．Skylake は 1 サイクルに最大 4 マイクロオペレーション，AMD Zen 3 は最大 8 をリタイアさせる．Intel 最適化マニュアル，AMD Family 19h ソフトウェア最適化ガイド．

例 8.6. プログラムは次のとおりである．

```
I1:  DIV R1, R2, R3      ; R1 <- R2 / R3
I2:  ADD R4, R5, R6      ; R4 <- R5 + R6
I3:  MUL R2, R4, R4      ; R2 <- R4 * R4
I4:  SUB R5, R1, R6      ; R5 <- R1 - R6
I5:  ADD R4, R4, R3      ; R4 <- R4 + R3
I6:  ADD R6, R2, R4      ; R6 <- R2 + R4
```

初期値は R1 = 5，R2 = 48，R3 = 4，R4 = 2，R5 = 7，R6 = 3 である．RAT のエントリはすべてレジスタファイルを指し，ROB の 2 つのポインタはともに ROB1 にある．

サイクル 1–6：発行． 1 サイクルに 1 命令が発行され，Ik は ROBk に入る．I1 はレジスタファイルから 48 と 4 を MD1 に複製し，R1 の RAT エントリを ROB1 に設定する．サイクル 2 で I1 がディスパッチされて MD1 を解放し，I2 が値 7 と 3 とともに AD1 に発行される．サイクル 3 で発行される I3 は，R4 の RAT エントリに ROB2 を見つける．ROB2 は完了していないので，I3 の両オペランドは ROB2 を待つ．I3 は MD1 を再び使える．Tomasulo のアルゴリズムだけであれば，MD1 はサイクル 8 まで I1 に占有されたままだったはずである．I4 は，サイクル 3 で I2 が解放した AD1 に発行される．第 1 オペランドは ROB1 を待ち，第 2 オペランドは 3 である．サイクル 4 で I2 が結果を書き込み，ROB2 が 10 を受け取って I3 がそれを取り込む．サイクル 5 で I5 が AD2 に発行され，R4 の RAT エントリがまだ ROB2 を示しているのを見つける．ROB2 は完了しているので，I5 は 10 を ROB から複製する．古い値 2 をまだ保持しているレジスタファイルからではない．続いて I5 は R4 を ROB5 にリネームする．サイクル 6 で I6 は AD3 に発行される．同じサイクルに I5 のディスパッチで解放される AD2 は，サイクル 7 からしか使えないからである．I6 は ROB3 と ROB5 を待つ．表 8.1，8.2 はサイクル 6 の終了時の状態を示す．AD1 が I4 を，AD3 が I6 を保持し，他の RS はすべて空きで，レジスタファイルは変化していない．

ヒント レジスタを 1 つずつ追うとよい．RAT エントリは書き込み命令が発行されるたびに変わり，読み出す命令は自身の発行サイクルで RAT が示すものを使う．

ヒント 使用中のエントリ数は（発行 − コミット）mod 8 である（空でも満杯でも 0 になる）．サイクル 6 の終了時は $(7-1) \bmod 8 = 6$，すなわち ROB1 から ROB6 である．

表 8.1: 例 8.6 のサイクル 6 と 10 の終了時の ROB．サイクル 6 の終了時にはコミットポインタは ROB1 に，サイクル 10 の終了時には ROB3 にある．発行ポインタはどちらでも ROB7 にある．

			サイクル 6		サイクル 10	
エントリ	命令	Reg	Value	Done	Value	Done
ROB1	DIV R1, R2, R3	R1		0	コミット済み	
ROB2	ADD R4, R5, R6	R4	10	1	コミット済み	
ROB3	MUL R2, R4, R4	R2		0	100	1
ROB4	SUB R5, R1, R6	R5		0	9	1
ROB5	ADD R4, R4, R3	R4		0	14	1
ROB6	ADD R6, R2, R4	R6		0		0

表 8.2: 例 8.6 の RAT とレジスタファイル．ダッシュは，RAT エントリがレジスタファイルを指すことを表す．

	サイクル 6		サイクル 10		最終
レジスタ	RAT	値	RAT	値	値
R1	ROB1	5	–	12	12
R2	ROB3	48	ROB3	48	100
R3	–	4	–	4	4
R4	ROB5	2	ROB5	10	14
R5	ROB4	7	ROB4	7	9
R6	ROB6	3	ROB6	3	114

サイクル 7–8：結果は出るがコミットはない． I5 はサイクル 7 に 14 を ROB5 に，I1 はサイクル 8 に 12 を ROB1 に書き込み，I6 と I4 がこれらの値を取り込む．ROB2 はサイクル 4 から，ROB5 はサイクル 7 から完了しているが，何もコミットされない．コミットポインタの位置にある ROB1 が，サイクル 8 の終わりまで完了しないからである．

サイクル 9–10：最初のコミット． サイクル 9 で I1 がコミットする．R1 は 12 になり，R1 の RAT エントリはまだ ROB1 を示しているので，レジスタファイルを指すように戻される．同じサイクルに I3 が 100 を ROB3 に書き込み，I4 がディスパッチされる．サイクル 10 で I2 がコミットして R4 は 10 になるが，R4 の RAT エントリは ROB5 を示しているので，8.7 節 の要求どおりそのままにされる．I4 が 9 を書き込み，I6 がディスパッチされる．

サイクル 11–14：残りのコミット． I6 はサイクル 11 に 114 を書き込む．I3，I4，I5，I6 はサイクル 11 から 14 に 1 サイクル 1 つずつコミットする．いずれも自身のレジスタへの最後の書き込み命令なので，それぞれが自分の RAT エントリを戻す．最終的に ROB は空になり，RAT のエントリはすべてレジスタファイルを指し，レジスタは R1 = 12，R2 = 100，R3 = 4，R4 = 14，R5 = 9，R6 = 114 という逐次実行の値を保持する．

サイクル 10 の終了時，レジスタファイルの R4 は 10 を保持する一方で，RAT は R4 を読む命令を，14 を保持する ROB5 に導く．どちらも正しい．10 は最後にコミットした命令 I2 の後の R4 の値であり，14 は最後に発行された書き込み命令 I5 の後の値である．

この例は 8.6.1 節 の外部からの見え方も示している．I4 がサイクル 9 の実行時に例外を発生させたとする．例外は ROB4 に記録され，I2 と I3 はそのままサイクル 10 と 11 にコミットし，サイクル 12 に I4 がコミットポインタに達して例外が受け付けられる．ハンドラが見るのは R1 = 12，R2 = 100，R4 = 10 であって，R4 = 14 ではない．I5 は I1 が結果を書き込むより前のサイクル 7 に 14 を ROB5 に書き込んでいたにもかかわらずである．外から見れば，実行はちょうど I4 の直前で停止したのである．

8.9 リオーダバッファのもとでのタイミング

ROB を備えたプロセッサのサイクルごとの状態は，第 7 章の Tomasulo のアルゴリズムと同様に，各命令が各ステップを通過するサイクルによって要約でき，そこにコミットの列が 1 つ加わる．記法は第 7 章のままとする．I_n，D_n，W_n は命令 n の発行，ディスパッチ，結果書き込みのサイクル，k_n はその実行時間，$P(n)$ はそのオペランドを生成する命令の集合であり，C_n をそのコミットのサイクルとする．8.8 節 の規約のもとでは，I_n は I_{n-1} より後で，必要な種類の RS と ROB エントリがともに空いている最初のサイクルである．命令 m のディスパッチで解放された RS はサイクル $D_m + 1$ から，m のコミットで解放された ROB エントリはサイクル $C_m + 1$ から空いている．実行ユニットが十分にあり CDB の奪い合

出典 Lesson 8，動画 31–34；H&P §3.6.

いもなければ，残りのサイクルは次式から定まる．

$$D_n = 1 + \max\Big(I_n,\ \max_{p\in P(n)} W_p\Big), \qquad W_n = D_n + k_n, \qquad C_n = \max\big(W_n + 1,\ C_{n-1} + 1\big), \tag{8.1}$$

ここで第 1 式は，実行ユニットが空いている場合の 式 (7.1) の下限である．実行とは異なり，コミットは 2 つの命令がデータを共有しない場合でも直前の命令に依存する．RS をいつ解放するか，1 サイクルに何命令コミットするかは設計上の仮定であり，これを変えれば表も変わる．第 7 章のように RS が結果書き込みではじめて解放されるなら，RS はサイクル $W_m + 1$ から空く．2 つの命令が同じサイクルにコミットしうるなら，$C_n = \max(W_n + 1,\ C_{n-1},\ C_{n-2} + 1)$ となる．

ヒント 表はプログラム順に 1 行ずつ，各行は左から右へ，コミットを最後にして埋めるとよい．

表 8.3: 例 8.6 のタイミング．実行の列は，演算が行われるサイクル D_n から $D_n + k_n - 1$ までを示す．

	命令	発行	実行	結果書き込み	コミット
I1	`DIV R1, R2, R3`	1	2–7	8	9
I2	`ADD R4, R5, R6`	2	3	4	10
I3	`MUL R2, R4, R4`	3	5–8	9	11
I4	`SUB R5, R1, R6`	4	9	10	12
I5	`ADD R4, R4, R3`	5	6	7	13
I6	`ADD R6, R2, R4`	6	10	11	14

表 8.3 は例に対する結果を示す．実行の列と結果書き込みの列は順序どおりでない．I5 は，先行する I1，I3，I4 より先に結果を書き込む．コミットの列は増加している．I2 と I5 は結果書き込みの 6 サイクル後にコミットする．ROB の先頭にある遅い除算に，そしてそれがサイクル 9 にコミットした後は 1 サイクル 1 命令というコミットの上限に阻まれるからである．それ以降の命令は，いずれも直前の命令のちょうど 1 サイクル後にコミットする．この待ちはどの計算も遅らせない．I3 と I5 は I2 の結果を，I6 は I3 と I5 の結果を，コミットを待つことなく得ている．ROB が遅らせるのは，結果がアーキテクチャ状態の一部になる時点だけである．

とはいえ ROB は，小さすぎれば実行を制限しうる．エントリが 4 つしかなければ，I5 は I1 がコミットした次のサイクルであるサイクル 10 より前には発行できず，オペランドがサイクル 5 から揃っているにもかかわらず，表 8.3 より数サイクル遅れて実行されることになる．したがって ROB は，遅い命令がコミットを待つ間に発行されるすべての命令を保持できるだけの大きさを持たなければならない．

ROB の大きさは，隠したい最長のストールに合わせて決める．1 サイクル 4 命令なら 224 エントリは約 56 サイクル分であり，Skylake の L3 ヒットは 42 サイクル，DRAM アクセスは 200 サイクル程度である（7-cpu.com）．

8.10 統合リザベーションステーション

図 8.3 では，実行ユニットの種類ごとに専用の RS の組があり，各組の大きさは設計時に固定される．プログラムによって演算の構成は異なるので，加算の多いプログラムでは，乗算器の RS が空いているのに加算器の RS がすべて使用中になり，発行が待たされることがある．命令はプログラム順に発行されるので，待たされている加算より後ろにある乗算でさえ，空いている乗算器の RS を使えない．

出典 Lesson 8，動画 35；H&P 第 3 章．

定義 8.7 (統合リザベーションステーション). すべての演算で共有される単一の RS の集まりを**統合リザベーションステーション**という．命令は，その集まりの中の空いている任意の RS に発行できる．

▷ unified reservation station

このとき発行が待たされるのは，すべての RS が使用中のときだけであり，同じ数の RS がより有効に使われる．代償はディスパッチにある．1 つの実行ユニットのために 1 つの組の RS から選ぶのではなく，ディスパッチ論理は，空いている実行ユニットごとに，そのユニットが担当する演算の，準備のできた命令を集まり全体から探し出し，その中から選ばなければならない．

Intel のスケジューラは統合型である（Skylake で 97 エントリ，第 7 章）．AMD の Zen コアは複数の分散した整数用スケジューラと，別の浮動小数点用スケジューラを持つ．各社の最適化マニュアル．

8.11 スーパースカラプロセッサ

第 6 章で見たように，プロセッサの IPC は，1 サイクルに扱える命令数によって制限される（6.8 節）．

出典 Lesson 8，動画 36；H&P §3.8．

定義 8.8 (スーパースカラプロセッサ). **スーパースカラ**プロセッサは，すべてのステップを 1 サイクルに複数の命令に対して行う．すなわち，複数の命令をフェッチ，デコード，発行し，複数を実行ユニットにディスパッチし，複数の結果を書き込み，各サイクルに複数の命令をコミットする．

▷ superscalar

そのためには，各ステップにそれぞれのハードウェアが必要になる．フェッチとデコードは複数の命令を一度に扱わなければならない．発行は同じサイクルに複数の命令をリネームし，そのそれぞれが自分より前の命令による RAT の更新を見なければならない．ディスパッチには複数の実行ユニットが要る．複数の結果を書き込むには複数の CDB が必要で，すべての RS のオペランドが自身のタグをそのすべてと比較しなければならない．コミットは，コミットポインタの位置の複数のエントリを調べ，複数のレジスタに書き込まなければならない．

これらのステップは，すべての命令が通る 1 本の鎖をなす．したがってプロセッサが持続できる 1 サイクル当たりの命令数は，最も制約の厳しいステップが扱える数を超えない．1 サイクルに 4 命令をフェッチ，デコード，ディスパッチしながら発行は 3 命令，コミットは 2 命令というプロセッサは，プログラムにどれだけ ILP があっても，長い目で見れば 1 サイクルに高々 2 命令しか完了させられない．1 つのステップを広げることに意味があるのは，そのステップがボトルネックである場合だけである．

8.12 用語

アウトオブオーダプロセッサのステップに対して本章で用いる名称は，普遍的なものではない．論文，教科書，プロセッサの資料では，いくつもの別名が使われている（表 8.4）．その中にはコミットを *complete* と呼ぶものもある．これに対し本章では，演算が完了するのはその実行が終わったときである．

出典 Lesson 8，動画 37；H&P 第 3 章．

注意 同じ語が別のステップを指すことがある．文献によっては *dispatch* が本講義でいう発行を，*issue* が本講義でいうディスパッチを指す．

表 8.4: 本講義と他の文献におけるステップの名称．

本講義	他の文献での名称
発行	issue，allocate，dispatch
ディスパッチ	dispatch，execute，issue
コミット	commit，complete，retire，graduate

8.13 何がアウトオブオーダなのか

アウトオブオーダ実行を行うプロセッサでも，すべてのステップが順序どおりでないわけではない（図 8.5）．

出典 Lesson 8，動画 38–40；H&P §3.6．

フェッチとデコード 予測された経路に沿って，命令をプログラム順に取り込む．

発行 プログラム順である．リネーミングがそれを要求する．各命令は，自分より前のすべての命令が残した RAT を見なければならず，ROB エントリもプログラム順に割り当てられるからである．

実行と結果書き込み 順序どおりでない．命令はオペランドが揃い次第開始し，その演算が終わったときに終了する．後の命令が先の命令を追い越すのは，この 2 つのステップだけである．

コミット 再びプログラム順である．実行が緩めた順序を ROB が回復する．

注意 ROB は何も並べ替えない．エントリはプログラム順に入り，プログラム順に出ていく．その名は実行に対する効果を指しており，キュー自体を指すのではない．

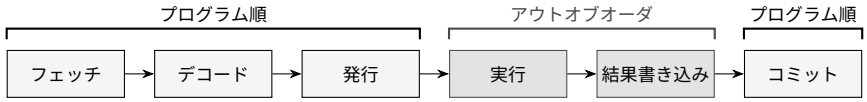


図 8.5: ROB を備えたアウトオブオーダプロセッサのステップ．プログラム順でないのは実行と結果書き込みだけである．

要点 — 章のまとめ

Tomasulo のアルゴリズムは，結果がブロードキャストされ次第それをレジスタに書き込むので，例外は正確でなく，予測を外した命令がレジスタを壊し，誤った経路の命令がファントム例外を発生させうる．リオーダバッファは，発行済みの命令を，コミットポインタと発行ポインタの間にプログラム順に保持する．RAT は ROB エントリを示し，結果は RS と ROB にブロードキャストされ，RS はディスパッチ時に解放される．レジスタファイルに書き込み，コミットするエントリをまだ示している RAT エントリを戻すのは，プログラム順に ROB エントリを 1 つずつ処理するコミットだけである．予測を外した分岐や例外を起こした命令への処置は，それがコミットするときに行う．ROB を空にし，RAT をレジスタファイルに戻し，RS と実行ユニットを解放して，正しいアドレスからフェッチをやり直す．タイミングの表にはコミットの列が加わり，これは 式 (8.1) に従う．統合リザベーションステーションは，1 つの RS の集まりをすべての演算で共有する．スーパースカラプロセッサはすべてのステップで 1 サイクルに複数の命令を扱い，最も狭いステップによって制限される．順序どおりでないのは，実行と結果書き込みだけである．

第 9 章

メモリオーダリング

講義動画	Lesson 9，動画 1–14
教科書	H&P [1] §3.5–3.6
キーワード	メモリを介した依存，ロード，ストア，コミット，ロードストアキュー，LSQ，ストアロードフォワーディング，ロードストアのアウトオブオーダ実行，ロードストアのインオーダ実行，再実行，リオーダバッファ，リザベーションステーション

第 8 章のリオーダバッファによって，アウトオブオーダプロセッサはレジスタを介するあらゆる依存を処理し，例外を正確に保てるようになった．しかしロードとストアは，メモリを介しても互いに依存する．あるロードが，先行するストアの書き込む位置を読むかどうかが分かるのは，両方のアドレスが計算された後である．第 7 章がロードとストアをプログラム順に実行していたのはこのためである．この章では，それらをアウトオブオーダに実行する方法を示す．ストアはコミット時にはじめてメモリに書き込み，ロードストアキューが未コミットのロードとストアをプログラム順に保持し，各ロードは自分の値をメモリから，あるいはそれを書いたストアから直接得る．

9.1 メモリを介した依存

出典 Lesson 9，動画 1–2；H&P §3.5–3.6．

第 8 章のプロセッサは，制御依存と，レジスタを介するあらゆる依存に対処する．分岐予測（第 4 章）は制御依存のコストの大半を取り除き，ROB は予測を外した経路の命令を取り消す．リネーミング（第 6 章）は，同じレジスタ名の再利用から生じる偽の依存（WAR と WAW）を除去する．Tomasulo のアルゴリズムによるスケジューリング（第 7 章）は，各命令を，それが読むレジスタ値が計算されるまで待たせる．そして ROB は正確な例外を実現する．

残るのはロードとストアである．これらは，命令がレジスタを介して互いに依存するのとまったく同じように，メモリ位置を介して互いに依存する（7.8 節）．

典型的な整数プログラムでは命令のおよそ 3 分の 1 がロードまたはストアであり，ストア 1 つにつきロードはおよそ 2 つである（H&P 付録 A，SPEC の命令構成）．

```
SW  R1, 0(R2)       ; MEM[R2] <- R1
ADD R4, R5, R6      ; R4 <- R5 + R6
LW  R3, 0(R4)       ; R3 <- MEM[R4]
```

R4 が R2 に等しいとき，かつそのときに限り，ロードはストアに対して RAW 依存を持つ．このような依存をレジスタを介した依存と区別する性質が 2 つある．

注意 メモリにも WAR 依存と WAW 依存はあるが，ストアはプログラム順にメモリへ書き込む（9.2 節）ので，仕掛けが必要なのは RAW だけである．

- 命令そのものからは見えない．レジスタ名はメモリ位置について何も語らず，メモリ位置はアドレスが計算されてはじめて，多くの場合その命令が発行された数サイクル後に分かる．
- メモリをリネーミングするものは存在しない．RAT が対応付けるのはレジスタ名だけなので，プロセッサはロードごとに，その値を読むべきストアを見つけるか，そのようなストアがないと判定しなければならない．

この章の残りでは 2 つの問いに答える．ストアが実際にメモリに書き込むのはいつか，そしてロードはどこで自分の値を見つけるか，である．

9.2 メモリが書かれるとき

出典 Lesson 9，動画 3；H&P §3.6．

ストアも他の命令と同様にオペランドが揃えば実行されるが，その時点ではメモリに書き込まない．ストアの実行はアドレスを計算し書き込む値を確定するだけであり，書き込み自体はストアがコミットするときに行われる．メモリへの書き込みは，レジスタへの書き込みと同様に取り消せない．ストアが実行時にメモリに書き込むとすると，その後に先行命令が例外を発生させたとき，ハンドラは，プログラム順ではまだ書かれていないはずの位置を見ることになり，また予測を外した経路のストアは，プログラムが変更しないメモリを変更してしまう．これに対しコミット時には，そのストアがプログラムの実行に属することが確定して

おり，しかもストアはプログラム順にコミットする．[1]

書き込みを遅らせることは，ロードにとって問題を生む．ロードはアドレスを計算し次第メモリを読むことができるが，プログラム順で先行するストアが，実行を終えてコミットを待っている，あるいはまだアドレスすら計算していないことがある．そのストアがロードの読む位置に書き込むなら，メモリはまだ古い値を保持しており，ロードは代わりにそのストアの値を得なければならない．したがってプロセッサは，まだコミットしていないすべてのストアのアドレスと値をプログラム順に保持し，すべてのロードがそれらを参照しなければならない．

> **要点**
> ストアはアウトオブオーダに実行されるが，メモリへの書き込みはコミット時にプログラム順で行う．それまでその値はプロセッサの内部にしか存在せず，ロードはそこを探さなければならない．

[1] コミットとは書き込みが取り消せなくなる時点であって，キャッシュに届く時点ではない．実際のコアでは，値はまずストアバッファに移り，その後 L1 キャッシュへ流し出される．他のコアがその値を遅れて見ることはあり，それを制約するのが第 21 章の一貫性モデルである．

第 3 章のパイプラインでは，MEM ステージがロードとストアをプログラム順に行う．

9.3 ロードストアキュー

まだコミットしていないロードとストアは，ROB と同じように働くキューに保持される．

出典 Lesson 9，動画 4–5；H&P §3.6．

定義 9.1 (ロードストアキュー). **ロードストアキュー**（LSQ）は，発行済みでまだコミットしていないすべてのロードとストアを，プログラム順に保持する．各エントリは 4 つのフィールドを持つ．

▷ load-store queue

L/S その命令がロードかストアか．
Addr メモリアドレス．計算された後に記録される．
Value ストアでは書き込む値，ロードでは得た値．
C エントリが完了したときに立つ．ロードでは値を得たとき，ストアではアドレスと値の両方を記録したときである．

ROB と同様に，LSQ は発行ポインタとコミットポインタを持つ循環バッファである（図 9.1）．ロードやストアは，発行されるときに ROB エントリに加えて発行ポインタの位置のエントリを受け取り，その LSQ エントリはコミット時に解放される．他の命令は LSQ を使わない．ロードはアドレスを記録し，続いて値を記録する．ストアは，ベースレジスタが得られ次第アドレスを，値レジスタが得られ次第値を，どちらの順でも記録する．ストアはコミットするまでエントリを保持し，そのときにはじめてメモリに書き込むので，LSQ にあるストアは，値がまだメモリに入っていないストアにちょうど一致する．

ROB もロードとストアを保持するが，キューを分けることでアドレスの探索を短く保てる．

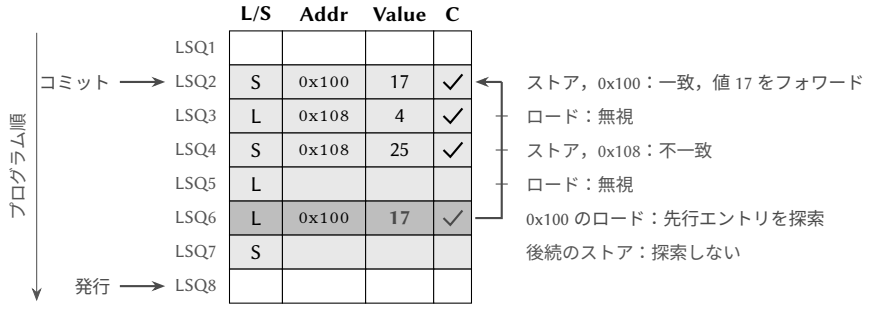


図 9.1: 8 エントリのロードストアキュー．LSQ2 から LSQ7 までが使用中である．LSQ6 のロードはアドレス 0x100 を計算した．その探索はロードと 0x108 へのストアを飛ばし，0x100 への最も近い先行ストアから値 17 を取る．

ロードはアドレスを計算すると，それを自身の LSQ エントリに記録し，自分のエントリからコミットポインタに向かって先行するエントリを探索する．位置を読むことはその内容を変えないので，ロードは飛ばされ，ストアだけが問題になる．探索は，同じアドレスを持つ最初のストアで，あるいはコミットポインタで終わる．

1. 同じアドレスのストアが見つかれば，この最も近い一致するストアが，プログラム順でロードの前にその位置へ書き込む最後の命令なので，その値こそロードが読むべき値である．ロードはそのストアの LSQ エントリから

値を複写し，ストアがまだ値を記録していなければそれを待ち，メモリにはまったくアクセスしない．このように未コミットのストアの値をロードに与えることを，**ストアロードフォワーディング**という．

▷ store-to-load forwarding

2. 一致するストアもアドレスの分からないストアも見つからないままコミットポインタに達すれば，ロードの前にその位置へ書き込む未コミットのストアは存在せず，メモリがすでに正しい値を保持しているので，ロードはメモリを読む．

探索が終わる前に，まだアドレスの計算されていないストアを通過した場合，そのストアはロードの読む位置に書き込むかもしれないし書き込まないかもしれず，プロセッサにはまだどちらとも分からない．これが難しい場合であり，次節ではその扱い方を示す．

注意 第 3 章のフォワーディングとは別物である．あちらはパイプラインステージの間で結果をバイパスするが，ここでは値を一方の命令の LSQ エントリから他方へ複写する．

9.4 ロードストアのアウトオブオーダ実行とインオーダ実行

アドレスの分からないストアを探索が通過してしまうロードに対しては，3 つの選択肢がある．

1. 第 7 章と同様に，ロードとストアを互いにプログラム順に実行する．
2. 先行するすべてのストアがアドレスを計算するまでロードを待たせ，その探索がアドレスの分からないストアに出会わないようにする．
3. ロードをそのまま実行し，誤った値は後で検出して訂正する．

出典 Lesson 9，動画 5–8；H&P §3.6．

最初の 2 つはロードを待たせ，3 つ目は待たせない代わりに検査を必要とする．ストアのアドレスはベースレジスタが得られるまで分からず，そのレジスタを遅い命令が計算する場合には長くかかりうる．次のプログラムでは，ストアのアドレスは乗算に依存するのに対し，ロードのアドレスはほとんど直ちに得られる．

分岐予測（第 4 章）と同じ型である．予測して先に進み，後で検査し，誤っていればそれ以降を取り消す．

```
I1:  MUL R1, R2, R3      ; R1 <- R2 * R3
I2:  ADD R1, R1, R4      ; R1 <- R1 + R4（アドレス）
I3:  SW  R5, 0(R1)       ; MEM[R1] <- R5
I4:  ADD R6, R6, R7      ; R6 <- R6 + R7（アドレス）
I5:  LW  R8, 0(R6)       ; R8 <- MEM[R6]
I6:  SUB R9, R8, R5      ; R9 <- R8 - R5
```

タイミングの規約は 8.8 節のものを用いる．MUL は 4 サイクル，ADD，SUB，ロード，ストアは 1 サイクルを要し，その 1 サイクルでロードはアドレスを計算して値を得，ストアはアドレスを計算する．値 R5 は最初から利用できる．I1 は結果をサイクル 6 に，I2 はサイクル 8 に書き込むので，ストア I3 が実行され，そのアドレスが分かるのはサイクル 9 である．I4 は結果をサイクル 6 に書き込むので，ロード I5 はサイクル 7 に実行できる（図 9.2）．

9.4.1 ロードストアのアウトオブオーダ実行

定義 9.2 (ロードストアのアウトオブオーダ実行). **ロードストアのアウトオブオーダ実行**（選択肢 3）では，ロードは，先行するストアがアドレスを計算しているかどうかにかかわらず，自身のアドレスが計算でき次第実行される．その LSQ 探索は，アドレスの分からないストアを，アドレスが異なるものとみなして通過する．

▷ out-of-order load-store execution

例では I5 はサイクル 7 に，その値を使う I6 はサイクル 9 に実行される．その代わり，すべてのストアは自身のアドレスを，自分を追い越したロードと照合しなければならない．I3 はサイクル 9 にアドレスを計算すると，後続の LSQ エントリの中から，同じアドレスを持ち C ビットの立ったロード，すなわちすでに値を得たロードを探す．

ヒント ロードは自分より前のストアを探し，ストアは自分より後のロードを探す．どちらの探索も自分と同じ種類は飛ばす．

- I5 のアドレスが異なれば何も起こらない．I5 は正しい値を得ており，I6 はサイクル 10 に結果を書き込む．
- アドレスが等しければ，I5 は，プログラム順では I3 が上書きする値を得てしまっているので，その値を直接または間接に使ったすべての命令が誤った結果を計算したことになる．プロセッサは I5 とそれ以降のすべての命令を取り消し，I5 のフェッチからやり直して再実行する．I3 がそれらに先行

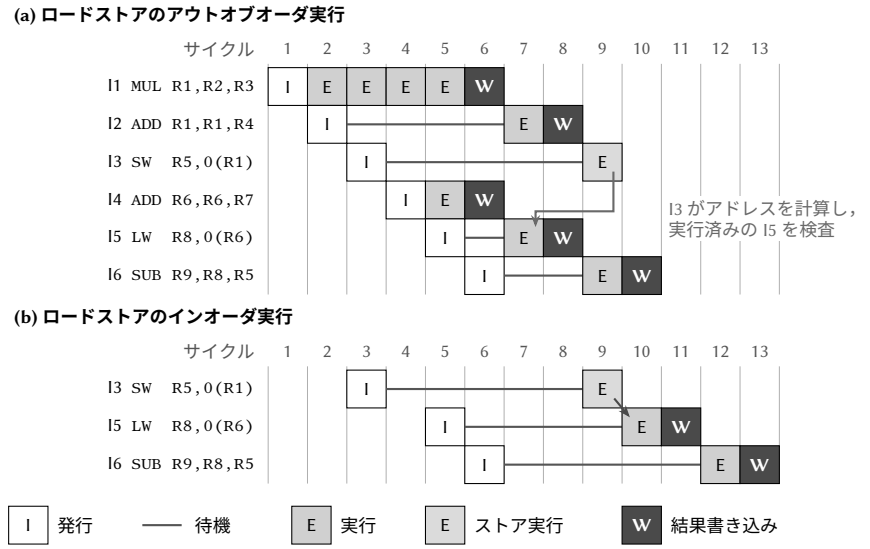


図 9.2: 9.4 節のプログラムを 2 つの方針で実行した場合．(a) I5 はストア I3 がアドレスを知る前に実行され，I3 はサイクル 9 に I5 を検査する．(b) I5 は I3 がアドレスを計算するまで待つ．I1，I2，I4 は (a) と同じである．

するので，いずれもコミットしておらず，ROB は予測を外した経路の命令と同じようにそれらを破棄する（8.5 節）．I5 が再び実行されると，その探索は I3 を見つけ，フォワーディングによって値を受け取る．

取り消しにはパイプラインの再起動の費用がかかり，分岐予測ミスと同程度である．分岐予測ミスは Skylake 級のコアで 15–20 サイクルである（Agner Fog，マイクロアーキテクチャマニュアル）．

9.4.2 ロードストアのインオーダ実行

定義 9.3 (ロードストアのインオーダ実行). **ロードストアのインオーダ実行**（選択肢 1）では，ロードとストアは互いにプログラム順に実行される．すなわち，ロードやストアは，先行するすべてのロードとストアが実行されるまで待つ．ここでストアは，アドレスが分かった時点で実行されたとみなす．他の命令は依然としてアウトオブオーダに実行される．

▷ in-order load-store execution

例では I5 は I3 を待ってサイクル 10 に実行され，I6 はサイクル 12 に実行されてサイクル 13 に結果を書き込む．どのロードも誤った値を得ることはなく，再実行も起こらない．しかし I5 は，そのアドレスが I3 のアドレスと異なる場合にも 3 サイクル待ち，そのときこの待ちは純粋な損失である．ここでは選択肢 2 も同じタイミングになる．I5 に先行するロードがないからである．一般には選択肢 2 のほうが待ちは短い．ロードは先行するロードには依存しないからである．

9.4.3 比較

ロードストアのアウトオブオーダ実行は，ロードが，アドレスのまだ分からない先行ストアの書き込む位置を読まない限りサイクルを稼ぎ，読む場合にはサイクルを失う．ロードとそれ以降の命令を取り消して再実行するコストは，待った場合よりも大きいからである．選択肢 1 と 2 は，アドレスの分からないストアに続くすべてのロードを，両者が衝突するかどうかにかかわらず遅らせる．実際のプログラムでは，そのようなストアの書き込む内容を本当に読むロードはごくわずかなので，ロードストアのアウトオブオーダ実行のほうが性能がよく，高性能プロセッサが採用しているのはこの方針である．[2]

> **要点**
> ロードストアのインオーダ実行はロードを再実行することがない代わりに，通常は依存しないストアをロードに待たせる．ロードストアのアウトオブオーダ実行はロードを直ちに実行させ，その正しさは，各ストアがアドレスを計算するときに完了済みの後続ロードを検査することに依存する．

[2] 実際のコアは，一律に決めるのではなくロードごとに決める．そのロードが衝突しそうかどうかを予測器が判定するのである．Alpha 21264 は，過去に取り消しを起こした PC を記録した表に載るロードだけを待たせ，その表を定期的に消去して，ロードが再び投機的になれるようにする．ストアセット [18] は，各ロードを，それと衝突したことのあるストアに対応付ける．Intel はこの仕組みをメモリディスアンビギュエーション（memory disambiguation）と呼び，2006 年から搭載している．

9.5 ストアロードフォワーディング

ストアロードフォワーディングは，各ロードと，その値を読むべきストアとを対応付ける．ロードストアのアウトオブオーダ実行を行うプロセッサは，この対応を両側から用いる．

出典 Lesson 9，動画 9；H&P 第 3 章．

ロードにとって 値は，同じアドレスを持つ最も近い先行ストア，すなわちプログラム順でロードの前にその位置へ書き込む最後のストアから来る．同じアドレスへのより古いストアは該当しない．その値はより近いストアに上書きされるからである．LSQ にこのアドレスの先行ストアがなければ，値はメモリから来る．

ストアにとって 値は，同じアドレスを持つ後続のすべてのロードに，そのアドレスへの次のストアの手前まで届く．そのストアより後のロードは，代わりにそのストアの値を受け取る．

注意 探索はロードから遡って行う．コミットポインタから順に前向きに探索すると，一致するストアのうち最も古いものが見つかってしまう．

ハードウェアがフォワードできるのは，1 つのストアの中に完全に収まるロードだけである．一部だけ重なる場合，ロードはストアが L1 に届くまで待つことになり，Skylake 級のコアで 12 サイクル前後かかる（Agner Fog，マイクロアーキテクチャマニュアル）．

ロードは探索（9.3 節）で第 1 の規則を，ストアは完了済みの後続ロードの検査（9.4 節）で第 2 の規則を適用する．図 9.3 は，2 つのアドレスに対する 7 つのロードとストアについてこの対応を示す．

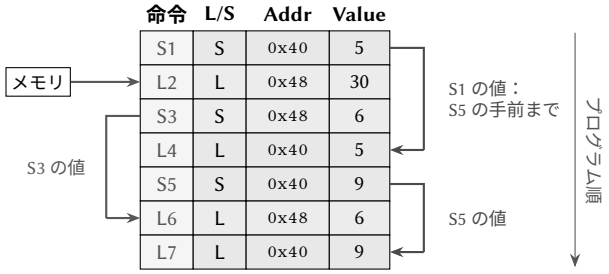


図 9.3: 0x40 と 0x48 への 7 つのロードとストアに対するストアロードフォワーディング．値は逐次実行の場合のものである．L4 は S1 の値を受け取り，その後ろにある S5 の値は受け取らない．L7 は S1 の値を上書きする S5 の値を受け取る．L2 は S3 に先行するのでメモリを読む．

9.6 ロードストアキューの例

ここで，図 9.3 のロードとストア（例 9.4）を，ロードストアのアウトオブオーダ実行のもとでトレースする．

出典 Lesson 9，動画 10；H&P §3.6．

例 9.4. あるプログラムのロードとストアは，プログラム順に次のとおりである．

```
S1:  SW  R1, 0(R2)      ; MEM[0x40] <- R1
L2:  LW  R3, 8(R2)      ; R3 <- MEM[0x48]
S3:  SW  R4, 8(R10)     ; MEM[R10+8] <- R4
L4:  LW  R5, 0(R2)      ; R5 <- MEM[0x40]
S5:  SW  R6, 0(R7)      ; MEM[R7] <- R6
L6:  LW  R8, 8(R2)      ; R8 <- MEM[0x48]
L7:  LW  R9, 0(R2)      ; R9 <- MEM[0x40]
```

ここで R2 = 0x40，R1 = 5，R4 = 6，R6 = 9 は最初から利用できる．R10 = 0x40 と R7 = 0x40 は S3 と S5 に先行する遅い命令が計算するので，この 2 つのストアがアドレスを知るのは遅く，その順序は表 9.1 のとおりである．メモリは最初 0x40 に 20 を，0x48 に 30 を保持している．

ヒント トレースを検算するには，まず逐次実行での値を書き出せばよい．それと異なる値で終わるロードがあれば，再実行が抜けている．

L4 と L6 は，アドレスのまだ分からないストアを通過するが，その後の S3 と S5 の検査では，これらとの衝突は見つからない．L7 も S5 を通過するが，ステップ 7 で S5 が 0x40 を計算すると，このアドレスで完了している L7 が見つかり，しかも両者の間に 0x40 へのストアはない．L7 が S1 から得た値 5 は，プログラム順では S5 に上書きされるので，L7 とそれ以降のすべての命令が取り消されて再実行され，ステップ 8 で L7 は S5 から 9 を受け取る．

エントリがプログラム順にコミットした後，メモリは 0x40 に 9 を，0x48 に 6 を保持し，ロードは R3 = 30，R5 = 5，R8 = 6，R9 = 9 を返している．これは逐次実行の結果と同じである．ロードストアのインオーダ実行であれば，L4 は S3

表 9.1: 例 9.4 の実行．各ステップで 1 つのエントリがアドレスを計算する．ロードは先行するストアを探索し，ストアは C ビットの立った後続ロードを検査する．

ステップ	エントリ	Addr	探索または検査	結果
1	S1	0x40	完了した後続ロードなし	値 5 を記録
2	L2	0x48	S1 は相違	メモリから 30
3	L4	0x40	S3 は不明，S1 が一致	S1 から 5
4	S3	0x48	L4 は相違	衝突なし
5	L6	0x48	S5 は不明，S3 が一致	S3 から 6
6	L7	0x40	S5 は不明，S3 は相違，S1 が一致	S1 から 5
7	S5	0x40	L6 は相違，L7 が一致	L7 を再実行
8	L7	0x40	S5 が一致	S5 から 9

を，L6 と L7 は S5 を待ち，二度実行される命令はなかったはずである．

9.7 LSQ と ROB とリザベーションステーション

ロードとストアも他の命令と同じステップを経るが，LSQ が各ステップに作業を加える（表 9.2）．ロードやストアが発行されるのは，ROB エントリと LSQ エントリの両方が空いているときだけである．LSQ エントリは，他の命令が ROB エントリとともに必要とするリザベーションステーションの代わりとなる．ロードは，算術演算の結果とまったく同じように，自身の ROB エントリの名前のもとで値を CDB にブロードキャストする．ストアはレジスタの値を生成しないので何もブロードキャストせず，その ROB エントリは，ストアがアドレスと値の両方を記録した時点で完了となる．

出典 Lesson 9，動画 11–14；H&P §3.6．

実際の規模：Skylake は 224 エントリの ROB に対して 72 個のロードと 56 個のストアを，Ice Lake は 352 に対して 128 個と 72 個を実行中に保持できる（Intel 最適化マニュアル）．

表 9.2: LSQ を持つプロセッサにおけるロードとストアのステップ．

ステップ	ロード	ストア
発行	空いている ROB エントリと LSQ エントリを必要とする（RS は不要）．ベースレジスタを RAT で引き，宛先レジスタの RAT エントリをそのロードの ROB エントリに設定する	空いている ROB エントリと LSQ エントリを必要とする（RS は不要）．ベースレジスタと値レジスタを RAT で引く．RAT エントリは変化しない
実行	アドレスを計算して LSQ に記録し，先行するストアを探索し，フォワードされた値を複写するかメモリを読み，C を立てる	ベースレジスタが得られたときアドレスを記録し，C の立った後続ロードを検査する．値は得られたときに記録する．両方を記録した時点で C を立てる
結果書き込み	値を CDB にブロードキャストし，ROB エントリがそれを受け取る	ブロードキャストするものはない
コミット	値をレジスタファイルに書き込み，RAT エントリがまだこの ROB エントリを指していればそれを消す．ROB エントリと LSQ エントリを解放する	値をメモリに書き込む．ROB エントリと LSQ エントリを解放する

どちらのキューもプログラム順なので，ROB のコミットポインタにあるロードまたはストアは LSQ の最も古いエントリでもあり，2 つのエントリは同時に解放される．ストアが LSQ を離れるのは，その値がメモリに書き込まれるのとちょうど同じ時点なので，後続のロードは，ロードが探す 2 つの場所のいずれかで必ずその値を見つける．コミット前はストアの LSQ エントリに，コミット後はメモリにある．

マルチコアではもう 1 つ規則が要る．すでに読み終えたロードも，コミットより前に他のコアがそのアドレスへ書き込んだ場合には取り消さなければならない（第 21 章）．

要点 — 章のまとめ

ロードとストアは，実行されてはじめて分かるメモリアドレスを介して互いに依存し，メモリはリネーミングされない．そのためストアは，コミット時にプログラム順でメモリに書き込む．ロードストアキューは，未コミットのロードとストアを，そのアドレス，値，完了ビットとともにプログラム順に保持する．ロードは，ストアロードフォワーディングによって，同じアドレスを持つ最も近い先行ストアから値を取り，そのようなストアがなければメモリから取る．アドレスの分からない先行ストアの扱いには 2 つの方針がある．ロードストアのインオーダ実行ではロードがそれらを待ち，ロードストアのアウトオブオーダ実行ではロードが先に進み，後に一致するアドレスを計算したストアが，そのロードとそれ以降のすべての命令を再実行させる．そのような衝突は稀なので，後者のほうが速い．ロードとストアは発行時に ROB エントリと LSQ エントリを必要とし，他の命令は ROB エントリと RS を必要とする．

第 10 章

コンパイラによる ILP

講義動画	Lesson 10，動画 1–20
教科書	H&P [1] §3.2，付録 H
キーワード	命令レベル並列性，サイクル当たり命令数，依存の連鎖，木の高さ削減，命令スケジューリング，インオーダ実行，if 変換，ループ展開，インライン展開，コード肥大化，命令数，ソフトウェアパイプライニング，トレーススケジューリング

第 6–9 章では，ハードウェアによってプロセッサのサイクル当たり命令数（IPC）を高める手法を見てきた．レジスタリネーミング，リオーダバッファを伴うアウトオブオーダ実行，ロードとストアのアウトオブオーダ実行により，IPC はプログラムの命令レベル並列性（ILP）に近づく．しかしハードウェアが利用できる並列性は，プログラムが実際に含んでいて，かつハードウェアの手の届く範囲にあるものに限られる．この章では，コンパイラがその両方をどのように改善するかを示す．木の高さ削減は ILP を制限する依存の連鎖を短くし，命令スケジューリングは独立な命令を隣り合わせに配置し，if 変換・ループ展開・インライン展開は，それらを隔てる分岐・呼び出し・リターンを除去する．

注意「コンパイラ ILP」は 2 つ目の ILP ではない．ILP はプログラムの性質であり（第 6 章），コンパイラはそのプログラムを変えるのである．

10.1 コンパイラはどのように IPC を高められるか

出典 Lesson 10，動画 1–2；H&P §3.2．

どのプロセッサでも，IPC は実行するプログラムの ILP によって上限が定まる（6.8 節）．IPC に対する 2 つの制限はプロセッサだけで決まるものではなく，コンパイラはそのどちらにも影響する．

プログラムの ILP ILP は，命令数を RAW 依存の最長の連鎖の長さで割ったものである（式 (6.1)，(6.2)）．連鎖がどれだけ長くなるかは，コンパイラがソースプログラムをどう翻訳したかに依存する．同じ計算を，より短い連鎖をなす命令で表現できることが多い．

プロセッサが見つけられる並列性 プロセッサが実行する命令を探すのは，最も古い未実行の命令に続く限られた数の命令の中だけである．アウトオブオーダプロセッサはリオーダバッファに収まる命令を対象とし（第 8 章），インオーダプロセッサは次の数命令だけを見て，待たなければならない最初の命令で止まる（第 6 章）．プログラム順でこのウィンドウより離れた位置にある独立な命令は，ILP がどれほど高くても同じサイクルには実行できない．コンパイラはそのような命令を近くに配置でき，また両者の間にある分岐，呼び出し，リターンを除去できる．

コンパイラはハードウェアのウィンドウよりはるかに広い範囲のプログラムを見ることができ，しかもプログラムの実行前に動作するので，プロセッサが 1 クロックサイクルの中では到底行えない解析にも時間をかけられる．この章で扱う技法を表 10.1 に示す．

コンパイラのウィンドウは関数全体であり，インライン展開後は呼び出し木全体である．プロセッサのウィンドウはリオーダバッファである（第 8 章）．

表 10.1: IPC に対する制限と，それに対処するコンパイラ技法．

制限	コンパイラ技法	節
長い依存の連鎖が ILP を下げる	木の高さ削減	10.2
独立な命令が離れている	命令スケジューリング	10.3
分岐が命令を隔てる	if 変換，トレーススケジューリング	10.4，10.7
ループの分岐が反復を隔てる	ループ展開，ソフトウェアパイプライニング	10.5，10.7
呼び出しとリターンが命令を隔てる	インライン展開	10.6

10.2 木の高さ削減

コンパイラは算術式を**式木**を経由して翻訳する．式木では，各演算がノードであり，その子がオペランドである．各演算は 1 つの命令になり，あるノードの命令はその子の命令の結果を読む．したがって，これらの命令の間の RAW 依存の最長の連鎖は木の**高さ**，すなわち根から葉への最長経路上の演算の個数に等しく，第 6 章の理想プロセッサでは木の 1 段が 1 サイクルを要する．

出典 Lesson 10，動画 3–4；H&P 付録 H．

素直な翻訳は，ソースプログラムがオペランドを書いた順に従う．R1 から R6 に保持された 6 つの変数の和を R10 に求める場合，次のコードが生成される．

1 段 1 サイクルは，すべての演算が 1 サイクルであることを仮定している．多サイクルの演算では 1 段がそのレイテンシ分かかるので，高さの影響はさらに大きい．

```
ADD R10, R1, R2      ; サイクル 1
ADD R10, R10, R3     ; サイクル 2
ADD R10, R10, R4     ; サイクル 3
ADD R10, R10, R5     ; サイクル 4
ADD R10, R10, R6     ; サイクル 5
```

どの加算も直前の加算の結果を使う．木は高さ 5 の連鎖に退化し（図 10.1a），5 つの命令はどのプロセッサでも 5 サイクルかかり，その ILP は 1 である．

定義 10.1 (木の高さ削減). 演算の結合法則と交換法則を用いて，式を，より段数の少ない式木を持つ等価な式に書き換えることを**木の高さ削減**という．演算の個数，したがって命令数は変わらない．

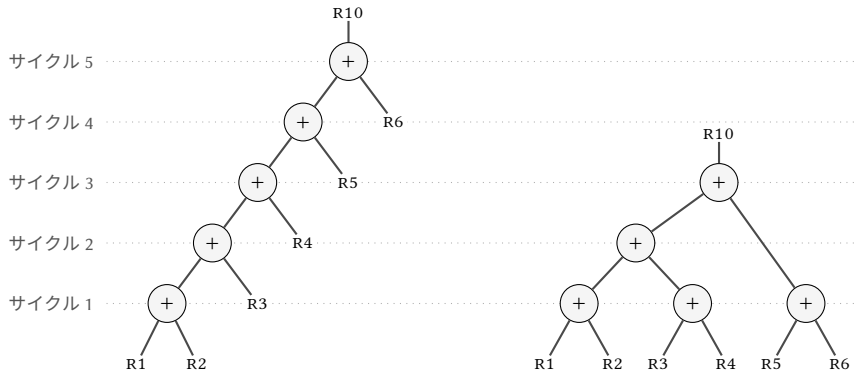
▷ tree height reduction

例 10.2. $((R1 + R2) + (R3 + R4)) + (R5 + R6)$ とグループ化すると，同じ和は次のようになる．

```
ADD R10, R1, R2      ; サイクル 1
ADD R11, R3, R4      ; サイクル 1
ADD R12, R5, R6      ; サイクル 1
ADD R10, R10, R11    ; サイクル 2
ADD R10, R10, R12    ; サイクル 3
```

対をなす 3 つの加算はどれも何にも依存せず，サイクル 1 で実行される．最初の 2 対の和がサイクル 2 に，最後の加算がサイクル 3 に続く（図 10.1b）．命令は依然として 5 つだが，それが 3 サイクルで済む．ILP は 1 から $5/3 \approx 1.67$ に上がる．その代償は，中間結果のための 2 つの追加のレジスタ R11 と R12 である．

再グループ化の予算はレジスタである．アーキテクチャ整数レジスタは x86-64 が 16 本，A64 と RISC-V が 31 本であり，それを超えるとコンパイラはスタックへ退避する．



(a) 左から順に：高さ 5　　(b) 平衡木：高さ 3

図 10.1: R1 から R6 までの和の式木．(a) 左から順に翻訳すると，5 つの加算の連鎖になる．(b) 例 10.2 の平衡木は高さ 3 である．加算の各段は理想プロセッサの 1 サイクルで実行される．

1 つの結合的な演算で結ばれた n 個のオペランドについて，左から順に翻訳すると高さは $n-1$ になるが，平衡木の高さは $\lceil \log_2 n \rceil$ である．加算と乗算は結合的であり，AND，OR，XOR も同様である．減算は $(a-b)-c \neq a-(b-c)$ なので結合的ではないが，符号を調整すれば減算を含む式も再グループ化できる．$a-b = a+(-b)$ であるから，各項が自身の符号を保つ限り項は自由にグループ化でき，マイナス符号に続くグループの中の符号は反転する．例えば $a-b-c-d = (a-b)-(c+d)$ となり，高さが 3 から 2 に減る．

注意 浮動小数点の加算は（丸めのため）結合的ではない．コンパイラがこれを再グループ化するのは，`-ffast-math` のように許可された場合だけである．

要点
木の高さ削減は，命令間の依存を変えるのであって命令数を変えるのではない．ILP を制限する連鎖を短くするので，1 サイクルに複数の命令を実行できるプロセッサで効果がある．

10.3 命令スケジューリング

プロセッサが独立な命令を同じサイクルに実行するのは，それらを見つけられた場合だけである．コンパイラは通常プログラムを文ごとに翻訳するので，ある文の命令の後に次の文の命令が続く．各文が 1 つの連鎖をなす場合，異なる文に属する独立な命令は連鎖の残りによって隔てられる．ウィンドウが両方の連鎖を覆うアウトオブオーダプロセッサならそれでも見つけられるが，ウィンドウがより短いアウトオブオーダプロセッサは見つけられず，待たなければならない最初の命令までしか後続を実行しないインオーダプロセッサも同様である．独立な命令を見つけやすくするコンパイラ技法が 3 つある．命令スケジューリング，ループ展開（10.5 節），トレーススケジューリング（10.7 節）である．

出典 Lesson 10，動画 5–7；H&P §3.2．

定義 10.3 (命令スケジューリング). プログラムの結果を変えずに命令を並べ替え，同じサイクルに実行できる命令がプログラム順で隣り合うようにすることを**命令スケジューリング**という．

注意 メモリもコンパイラを縛る．ロードをストアより前に移動できるのは，2 つのアドレスが異なると証明できる場合だけである．同じ型のポインタ 2 つは同じ位置を指しうるので，異なると証明できない限り順序はそのままになる．

▷ instruction scheduling

インオーダプロセッサでは，スケジューリングは，そのままではプロセッサが待つことになるサイクルを，待っている命令に依存しない命令で埋める．1 サイクルに最大 w 命令を実行するプロセッサ向けにスケジューリングするには，コンパイラは各命令について理想プロセッサがそれを実行するサイクルを求め（式 (6.2)），そのサイクルの順に，高々 w 命令ずつのグループにして命令を並べればよい．このとき各命令は，自分が読む結果を生成する命令より後に置かれたままであるが，それに加えて，並べ替えたプログラムは元のプログラムと同じ結果を計算しなければならない．WAW 依存や WAR 依存を破る移動は，ある命令が使う値を変えてしまう．そこでコンパイラは，元の順序を保つか，あるいは次のように補正する．

宛先レジスタ 移動する命令が，後の命令がまだ読む古い値を持つレジスタを上書きしてしまう場合，その命令は代わりに，使われていない別のレジスタに結果を書き，その結果を読む命令もそれに合わせて変更する．

アドレスのオフセット 添字の増分のようにレジスタに定数を加える命令が，そのレジスタをアドレスのベースとして使う命令より前に移動する場合，アドレスの変位をその定数だけ減らす．`SW R3, 0(R1)` は `ADDI R1, R1, 4` の後に置かれると `SW R3, -4(R1)` になる．

ヒント ある命令を，c を加える `ADDI` より後ろへ移すと，そのレジスタを使う変位はすべて c 減る．逆向きに移せば c 増える．

例 10.4. a から h を R1 から R8 に保持するとき，文 $x = (a+b)\times c - d$ と $y = (e-f)\times g + h$ は次のように翻訳される．

```
I1:  ADD R9, R1, R2      ; a + b
I2:  MUL R9, R9, R3      ; (a + b) * c
I3:  SUB R9, R9, R4      ; x <- (a + b) * c - d
I4:  SUB R10, R5, R6     ; e - f
I5:  MUL R10, R10, R7    ; (e - f) * g
I6:  ADD R10, R10, R8    ; y <- (e - f) * g + h
```

これらの命令は I1–I3 と I4–I6 の 2 つの連鎖をなすので，理想プロセッサはサイクル 1 に I1 と I4 を，サイクル 2 に I2 と I5 を，サイクル 3 に I3 と I6 を実行する．ILP は 2 である．すべての命令が 1 サイクルで実行される幅 2 のインオーダプロセッサは，I2 が I1 を待たなければならないのでサイクル 1 には I1 だけを，サイクル 2 には I2 だけを実行する．サイクル 3 では I3 を I4 と一緒に実行するが，その後は I5 と I6 が再び 1 サイクルずつ必要になる．合計 5 サイクル，IPC は $6/5 = 1.2$ である（図 10.2a）．I1，I4，I2，I5，I3，I6 の順にスケジューリングすると，隣り合う命令の組はすべて独立になり，同じプロセッサが 3 サイクルで済む．IPC は 2 であり，ILP に等しい（図 10.2b）．2 つの文は異なるレジスタを使うので，この並べ替えは WAW 依存も WAR 依存も破らない．

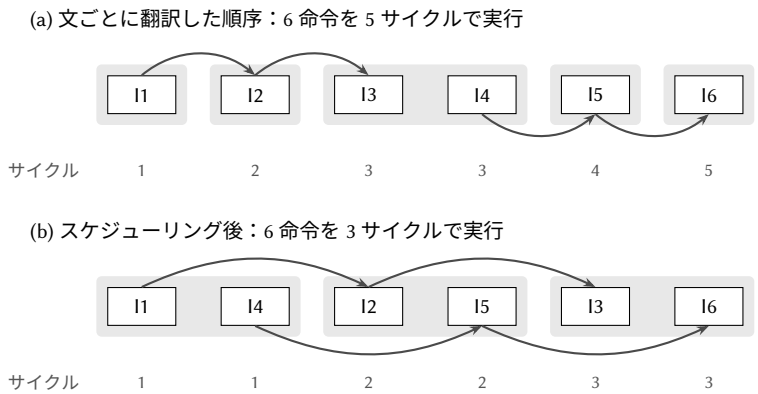


図 10.2: 幅 2 のインオーダプロセッサにおける 例 10.4 のプログラム．弧は RAW 依存であり，網掛けのグループは同じサイクルに実行される．(a) 文の順序のままでは，連鎖 I1–I3 が I4 を足止めする．(b) スケジューリング後は，両方の連鎖が毎サイクル進む．

例 10.5. 次のループは

```
for (i = 0; i < n; i++)
    a[i] = a[i] + x;
```

整数配列の n 個の要素それぞれに x を加える．R1 が現在の要素を指し，R2 が x を保持し，R5 が最後の要素の次のアドレスを保持するとき，次のように翻訳される．

翻訳後のループに *i* はない．コンパイラは帰納変数としてアドレスだけを残し，それを R5 の終端と比較する（強度低減）．

```
Loop: LW   R3, 0(R1)      ; R3 <- a[i]
      ADD  R3, R3, R2     ; R3 <- a[i] + x
      SW   R3, 0(R1)      ; a[i] <- R3
      ADDI R1, R1, 4      ; 次の要素へ
      BNE  R1, R5, Loop   ; 末尾まで繰り返す
```

完全な分岐予測を持つ幅 4 のインオーダプロセッサは，ADD が LW を待つのでまず LW だけを，次に SW が ADD を待つので ADD だけを実行する．SW と ADDI は一緒に実行され，ADDI を待つ BNE は次の反復の LW と一緒に実行される．1 反復は 3 サイクルかかり，CPI は 3/5 = 0.6 である．ADDI を LW の直後に繰り上げると，ADDI はもはや連鎖の後ろで待たされず，増分の後に置かれることになった SW は変位 –4 を必要とする．

```
Loop: LW   R3, 0(R1)
      ADDI R1, R1, 4
      ADD  R3, R3, R2
      SW   R3, -4(R1)     ; R1 はすでに進んでいる
      BNE  R1, R5, Loop
```

これは理想サイクルの順序であり，分岐だけは反復の末尾に留めてある．LW と ADDI が一緒に実行され，ADD が続き，SW と BNE，そして次の反復の LW と ADDI が次のサイクルの 4 つの枠をすべて埋める．1 反復は 2 サイクルとなり，CPI は 2/5 = 0.4 である．

注意 –4 は要素数ではなくバイト数である．変位は，要素の大きさによらず，ベースレジスタが増えた定数ちょうどで補正する．

このようにスケジューリングが効くのは，インオーダプロセッサ，プログラム中の連鎖に比べてウィンドウが短いアウトオブオーダプロセッサ，そして幅より多くの命令が準備できているときのアウトオブオーダプロセッサである．どの命令が先に実行されるかを順序が決めるからである（第 7 章）．

静的なスケジュールは仮定したレイテンシに固定される．ロードがキャッシュにヒットする限り順序は正しいが，ミスするとインオーダプロセッサは止まり，反応できるのは第 7–9 章のハードウェアだけである．

10.4 スケジューリングと if 変換

命令スケジューリングが命令を動かせるのは，全体がひとまとまりに実行されるコードの範囲の中だけであり，条件分岐はその範囲を終わらせる．分岐に続く命令は，その一方の経路でしか実行されない．それを分岐の前に移動すれば，もう一方の経路でも実行されてしまう．また 2 つの経路の命令は，どちらか一方の経

出典 Lesson 10，動画 8–9；H&P §3.2.

路しか実行されないので，同じグループに置くことはできない．したがってコンパイラは，分岐の前後の命令が独立であっても，分岐をまたいで命令をグループ化できない．

小さな if-then-else に対しては，if 変換（第 5 章）がこの境界を取り除く．変換後のコードは，条件と両方の部分を無条件に計算し，条件付きムーブで結果を選択する．全体が分岐のない 1 つの列になっており，コンパイラはそれを他のコードと同じようにスケジューリングできる．条件と 2 つの部分を計算する命令は選択に依存しないので，それら自身の依存に従って先頭のグループに置かれ，選択がその後に続く．例 5.4 の if 変換後のコードは，すでにこの形をしている．最初の 4 つの命令は互いに独立であり，3 つの条件付きムーブはそれらにしか依存しないので，幅 4 のインオーダプロセッサは 7 つの命令すべてを 2 サイクルで実行する．

この範囲を基本ブロックという．整数コードでは 5 命令に 1 回ほど分岐があり（第 3 章），1 つのブロックの中だけではスケジューリングの余地は小さい．

10.4.1 ループ

ループは各反復の末尾に先頭へ戻る分岐を持つが，この分岐を if 変換しても役には立たない．これを除去するということは，ループが取りうる反復回数と同じだけ本体を実行するということであり，各コピーには，その反復が実行されるべきだったかどうかを示す述語がそれぞれ必要になる．これらの述語を計算し適用する命令のコストは，スケジューリングで得られる利得を上回る．さらに，ループの分岐は正確に予測され，これにプレディケーションを適用すると，ループの後のコードを毎反復実行することになる（表 5.1）．したがってループの分岐は残り，命令スケジューリングを 1 回の反復の中に閉じ込める．幅 4 のインオーダプロセッサで 1 反復 2 サイクルとなる 例 10.5 のスケジューリング後の本体は，そのプロセッサにとって最良の順序である．本体が短い場合，ループの独立な命令の大半は異なる反復にあり，そこではコンパイラはそれらを近づけられない．ループ展開がこの制限に対処する．

10.5 ループ展開

本体の短いループでは，実行される命令の大きな割合がループのオーバーヘッドであり，本来の処理はループの分岐で隔てられた多数の反復に分散する．例 10.5 のループでは，5 つの命令のうち 3 つが本来の処理を行い，ADDI と BNE はオーバーヘッドである．

出典 Lesson 10，動画 10–15；H&P §3.2．

定義 10.6 (ループ展開). 本体を k 回繰り返して添字を調整し，添字を k ずつ進めることによって，新しいループの 1 反復が元のループの k 反復分の処理を行うようにループを書き換えることを**ループ展開**という．この k を**展開係数**という．1 回展開すれば $k = 2$，2 回展開すれば $k = 3$ である．

▷ loop unrolling

n が偶数のとき，$k = 2$ で展開するとループは

```
for (i = 0; i < n; i += 2) {
    a[i]   = a[i]   + x;
    a[i+1] = a[i+1] + x;
}
```

となり，次のように翻訳される．

```
Loop: LW   R3, 0(R1)       ; a[i] <- a[i] + x
      ADD  R3, R3, R2
      SW   R3, 0(R1)
      LW   R3, 4(R1)       ; a[i+1] <- a[i+1] + x
      ADD  R3, R3, R2
      SW   R3, 4(R1)
      ADDI R1, R1, 8       ; 2 要素先へ
      BNE  R1, R5, Loop
```

各反復は 8 つの命令で 2 要素を処理し，そのうち添字の増分と分岐は 1 回ずつしか実行されない．本体の 2 つのコピーは隣接していて間に分岐がないので，2 つ目のコピーが中間結果を自分専用のレジスタに保持するようにすれば，コンパイラは両者をまとめてスケジューリングできる（10.5.2 節）．

n が k の倍数でない場合，コンパイラは残りの要素のためのコード—通常は元のループのコピー—を追加する．

10.5.1 命令数と ILP

展開はクロック周期には影響しないが，性能の鉄則（式 (2.3)）の他の 2 つの因子を変える．元のループは 1 要素当たり 5 命令を実行し，展開後のループは 2 要素当たり 8 命令，すなわち 1 要素当たり 4 命令を実行する．オーバーヘッドの命令が実行される回数は半分になり，命令数は 20 % 減る．係数 k では，ループは 1 要素当たり $(3k+2)/k$ 命令を実行し，これは本来の処理だけの 3 に近づく．

ILP も上がる．理想プロセッサ（式 (6.2)）では，連続する反復の ADDI が 1 反復 1 サイクルの連鎖をなす．各反復の LW は前の反復の ADDI から R1 を読み，自分の反復の ADDI と同じサイクルに実行され，その ADD と SW は続く 2 サイクルに実行される．展開後の本体で 2 つ目のコピーが R3 を再利用することによる WAW 依存と WAR 依存は，リネーミングが除去する．したがって理想プロセッサは 1 サイクルに 1 反復，すなわち k 要素を実行し，長いループの ILP は 1 反復当たりの命令数に近づく．元のループでは 5，展開後のループでは 8 である．これは，反復が互いに添字を通じてしか依存しないことを前提としている．`s = s + a[i]` のように累算するループでは，s への加算がどちらの版でも 1 要素 1 サイクルの連鎖をなす．

ヒント $3+2/k$ と読めばよい．1 要素当たりのオーバーヘッドは k を倍にするごとに半分になるので，利得の大半は最初の数段階にある．

10.5.2 命令当たりサイクル数

CPI も下がるかどうかはプロセッサに依存する．表 10.2 は，例 10.5 の幅 4 のインオーダプロセッサと理想プロセッサ（どちらも分岐予測は完全とする）について，ループの 4 つの版を比較したものである．最初の 2 つの版は 例 10.5 のものである．展開しただけでスケジューリングしていない版では，ループは依然として LW–ADD–SW の連鎖を順に実行する．2 つ目の LW が 1 つ目の SW と，ADDI が 2 つ目の SW と，BNE が次の反復の LW と一緒に実行されるので，8 命令からなる 1 反復は 5 サイクルかかる．CPI は 0.6 から 5/8 = 0.625 に上がるが，1 要素当たりの時間は命令数とともに 3 サイクルから 2.5 サイクルに減る．理想サイクルの順にスケジューリングすると，展開後の本体は次のようになる．

ヒント このような累算は，k 個の独立なアキュムレータに分けて展開し，ループの後で合計する．浮動小数点では 10.2 節の再グループ化の許可が必要である．

```
Loop: LW   R3, 0(R1)
      LW   R4, 4(R1)       ; 2つ目のコピーは R4 を使う
      ADDI R1, R1, 8
      ADD  R3, R3, R2
      ADD  R4, R4, R2
      SW   R3, -8(R1)      ; R1 はすでに進んでいる
      SW   R4, -4(R1)
      BNE  R1, R5, Loop
```

ループの分岐の予測が外れるのは，実行 1 回につきおよそ 1 回，ループを抜けるときである．端数処理のループがあっても高々もう 1 回増えるだけである．

10.3 節の補正が両方とも必要になる．2 つ目の LW は，1 つ目のコピーの ADD と SW より前に来るが，これらはまだ R3 を必要とするので，2 つ目のコピーは自分の値を R4 で計算する．また両方のストアが増分の後に来るので，その変位を 8 だけ減らす．プロセッサは最初のサイクルに 2 つの LW と ADDI を，2 サイクル目に 2 つの ADD を実行し，3 サイクル目を 2 つの SW，BNE，そして次の反復の最初の LW が埋める．それ以降はグループが反復と一致しなくなり，定常状態では 16 命令，すなわち 2 反復が 5 サイクルかかる．1 要素当たり 1.25 サイクル，CPI は 5/16 = 0.3125 である．スケジューリングした元のループと比べると，1 要素当たりの時間は 2 サイクルから 1.25 サイクルに減り，高速化率は 1.6 となる．そのうち 5/4 = 1.25 が命令数に，0.4/0.3125 = 1.28 が CPI に由来する．

表 10.2: $k = 2$ で展開する前と後の 例 10.5 のループを，コンパイルしたままの版とスケジューリングした版について比較する．1 要素当たりの命令数と，分岐予測が完全な 2 つのプロセッサにおける 1 要素当たりのサイクル数および CPI を示す．

		幅 4 のインオーダ		理想	
版	命令数	サイクル	CPI	サイクル	CPI
元のループ	5	3.00	0.600	1.00	0.200
スケジューリング後	5	2.00	0.400	1.00	0.200
展開後	4	2.50	0.625	0.50	0.125
展開・スケジューリング後	4	1.25	0.3125	0.50	0.125

このようにインオーダプロセッサでは，展開はそれが可能にするスケジューリングを通じて CPI を下げる．ループの分岐が 2 つの反復の命令を隔てなくなり，コンパイラが両者を交互に並べられるからである．理想プロセッサでは命令の順序は問題にならない．そこで CPI が下がるのは ILP が 5 から 8 に上がるからであり，命令数の減少と合わせて 1 要素当たりの時間は半分になる．大きなウィンドウを持つアウトオブオーダプロセッサも，自身の幅までは同じように利益を得る．1 サイクルに 1 命令を完了するプロセッサは，どの版でも CPI が 1 であり，命令数による 1.25 倍しか得られない．したがってループ展開は，どのプロセッサでも命令数を下げ，1 サイクルに複数の命令を実行するプロセッサでは CPI も下げる．

10.5.3 展開の欠点

本体は k に比例して大きくなり，5 命令から $3k+2$ 命令になるが，利得は頭打ちになる．1 要素当たりの命令数は 3 に近づき，実在のプロセッサは ILP がどれほど高くても IPC を自身の幅より上げられない．$k=4$（14 命令，1 要素当たり 3.5 命令）から $k=8$（26 命令，1 要素当たり 3.25 命令）にすると，本体はほぼ 2 倍になるのに，実行される命令は 7 % しか減らない．これは第 2 章の収穫逓減の一例である．

定義 10.7 (コード肥大化)．ループ展開のような変換によってプログラムのサイズが増大することを**コード肥大化**という．

▷ code bloat

コードが大きくなればより多くのメモリを占め，キャッシュの章で見るように，その命令をフェッチする際のキャッシュミスが増えて，利得を打ち消しうる．さらに 2 つの問題が反復回数に関わる．n が k の倍数でない場合，残りの反復のために追加のコードが必要になる．また展開は，展開後のループの 1 反復が始まる時点で残りの反復回数が分かっていることを前提とする．値が 0 である最初の要素で止まる探索のように，脱出が本体の中で決まるループでは，単純に 1 反復当たり本体の k 個のコピーを実行することはできない．後ろのコピーが，元のループなら実行しない反復を行ってしまう可能性があるからであり，追加の脱出判定が必要になる．そのためコンパイラは，小さな係数で，しかも本体の短いループだけを展開する．

> **要点**
> ループ展開は，添字の増分とループの分岐を元のループの k 反復につき 1 回だけ実行するので，どのプロセッサでも命令数を下げる．さらに，1 サイクルに複数の命令を実行するプロセッサでは CPI も下げる．インオーダプロセッサではコンパイラが複数の反復の命令をまとめてスケジューリングできるようになるからであり，並列性を自分で見つけるプロセッサでは，反復が独立なループの ILP が上がるからである．その代償は，k とともに増大するコードと，反復回数が k の倍数でない場合や事前に分からない場合の追加のコードである．

10.6 インライン展開

分岐が if-then-else の 2 つの経路を隔てるのと同じように，呼び出しは呼び出し側の命令と呼び出される関数の命令とを隔てる．さらに，呼び出しのたびに，関数本来の処理を何も行わない命令が実行される．呼び出しとリターンそのもの，呼び出し規約で定められたレジスタへの引数のコピーとそこからの結果のコピー，そしてしばしばスタックへのレジスタの退避と復帰である．

定義 10.8 (インライン展開)．呼び出しを，呼び出される関数の本体を呼び出し側のオペランドに合わせたコピーで置き換えることを**インライン展開**という．

例 10.9．次の関数を

```
int sqdiff(int x, int y) { int d = x - y; return d * d; }
```

まず効いてくるのはフロントエンドである．Skylake はデコード済みマイクロオペレーションを 1536 個，Zen 2 は 4096 個キャッシュし，それを超えて展開したループはデコーダに戻る．Intel 最適化マニュアル；AMD Family 17h ソフトウェア最適化ガイド．

出典 Lesson 10，動画 16–18；H&P 付録 A．

実際の呼び出し規約はもっと多くをレジスタで渡す．System V AMD64 ABI は整数引数の最初の 6 つ，AArch64 は最初の 8 つであり，残りはスタックに積まれる．

▷ function call inlining

注意 C と C++ の `inline` はリンケージの指定であって命令ではない．GCC は `-O2` 以降，`inline` と書かれていない関数も展開し，強制できるのは `always_inline` だけである．

`c = sqdiff(a, b)` として呼び出す．*a*，*b*，*c* は R8，R9，R10 にある．呼び出し規約は引数を R4 と R5 で渡し，結果を R2 で返す．

```
        ADD  R4, R8, R0      ; x <- a
        ADD  R5, R9, R0      ; y <- b
        CALL SqDiff
        ADD  R10, R2, R0     ; c <- 結果
        ...
SqDiff: SUB  R2, R4, R5      ; d <- x - y
        MUL  R2, R2, R2      ; 結果 <- d * d
        RET
```

インライン展開すると，呼び出しは次のようになる．

```
        SUB  R10, R8, R9     ; c <- a - b
        MUL  R10, R10, R10   ; c <- c * c
```

呼び出しは 1 回につき 7 命令を実行するのに対し，インライン展開後のコードは 2 命令である．引数と結果のコピーは，*a* と *b* から *c* への依存の連鎖も長くする．理想プロセッサでは，呼び出しは 4 サイクル，インライン展開後のコードは 2 サイクルかかる（図 10.3）．

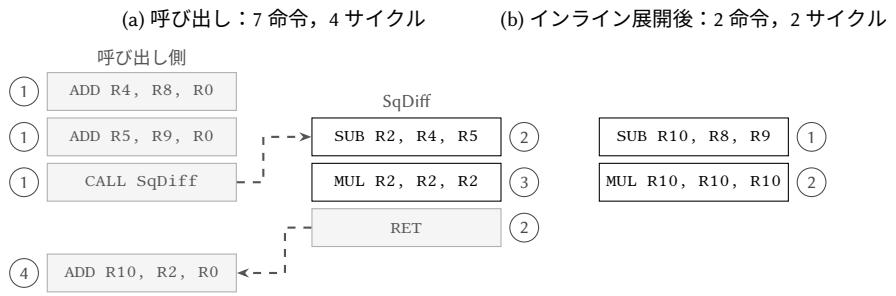


図 10.3: インライン展開の前と後の 例 10.9．網掛けの命令は呼び出しを実現するだけの命令であり，破線の矢印は制御の移動を，丸で囲んだ数字は理想プロセッサが各命令を実行するサイクルを表す．

このようにインライン展開は，呼び出し，リターン，引数と結果のコピーがなくなるので命令数を下げ，本体が呼び出し側のオペランドを直接読み，結果を呼び出し側が必要とする場所に直接届けるので依存の連鎖を短くする．さらに，本体は呼び出し側のコードの一部になる．コンパイラはその命令を呼び出し側の命令とまとめてスケジューリングでき，かつての呼び出しをまたいで木の高さ削減を適用でき，プロセッサは呼び出しとリターンを予測する必要がなくなる（第 4 章）．[1]

[1] インライン展開は何よりも，他の最適化を可能にする変換である．本体が呼び出し側に入ると，引数はそこではしばしば定数として分かっているので，定数伝播やデッドコード除去，そしてこの章の技法が，かつての境界をまたいで適用できる．そのためコンパイラは早い段階でインライン展開し，その後で最適化する．

10.6.1 インライン展開によるコード肥大化

インライン展開は本体のコピーを呼び出し箇所ごとに別々に置くので，多数の箇所から呼ばれる大きな関数ではコードが相当に増える．

例 10.10. 本体が 20 命令とリターンからなる関数が 10 箇所から呼ばれ，各箇所が呼び出しに 4 命令（引数のコピー 2 つ，呼び出し，結果のコピー 1 つ）を費やすとする．インライン展開しなければ，コードは $10 \times 4 + 21 = 61$ 命令を占め，呼び出し 1 回につき $4 + 21 = 25$ 命令が実行される．すべての箇所でインライン展開すると，関数と呼び出しの列は消えるが，各箇所が本体の 20 命令を受け取る．合計 200 命令，コードは 3 倍以上になるのに，節約できるのは呼び出し 1 回当たり 25 命令のうち 5 命令（20 %）である．本体が 2 命令の 例 10.9 の関数なら，同じ 10 箇所でも，インライン展開しない場合は $10 \times 4 + 3 = 43$ 命令，展開した場合は $10 \times 2 = 20$ 命令しか必要としない．コードは小さくなり，呼び出し 1 回につき実行される命令は 7 から 2 になる．

ループ展開と同様に，コードが大きくなるとキャッシュミスが増え，利得を打ち消しうる．そのためコンパイラがインライン展開するのは，本体に比べて呼び

1 次命令キャッシュは小さいままである．Skylake も Zen 3 も 32 KB であり，インライン展開されたコピーはそこを奪い合う（Intel と AMD の最適化マニュアル）．

出しのオーバーヘッドが大きい小さな関数と，呼び出し箇所が 1 つだけでそのコピーが元の関数を置き換える関数である．多数の箇所から呼ばれる大きな関数は，呼び出しのまま残る．

要点
インライン展開は，呼び出し，リターン，引数と結果の受け渡しを除去して命令数を下げ，依存の連鎖を短くし，さらにコンパイラが本体を呼び出し側とまとめて最適化できるようにする．小さな関数では効果があるが，多数の箇所から呼ばれる大きな関数では，コード肥大化が利得を上回る．

10.7 その他のコンパイラ技法

この章の技法は，IPC を高めるコンパイラ変換という，より大きな体系の例である．命令スケジューリングを，それを閉じ込めている境界の向こうまで押し広げる技法がさらに 2 つあるが，ここでは概略を述べるにとどめる．**ソフトウェアパイプライニング**では，コンパイラはループをパイプラインのように扱う．反復が重なるように連続する反復の命令をスケジューリングし，各サイクルが複数の反復の独立な命令を組み合わせつつ，結果は元のループのものと同じに保たれる [19]．もう 1 つの**トレーススケジューリング**は，分岐を除去せずに分岐をまたいでスケジューリングする．コンパイラはプロファイルを用いて，複数の分岐を通る最も頻繁に実行される経路を選び，この経路を 1 つのコードブロックとしてスケジューリングし，実行がその経路から外れたときに結果を訂正する補償コードへ分岐する判定を追加する [20]．コンパイラによるスケジューリングを突き詰めると，操作を並列なステップにグループ化することを完全にコンパイラに委ねるプロセッサ，すなわち第 11 章の VLIW プロセッサに行き着く．[2]

出典 Lesson 10，動画 19–20；[19]；[20]．

▷ software pipelining

▷ trace scheduling

要点 — 章のまとめ
IPC は，プログラムの ILP と，プロセッサがウィンドウの中で見つけられる独立な命令とによって制限され，コンパイラはそのどちらにも影響する．木の高さ削減は式を再グループ化してその命令がより短い連鎖をなすようにし，命令数を変えずに ILP を上げる．命令スケジューリングは独立な命令を隣り合わせにし，命令が互いを追い越す箇所では宛先レジスタとアドレスのオフセットを変更する．これはインオーダプロセッサやウィンドウの小さいプロセッサで効く．分岐はこれを閉じ込めるが，if 変換は小さな if-then-else から分岐を除去する．ループ展開は，添字の増分とループの分岐を k 反復につき 1 回だけ実行して命令数を下げる．コンパイラが複数の反復の命令をまとめてスケジューリングできるようになることと，反復が独立なループの ILP が上がることから，CPI も下げる．インライン展開は，呼び出し，リターン，引数のコピーを除去する．どちらもコード肥大化を招くので，コンパイラは短いループと小さな関数に限ってこれらを用いる．ソフトウェアパイプライニングとトレーススケジューリングは，スケジューリングをループの反復をまたいで，また分岐をまたいで拡張する．

[2] 最もよく知られた VLIW 型の命令セットである IA-64（第 11 章）は，ソフトウェアパイプライニングをハードウェアで支援する．その回転レジスタは反復ごとにレジスタを付け替え，回転述語がパイプラインの各段を有効・無効にし，専用のループ分岐が立ち上げと立ち下げを行う．そのため，展開なら複数のコピーを要する処理を本体 1 つで行える．

第 11 章

VLIW

講義動画	Lesson 11，動画 1–9
教科書	H&P [1] §3.7，付録 H
キーワード	VLIW，アウトオブオーダ・スーパースカラ，インオーダ・スーパースカラ，NOP，後方互換性，不規則なコード，コード肥大化，完全プレディケーション，分岐ヒント，ストップビット，Itanium，EPIC，DSP

第 10 章では，コンパイラがプログラムを書き換えることで，プロセッサが 1 サイクルに完了する命令の数を増やせることを示した．コンパイラは依存の連鎖を短くし，独立な命令を隣り合わせに配置し，分岐と呼び出しを除去する．VLIW（very long instruction word）プロセッサは，この考え方を突き詰めたものである．どの操作を並列に実行するかをコンパイラだけが決め，その決定を命令に符号化するので，ハードウェアには第 7–9 章のアウトオブオーダプロセッサが持つ依存追跡の機構が一切必要ない．この章では，VLIW をスーパースカラの設計と比較し，その利点と欠点を検討し，この手法が失敗した分野と成功した分野を示す．

11.1 スーパースカラプロセッサと VLIW プロセッサ

スーパースカラプロセッサ（第 8 章）は，1 サイクルに最大 N 個の命令を実行する．設計による違いは，一緒に実行できる N 個の命令を見つけるためにいくつの命令を調べるか，そしてその探索のどれだけをコンパイラに任せるかにある（図 11.1，表 11.1）．アウトオブオーダの設計は，リネーミング，ウィンドウ内のスケジューリング，リオーダバッファのための複雑なハードウェアと引き換えに，どのようなプログラムの並列性も自力で見つける（第 6–8 章）．インオーダの設計は，他の命令を待たなければならない最初の命令で止まるので，独立な命令を隣り合わせに配置するコンパイラの助けを必要とする（第 10 章）．VLIW プロセッサは，コンパイラが互いに独立にした最大 N 個の操作からなる 1 つの命令を，何の検査もせずに実行する．

出典 Lesson 11，動画 1–3；H&P §3.7；[21]．

注意 グループ化を誤ると，遅くなるのではなく結果が誤る．検査が一切ないので，依存する 2 つの操作も同じ命令の中でそのまま実行される．正しさはコンパイラに懸かっている．

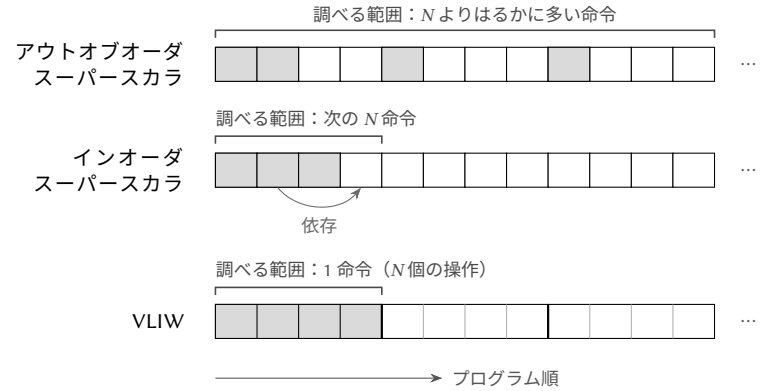


図 11.1: 1 サイクルを最大 $N = 4$ 個の操作で埋めるために，各設計が調べる範囲．網掛けの箱がこのサイクルで実行される．インオーダの設計は，グループ内の先行する命令に依存する最初の命令で止まる．

定義 11.1 (VLIW). **VLIW**（very long instruction word）アーキテクチャとは，各命令が最大 N 個の操作からなり，それらの操作が同じサイクルで並列に実行される命令セットアーキテクチャである．各操作は通常の ISA の 1 命令に相当する．1 つの命令を構成する操作を選び，それらが独立であることを保証するのはコンパイラの役割であり，プロセッサは依存の検査を一切行わない．

▷ very long instruction word

実際の幅：IA-64 のバンドルは 128 ビットで，41 ビットの操作 3 つと，その実行ユニットの種類とストップの位置を示す 5 ビットのテンプレートからなる（Intel Itanium アーキテクチャマニュアル）．

N 個の操作を収める命令は通常の命令のおよそ N 倍の長さになり，これがこのアーキテクチャの名前の由来である．1 命令が最大 N 個の通常の命令の仕事を

するので，1 サイクルに 1 命令を完了する VLIW プロセッサは，IPC が最大 N のスーパースカラプロセッサに相当する（第 6 章）．

注意 VLIW プログラムの命令数と CPI は，通常の ISA のものとは比較できない．サイクル当たりの操作数で比較すること．

表 11.1: 1 サイクルに最大 N 個の操作を実行するスーパースカラプロセッサと VLIW プロセッサ．

	アウトオブオーダ	インオーダ	VLIW
サイクル当たりの実行	$\leq N$ 命令	$\leq N$ 命令	1 命令（$\leq N$ 操作）
調べる範囲	$\gg N$ 命令	次の N 命令	1 命令
ハードウェア	複雑	より単純	最も単純
コンパイラ	助けになりうる	助けが必須	すべてを担う

例 11.2 (命令への操作の詰め込み). 次の断片は配列の 4 要素を加算し，その和をストアする．コメントは RAW 依存（第 3 章）を示す．4 つのロードは R8 を読むだけであり，互いに独立である．

```
I1: LW  R1, 0(R8)
I2: LW  R2, 4(R8)
I3: LW  R3, 8(R8)
I4: LW  R4, 12(R8)
I5: ADD R5, R1, R2     ; I1, I2 と RAW
I6: ADD R6, R3, R4     ; I3, I4 と RAW
I7: ADD R7, R5, R6     ; I5, I6 と RAW
I8: SW  R7, 16(R8)     ; I7 と RAW
```

VLIW コンパイラは操作をプログラム順に取り出し，それぞれを，依存するすべての操作より後にあり，かつ空きスロットが残っている最も早い命令に配置する．余ったスロットは NOP で埋める（表 11.2）．通常の ISA なら 8 命令が必要なところ，スロットが 4 つなら 8 個の操作は 4 命令，すなわち 4 サイクルに収まり，スロットが 2 つなら 5 命令を占める．

ヒント 下界は $\lceil n/N \rceil$ 命令であり，かつ最長の依存連鎖の操作数を下回らない．ここでは $\max(2, 4) = 4$ である．ただしこの下界に達するとは限らない．2 スロットでは下界は 4 だが，I5 と I6 は合わせて 4 つのロードすべてを必要とし，それらが最初の 2 命令を埋めるので，どのスケジュールでも 5 命令を下回れない．

表 11.2: 例 11.2 の断片を，操作スロットが 4 つの VLIW ISA と 2 つの VLIW ISA に向けてコンパイルした結果．各行が 1 命令である．サイズは 32 ビットのスロットを仮定している．

命令	4 スロット	2 スロット
1	I1, I2, I3, I4	I1, I2
2	I5, I6, NOP, NOP	I3, I4
3	I7, NOP, NOP, NOP	I5, I6
4	I8, NOP, NOP, NOP	I7, NOP
5	—	I8, NOP
NOP スロット	16 個中 8 個	10 個中 2 個
コードサイズ	512 ビット	320 ビット

11.2 利点と欠点

並列性の探索をコンパイラに任せることには 3 つの利点がある．

出典 Lesson 11，動画 4–5；H&P §3.7，付録 H．

- **コンパイラには時間が十分にある．** アウトオブオーダプロセッサは，毎クロックサイクルのごく一部の時間で，しかも自身のウィンドウの中だけで独立な命令を見つけなければならない．コンパイラは一度だけ実行され，プログラム全体を解析できる．
- **ハードウェアが単純になる．** リネーミング，ウィンドウ内のスケジューリング，リオーダバッファがないので，VLIW プロセッサは同じサイクル当たり操作数を実現するのに必要な面積と電力が少なくて済み，エネルギー効率を高くできる．
- **ループと規則的なコードに適する．** 配列をループで処理する場合，依存は少なく，コンパイル時に把握できる．ループ展開（第 10 章）が反復間の分岐を除去するので，独立な操作がスロットを埋める．

一方で，欠点は 4 つある．

- **レイテンシがコンパイル時に分からない．**コンパイラは，各操作がそのサイクル内に完了するものとして操作をグループ化する．ある操作が 1 サイクルで終わらないと–例えばキャッシュミス（第 12 章）を起こしたロード–後続のすべての操作が，独立なものまで含めてそれを待つ．一方，アウトオブオーダプロセッサはそれに依存しない命令の実行を続ける．
- **不規則なコードは難しい．**コンパイラが見るのはプログラムであって，その実行ではない．2 つの操作が独立であることを証明できないとき–同じ位置を指しうるポインタを介したストアとロードや，データに依存する分岐の両側にある操作–コンパイラはそれらを別々の命令に置かなければならない．一方，アウトオブオーダプロセッサは実行時に実際のアドレスを知り，分岐を予測する（第 4 章と第 9 章）．
- **コード肥大化．**独立な操作が N 個未満しかないたびに NOP がスロットを埋め，プログラムが大きくなる．表 11.2 の 4 スロット版は半分が NOP で 512 ビットを占めるが，通常の 32 ビット命令 8 個なら 256 ビットで済む．命令の圧縮（11.3 節）はこのオーバーヘッドを減らす．
- **新しいプロセッサには再コンパイルしたプログラムが必要である．**操作のグループ化とスロット数はコンパイル時に固定されるので，後方互換性（後述）が損なわれる．

コンパイラが仮定するしかない値：Skylake ではロードは L1 から 4 サイクル，L2 から 12 サイクル，L3 からおよそ 40 サイクルかかり（Intel 最適化マニュアル），DRAM からはおよそ 200–300 サイクル（≈ 60–80 ns）かかる（7-cpu.com）．

ヒント 最初の 2 つは，コンパイラには分からないもの，すなわち実際のレイテンシ，実際のアドレス，実際の分岐方向である．

定義 11.3 (後方互換性). プロセッサが**後方互換性**を持つとは，同じファミリの以前のプロセッサ向けにコンパイルされたプログラムを，変更せずに実行できることである．

▷ backward compatibility

スーパースカラプロセッサでは，後方互換性に性能向上が伴う．幅の広いアウトオブオーダプロセッサは，古いバイナリからも再コンパイルなしでより多くの並列性を見つける．スロット数を増やした新しい VLIW プロセッサは，命令形式が変わったために古い VLIW バイナリをまったく実行できないか，実行できても，そのバイナリのコンパイル対象だったプロセッサよりサイクル当たりの操作数が増えない．[1]

例 11.4 (幅の広い VLIW プロセッサ上の古いバイナリ). 2 スロット形式も受け付ける 4 スロットのプロセッサは，例 11.2 の 2 スロット版を，古いプロセッサと同じく 5 サイクルで実行する．4 サイクルで済むのは再コンパイルした 4 スロット版だけである．8 個の通常の命令は，同じ数え方をすると，再コンパイルなしで，幅 2 のアウトオブオーダプロセッサでは 5 サイクル，幅 4 のものでは 4 サイクルかかる．

[1] 欠けている互換性をソフトウェアで補うこともできる．Transmeta の Crusoe（2000 年）は x86 命令を直接実行せず，コードモーフィングソフトウェアと呼ばれる変換器が x86 コードのブロックを 4 操作からなる 128 ビットの VLIW 命令に変換し，それをキャッシュした [22]．後継の Efficeon は 256 ビットの命令を用いた．Nvidia の Denver（2014 年）は Arm 命令をハードウェアでデコードし，頻繁に実行される Arm コードだけをネイティブの VLIW 命令に変換する．

> **要点**
> VLIW は，ハードウェアの複雑さをコンパイラへの依存と引き換えにする．コードが規則的で，まさにそのプロセッサ向けにコンパイルされている場合には見合うが，並列性や操作のレイテンシが実行時にしか分からない場合や，古いバイナリを新しいプロセッサでより速く実行しなければならない場合には不利になる．

11.3 VLIW 命令

出典 Lesson 11，動画 6；H&P 付録 H．

VLIW 命令は N 個の操作スロットの並びである（図 11.2）．各スロットは通常の ISA の命令と同じように符号化され，通常の命令コード全体から選ばれた固有の命令コードと，固有のオペランドを持つ．

1 つの命令に含まれる操作は独立でなければならない．どの操作も他の操作の結果を使ってはならず，2 つの操作が同じ位置に書き込んではならない．ハードウェアはこれを検査しないので，正しくグループ化する責任はコンパイラにある．連続する命令の操作どうしは依存してよい．表 11.2 で I5 が I1 に依存しているのがその例である．

注意 独立性が要求されるのは 1 つの命令の中だけであり，命令と命令の間では要求されない．

コンパイラは，例 11.2 のように命令スケジューリングによって並列性を引き出す．第 10 章の他の技法もこれを助ける．木の高さ削減は依存の連鎖を短くして，より多くの操作が 1 つの命令を共有できるようにし，ループ展開とインライン展開は，操作を隔ててしまう分岐，呼び出し，リターンを除去する．VLIW

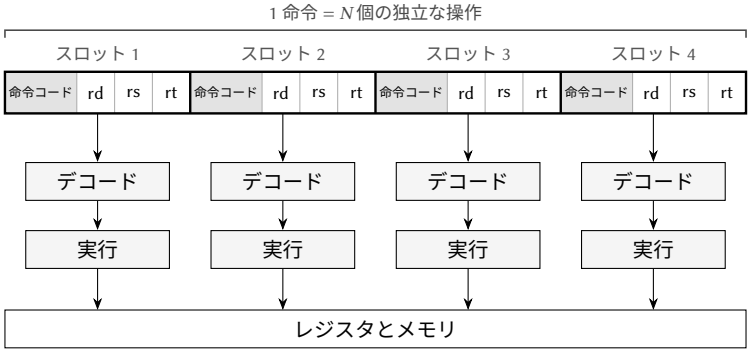


図 11.2: 4 つの操作スロットを持つ VLIW 命令．各スロットは他のスロットと独立にデコードされ，実行される．スロット間の依存を検査するハードウェアはない．

プロセッサにはリネーミングのハードウェアがないので，偽の依存（第 3 章）はコンパイラ自身が，他で使われていないレジスタに結果を書き込むことで除去する．そのためのレジスタを，命令セットは数多く用意している．

IA-64 はコンパイラがリネーミングに使うレジスタを，整数用と浮動小数点用にそれぞれ 128 本与える．MIPS では 32 本，x86-64 では 16 本である（第 5 章）．

分岐の後の操作は分岐の一方の経路でしか実行されないので，分岐はグループ化を制限する．そのため VLIW の命令セットはプレディケーション（第 5 章），理想的には完全プレディケーションを十分にサポートし，コンパイラが分岐を if 変換して両方の経路の操作をグループ化できるようにする．残った分岐には**分岐ヒント**を付けることができ，コンパイラはこれによって，分岐がどちらに進むと予想しているかをプロセッサに伝える．

▷ branch hint

固定スロットは，独立な操作が N 個未満しかないたびに領域を無駄にする．圧縮された命令は操作を NOP なしで順に並べて格納し，一緒に実行される各グループの最後の操作に**ストップビット**で印を付ける．表 11.2 の 4 スロット版のプログラムでは I4，I6，I7，I8 がストップビットを持ち，操作当たり 33 ビットとすると，コードは 512 ビットではなく 264 ビットを占める．

▷ stop bit

11.4 実際の VLIW プロセッサ

VLIW の手法がどこで失敗し，どこで成功するかを，2 種類のプロセッサが示している．

出典 Lesson 11，動画 7–9；H&P §3.7，付録 H．

11.4.1 Itanium

VLIW 風の命令を汎用コンピューティングに持ち込もうとした最も有名な試みは，Intel が IA-64 命令セットのために開発したプロセッサファミリ **Itanium** である．設計者たちはこの手法を EPIC（explicitly parallel instruction computing）と呼んだ [23]．しかし，この命令セットには多くの追加機能が積み重なり，そのハードウェアが非常に複雑になったため，VLIW の主な利点が失われた．不規則なコードからはやはりコンパイラがほとんど並列性を見つけられず，Itanium がアウトオブオーダプロセッサに取って代わることはなかった．

▷ Itanium

IA-64 も同じようにストップを用いて，独立な命令のグループの終わりを示す（11.3 節）．

11.4.2 DSP

DSP（digital signal processor）とは，音声，映像，無線などのデジタル信号の処理に特化したプロセッサである．この市場は VLIW の長所（11.2 節）を生かせる．

- 同じ計算を多数のサンプルに適用するループがコードの大半を占める．コンパイラが並列性を十分に見つけられる，規則的なコードである．
- プログラムは通常，それが動作する機器の特定の DSP 向けにコンパイルされるので，バイナリの後方互換性はほとんど問題にならない．

▷ digital signal processor

H&P は VLIW の手法の例として TI の C6x 系 DSP を挙げている．

注意 C6x はストップビットとは逆の印を付ける．操作の最下位ビットが 1 なら次の操作と並列に実行するという意味であり，グループはそのビットが 0 のところで終わる．32 ビットの操作 8 つで 256 ビットのパケットになる（TI C6000 リファレンスガイド）．

- DSP は電力の限られた機器に搭載されるので，エネルギー効率の高いハードウェアが効果を発揮する．

要点 — 章のまとめ
アウトオブオーダのスーパースカラプロセッサは N よりはるかに多い命令を調べ，コンパイラの助けを必要としない．インオーダのものは N 個の命令を調べ，その助けを必要とする．VLIW プロセッサは最大 N 個の独立な操作からなる 1 命令を調べ，全面的にコンパイラに頼る．VLIW のハードウェアは単純でエネルギー効率が高く，規則的なループのコードで良い性能を示す．しかし新しいプロセッサごとにバイナリを再コンパイルしなければならず，不規則なコードからはほとんど並列性が得られず，1 つのロードのキャッシュミスが後続のすべての操作を遅らせ，ストップビットで圧縮しない限り NOP がコードを肥大化させる．Itanium は汎用コンピューティングで失敗し，VLIW が適するのは DSP である．第 12 章では，サイクル当たり命令数からメモリへと話題を移す．

第 12 章

キャッシュの復習

講義動画	Lesson 12，動画 1–45
教科書	H&P [1] §2.1，§B.1–B.2
キーワード	局所性の原理，時間的局所性，空間的局所性，キャッシュヒット，キャッシュミス，AMAT，ミスペナルティ，ブロックオフセット，タグ，有効ビット，ダイレクトマップキャッシュ，セットアソシアティブキャッシュ，LRU，ダーティビット

第 3–11 章では，プロセッサが毎秒完了する命令の数を増やす手法を見てきた．しかし，どの命令もメモリからフェッチされなければならず，多くの命令はさらにデータをロードまたはストアする．そして第 1 章で見たように，メモリの速度はプロセッサの速度からますます引き離されていく（メモリウォール）．この章では，この差を埋める機構であるキャッシュを復習する．キャッシュは小容量で高速なメモリであり，プログラムは局所性をもってメモリにアクセスするので，大容量で低速な主記憶を待たずに大半のアクセスに応えられる．続いて，キャッシュの性能をどのように測るか，キャッシュをどのように構成し探索するか，ミスの際にどのブロックを追い出すか，書き込みをどのように扱うかを述べる．仮想記憶（第 13 章）と第 14 章の高度なキャッシュ技法は，この内容の上に直接成り立つ．

12.1 局所性の原理

キャッシュが機能するのは，プログラムのメモリアクセスがランダムとはほど遠いからである．

出典 Lesson 12，動画 1–6；H&P §2.1；[24]．

定義 12.1 (局所性の原理). プログラムが近い将来に行うことは，最近行ったことに近い可能性が高い，という観察を**局所性の原理**という．

▷ locality principle

メモリ参照において，「近い」には 2 つの意味がある．プログラムがアドレス X にアクセスしたところだとする．

定義 12.2 (時間的局所性と空間的局所性). 同じアドレス X に近いうちに再びアクセスする可能性が高いとき，プログラムは**時間的局所性**を示す．X の近くのアドレスに近いうちにアクセスする可能性が高いとき，プログラムは**空間的局所性**を示す．

▷ temporal locality
▷ spatial locality

どちらの局所性もプログラムの構造から生じる．ループは同じ命令を繰り返し実行し（時間的局所性），それらの命令は連続したアドレスからフェッチされる（空間的局所性）．反復のたびに更新される変数には同じアドレスで何度もアクセスし，順に処理される配列の要素は連続したアドレスに並んでいる．

例 12.3. 次のループは，アドレス R4 のバッファからアドレス R5 のバッファへ R6 バイトをコピーする．

```
Loop:  LB   R3, 0(R4)      ; R3 <- 転送元のバイト
       SB   R3, 0(R5)      ; 転送先のバイト <- R3
       ADDI R4, R4, 1      ; 次の転送元バイト
       ADDI R5, R5, 1      ; 次の転送先バイト
       ADDI R6, R6, -1     ; 残りのバイト数を 1 減らす
       BNE  R6, R0, Loop   ; 全バイトについて繰り返す
```

6 つの命令フェッチは両方の局所性をもつ．フェッチは連続したアドレスに向かい，毎回の反復で同じ 6 つのアドレスがフェッチされる．データアクセスは，各バッファを順にたどる 2 本の流れをなす．どのバイトもちょうど 1 回だけ読まれるか書かれるので，データには時間的局所性がない．それでもデータには空間的局所性がある．アクセスは遠く離れているかもしれない 2 つのバッファの間を行き来するが，各アクセスは同じバッファへの直前のアクセスに隣接している．空間的局所性が要求するのは**最近**アクセスしたアドレスの近くへのアクセスであ

り，最後にアクセスしたアドレスの近くである必要はない．

注意 空間的局所性は距離の問題であり，規則性の問題ではない．一定の大きなストライドをもつアクセス列は完全に規則的だが，空間的局所性をもたない．

12.2 キャッシュ

局所性は，大容量のメモリへの高速なアクセスをプロセッサに与える方法を示唆する．フェッチ，ロード，ストアのたびに主記憶にアクセスするのは遅い．主記憶全体をプロセッサと同じ速さにすることもできない．高速なメモリはビット当たりのコストが高く，大容量のメモリほど本質的にアクセスが遅いからである．残る選択肢は，プロセッサ自身の中に少量の高速なメモリを置き，最近アクセスされたデータとその周辺のデータをそこに入れておくことである．時間的局所性と空間的局所性により，大半のアクセスは目的のデータをそこで見つけられる．

出典 Lesson 12，動画 7–9；H&P §B.1；[25]．

定義 12.4 (キャッシュ). **キャッシュ**とは，より大容量で低速なメモリのうち最近アクセスされた部分の写しを保持し，大半のアクセスをその写しで処理できるようにする，小容量で高速なメモリである．

▷ cache

プロセッサのすべてのメモリアクセスは，まずキャッシュ内でデータを探す（図 12.1）．データがあれば，そのアクセスは**キャッシュヒット**であり，データはキャッシュ内で読み書きされる．

▷ cache hit

そうでなければ，そのアクセスは**キャッシュミス**である．データを主記憶から取得してキャッシュにコピーし，そのデータへの以降のアクセスと，空間的局所性によりその近傍へのアクセスがヒットするようにする．キャッシュはすべてのアクセスで参照されるので，探索そのものが非常に高速でなければならない．これが，コストに加えてキャッシュを小さくしなければならない 2 つ目の理由である．小さなキャッシュにすべてを保持することはできないので，ミスは避けられない．[1]

▷ cache miss

[1] この考えは Wilkes が 1965 年に提案した「スレーブメモリ」にさかのぼる [25]．商用機として最初にこれを備えたのは IBM System/360 Model 85（1968 年）であり，その 16–32 KB のバッファは，それを記述した論文でキャッシュと名付けられた [26]．

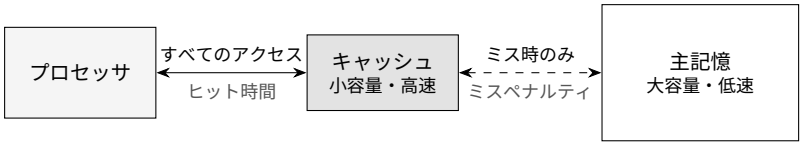


図 12.1: すべてのメモリアクセスはキャッシュに向かい，ミスだけが主記憶へ進む．

12.3 キャッシュの性能

キャッシュの効果は，メモリアクセス 1 回にかかる平均時間で測る．この時間は次の量で決まる．

出典 Lesson 12，動画 10–12；H&P §B.2．

定義 12.5 (ヒット率，ミス率). **ヒット率**はメモリアクセスのうちキャッシュヒットである割合，**ミス率**はミスである割合である．ヒット率 + ミス率 = 1 である．

▷ hit rate
▷ miss rate

定義 12.6 (ヒット時間，ミスペナルティ). **ヒット時間**とは，キャッシュを探索してその中のデータにアクセスするのに必要な時間である．**ミスペナルティ**とは，ミスが**追加で**要する時間，すなわち主記憶からデータを取り出してキャッシュに格納する時間である．

▷ hit time
▷ miss penalty

ミスを検出する前にキャッシュを探索しなければならないので，ミスもヒット時間を払う．したがってミスにはヒット時間とミスペナルティの和がかかる．2 つの場合をその頻度で重み付けすると，**平均メモリアクセス時間**（AMAT）が得られる．

▷ average memory access time

$$\begin{aligned}\text{AMAT} &= (1 - \text{ミス率}) \times \text{ヒット時間} + \text{ミス率} \times (\text{ヒット時間} + \text{ミスペナルティ}) \\ &= \text{ヒット時間} + \text{ミス率} \times \text{ミスペナルティ}.\end{aligned} \tag{12.1}$$

注意 ミスペナルティはミスの余分な時間である．ミスの総時間からヒット時間を引くこと．

AMAT を小さくするには 3 つのことが必要である．

- ヒット時間が短いこと．キャッシュは小さく単純でなければならない．
- ミス率が低いこと．キャッシュは，プログラムがこれから使うデータを保持できるだけ十分に大きく，かつ巧みに構成されていなければならない．

- ミスペナルティが小さいこと．ミスペナルティは主記憶のアクセス時間に支配され，大きい．第 14 章では，キャッシュのレベルを増やすことでこれを減らす．

最初の 2 つの目標は相反する．大きなキャッシュはミスが少ないが，探索に時間がかかる．最適なキャッシュサイズは両者のバランスをとったものであり，そのバランスを測る尺度が AMAT である．

例 12.7. 主記憶によるミスペナルティは 60 ns である．3 つの候補キャッシュを比較する．

3 GHz ではこのミスペナルティは 180 サイクルであり，4 命令幅のプロセッサ（第 6 章）は待つ間におよそ 700 命令分の発行機会を失う．

キャッシュ	ヒット時間	ミス率	AMAT
小	1 ns	6 %	$1 + 0.06 \times 60 = 4.6$ ns
中	1.5 ns	3 %	$1.5 + 0.03 \times 60 = 3.3$ ns
大	3 ns	2 %	$3 + 0.02 \times 60 = 4.2$ ns

中程度のキャッシュが最良である．大きなキャッシュはミスが少ないが，ヒット時間が長いことによるコストが，減らせるミスの分を上回る．3 つとも，毎回主記憶にアクセスする場合より 1 桁優れている．

注 12.8. キャッシュが引き合うのは，ヒットが主記憶へのアクセスよりずっと速く，かつミスが稀な場合だけである．ヒット時間が主記憶のアクセス時間に近ければ，ヒットでさえキャッシュなしのアクセスとほぼ同じくらい遅い．ミスが頻繁なら，ミスペナルティが支配的になる．どちらの場合も，AMAT は主記憶だけのアクセス時間に近づくか，それを上回ることさえある．

12.3.1 実際のプロセッサのキャッシュ

実際のプロセッサは複数のレベルのキャッシュをもつ．最初に参照され，プロセッサの読み書きに直接応えるキャッシュが最も小さく高速であり，そのミスだけが，より大きな後続のキャッシュレベル（L2，L3；第 14 章），そして最後に主記憶へと進む．この第 1 レベルを **L1 キャッシュ**と呼ぶ．L1 キャッシュは典型的には 16–64 KB（1 KB = 2^{10} バイト）であり，アクセスの約 90 % がヒットするだけの大きさと，ヒットが 1–3 クロックサイクルで済むだけの小ささをもつ．

出典 Lesson 12，動画 13；H&P §2.1．

▷ L1 cache

最近のプロセッサの規模：Skylake はコアごとに 32 KB の L1 と 256 KB の L2 をもち，8 MB の L3 を共有する．Zen 3 では 32 KB，512 KB，32 MB である（7-cpu.com）．

12.4 ブロックとライン

データがキャッシュにあるかどうかを素早く判定するため，キャッシュは固定サイズのエントリからなる表として構成される．データは主記憶とキャッシュの間を 1 バイトずつではなく，ブロックと呼ばれる単位で移動する．

出典 Lesson 12，動画 14–18；H&P §B.1．

定義 12.9 (キャッシュブロック). **キャッシュブロック**とは，ひとまとまりとしてキャッシュにコピーされ，またキャッシュから追い出される，メモリ上の連続したバイトの並びである．ブロックのバイト数を**ブロックサイズ**という．

▷ cache block

▷ block size

ブロックサイズは，空間的局所性とキャッシュの容量とのトレードオフである．

- ブロックが非常に小さい（極端な場合は 1 バイト）と，空間的局所性により起こりやすい次のバイトへのアクセスが再びミスし，バイトごとに管理情報が必要になる．
- ブロックが非常に大きいと，ブロックの大部分はすぐには使われないデータになる．それは，最近使われた他のデータを保持できたはずの領域を占め，キャッシュに入るブロックの数が減る．

典型的なブロックサイズは 32–128 バイトである．キャッシュが保持できるブロックの数は，キャッシュのサイズをブロックサイズで割ったものである．64 バイトのブロックをもつ 16 KB のキャッシュは $2^{14}/2^6 = 256$ 個のブロックを保持する．

実際のブロックサイズ：現行の x86-64 プロセッサと大半の Arm コアは 64 バイト，Apple の M1 と M2 は 128 バイトである（Intel・Arm 最適化マニュアル，Apple の `hw.cachelinesize`）．

12.4.1 ブロックの開始位置

ブロックは任意のアドレスから始まるわけではない．もしそうなら，アドレス 100–227 を覆うブロックと 200–327 を覆うブロックが同時にキャッシュに存在しうる．すると 1 つのアドレスが複数のキャッシュ内ブロックに含まれ，あるアド

レスがキャッシュされているかを調べるにはアドレス範囲の比較が必要になる．そこで，メモリはブロックサイズの倍数から始まる，重なりのない固定のブロックに分割される．128 バイトのブロックなら，ブロックはアドレス 0，128，256，... から始まる（図 12.2）．こうすれば，どのアドレスもちょうど 1 つのブロックに属する．

定義 12.10 (キャッシュライン). **キャッシュライン**とは，ブロックを 1 つ保持できるキャッシュ内のスロットである．メモリはブロックに，キャッシュはラインに分割される．ラインはブロックと同じ大きさをもち，時によって異なるブロックを保持する．

▷ cache line

ブロックサイズ，したがってラインのサイズは，常に 2 のべき乗である．そうして初めて，次節で示すように，アドレスが属するブロックとブロック内の位置を，除算ではなくアドレスのビットの選択によって求められる．

注意 本章はブロック（メモリ側）とライン（キャッシュ側の置き場）を区別するが，H&P はブロックとラインを同義に用い，ベンダのマニュアルはどちらもキャッシュラインと呼ぶ．

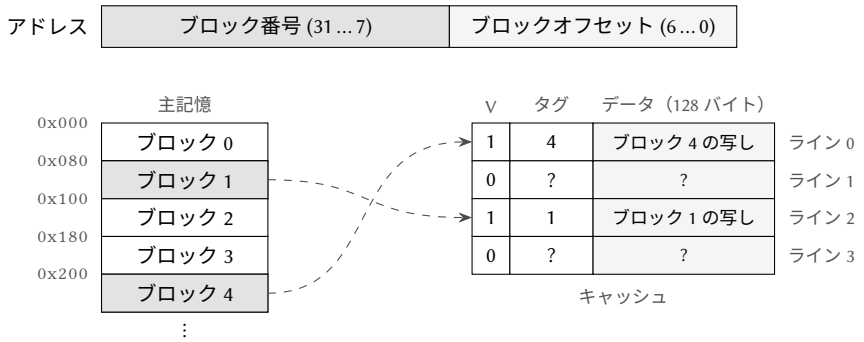


図 12.2: 128 バイトのブロックに分割された主記憶と，4 ラインのキャッシュ．ライン 0 と 2 はブロック 4 と 1 の写しを保持し，ライン 1 と 3 は有効なものを何も保持していない．

12.5 ブロック番号，タグ，有効ビット

ブロックサイズが 2^k バイトのとき，アドレス A は 2 つのフィールドに分かれる．

出典 Lesson 12，動画 19–23；H&P §B.1．

定義 12.11 (ブロックオフセットとブロック番号). アドレスの下位 k ビットは**ブロックオフセット**であり，そのバイトのブロック内での位置を表す．残りの上位ビットは**ブロック番号**であり，ブロックを識別する．

▷ block offset
▷ block number

$$\text{ブロック番号} = \lfloor A/2^k \rfloor, \qquad \text{ブロックオフセット} = A \bmod 2^k. \tag{12.2}$$

例 12.12. 128 バイトのブロック（$k = 7$）では，アドレス 0x0040A7C6 のブロックオフセットは 0xC6 & 0x7F = 0x46 = 70，ブロック番号は 0x0040A7C6 ≫ 7 = 0x814F である．このバイトはブロック 0x814F の位置 70 にあり，このブロックはアドレス 0x0040A780 から 0x0040A7FF までを覆う．

12.5.1 タグ

ラインは時によって異なるブロックを保持するので，各ラインは現在どのブロックを保持しているかを記録しなければならない．この記録がラインの**タグ**である．タグは，ラインにあるブロックのブロック番号，あるいは以降の節で示すように，ブロック番号のうちラインの位置からは分からない部分である．探索では，アドレスのブロック番号を計算し，そのブロックが置かれうるラインのタグと比較する．一致すればヒットであり，ブロックオフセットがライン内のバイトを選ぶ．

▷ tag

12.5.2 有効ビット

プロセッサの電源投入時，キャッシュのラインには任意のビット，したがって任意のタグが入っている．たまたまブロック番号と一致したタグは，一度も読み込まれていないデータに対するヒットを生んでしまう．

定義 12.13 (有効ビット). ラインの**有効ビット**は，そのラインがブロックを保持しているかどうかを示す．起動時にすべての有効ビットをクリアし，ラインにブロックを読み込んだときにそのラインの有効ビットをセットする．

▷ valid bit

したがって，ライン，あるいはキャッシュ全体を空にするには有効ビットをクリアするだけでよく，タグとデータはそのままにしておける．唯一の例外は，キャッシュ内で書き込まれたがまだ主記憶には書き込まれていないブロックであり（12.11 節），これは先に書き戻さなければならない．

x86 には両方の操作がある．WBINVD はダーティなブロックを書き戻してから有効ビットをクリアし，INVD はクリアしてデータを捨てる．

探索がヒットとなるのは，ラインが有効で，かつ格納されたタグがアドレスのタグフィールドと一致するときだけである．

$$\text{ヒット} \;=\; V \wedge (\text{格納されたタグ} = \text{アドレスのタグ}). \tag{12.3}$$

本節のように，ブロック番号のどの部分もラインの選択に使われない限り，タグフィールドはブロック番号全体である．以降の節の構成では，ブロック番号のビットの一部をラインの選択に使い，残りのビットだけをタグに保持する．

> **要点**
> アドレスはブロック番号とオフセットからなる．タグはラインがどのブロックを保持しているかを，有効ビットはそもそもブロックを保持しているかを示し，オフセットはバイトを選ぶ．次節の主題であるブロックを置ける場所が，**どの**ラインを探索しなければならないかを決める．

12.6 ブロックを置ける場所

キャッシュは，あるブロックを置けるラインがいくつあるかによって異なる．この数をキャッシュの**連想度**という．連想度は，探索で比較しなければならないタグの数と，異なるブロックが同じラインを奪い合う頻度を決める．次の 3 つの構成が区別される（図 12.3）．

出典 Lesson 12，動画 24；H&P §B.1.

▷ associativity

ダイレクトマップ 各ブロックはちょうど 1 つのラインにしか置けない（12.7 節）．

セットアソシアティブ 各ブロックは，セットをなす N 個のラインのどれにでも置ける（12.8 節）．

フルアソシアティブ 各ブロックはキャッシュのどのラインにでも置ける（12.9 節）．

典型的な連想度：Skylake は L1 が 8 ウェイ，L2 が 4 ウェイ，L3 が 16 ウェイ．Zen 3 は L1 が 8 ウェイ，L2 が 8 ウェイ，L3 が 16 ウェイである（7-cpu.com）．

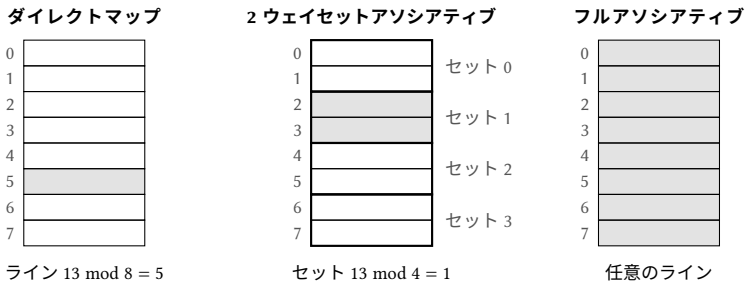


図 12.3: 各構成の 8 ラインのキャッシュにおいて，ブロック番号 13 を置けるライン．

12.7 ダイレクトマップキャッシュ

3 つの構成のうち最も単純なものは，すべてのブロックにキャッシュのラインを 1 つだけ割り当てる．**ダイレクトマップキャッシュ**では，どのブロックにもそれを置けるラインがちょうど 1 つあり，そのラインはブロック番号の下位ビットで選ばれる．

出典 Lesson 12，動画 25–28；H&P §B.1.

▷ direct-mapped cache

定義 12.14 (インデックス). **インデックス**とは，ブロックをキャッシュのどこに置けるかを選ぶアドレスのビット群である．2^n 個のラインをもつダイレクトマップキャッシュでは，インデックスはブロック番号の下位 n ビットからなる．

▷ index

したがってアドレスは 3 つのフィールドに分かれる（図 12.4）．最下位の k ビットがオフセット，その上の n ビットがインデックス，残りのすべてのビットがタグである．ラインにはブロック番号全体ではなく，このタグのビットだけを格納する．ライン i に置けるブロックはすべてインデックス i をもち，ブロック番号のインデックス部分はライン自体から分かるからである．

探索では，インデックスで選ばれた 1 つのラインを読み，そのタグをアドレスのタグと比較し，有効ビットを調べる（式 (12.3)）．

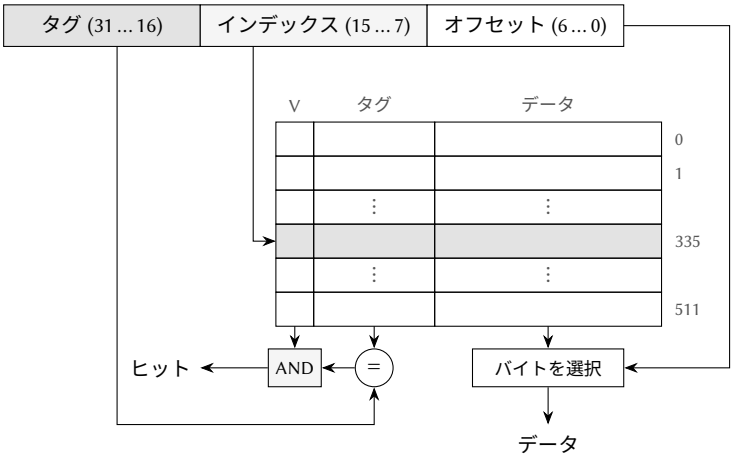


図 12.4: 128 バイトのブロックをもつ 64 KB のダイレクトマップキャッシュにおける探索．インデックスが 512 個のラインの 1 つを選び，1 つの比較器がそのタグを調べる．

例 12.15. 128 バイトのブロックと 32 ビットのアドレスをもつ 64 KB のダイレクトマップキャッシュには $512 = 2^9$ 個のラインがあり，オフセットはビット 6–0，インデックスはビット 15–7，タグはビット 31–16 である．例 12.12 のアドレス 0x0040A7C6（ブロック番号 0x814F）のインデックスは $0x814F \,\&\, 0x1FF = 0x14F = 335$，タグは $0x814F \gg 9 = 0x0040$ である．ライン 335 が有効でタグ 0x0040 を保持していれば，このアクセスはヒットする．

12.7.1 利点と欠点

ダイレクトマップキャッシュの長所は，調べるラインが 1 つだけで済むことである．比較器は 1 つで足りるので，探索は高速かつ安価で，エネルギー効率も良い．ヒット時間は，与えられたサイズのキャッシュとして可能な限り短い．

短所は，同じインデックスをもつブロックが 1 つのラインを奪い合うことである．プログラムがそのような 2 つのブロックに交互にアクセスすると，各アクセスがもう一方のアクセスに必要なブロックを追い出し，キャッシュの残りが空であってもすべてのアクセスがミスする．こうした競合がミス率を高める．

この奪い合いで生じるミスを競合性ミスといい，第 14 章で分類されるミスの 3 つの原因の 1 つである．

例 12.16. 16 バイトのブロックをもつ 8 ラインのダイレクトマップキャッシュでは，オフセットはビット 3–0，インデックスはビット 6–4，タグはビット 31–7 である．あるプログラムがアドレス 0x1000（ブロック 0x100）と 0x1084（ブロック 0x108）に交互にアクセスする．どちらのブロックもインデックスは 0 で，タグはそれぞれ 0x20 と 0x21 である．各アクセスは直前のアクセスが読み込んだブロックを置き換えるので，プログラムがどれだけ長く続いても，すべてのアクセスがミスとなる．

12.8 セットアソシアティブキャッシュ

セットアソシアティブ構成は，ラインが 1 つだけという制約を緩める．

出典 Lesson 12，動画 29–31；H&P §B.1．

定義 12.17 (セットアソシアティブキャッシュ). **セットアソシアティブキャッシュ**は，ラインを N 個ずつの**セット**に分け，ブロックを特定の 1 つのセットのどのラインにでも置けるようにする．このようなキャッシュを N ウェイセットアソシアティブという．

▷ set-associative cache

このときインデックスはラインではなくセットを選ぶ．2^s 個のセットをもつキャッシュのインデックスは s ビットであり，タグは再びインデックスより上のすべてのビットからなる．探索では，アドレスのタグを，選ばれたセットの N 個すべてのラインのタグと並列に比較し，有効なラインのいずれかが一致すればヒットとなる（図 12.5）．データは一致したラインから得られる．

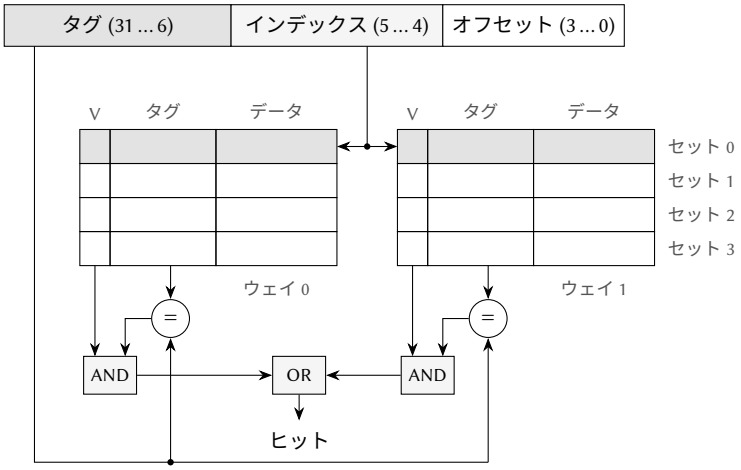


図 12.5: 4 セットの 2 ウェイセットアソシアティブキャッシュ（例 12.18 の構成）における探索．インデックス（ここではその例と同じ 0）がセットを選び，セット内の 2 つのラインはそれぞれ，有効でタグが一致すればヒットとなる．

例 12.18. 例 12.16 の 8 ラインを 2 ウェイセットアソシアティブキャッシュとして構成する．セットは 4 個なので，インデックスはビット 5–4，タグはビット 31–6 である．ブロック 0x100 はタグ 0x40 でセット 0 に，ブロック 0x108 はタグ 0x42 でセット 0 に入る．2 つのブロックは依然として同じセットを共有するが，セットには 2 つのラインがある．各ブロックへの最初のアクセスがミスしてそのブロックを読み込んだ後は，以降のアクセスはすべてヒットする．

同じサイズのダイレクトマップキャッシュと比べると，N ウェイセットアソシアティブキャッシュには N 個の比較器と，一致したラインのデータの選択が必要であり，そのためヒット時間は長くなる．しかし競合が少ないので，ミス率は低い．

32 KB 8 ウェイの L1 が多い理由：64 セット ×64 バイトのブロックで 4 KB となり，インデックスとオフセットがページオフセットに収まるので，アドレス変換の前に探索を始められる（第 14 章）．

12.9 フルアソシアティブキャッシュ

もう一方の極端では，ブロックをどこにでも置ける．

定義 12.19 (フルアソシアティブキャッシュ). **フルアソシアティブキャッシュ**では，どのブロックもどのラインにでも置ける．

出典 Lesson 12，動画 32–33；H&P §B.1．

▷ fully associative cache

選ぶものがないので，インデックスは存在しない．タグはブロック番号全体であり，探索ではそれをキャッシュ内のすべてのラインのタグと比較する．空いているラインがある限り 2 つのブロックが 1 つのラインを奪い合うことはないが，ラインごとに比較器が必要になるので，大きなフルアソシアティブキャッシュは低速で高価になる．

フルアソシアティブの構造は小さいものに限られる．32–64 エントリの L1 TLB（第 13 章）にはフルアソシアティブのものが多いが，キャッシュはそうではない．

12.9.1 3 つの構成に共通の枠組み

ダイレクトマップキャッシュとフルアソシアティブキャッシュは，セットアソシアティブキャッシュの両極端である．ダイレクトマップキャッシュは，ラインと同数のセットをもつ 1 ウェイセットアソシアティブキャッシュであり，フルアソシアティブキャッシュは，すべてのラインを含むただ 1 つのセットをもつ．アド

レスが a ビットのとき，どの構成についても次が成り立つ．

$$\begin{aligned}\text{オフセットビット数} &= \log_2(\text{ブロックサイズ}),\\ \text{インデックスビット数} &= \log_2(\text{セット数}) = \log_2\bigl(\text{ライン数}/N\bigr),\\ \text{タグビット数} &= a - \text{インデックスビット数} - \text{オフセットビット数},\end{aligned} \tag{12.4}$$

ここで，ダイレクトマップキャッシュでは $N = 1$（インデックスビット数 $= \log_2$(ライン数)），フルアソシアティブキャッシュでは $N =$ ライン数（インデックスは 0 ビット）である．例 12.15 の 128 バイトのブロックをもつ 64 KB のキャッシュでは，タグはダイレクトマップの 16 ビットからフルアソシアティブの 25 ビットまで増える．その中間の場合は 12.12 節 の表 12.4 に示す．

ヒント C バイトのキャッシュのタグは $a - \log_2(C/N)$ ビットである．N を 2 倍にするたびに，1 ビットがインデックスからタグに移る．

> **要点**
> 連想度はヒット時間とミス率のトレードオフを決める．セット当たりのウェイ数が多いほど，毎回のアクセスでのタグ比較は増えるが，競合は減る．ブロックが追い出されるのは，そのセットのすべてのラインが他のブロックで占められているときだけだからである．

12.10 キャッシュの置換

ミスの際には，新しいブロックを，それに許されたラインのいずれかに置かなければならない．それらのラインのうち有効でないものがあれば，そのラインを使う．すべてが有効なブロックを保持していれば，1 つのブロックを追い出さなければならない．ダイレクトマップキャッシュには選択の余地がない．セットアソシアティブキャッシュとフルアソシアティブキャッシュでは，追い出すブロックを**置換アルゴリズム**が選ぶ．

出典 Lesson 12，動画 34–36；H&P §B.1．

▷ replacement policy

ランダム セット内のブロックを無作為に選んで追い出す．単純だが，ブロックの使われ方を無視する．

FIFO （first in, first out）キャッシュに最も長く存在しているブロックを，まだ使われているかどうかにかかわらず追い出す．

LRU 最後にアクセスされたのが最も昔であるブロックを追い出す．

定義 12.20 (LRU). **LRU** 方式は，置き換えてよいブロックのうち，最も長い間アクセスされていないものを追い出す．

▷ least recently used

LRU は時間的局所性を利用する．最近アクセスされたブロックは近いうちに再びアクセスされる可能性が高いので，最も長い間使われていないブロックが，必要とされる可能性の最も低いブロックである．

12.10.1 カウンタによる LRU の実装

LRU は，ラインごとのカウンタで実装できる．N ウェイセットアソシアティブキャッシュでは，各ラインが $\log_2 N$ ビットのカウンタをもつ．起動時に各セットのカウンタを任意の順序で値 $0, 1, \ldots, N-1$ に初期化し，以下の更新規則はこの状態を保つ．すなわち，各セットでどの値もちょうど 1 回ずつ現れる．カウンタが 0 のラインが最も長い間使われていないラインであり，カウンタが $N-1$ のラインが直近に使われたラインである．カウンタが c であるラインへのアクセスのたびに，次を行う．

1. セット内で c より大きいカウンタをすべて 1 減らす．
2. アクセスしたラインのカウンタを $N-1$ にする．

ヒント セットのカウンタは常に $0, \ldots, N-1$ の並べ替えである．トレースの 1 行に同じ値が 2 つ現れたら，更新が抜けている．

ミスの際は，カウンタが 0 のラインを追い出す．新しいブロックがそこに入った後は，そのアクセスを他のアクセスと同様に扱い，上のようにカウンタを更新する．表 12.1 は 4 ウェイのセット 1 つを追ったものである．

この実装のコストは，ラインごとの $\log_2 N$ ビットと，より重要なことに，すべてのアクセスで生じる動作である．ヒットでさえセットの N 個のカウンタすべてを変更しうる．連想度の高いキャッシュでは，これはかなりのハードウェアとエネルギーを要し，第 14 章の，より安価な LRU の近似の動機となる．

注意 カウンタはヒットを含むすべてのアクセスで更新すること．ブロックをキャッシュに読み込んだときにだけ更新すると FIFO になる．

表 12.1: ブロック A〜D を保持する 4 ウェイのセット 1 つの LRU カウンタ．4 つのラインそれぞれについて，ブロック: カウンタの形で示す．

アクセス	結果	ライン 0	ライン 1	ライン 2	ライン 3
（初期状態）		A:0	B:1	C:2	D:3
C	ヒット	A:0	B:1	C:3	D:2
A	ヒット	A:3	B:0	C:2	D:1
E	ミス，B を追い出す	A:2	E:3	C:1	D:0
D	ヒット	A:1	E:2	C:0	D:3
B	ミス，C を追い出す	A:0	E:1	B:3	D:2

12.11 書き込み方針

出典 Lesson 12，動画 37–40；H&P §B.1．

書き込みには，読み出しでは生じない問題が 2 つある．

1. **書き込みミスの際にブロックをキャッシュに持ち込むか．**持ち込むキャッシュは**ライトアロケート方式**に従う．ノーライトアロケートのキャッシュは値を主記憶だけに書き込む．大半のキャッシュはライトアロケートである．局所性により，書き込まれたばかりのブロックは近いうちに再び読み書きされる可能性が高いからである．
2. **主記憶が書き込まれた値を受け取るのはいつか．**これには 2 つの方式がある．

▷ write-allocate policy

定義 12.21 (ライトスルーとライトバック)**.** **ライトスルー方式**では，書き込みはキャッシュ内のブロックを更新し，ただちに主記憶内のブロックも更新する．

ライトバック方式では，書き込みはキャッシュ内のブロックだけを更新し，主記憶はそのブロックがキャッシュから追い出されるときに更新される．

▷ write-through policy

▷ write-back policy

ライトスルーでは主記憶が常に最新に保たれるが，そのかわりすべての書き込みが主記憶のアクセス時間を払うことになり，これはまさにキャッシュが避けようとしたものである．そのためキャッシュはライトバックを用いる．ブロックがキャッシュにある間の多数の書き込みは，追い出し時の高々 1 回の主記憶への書き込みで済む．ライトバックのキャッシュは通常ライトアロケートでもある．そうでなければ，キャッシュにないブロックへの書き込みは，そのブロックがどれほど頻繁に書き込まれても，すべて主記憶に向かうことになる．

しかし，追い出すブロックをすべて書き戻すと，読み出されただけでメモリ上の写しと同一のブロックまで書き込むことになる．

注意 2 つの選択は独立である．ライトスルーかライトバックかはいつメモリを更新するかを，ライトアロケートかどうかは書き込みミスで何をするかを決める．

定義 12.22 (ダーティビット)**.** ラインの**ダーティビット**は，そのブロックが読み込まれてから書き込まれたかどうかを示す．ブロックを読み込んだときにクリアし，ブロックが書き込まれたときにセットする．追い出しの際，ダーティなブロックは主記憶に書き込み，クリーンなブロックは単に破棄する．

▷ dirty bit

例 12.23. ブロック X と Y はダイレクトマップキャッシュの同じラインに対応し，このラインは最初は無効である．表 12.2 は，ライトアロケートのライトバック方式のもとでの一連のアクセスを追ったものである．Y への連続した 2 回の書き込みは，ステップ 6 での 1 回のメモリ書き込みで済み，ステップ 8 で Y を追い出してもメモリ書き込みは生じない．Y はステップ 7 で読み込まれた後に書き込まれていないからである．[2] ライトバックに必要なメモリ書き込みは合計 3 回であり，ライトスルーなら書き込みごとに 1 回，すなわち 4 回必要になる．

[2] ライトスルーは過去のものではない．Pentium 4 と AMD の Bulldozer は L1 データキャッシュをライトスルーとし，書き込みをバッファで吸収した．このような L1 はダーティビットも書き戻しも要らない．どちらもその後はライトバックの L1 に戻った（Intel・AMD の最適化マニュアル）．

12.12 全体のまとめ

出典 Lesson 12，動画 41–45；H&P §B.1．

キャッシュはいくつかの設計上の選択によって完全に記述され，その構造はそこから導かれる．

キャッシュサイズとブロックサイズ ライン数とオフセットのビット数が決まる．
連想度 セット数，したがってインデックスとタグのビット数が決まる（式 (12.4)）．
置換アルゴリズム LRU は各ラインに $\log_2 N$ ビットのカウンタを加える（ダイレクトマップキャッシュでは不要）．
書き込み方針 ライトバックは各ラインにダーティビットを加える．

表 12.2: ライトバック方式のもとでの 1 つのライン（例 12.23）．D はアクセス後のダーティビットである．

ステップ	アクセス	結果	メモリ操作	アクセス後のライン
1	X 読み出し	ミス	X 読み込み	X，D=0
2	X 書き込み	ヒット	—	X，D=1
3	Y 読み出し	ミス	X 書き戻し，Y 読み込み	Y，D=0
4	Y 書き込み	ヒット	—	Y，D=1
5	Y 書き込み	ヒット	—	Y，D=1
6	X 書き込み	ミス	Y 書き戻し，X 読み込み	X，D=1
7	Y 読み出し	ミス	X 書き戻し，Y 読み込み	Y，D=0
8	X 読み出し	ミス	X 読み込み	X，D=0

こうして各ラインは，データに加えて有効ビット，ダーティビット，LRU カウンタ，タグを格納する．図 12.6 は，1 回のアクセスでこれらがどのように使われるかを示す．ヒットはデータを読み書きし，LRU カウンタを更新する．ミスはまず追い出すラインを選び，それが有効かつダーティであれば書き戻してから，新しいブロックを読み込む．その後，アクセスはヒットと同様に進む．

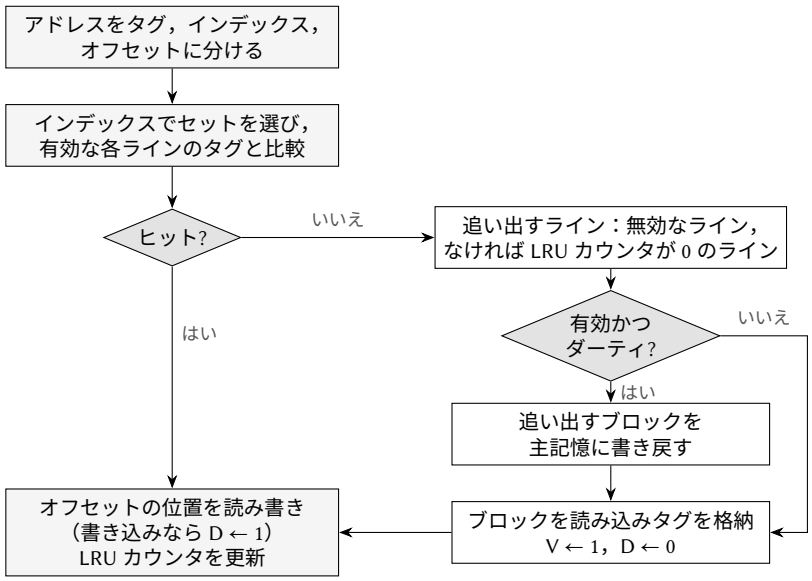


図 12.6: LRU 置換とライトバック方式を用いるセットアソシアティブキャッシュにおける 1 回のメモリアクセス．

例 12.24. 例 12.18 の 2 ウェイセットアソシアティブキャッシュ（16 バイトのブロック，インデックスはビット 5–4，タグはビット 31–6）に，LRU 置換とライトバック方式を加えたものを考える．表 12.3 は，そのセット 1 を 5 回のアクセスにわたって追ったものである．セットの 2 つのラインは終始有効である．最初のアクセスはヒットし，カウンタを更新するだけである．2 番目のアクセスはミスし，カウンタが 0 のウェイ 1 が追い出される．そのブロックはダーティなので，新しいブロックを読み込む前に書き戻される．4 番目のアクセスは書き込みミスであり，追い出されるブロックはクリーンなので何も書き戻されず，読み込まれたブロックはただちにダーティになる．最後のアクセスは，3 番目のアクセスが書き込んだタグ 0x40 のブロックを追い出し，それを書き戻す．

ヒント ライトバックでのメモリ転送は，ミス 1 回につき読み出し 1 回，ダーティなブロックの追い出し 1 回につき書き込み 1 回である．ヒットは何も生じない．

例 12.25. 表 12.4 は，128 バイトのブロック，32 ビットのアドレス，LRU 置換，ライトバックを用いる 64 KB のキャッシュの構成を，いくつかの連想度について求めたものである．8 ウェイのキャッシュでは，アドレス 0x0040A7C6（ブロック番号 0x814F）のインデックスは $0x814F \,\&\, 0x3F = 15$，タグは $0x814F \gg 6 = 0x205$ である．その 512 個のラインには $512 \times 24 = 12\,288$ ビット，すなわち 1536 バイトのメタデータが必要であり，これは 64 KB のデータの約 2.3 % にあたる．

管理情報はブロックを小さくしない理由の 1 つである．同じ 8 ウェイキャッシュを 16 バイトブロックにすると 4096 ライン ×24 ビット，すなわち 12 KB となり，データの 19 % を占める．

表 12.3: LRU 置換とライトバックを用いる 2 ウェイセットアソシアティブキャッシュのセット 1（例 12.24）．各アクセス後の各ウェイのタグ，ダーティビット，LRU カウンタを示す．メモリ操作のものも含め，タグはすべて 16 進数である．

		ウェイ 0			ウェイ 1			
アクセス	結果	タグ	D	LRU	タグ	D	LRU	メモリ操作
（初期状態）		40	0	0	43	1	1	
読み出し 0x1018	ヒット	40	0	1	43	1	0	—
読み出し 0x1054	ミス	40	0	0	41	0	1	43 書き戻し，41 読み込み
書き込み 0x101C	ヒット	40	1	1	41	0	0	—
書き込み 0x1098	ミス	40	1	0	42	1	1	42 読み込み
読み出し 0x1058	ミス	41	0	1	42	1	0	40 書き戻し，41 読み込み

第 13 章では同じ考え方–ブロック，タグ，置換–を仮想アドレスの変換に適用し，第 14 章では，ここで述べた基本的なキャッシュのヒット時間，ミス率，ミスペナルティを下げる技法を扱う．

表 12.4: 128 バイトのブロックと 32 ビットのアドレスをもつ 64 KB のキャッシュのアドレスフィールドと，ライン当たりのメタデータのビット数（有効ビット + ダーティビット + LRU カウンタ + タグ）．

構成	セット数	オフセット	インデックス	タグ	ライン当たりのメタデータ
ダイレクトマップ	512	7	9	16	1 + 1 + 0 + 16 = 18
2 ウェイ	256	7	8	17	1 + 1 + 1 + 17 = 20
8 ウェイ	64	7	6	19	1 + 1 + 3 + 19 = 24
フルアソシアティブ	1	7	0	25	1 + 1 + 9 + 25 = 36

要点 — 章のまとめ

プログラムは時間的局所性と空間的局所性をもってメモリにアクセスするので，最近使われたブロックを保持する小容量で高速なキャッシュが大半のアクセスに応える．その効果は AMAT = ヒット時間 + ミス率 × ミスペナルティ（式 (12.1)）で測られる．メモリは 2 のべき乗の大きさのブロックに分割されてキャッシュのラインに置かれる．アドレスはタグ，インデックス，オフセットに分かれ，ラインが有効でそのタグが一致すればヒットとなる．ダイレクトマップ，セットアソシアティブ，フルアソシアティブの各キャッシュは，ブロックが占めうるラインの数が異なり，ヒット時間とミス率をトレードオフする．ミスの際には LRU が最も長い間使われていないブロックを追い出し，ダーティビットを用いるライトバック方式は，変更されたブロックが追い出されるときにだけブロックをメモリに書き込む．

第 13 章

仮想記憶

講義動画	Lesson 13，動画 1–27
教科書	H&P [1] §B.4–B.5
キーワード	仮想記憶，物理メモリ，ページ，ページフレーム，ページテーブル，ページフォールト，仮想ページ番号，ページオフセット，多段ページテーブル，ページサイズ，内部断片化，TLB，TLB ミス，ページテーブルウォーク

第 12 章では，プロセッサが生成するアドレスをすべて主記憶上の位置として扱った．実際のシステムでは，プログラムが使うアドレスは**仮想**である．すなわち，どのプログラムも自分専用の大きなアドレス空間を見ており，オペレーティングシステムとプロセッサが協力して，そのうちプログラムが実際に使う部分を搭載されたメモリに対応づける．この章では，なぜ仮想記憶が必要か，ページテーブルがどのように仮想アドレスを物理アドレスに変換するか，それらのテーブルがどれほど大きく，多段ページテーブルがどのようにそれを小さく保つか，そして，プロセッサが最近の変換を TLB に保持しなければ，変換がすべてのメモリアクセスを遅くしてしまうのはなぜかを述べる．第 14 章は，この TLB を第 12 章のキャッシュと組み合わせる．

13.1 なぜ仮想記憶が必要か

出典 Lesson 13，動画 1–3；H&P §B.4；[27]．

ハードウェアとソフトウェアでは，メモリの見え方が異なる．ハードウェアにとって，メモリとはマシンに搭載されているもの，すなわちメモリモジュールが提供するバイト数だけの配列である．プログラムにとって，メモリとは 32 ビットプロセッサなら 2^{32} バイト，64 ビットプロセッサなら 2^{64} バイトのアドレス空間であり，そのすべてがそのプログラムのものである．コード，データ，スタックは，搭載されたメモリの量にも，同時に走る他のプログラムにも関係なく，プログラムとコンパイラが選んだアドレスに置かれる．

この 2 つの見方は，2 つの点で食い違う．

- 1 つのプログラムのアドレス空間は，搭載されたメモリよりはるかに大きい．32 ビットのアドレス空間は 4 GB であり，マシンがそれだけのメモリをもつとは限らない．64 ビットのアドレス空間は 16 EB（1 EB = 2^{60} バイト）で，どのマシンがもつ量よりも何桁も大きい．
- 多数のプログラムが同時に走り，そのそれぞれが，アドレス空間全体を所有しているかのように同じアドレスを使う．

注意 64 ビットすべてを変換するプロセッサは存在しない．x86-64 は，ビット 63–48 がビット 47 の繰り返しであるカノニカルアドレスを要求するので，使える空間は 16 EB ではなく 256 TB である．

定義 13.1 (仮想記憶)．すべてのプログラムに，大きな自分専用のメモリという見え方を与え，そのうちプログラムが実際に使う部分をマシンに搭載されたメモリに対応づける機構を**仮想記憶**という．

▷ virtual memory

各プログラムの各部分をどこに置くかはオペレーティングシステムが決め，プロセッサはその決定をすべてのメモリアクセスのすべてのアドレスに適用する．これにより，プログラムはメモリの大きさも，一緒に走ることになる他のプログラムも知らずに書ける．また，オペレーティングシステムが取り計らわない限り，あるプログラムが別のプログラムのメモリにアクセスすることはできず，さらに，プログラム全体では搭載量を超えるメモリを使うことができ，あふれた分はディスクに置かれる（13.4 節）．[1]

13.2 物理メモリと仮想アドレス空間

2 つの見方は，2 種類のアドレスに対応する．

定義 13.2 (物理メモリ)．マシンに実際に搭載されているメモリを，そのマシンの**物理メモリ**という．その各バイトには 0 から物理メモリの大きさより 1 小さい値まで連番が付けられ，バイトを識別するこの番号を，そのバイトの**物理アドレス**

[1] この着想の歴史は古い．1962 年のマンチェスター Atlas は，16K ワードのコア記憶と 96K ワードの磁気ドラムを 512 ワードのページからなる 1 つのアドレス空間として見せ，32 個の連想的なページアドレスレジスタでアドレスを変換した．13.10 節 の TLB の先祖である．

出典 Lesson 13，動画 4–5；H&P §B.4．

▷ physical memory

▷ physical address

という．

物理メモリの大きさを決めるのは搭載されたモジュールであって，プロセッサのアドレスの幅ではない．32 ビットのマシンは 4 GB をアドレス指定できるが，搭載されているのは 2 GB かもしれない．64 ビットのマシンはおそらく数十ギガバイトをもつが，これはアドレス指定可能な 2^{64} バイトのごく一部である．この章の例では，物理アドレスのビット数は搭載されたメモリが要求するちょうどの数とする．2 GB なら 31 ビットである．

定義 13.3 (仮想アドレス空間)．プログラムの**仮想アドレス空間**とは，物理メモリとは無関係に，そのプログラムが使えるアドレスの全範囲，すなわち n ビットプロセッサでは 0 から $2^n - 1$ までである．その中のアドレスを**仮想アドレス**という．

▷ virtual address space

▷ virtual address

プログラムが扱うアドレスはすべて仮想である．プログラムカウンタも，データ中に格納されたポインタも，ロードとストアのためにプロセッサが計算するアドレスもそうである．プログラムが使うのは，仮想アドレス空間のうちのいくつかの領域だけである（図 13.1 の左と右）．

- 低位アドレスにある**コード**とグローバルな**データ**．
- その上にある**ヒープ**．プログラムが動的にメモリを確保するにつれて高位アドレスの方向へ伸びる．
- アドレス空間の最上部にある**スタック**．低位アドレスの方向へ伸びる．

ヒープとスタックの間には，プログラムが決して使わない膨大なアドレスの範囲がある．走っているプログラムはどれもこのように配置された自分専用の仮想アドレス空間をもつので，異なるプログラムがまったく異なる内容に対して同じ仮想アドレスを使うことになる．

13.3 ページとページテーブル

仮想アドレスごとに対応づけを記録すると，アドレスの数だけテーブルのエントリが必要になる．そこで仮想記憶は，キャッシュが個々のバイトではなくブロックを移動させるのと同じように，固定サイズのアドレスの範囲を対応づける．

出典 Lesson 13，動画 6–7；H&P §B.4．

定義 13.4 (ページとページフレーム)．仮想アドレス空間は等しい大きさの連続した範囲に分割され，その 1 つ 1 つを**ページ**という．物理メモリも同じ大きさの範囲に分割され，その 1 つ 1 つを**ページフレーム**，あるいは単にフレームという．その大きさは 2 のべき乗で，典型的には 4 KB であり，どのページもどのフレームにもちょうど収まる．

▷ page

▷ page frame

定義 13.5 (ページテーブル)．プログラムの**ページテーブル**は，その仮想アドレス空間のページごとに，そのページが物理メモリにあるかどうかと，ある場合はどのフレームがそれを保持しているかを記録する．

▷ page table

オペレーティングシステムは，走っているプログラムごとに 1 つのページテーブルを管理し，そのプログラムのページをフレームに置いたり取り除いたりするたびに更新する．走っているプログラムのページテーブルは，オペレーティングシステムが設定したレジスタを通じて見つけられ，そのプログラムのアドレスを変換するために読まれる．読むのは通常プロセッサ自身であり，システムによってはオペレーティングシステムである（13.10 節）．図 13.1 は，2 つのプログラムについて，こうして得られる対応づけを示す．

このレジスタは x86-64 では CR3，AArch64 では TTBR0/TTBR1 である．コンテキストスイッチがこれを入れ替えると，対応づけ全体が一度に変わる．

- どのページもどのフレームにでも置ける．仮想アドレス空間で隣り合うページが物理メモリで隣り合っている必要はなく，異なるプログラムのフレームは互いに入り混じる．
- プログラムが決して使わないページは対応づけられず，フレームを占めない．物理メモリを消費するのは，アドレス空間のうち使われる領域だけである．
- 2 つのプログラムのページテーブルが，同じフレームにページを対応づけることもある．これによりプログラムはメモリを共有でき，例えば同じプログラムが 2 つ走っているとき，そのコードを 1 つのコピーで済ませられる．
- 対応づけられたページがすべて物理メモリにある必要はない．残りはディスクにある（13.4 節）．

ページテーブルは，使われるかどうかによらず仮想アドレス空間の**すべての**

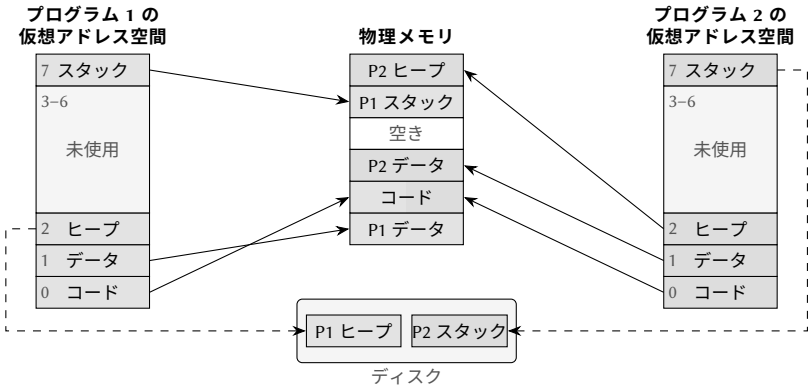


図 13.1: 2 つのプログラムの仮想アドレス空間（ページ 0–7，下が低位アドレス）を物理メモリの 6 つのフレームに対応づけたもの．両プログラムのページ 0 は同じフレームに対応づけられている．各プログラムは 1 ページをディスクにもち（破線），使われないページ 3–6 は対応づけられていない．

ページを記述する．最も単純な形（13.6 節）では，そのそれぞれにエントリをもつ．したがって，記述するページの数を決めるのは仮想アドレス空間の大きさとページの大きさであり，物理メモリの大きさは，1 つのエントリが指せるフレームの数を決めるにすぎない．

注意 ページテーブルの添字は仮想ページであって，フレームではない．物理メモリが少なければフレームは少なくなるが，ページテーブルが記述すべきページの数は変わらない．

13.4 ページフォールト

走っているプログラム全体では，通常，物理メモリのフレーム数より多くのページが対応づけられる．収まりきらないページはディスクに置かれ，物理メモリは最近使われたページを保持する．

出典 Lesson 13，動画 8；H&P §B.4．

定義 13.6 (ページフォールト). ページテーブルがフレームに対応づけていないページへのアクセスは**ページフォールト**，すなわち制御をオペレーティングシステムに移す例外を引き起こす．

▷ page fault

そのページがディスクにある場合，オペレーティングシステムは 3 つの手順でページフォールトを処理する．

1. そのページのためにフレームを 1 つ選ぶ．空きフレームがなければ，最近使われていないページを追い出す．そのページが読み込まれてから書き込まれていれば，先にディスクに書き戻す．追い出されたページのページテーブルは，そのページがもうメモリにないことを示すように更新する．
2. フォールトしたページをディスクからそのフレームに読み込み，ページをこのフレームに対応づけるようにページテーブルを更新する．
3. フォールトを起こした命令を再実行する．今度はそのページがメモリに見つかる．これには正確な例外（第 8 章）が必要である．

ディスクアクセスにはミリ秒，すなわち数百万クロックサイクルがかかるので，ページフォールトはキャッシュミスよりはるかに高くつく．ページフォールトは稀でなければならず，追い出すページを，オペレーティングシステムがソフトウェアで入念なアルゴリズムを用いて選ぶだけの価値がある．

3 GHz では，10 ms のディスクシークは 3000 万サイクルであり，100 µs の NVMe SSD 読み出しでさえ 30 万サイクルである．

すべてのページフォールトがディスクを伴うわけではない．アドレスが，そのプログラムに割り当てられたどの領域にも属さない場合，読み込むものは何もなく，オペレーティングシステムはそのアクセスを誤りとして扱い，通常はプログラムを終了させる．

ディスクに触れない場合はもう 1 つある．Linux は，ゼロクリアしたフレームを割り当てるだけ，あるいは共有ページを複製するだけで済むフォールトをマイナーフォールトと数え，記憶装置を読むものをメジャーフォールトとする．

注 13.7. 物理メモリは，ディスク上のページのキャッシュとして働く．ページは第 12 章のブロックに，フレームはラインに，ページフォールトはキャッシュミスに，変更されたページだけをディスクに書き戻すことはダーティビットをもつライトバックキャッシュに対応する．どのページもどのフレームにでも置けるので配置はフルアソシアティブであるが，タグを探索する必要はない．ページテーブルはページを添字として直接引けるからである．

13.5 アドレス変換

ページはページサイズの倍数から始まるので，仮想アドレスは，キャッシュアドレスのブロック番号とブロックオフセットとちょうど同じように，2 つのフィールドに分かれる．

出典 Lesson 13，動画 9–10；H&P §B.4．

定義 13.8 (仮想ページ番号とページオフセット). 2^k バイトのページでは，仮想アドレスの下位 k ビットが**ページオフセット**，すなわちそのバイトのページ内での位置を表す．残りの上位ビットが**仮想ページ番号**（VPN）であり，ページを識別する．

▷ page offset
▷ virtual page number

フレームも同じように番号づけられる．物理アドレス $j \times 2^k$ から始まるフレームの**物理フレーム番号**（PFN）は j である．ページテーブルは VPN を PFN に対応づけ，変換はオフセットを保ったまま前者を後者で置き換える（図 13.2）．

▷ physical frame number

$$\begin{aligned} \text{VPN} &= \lfloor VA/2^k \rfloor, & \text{オフセット} &= VA \bmod 2^k, \\ \text{PFN} &= \text{ページテーブル}[\text{VPN}], & PA &= \text{PFN} \times 2^k + \text{オフセット}, \end{aligned} \tag{13.1}$$

ここで VA と PA はそれぞれ仮想アドレスと物理アドレスである．ページとそのフレームは同じ大きさであり，バイトはその中での位置を保つので，オフセットはそのまま通り抜ける．2 進表現では，物理アドレスは PFN のビットの後にオフセットのビットが続いたものである．VPN は仮想アドレスより k ビット少なく，PFN は物理アドレスより k ビット少ないので，仮想アドレスと物理アドレスの幅が異なれば，この 2 つの番号の幅も異なる．そのページに対する PFN がページテーブルにない場合，そのアクセスは代わりにページフォールトを引き起こす．

注意 変換はバイトをそのページの外へ動かさないが，アラインされていないアクセスはページ境界をまたぐことがある．その 2 つの部分は別々のページに属し，2 回の変換を要し，離れたフレームに落ちることもある．

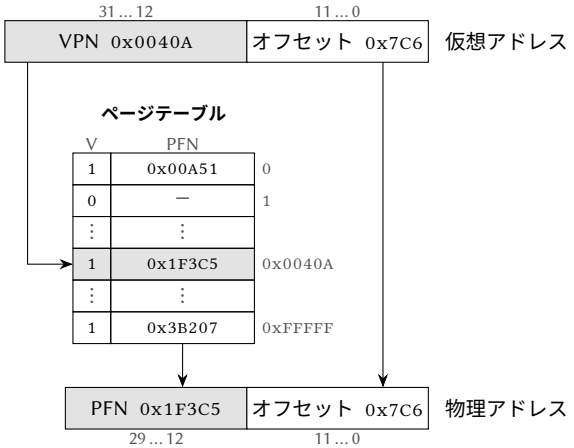


図 13.2: 4 KB のページと 30 ビットの物理アドレスのもとでの仮想アドレス 0x0040A7C6 の変換（例 13.9）．VPN がページテーブルエントリを選び，その PFN が VPN を置き換え，オフセットはそのまま写される．V = 0 のエントリであればページフォールトとなる．

例 13.9. 4 KB のページ（$k = 12$）をもつ 32 ビットプロセッサが 1 GB の物理メモリをもつとすると，物理アドレスは 30 ビットである．ページオフセットはビット 11–0，VPN はビット 31–12（20 ビット）であり，PFN は 18 ビットである．仮想アドレス 0x0040A7C6 の VPN は 0x0040A，オフセットは 0x7C6 である．図 13.2 のページテーブルはページ 0x0040A をフレーム 0x1F3C5 に対応づけるので，物理アドレスは $\text{0x1F3C5} \times 2^{12} + \text{0x7C6} = \text{0x1F3C57C6}$ である．仮想アドレス 0x000017FF はページ 0x00001 に属し，同じページテーブルはこれをメモリにないと記している．このアドレスへのアクセスはページフォールトを引き起こす．

ヒント 4 KB のページで 16 進表記のアドレスを使うと，下 3 桁がオフセットである．変換はその手前の桁を置き換える．

要点
変換は仮想ページ番号を物理フレーム番号に置き換え，ページオフセットをそのまま残す．フレーム番号を与えるのは，オペレーティングシステムが管理するページテーブルである．そのページに対するフレーム番号がなければ，そのアクセスはページフォールトを引き起こす．

13.6 単一レベルページテーブルの大きさ

最も単純なページテーブルは，1 つの配列である．

出典 Lesson 13，動画 11–12；H&P §B.4．

定義 13.10 (単一レベルページテーブル)．**単一レベルページテーブル**とは，仮想ページごとに 1 つのエントリをもち，VPN で直接添字づけされる配列である．あるページに対応するその要素を，そのページの**ページテーブルエントリ**（PTE）という．

▷ flat page table

▷ page table entry

PTE は，そのページの PFN といくつかの状態ビット，すなわちページが物理メモリにあるときだけセットされる有効ビットと，そのページにどのようにアクセスしてよいか（読み出し，書き込み，実行）を記録するビットを保持する．PTE の大きさは典型的にはアドレスと同程度，32 ビットプロセッサで 4 バイト，64 ビットプロセッサで 8 バイトであり，PFN と状態ビットを収めるだけの余地がある．どの仮想ページにもエントリがあるので，

x86-64 の PTE は 8 バイトで，PFN のほかに存在ビット，読み書き，ユーザ/スーパバイザ，アクセスビット，ダーティビット，そして最上位の実行禁止ビットをもつ．

$$\text{単一レベルページテーブルの大きさ} = \frac{\text{仮想アドレス空間の大きさ}}{\text{ページサイズ}} \times \text{PTE の大きさ}. \tag{13.2}$$

式 (13.2) に現れない量が 2 つある．物理メモリの大きさは，エントリのうち PFN が占めるビット数に影響するだけである．プログラムが実際に使うメモリの量はまったく関係しない．使われないページにもエントリがあるからである．走っているプログラムはどれも，この大きさのページテーブルを必要とする．

ヒント 指数で考えるとよい．n ビットのアドレス空間，2^k バイトのページ，2^s バイトのエントリなら，テーブルは 2^{n-k+s} バイトである．ページのビットを引き，エントリのビットを足す．

例 13.11．32 ビットの仮想アドレス，4 KB のページ，4 バイトのエントリでは，単一レベルページテーブルは $2^{32}/2^{12} = 2^{20}$ 個のエントリをもち，走っているプログラム 1 つにつき $2^{20} \times 4$ バイト = 4 MB を占める．数キロバイトしか使わないプログラムでも同じである．64 ビットの仮想アドレス，4 KB のページ，8 バイトのエントリなら 2^{52} 個のエントリをもち，2^{55} バイト（32 PB），すなわちどのメモリにもディスクにも収まらない量を占めることになる．64 ビットのアドレスのうち 48 ビットしか使わない場合でも，単一レベルページテーブルは $2^{36} \times 8 = 2^{39}$ バイト，すなわちプログラム 1 つにつき 512 GB を占める．

13.7 多段ページテーブル

単一レベルページテーブルが大きいのは，プログラムがアドレス空間の下端と上端しか使わないにもかかわらず，空間全体に対するエントリをもつからである．多段ページテーブルは，その間の使われない範囲に対するエントリを保持しないようにする．

出典 Lesson 13，動画 13–17；H&P §B.4–B.5．

定義 13.12 (多段ページテーブル)．**多段ページテーブル**とは，小さなテーブルからなる木である．VPN は段ごとに 1 つのフィールドに分割される．最初のフィールドは外側ページテーブルに添字を与え，そのエントリは次の段のテーブルを指す．最後の段のテーブル，すなわち内側ページテーブルのエントリが PFN を保持する．担当するアドレス空間の部分がまったく使われていないエントリは無効と記され，その下にはテーブルが存在しない．

▷ multi-level page table

2 段の場合，仮想アドレスは外側の添字 p_1，内側の添字 p_2，ページオフセットに分かれ（図 13.3），変換は 2 つの手順を踏む．

1. 外側ページテーブルのエントリ p_1 が，内側ページテーブルの位置を与える．
2. その内側ページテーブルのエントリ p_2 が PFN を与え，これが式 (13.1) のようにオフセットと組み合わされる．

どちらかのエントリが無効なら PFN は得られず，そのアクセスはページフォールトを引き起こす．テーブルの大きさは，通常，1 つがちょうど 1 ページに収まるように定められる．

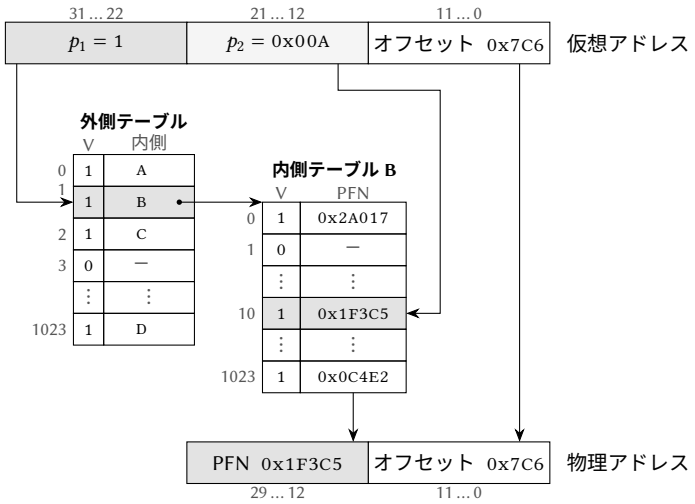


図 13.3: 仮想アドレス 0x0040A7C6 の 2 段の変換（例 13.13）．外側のエントリ 1 は内側テーブル B を指す．A，C，D のエントリは図示していない内側テーブルを指す．エントリ 3 のような無効な外側エントリには，内側テーブルがそもそも存在しない．

例 13.13. 例 13.9 の 20 ビットの VPN を，10 ビットの外側の添字（ビット 31–22）と 10 ビットの内側の添字（ビット 21–12）に分割する．4 バイトのエントリなら，どのテーブルも 2^{10} 個のエントリをもち 4 KB，すなわちちょうど 1 ページを占める．仮想アドレス 0x0040A7C6 の VPN は $0x0040A = 0000000001\,0000001010_2$ なので，$p_1 = 1$，$p_2 = 0x00A = 10$ である．外側のエントリ 1 は内側テーブル B を指し，B のエントリ 10 が PFN 0x1F3C5 を保持する．物理アドレスは単一レベルページテーブルの場合と同じ 0x1F3C57C6 である．多段ページテーブルは，同じ対応づけを別の形で格納しているにすぎない．

13.7.1 多段ページテーブルの大きさ

外側ページテーブルは常に存在するが，内側ページテーブルが存在するのは，その担当するアドレス空間の部分で少なくとも 1 つのページが使われている場合だけである．例 13.13 の構成では，内側テーブル 1 つが 2^{10} ページ，すなわち仮想アドレス空間の 4 MB を担当する．

例 13.14. あるプログラムが，32 ビットのアドレス空間の下端の 10 MB をコード，データ，ヒープに（アドレス 0x00000000–0x009FFFFF），上端の 1 MB をスタックに（0xFFF00000–0xFFFFFFFF）使うとする．例 13.13 の 2 段ページテーブルでは，下端の 10 MB はそれぞれ 4 MB を担当する外側エントリ 0，1，2 の範囲に，スタックは外側エントリ 1023 の範囲に入る．ページテーブルは外側テーブル 1 つと内側テーブル 4 つからなり，単一レベルページテーブルの 4 MB ではなく 5 × 4 KB = 20 KB である．

2 段ページテーブルが単一レベルのものより大きくなるのは，すべての外側エントリに内側テーブルが必要な場合，すなわち 4 MB の 1024 個の範囲のそれぞれで少なくとも 1 つのページが使われている場合だけである．そのとき 4 KB+1024×4 KB ≈ 4.004 MB を占め，単一レベルのテーブルより外側テーブルの分だけ大きくなる．実際のプログラムはアドレス空間の連続したいくつかの領域を使うので，必要な内側テーブルはわずかである．[2]

13.7.2 64 ビットアドレスのためのより多くの段

64 ビットのアドレス空間には 2 段では足りない．8 バイトのエントリからなる 4 KB のテーブルは 2^9 個のエントリをもつので，1 ページに収まるテーブルから

[2] 同じ問題への別の答えが逆ページテーブルである．ページごとではなくフレームごとに 1 エントリをもち，VPN のハッシュで引く．大きさは物理メモリで決まりアドレス空間とともに増えることはないが，衝突すると連鎖をたどる必要があり，ページの共有にも別の仕掛けがいる．PowerPC や PA-RISC が用いた．H&P §B.4 がこの考え方を述べている．

なる各段が消費できるのは VPN の 9 ビットだけである．48 ビットの仮想アドレスでは VPN は 36 ビットである．9 ビットの内側テーブルをもつ 2 段ページテーブルなら 27 ビットの外側の添字が必要で，外側テーブルはプログラム 1 つにつき $2^{27} \times 8$ バイト = 1 GB になる．代わりに 36 ビットを 9 ビットずつ 4 つのフィールドに分割すると，どのテーブルも 1 ページに収まる 4 段ページテーブルが得られる．x86-64 プロセッサはこの構成を用いる．段を 1 つ増やすごとに，仮想アドレスをさらに 9 ビット分収められるようになる．

x86-64 は 4 つの段を PML4，PDPT，PD，PT と呼ぶ．Ice Lake 以降のプロセッサは，57 ビットすなわち 128 PB のアドレスのために 5 段目（CR4.LA57）を備える．

例 13.15. このような 4 段ページテーブルでは，最後の段のテーブル 1 つが 2^9 ページ，すなわちアドレス空間の 2 MB を担当し，第 3 段のテーブルはその 2^9 倍の 1 GB を，第 2 段のテーブルは 512 GB を，そして第 1 段のただ 1 つのテーブルは 2^{48} バイト（256 TB）すべてを担当する．アドレス空間の下端で 8 MB，上端で 8 MB を使うプログラムは，それぞれの端について最後の段のテーブル 4 つと第 3 段および第 2 段のテーブルを 1 つずつ必要とし，それらはすべて第 1 段のただ 1 つのテーブルの下にぶら下がる．合計で $1 + 2 \times (4 + 1 + 1) = 13$ 個のテーブル，すなわち 13×4 KB = 52 KB である．単一レベルページテーブルなら 512 GB を占めることになる（例 13.11）．

> **要点**
> ページテーブルは仮想アドレス空間全体を記述しなければならないが，多段ページテーブルが記憶領域を必要とするのは，使われている部分についてだけである．その代償は，変換のたびに段の数だけテーブルを読むことである．

13.8 ページサイズの選択

出典 Lesson 13，動画 18；H&P §B.4.

ページをどれだけ大きくすべきかは，2 つの効果の折り合いで決まる．

- **ページが大きい**ほどページの数は少なくなる．PTE が減ってページテーブルが小さくなり，さらに 13.10 節 で示すように，プロセッサが保持する変換 1 つが覆うメモリが増える．
- **ページが小さい**ほど，部分的にしか使われないページで無駄になるメモリが減る．

1 つのページと 1 つのフレームのバイト数である**ページサイズ**は，この両者の均衡をとらなければならない．

▷ page size

定義 13.16 (内部断片化). メモリはページ単位で対応づけられる．したがって，大きさがページサイズの倍数でない領域は，最後のページの一部を使わないまま残し，その部分は他のどのプログラムも使えない．このようにして失われるメモリを**内部断片化**という．

▷ internal fragmentation

大きさが任意の領域は平均して半ページを使わないまま残すので，内部断片化はページサイズに比例して増える．

注意 ページングは外部断片化，すなわち大きさの異なる割り当ての間に残る無駄を取り除く．どのページもどのフレームにも収まるからである．残るのは最後のページの内側，内部断片化の方である．

例 13.17. 4 バイトの PTE をもつ 32 ビットプロセッサ上のあるプログラムが，3 つの領域（コードとデータ，ヒープ，スタック）をもち，そのそれぞれが部分的にしか使われないページで終わっているとする．表 13.1 は 3 つのページサイズを比較したものである．4 KB のページから 4 MB のページにすると，単一レベルページテーブルは 2^{10} 分の 1，すなわち 4 MB から 4 KB に縮むが，内部断片化の期待値は同じ倍率で，6 KB から 6 MB に増える．

表 13.1: 例 13.17 のプログラムについての，ページテーブルの大きさと内部断片化の期待値（領域当たり半ページ）．

ページサイズ	ページ数	単一レベルページテーブル	内部断片化
4 KB	2^{20}	4 MB	3×2 KB = 6 KB
64 KB	2^{16}	256 KB	3×32 KB = 96 KB
4 MB	2^{10}	4 KB	3×2 MB = 6 MB

ページはキャッシュブロック（第 12 章）よりはるかに大きい．ページごとに PTE が必要であり，ディスクから持ち込むページごとにディスクアクセスがかか

るからである．通常のページサイズは 4 KB である．プロセッサは，はるかに大きなページも備えている．オペレーティングシステムは，プログラムが完全に埋める大きな領域にこれを使うことができ，その場合，内部断片化はほとんど問題にならない．

x86 では 1985 年の 386 以来 4 KB がページサイズである．AArch64 では，オペレーティングシステムが 4，16，64 KB の粒度から選べる．x86-64 は 2 MB と 1 GB のページも備える．

13.9 変換を伴うメモリアクセス時間

出典 Lesson 13，動画 19–21；H&P §B.4．

プログラムが行うメモリアクセスは，命令フェッチであれロードやストアであれ，すべて仮想アドレスを使うので，物理メモリにアクセスする前に変換されなければならない．ページテーブル自体もメモリに置かれているので，変換はメモリ読み出しからなる．2 段ページテーブルの場合，プロセッサは `LW R1,4(R2)` のようなロードを 4 つの手順で実行する．

1. R2 に 4 を加えて仮想アドレスを求め，その VPN を取り出す．
2. 外側の PTE の物理アドレスを（外側テーブルのアドレスに p_1 と PTE の大きさの積を加えて）計算し，その PTE を読む．
3. 同じようにして，外側の PTE にあるテーブルのアドレスと p_2 から内側の PTE のアドレスを計算し，その PTE を読む．
4. PFN にオフセットを付け加え，この物理アドレスにあるデータにキャッシュを通じてアクセスする．

PTE の読み出しはいずれも通常のメモリアクセスであり，キャッシュを通り，ヒットすることもミスすることもある．PTE がキャッシュに保持されていなければ，1 回ごとに主記憶への完全なアクセスがかかる．L 段のページテーブルでは，プログラムのアクセス 1 回が $L+1$ 回のメモリアクセスになる．PTE の読み出しがデータアクセスと同じ平均メモリアクセス時間（AMAT）をもつとすれば，

$$\text{変換を伴うアクセス時間} \approx (L+1)\times \text{AMAT}. \tag{13.3}$$

x86 プロセッサは通常のキャッシュとは別に，上位段のエントリを専用のページング構造キャッシュに保持する．そのためウォークはたいてい最下段の読み出しだけで済む．Intel SDM 第 3 巻．

例 13.18. あるキャッシュのヒット時間が 2 サイクル，ミス率が 5 %，ミスペナルティが 40 サイクルなら，$\text{AMAT} = 2 + 0.05\times 40 = 4$ サイクルである（式 (12.1)）．変換がなければメモリアクセスは平均 4 サイクルだが，単一レベルページテーブルでは $2\times 4 = 8$ サイクル，4 段ページテーブルでは $5\times 4 = 20$ サイクルになる．個々のアクセスのばらつきは大きい．上位 3 段の PTE がヒットし（3×2 サイクル），最後の段の PTE がミスし（$2 + 40$ サイクル），データがヒットする（2 サイクル）ロードは 50 サイクルかかり，これは変換がなければキャッシュヒットで済んだ時間の 25 倍である．

上位の段のわずかなエントリはどの変換でも読まれるので，たいていキャッシュに保持される．数の多い最後の段のエントリはより頻繁にミスする．いずれにせよ，より速く変換する方法がなければ，仮想記憶はすべての命令フェッチ，ロード，ストアを大きな倍率で遅くしてしまう．

13.10 TLB

出典 Lesson 13，動画 22–24；H&P §B.4．

▷ translation lookaside buffer

あるページにアクセスしたプログラムは，近いうちに同じページに再びアクセスする可能性が高いので，ある時点で必要な変換はわずかであり，それが何度も繰り返し使われる．そこでプロセッサは，それをキャッシュする．

定義 13.19 (TLB). **TLB** とは，最近の変換を保持する小容量で高速なキャッシュである．各エントリは，有効ビット，VPN から取ったタグ（フルアソシアティブの TLB では VPN 全体），そのページが対応づけられている PFN，およびその PTE のアクセス権ビットを保持する．アドレスの VPN が有効なエントリと一致すれば，PFN はそのエントリから得られ，ページテーブルは読まれない．

TLB は第 12 章の意味でのキャッシュであるが，データキャッシュよりはるかに小さくできる．理由は 3 つある．

- 格納するのは変換だけでデータではない．1 エントリは数バイトで済む．
- 1 つのエントリが，1 つのキャッシュブロックではなく 1 ページ全体に対応するので，わずかなエントリでプログラムが現に使っているメモリを覆える．

ヒント TLB が一度に覆えるメモリをそのリーチという．エントリ数 × ページサイズであり，4 KB の 64 エントリなら 256 KB，同じ 64 エントリでも 2 MB のページなら 128 MB になる．

- 格納するのは変換の最終結果であって，個々の段のエントリではない．ページテーブルが何段であろうと，TLB ヒットは 1 回の探索で済む．

例 13.20．64 バイトのブロックをもつ 128 KB のデータキャッシュには 2048 本のラインがある．4 KB のページなら，128/4 = 32 個の TLB エントリでキャッシュが保持するのと同じだけのメモリを覆える．キャッシュされたブロックがすべて別々のページにある場合には 2048 個が必要になるが，空間的局所性により実際に必要な数はこの上限をはるかに下回る．

13.10.1 TLB ミス

▷ TLB miss

アドレスの VPN に一致する有効な TLB エントリがないとき，**TLB ミス**が生じる．このとき変換はページテーブルから得なければならない．

▷ page table walk

定義 13.21 (ページテーブルウォーク)．**ページテーブルウォーク**とは，13.7 節で述べたように，あるアドレスのページテーブルエントリを，外側ページテーブルから PFN を保持するエントリまで段ごとに読むことである．

TLB ミスの後，ウォークが PFN を見つけ，その変換が TLB に書き込まれ（TLB が満杯なら既存のエントリを置き換え），アクセスが進む（図 13.4）．

注意 TLB ミスはページフォールトではない．ウォークはたいてい有効な PTE を見つける．

TLB はページテーブルと一致していなければならない．オペレーティングシステムが対応づけを変更したとき，例えばページを追い出したときには，そのページの TLB エントリを無効化する．また，プロセッサが別のプログラムに切り替わるときには，前のプログラムのエントリを無効化するか，プログラムの識別子をタグとして付けておく．[3]

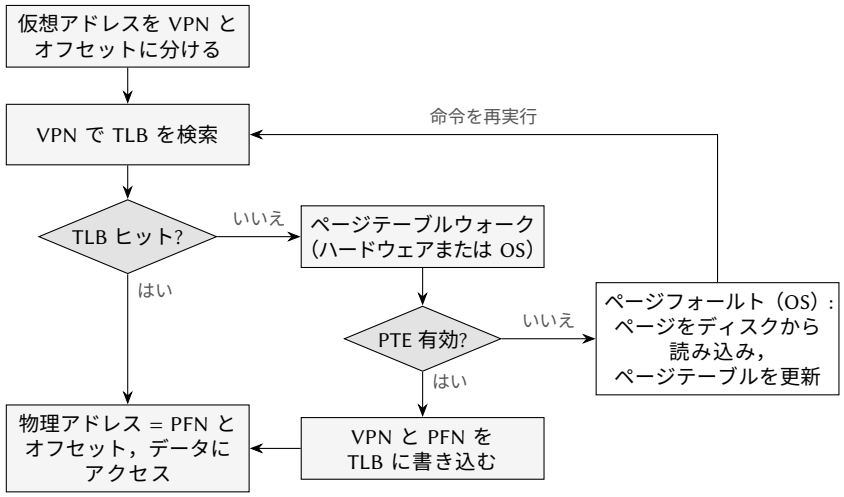


図 13.4: メモリアクセスにおける変換．TLB ヒットならただちに PFN が得られ，TLB ミスならページテーブルウォークが必要になる．PTE が無効であればページフォールトとなり，ページがメモリに入った後に命令が再実行される．

ウォークは 2 通りの方法で行える．

ソフトウェアによる方法 TLB ミスが例外を起こし，オペレーティングシステム内のハンドラがページテーブルをたどって変換を TLB に書き込む．ハードウェアは単純で済み，ページテーブルをどのように構成してもよいが，TLB ミスのたびに例外の代償を払う．MIPS プロセッサは TLB ミスをこの方法で処理する．

ハードウェアによる方法 プロセッサがページテーブルウォーカ，すなわち PTE を読んで自ら TLB を埋める論理回路を内蔵する．TLB ミスの処理ははるかに速いが，ページテーブルはハードウェアが想定する形式でなければならない．x86 系のような高性能プロセッサは，ページテーブルウォークをハードウェアで行う．

[3] このタグを x86 では PCID，Arm では ASID という．これがあればコンテキストスイッチで TLB を捨てずに済む．難しいのは 1 つの対応づけを無効化する場合である．マルチコアプロセッサでは，そのエントリを保持しうるすべてのコアがそれを捨てなければならないので，オペレーティングシステムはプロセッサ間割り込みを送り，その応答を待つ．この *TLB シュートダウン* のために，メモリの対応づけを解除する費用は対応づける費用よりはるかに大きい．

要点
どのメモリアクセスにも変換が必要である．TLB は，よくある場合である TLB ヒットを 1 回の高速な探索で済ませる．ページテーブルウォークは，ハードウェアによるものであれオペレーティングシステムによるものであれ TLB ミスのときだけ必要であり，ページフォールトハンドラは，ウォークが PFN を見つけられなかったときだけ必要である．

13.11 TLB の構成と性能

出典 Lesson 13，動画 25–26；H&P §B.4．

TLB の構成は，その役割から導かれる．

連想度 TLB ミスは 1 回ごとに数回のメモリ読み出しからなるウォークを要するので，TLB はフルアソシアティブか，非常に連想度の高いセットアソシアティブとする．これだけ少ないエントリ数であれば比較器の費用は許容でき，変換どうしがエントリを奪い合うこともほとんどない．

サイズ TLB のヒット率はキャッシュと同程度であるべきなので，キャッシュが保持するデータのページを覆えなければならない（例 13.20）．これには 64 個から 512 個ほどのエントリが必要である．すべてのアクセスで参照されるので，L1 キャッシュのヒットと同程度に高速でもなければならず，そのためこれ以上大きくできない．

Intel の最適化マニュアルによれば Skylake は，4 KB ページ用の第 1 レベルデータ TLB が 64 エントリ，命令用が 128 エントリ，その後ろに 1536 エントリ 12 ウェイの第 2 レベル TLB をもつ．

階層 それで十分なメモリを覆えない場合は，キャッシュを階層化するのと同じように（第 14 章），数百から数千のエントリをもつ，より大きく低速な第 2 レベルの TLB を第 1 レベルの後ろに置く．これは第 1 レベルでミスしたときにだけ参照され，両方の TLB がミスしたときにだけウォークが必要になる．

例 13.22. 例 13.18 のプロセッサ（どのメモリ読み出しも AMAT は 4 サイクル，4 段ページテーブル）に，1 サイクルで参照でき，アクセスの 1 % でミスする TLB を与える．どのアクセスも探索とデータアクセスの代償を払い，100 回に 1 回は $4 \times 4 = 16$ サイクルのウォークの代償も払うので，アクセス 1 回は平均して 20 サイクルではなく $1 + 0.01 \times 16 + 4 = 5.16$ サイクルかかる．変換の費用は 16 サイクルではなく 1.16 サイクルとなり，その大半は探索の分である．第 14 章では，この探索をキャッシュの探索と重ね合わせる．

この式は 式 (12.1) と同じ形をしている．探索時間 + ミス率 × ペナルティであり，ウォークがペナルティにあたる．TLB はキャッシュであり，同じように評価する．

要点 — 章のまとめ
仮想記憶は，各プログラムの仮想アドレス空間のページを物理メモリのフレームに対応づけ，残りをディスクに置く．フレームのないページへのアクセスはページフォールトである．変換は，仮想ページ番号を，ページテーブルから得たフレーム番号で置き換え，オフセットはそのまま残す（式 (13.1)）．多段ページテーブルは，単一レベルのテーブルが使われないアドレス空間に費やすエントリを不要にする（式 (13.2)）．ページサイズはテーブルの大きさと内部断片化のトレードオフを決める．どの段もアクセス 1 回につきメモリ読み出しを 1 回加えるので，小さく連想度の高い TLB が変換をキャッシュし，ページテーブルウォークは TLB ミスのときだけ行われる．

第 14 章

高度なキャッシュ

講義動画	Lesson 14，動画 1–40
教科書	H&P [1] §2.3，§B.1，§B.3
キーワード	パイプライン化キャッシュ，VIPT キャッシュ，エイリアシング，ウェイ予測，NMRU，疑似 LRU，競合性ミス，プリフェッチ，ループ交換，ミスアンダーミス対応，MSHR，キャッシュ階層，大域ミス率，包含性

第 12 章では基本的なキャッシュを述べ，平均メモリアクセス時間でそれを測った．第 13 章では，仮想記憶をもつシステムでどのメモリアクセスにも必要となるアドレス変換を加えた．この章では，実際のキャッシュを高速にする技法をまとめる．技法は，平均メモリアクセス時間のどの項を下げるかによって整理する．すなわち，ヒット時間（パイプライン化キャッシュ，TLB に対するキャッシュの置き方，ウェイ予測，安価な置換アルゴリズム），ミス率（ブロックの大型化，プリフェッチ，ループ交換），そしてミスペナルティ（重なったミスの処理と，何よりも多段のキャッシュ階層）である．

14.1 キャッシュ性能の改善

出典 Lesson 14，動画 1–2；H&P §2.3．

改善すべき量は，第 12 章の平均メモリアクセス時間（AMAT）である．

$$\text{AMAT} = \text{ヒット時間} + \text{ミス率} \times \text{ミスペナルティ}. \tag{14.1}$$

3 つの項のそれぞれに固有の攻め方があり，表 14.1 にはこの章の技法を，対象とする項ごとに挙げる．代償を伴わないものはほとんどない．一方の項を下げる技法は，しばしば別の項を上げる—キャッシュを小さくすればヒット時間は短くなるがミス率は高くなる—ので，AMAT 全体が減る場合にのみ価値がある．

表 14.1: この章の技法を，式 (14.1) のどの項を下げるかで分類したもの．

項	技法	節
ヒット時間	パイプライン化キャッシュ，VIPT キャッシュ，ウェイ予測，NMRU 置換と PLRU 置換	14.2–14.6
ミス率	ブロックの大型化，プリフェッチ，ループ交換	14.7–14.10
ミスペナルティ	ミスアンダーミス対応，キャッシュ階層	14.11–14.14

14.2 パイプライン化キャッシュ

ヒット時間を下げる最も直接的な方法は，キャッシュを小さく単純に—たとえばダイレクトマップに—保つことである．デコードすべきラインが少なく，比較すべきタグも少なくて済む．しかし小さなキャッシュはミスが多く，ダイレクトマップキャッシュは競合を多く受けるので，ヒットで節約した時間はミスで容易に失われる．この節と続く 4 つの節の技法は，ミス率を犠牲にすることなく，ヒット時間またはその代償を減らす．

出典 Lesson 14，動画 3–4；H&P §2.3．

第 12 章で述べたように，L1 のヒットには 1–3 クロックサイクルかかる．数サイクルを要する探索が，完了するまでキャッシュを占有するとすれば，どのメモリアクセスも直前のアクセスを待たなければならない．アクセスから見たヒット時間は自身の探索時間にこの待ち時間を加えたものとなり，連続するロードとストアはプロセッサを止めてしまう．探索が 1 サイクルに収まるキャッシュにはこの問題はない．2–3 サイクルかかる L1 キャッシュは，通常パイプライン化される．

注意 ベンダが公表する L1 のレイテンシは，配列アクセス単体ではなく，ロードから値が使えるまで（load-to-use）の値である．Skylake は単純なアドレスで 4 サイクル，インデックス付きで 5 サイクルとする．Intel 最適化マニュアル．

定義 14.1 (パイプライン化キャッシュ). **パイプライン化キャッシュ**とは，探索を

▷ pipelined cache

ラッチで区切られた段に分割し，1 回の探索には数クロックサイクルかかるが，毎サイクル新しい探索を開始できるようにしたキャッシュである．

セットアソシアティブキャッシュでは，探索は自然に 3 つの手順に分かれ，そのそれぞれが段になりうる（図 14.1）．

1. インデックスがセットを選び，そのセットのラインの有効ビットとタグを読み出す．
2. タグをアドレスのタグと比較して，ヒットかミスかを決めるとともに一致したウェイを特定し，データの読み出しを開始する．
3. データの読み出しが完了し，一致したウェイのデータが選択され，ブロックオフセットがブロックから要求されたバイトを取り出して引き渡す．

ミスは第 2 段の終わりに判明する．パイプライン化は，1 回の探索にかかるサイクル数を減らすものではない．そのサイクル数がアクセスを発行できる頻度を制限しないようにするものである．あるアクセスがタグを比較している間に，次のアクセスはすでに自身のセットを読み出しており，これは第 3 章のパイプラインで命令が重なるのとまったく同じである．そしてクロック周期は，探索全体ではなく最も遅い段だけを収めればよい．

タグの確認と並行してすべてのウェイのデータを読むことが，探索を短く保っている．大きな L2 や L3 はまずタグを読み，一致したウェイのデータだけを後から読む．遅いが消費電力ははるかに小さい．

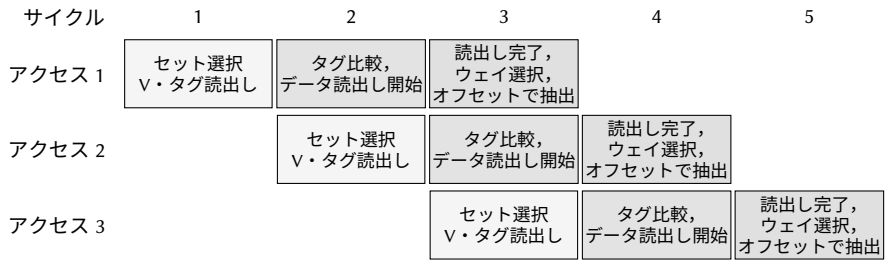


図 14.1: 探索を 3 段に分割したキャッシュへの，連続する 3 回のアクセス．各アクセスには 3 サイクルかかるが，新しいアクセスが毎サイクル開始される．

例 14.2. ある L1 キャッシュの探索論理の遅延は 0.75 ns であり，プロセッサの他の段は 0.3 ns のクロック周期を許す．パイプライン化しなければ，プロセッサ全体のクロック周期が 0.75 ns 以上に伸びるか，各アクセスが 3 サイクルにわたってキャッシュを占有してメモリアクセスが 3 サイクルに 1 回しか開始できなくなるかである．ラッチのオーバーヘッドの余地を残して約 0.25 ns の 3 段に分割すれば，探索は 0.3 ns のクロックに収まる．データはアクセス開始の 3 サイクル後に到着し，アクセスは毎サイクル開始できる．

パイプライン化が与えるのは毎サイクル 1 アクセスである．それ以上はポートとバンクによる．Skylake の L1 データキャッシュは 1 サイクルに 2 つのロードと 1 つのストアを処理する．Intel 最適化マニュアル．

出典 Lesson 14，動画 5–7；H&P §B.3.

14.3 アドレス変換とヒット時間

仮想記憶（第 13 章）のもとでは，プロセッサは仮想アドレスを発行するが，主記憶のデータは物理アドレスで位置づけられる．キャッシュの探索は，変換の後に行うことも，前に行うことも，一部を変換と並列に行うこともでき（図 14.2），この選択がヒット時間に影響する．

14.3.1 物理アドレスキャッシュ

物理アドレスで探索されるキャッシュを**物理アドレスキャッシュ**という．どのアクセスもまず TLB で仮想アドレスを変換し，得られた物理アドレスだけが探索に使われる（図 14.2a）．したがってヒット時間には変換が含まれる．

▷ physically accessed cache

$$\text{ヒット時間} = \text{TLB の探索時間} + \text{キャッシュの探索時間}. \tag{14.2}$$

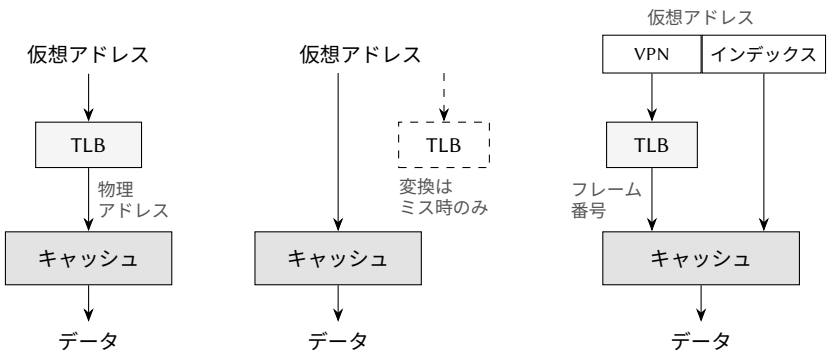
TLB は小さく高速だが，その遅延はヒットを含むすべてのアクセスで支払われ，まさに我々が減らそうとしている量に上乗せされる．

TLB は追いつくために小さくなければならない．Skylake の第 1 レベルデータ TLB は 4 KB ページ用に 64 エントリであり，その後ろに第 2 レベルの 1536 エントリがある．Intel 最適化マニュアル．

14.3.2 仮想アドレスキャッシュ

定義 14.3 (仮想アドレスキャッシュ). **仮想アドレスキャッシュ**は，インデックス

▷ virtually accessed cache



(a) 物理アドレスキャッシュ　(b) 仮想アドレスキャッシュ　(c) VIPT

図 14.2: TLB とキャッシュ探索を組み合わせる 3 つの方法．(b) では変換はミス時にだけ必要なので，TLB はヒットの経路に入らない（権限の確認のためには依然として参照される）．(c) では，インデックスがすでにセットを選んでいる間に TLB が仮想ページ番号を変換する．

もタグも仮想アドレスから取る．アドレスが変換されるのは，ブロックを主記憶から取得しなければならないミスのときだけである．

ヒット時には TLB は経路に入らず，ヒット時間はキャッシュの探索時間そのものである（図 14.2b）．この利点には問題が伴う．

- **権限．**ページの TLB エントリは，そのページを読めるか，書けるか，実行できるかも記録しており，権限はすべてのアクセスで確認しなければならない．そのため実際には，ヒット時にも TLB が参照される．
- **コンテキストスイッチ．**各プロセスは自分の仮想アドレス空間をもつので，同じ仮想アドレスが異なるプロセスでは異なるデータを指す．オペレーティングシステムがプロセッサをあるプロセスから別のプロセスへ切り替える（コンテキストスイッチ）と，キャッシュ内のブロックは古いプロセスの仮想アドレスで識別されており，新しいプロセスがそれらを見つけてはならない．すべての有効ビットを落としてキャッシュをフラッシュするか，すべてのタグにアドレス空間の識別子を付け加えるかである（H&P §B.3）．コンテキストスイッチは毎秒何度も起こり，フラッシュする方式では，次に走るプロセスが毎回，空のキャッシュとミスの多発から始めることになる．
- **エイリアシング．**これは 14.4 節 の主題である．

14.3.3 仮想インデックス物理タグキャッシュ

両者を組み合わせると，短いヒット時間を保ちながらフラッシュを避けられる．

定義 14.4 (VIPT キャッシュ)．**仮想インデックス物理タグキャッシュ**（VIPT キャッシュ）は，インデックスを仮想アドレスから，タグを物理アドレスから取る．

▷ virtually indexed physically tagged cache

仮想アドレスのインデックスビットが直ちにセットを選ぶ（図 14.3）．そのセットの有効ビットとタグが読み出されている間に，TLB が仮想ページ番号を物理フレーム番号に変換し，物理的な値であるセットのタグが TLB からのフレーム番号と比較される．

- ヒット時間は本質的にキャッシュの探索時間である．TLB の探索はセットの選択と並列に走り，タグが比較される時点までにフレーム番号を届ければよく，小さな TLB なら通常これができる．
- コンテキストスイッチでキャッシュをフラッシュする必要がない．タグはブロックの由来する物理メモリを表しており，どのプロセスが走っていてもそれは正しいままである．
- いずれにせよ TLB はすべてのアクセスで参照されるので，権限の確認に追加の費用はかからない．

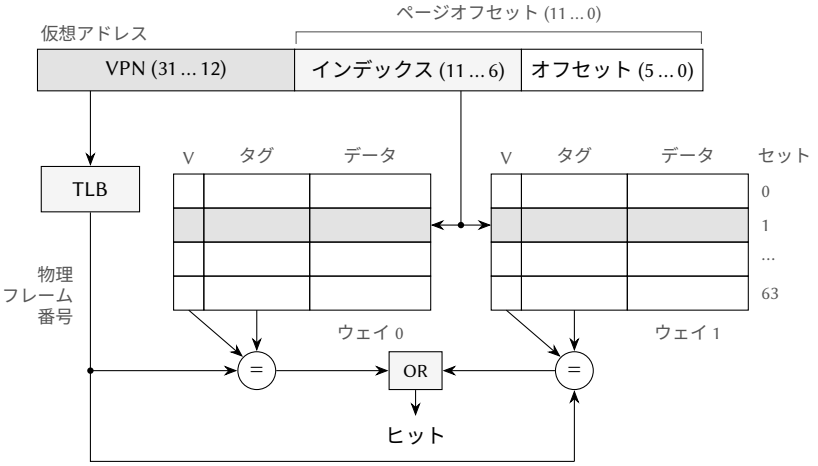


図 14.3: 4 KB のページ，64 バイトのブロック，64 セット（8 KB）をもつ 2 ウェイ VIPT キャッシュにおける探索．インデックスは仮想アドレスのページオフセットから取られる．タグは物理フレーム番号であり，TLB の出力と比較される．ラインは，有効ビットが立っていてタグが一致すればヒットする．

14.4 エイリアシング

仮想記憶では，同じ物理メモリが複数の仮想アドレスに現れうる．

出典 Lesson 14，動画 8–11；H&P §B.3．

定義 14.5 (エイリアシング). 同じアドレス空間内の，あるいは異なるアドレス空間の 2 つ以上の仮想アドレスが同じ物理アドレスに対応づけられる状況を**エイリアシング**という．このとき，それらの仮想アドレスは互いにエイリアスである．

▷ aliasing

エイリアシングはよく起こる．メモリを共有するプロセスは，通常は異なる仮想アドレスで同じフレームを見るし，1 つのプロセスが 1 つのフレームを 2 つのアドレスに対応づけることもある．仮想アドレスキャッシュでは，2 つのエイリアスは単に 2 つの異なるアドレスである．一般にインデックスもタグも異なるので，キャッシュは同じ物理ブロックの写しをエイリアスごとに 1 つずつ保持でき，写し同士を結びつけるものは何もない．

例 14.6. 仮想アドレス V1 と V2 は物理アドレス P のエイリアスであり，仮想アドレスキャッシュでは両者のインデックスが異なる．P のメモリには値 5 が入っている．表 14.2 は 4 回のアクセスを追う．最後のアクセスは V2 を通して 5 を読むが，同じメモリには V1 を通して 7 が書き込まれている．ライトスルー方式であれば主記憶は 7 を保持するが，それでも V2 用の古い写しが返される．

表 14.2: ライトバック方式の仮想アドレスキャッシュにおける 2 つのエイリアス（例 14.6）．各アクセスの後の，2 つの写しと主記憶の内容．

手順	アクセス	結果	V1 の写し	V2 の写し	P のメモリ
1	V1 を読む	ミス	5	—	5
2	V2 を読む	ミス	5	5	5
3	V1 に 7 を書く	ヒット	7	5	5
4	V2 を読む	ヒット，5 を返す	7	5	5

正しく動作するためには，どの書き込みも同じ物理ブロックのキャッシュ内の他のすべての写しを見つけ出し，更新するか無効化しなければならないが，キャッシュにはそれらを効率的に探す手段がない．したがってエイリアシングは仮想アドレスキャッシュのもう 1 つの問題である．[1]

14.4.1 VIPT キャッシュにおけるエイリアシング

VIPT キャッシュではタグが物理的なので，ブロックの 2 つの写しは同じタグをもつことになる．それでも両者は異なるセットに入りうる．インデックスが仮想アドレスから取られるからである．それが起こりうるかどうかは，インデックス

[1] オペレーティングシステムの側で 2 つ目の写しを防ぐこともできる．ページカラーリングでは，エイリアスをインデックスビットの一致する仮想アドレスにしか割り当てないので，エイリアスは同じセットを選ぶ．代償は，どのフレームをページに与えるかの自由度である（H&P §B.3）．

ビットがどこにあるかで決まる．変換は仮想ページ番号をフレーム番号に置き換え，ページオフセットはそのまま残す．したがってエイリアスは，仮想ページ番号こそ異なるが，ページオフセットは常に等しい．

- インデックスが仮想ページ番号のビットを含む場合，2 つのエイリアスは異なるセットを選びうるので，キャッシュは 2 つの写しを保持でき，例 14.6 の問題が残る．
- インデックスがページオフセットのビットだけからなる場合，エイリアスは常に同じセットを選ぶ．そこには自分のブロックの物理タグをもつラインが 1 つあるだけなので，写しはたかだか 1 つしか存在せず，エイリアシングは無害である．このとき仮想アドレスのインデックスビットは物理アドレスのインデックスビットでもあり，キャッシュは物理アドレスキャッシュとまったく同じに，ただしより高速に振る舞う．

14.4.2 エイリアシングの回避

したがって VIPT キャッシュは，ブロックオフセットとインデックスがページオフセットに収まるとき，エイリアシングから自由である．

$$\text{オフセットビット数} + \text{インデックスビット数} \;\le\; \text{ページオフセットビット数}. \tag{14.3}$$

左辺は $\log_2$(セット数 × ブロックサイズ)，右辺は $\log_2$(ページサイズ) なので，この条件は セット数 × ブロックサイズ ≤ ページサイズ と読める．両辺に連想度 N を掛けると，キャッシュサイズの上界が得られる．

$$\text{キャッシュサイズ} \;\le\; N \times \text{ページサイズ}. \tag{14.4}$$

ブロックサイズは消えた．VIPT キャッシュを制限するのは連想度とページサイズだけである．

ヒント バイトではなくビットで数える．4 KB ページで変換を受けないのはビット 0–11 だけなので，オフセットビット数 + インデックスビット数 ≤ 12 である．

例 14.7. 4 KB のページではページオフセットは 12 ビットである．64 バイトのブロック（オフセット 6 ビット）をもつ VIPT キャッシュは，インデックスを最大 6 ビット，すなわち 64 セットまでしか使えない．2 ウェイならこれは図 14.3 の 8 KB のキャッシュであり，4 ウェイなら 16 KB である．16 KB の 2 ウェイキャッシュは 128 セットを必要とし，その 7 番目のインデックスビットは仮想アドレスのビット 12，すなわち仮想ページ番号の最下位ビットとなるので，エイリアスが異なるセットに置かれうる．

14.4.3 実際の VIPT キャッシュ

ページサイズはアーキテクチャとオペレーティングシステムが定めるもので，典型的には 4 KB である．式 (14.4) により，VIPT であり，かつエイリアシングから自由であろうとする L1 キャッシュは，連想度を上げることによってしか大きくできない．ダイレクトマップなら 4 KB が限界であり，32 KB のキャッシュには 8 ウェイが必要である．Intel の Core 2，Nehalem，Sandy Bridge，Haswell はいずれも 32 KB の 8 ウェイ L1 データキャッシュをもち，ちょうどこの上界にある．したがって実際の L1 キャッシュは，ヒット時間だけでは正当化できないほど連想度が高く，そのことが次の 2 つの節の技法を重要にしている．

ページを大きくすれば上界も上がる．Apple の M シリーズは 16 KB ページを用い，M1 の高性能コアは 128 KB の L1 データキャッシュをもつ．Apple silicon CPU 最適化ガイド．

> **要点**
> VIPT キャッシュは，TLB が並列に変換する間に仮想アドレスでインデックスし，物理タグを比較する．ヒット時間は仮想アドレスキャッシュとほぼ同じで，コンテキストスイッチでのフラッシュを必要とせず，キャッシュサイズ ≤ N × ページサイズ である限りエイリアシングの影響を受けない．

14.5 ウェイ予測

連想度はヒット時間とミス率を逆向きに動かす．セット当たりのウェイが多いほど競合は減ってミス率は下がり—VIPT キャッシュでは許されるサイズも大きくなる（式 (14.4)）—が，探索ではより多くのタグを比較し，より多くのラインから

出典 Lesson 14，動画 12–15；H&P §2.3．

選択しなければならないので，ヒット時間は長くなる．ダイレクトマップキャッシュはヒット時間が最も短く，ミス率が最も高い．目標は，セットアソシアティブキャッシュのミス率を，ダイレクトマップキャッシュのヒット時間で得ることである．

定義 14.8 (ウェイ予測). **ウェイ予測**を行うセットアソシアティブキャッシュは，選ばれたセットのどのラインが要求されたブロックを保持しているかを推測し，まずそのラインだけを調べる．そのタグが一致すれば，アクセスはダイレクトマップキャッシュのヒット時間で完了する．そうでなければ，通常のセットアソシアティブキャッシュと同様にセットの他のラインを調べる．

▷ way prediction

サイズ C の N ウェイキャッシュの最初の確認はセット当たり 1 つのラインを調べるので，同じセットをもつサイズ C/N のダイレクトマップキャッシュのように振る舞い，そのようなキャッシュのヒット率が，推測の当たる頻度を見積もる．推測がセット内で最も最近アクセスされたラインであれば，この見積もりは厳密である．そのラインはセット内で最も最近アクセスされたブロックを保持しており，それはまさにサイズ C/N のダイレクトマップキャッシュが保持するブロックだからである．推測が外れれば，素早い確認に加えて通常の探索の分も費やし，ミスもまた通常の探索の後にしか判明しない．予測されたラインがヒットするアクセスの割合を p，1 つのラインを調べる時間を t_1，セット全体の探索時間を t_N とすると，平均探索時間は次で与えられる．

$$t = pt_1 + (1-p)(t_1 + t_N) = t_1 + (1-p)\,t_N. \tag{14.5}$$

例 14.9. ある L1 キャッシュの後ろには L2 キャッシュがあり，L1 のミスペナルティは 20 ns である．3 つの L1 の設計を比較する．4 KB のダイレクトマップキャッシュは，ウェイ予測が最初に調べる 8 ウェイキャッシュの部分集合なので，予測されたラインは全アクセスの $p = 80\,\%$ でヒットし，残りの 20 %（他のラインでのヒットとすべてのミス）は $0.8 + 1.5 = 2.3$ ns を要する．

L1 キャッシュ	探索時間	ミス率	AMAT
4 KB ダイレクトマップ	0.8 ns	20 %	$0.8 + 0.20 \times 20 = 4.8$ ns
32 KB 8 ウェイ	1.5 ns	5 %	$1.5 + 0.05 \times 20 = 2.5$ ns
32 KB 8 ウェイ，予測あり	0.8 または 2.3 ns	5 %	$1.1 + 0.05 \times 20 = 2.1$ ns

式 (14.5) により，予測ありの平均探索時間は $0.8 + 0.2 \times 1.5 = 1.1$ ns である．ウェイ予測は 8 ウェイキャッシュのミス率を保ったまま，その平均探索時間をダイレクトマップキャッシュのそれに近づける．

14.6 置換アルゴリズムとヒット時間

置換アルゴリズムはミス率だけでなくヒット時間にも影響する．第 12 章のアルゴリズムのうち 2 つが両極端をなす．

出典 Lesson 14，動画 16–19；H&P §B.1．

ランダム置換 ヒット時には何もせず，ミス時に犠牲を無作為に選ぶ．ヒット時間には何も加えないが，これから使われるブロックを追い出してしまうことがあるので，ミス率は高くなる．

LRU はミス率が低いが，各ラインに $\log_2 N$ ビットのカウンタを保持し，ヒットであれミスであれ 1 回のアクセスがセットの N 本すべてのラインのカウンタを変えうる．すべてのアクセスでのこの活動は論理とエネルギーとヒット時間を費やし，連想度が高いほどその度合いは大きい．

以下のアルゴリズムは，はるかに少ない管理情報で LRU のミス率に近づく．

14.6.1 NMRU

定義 14.10 (NMRU). **NMRU** 方式は，各セットについてどのラインが最も最近アクセスされたかだけを記録し，ミス時にはそれ以外のラインの 1 つを無作為に選んで追い出す．

▷ not most recently used

時間的局所性により，最も最近使われたブロックは次にアクセスされる可能性が最も高い．それを追い出すことは，ランダム置換がなしうる最も有害な選択で

あり，NMRU はまさにその選択を除外する．費用はセット当たり $\log_2 N$ ビットのポインタ 1 つであり，1 回のアクセスが変えるのはたかだかこのポインタだけである．8 ウェイキャッシュには，LRU の $8 \times 3 = 24$ ビットではなく，セット当たり 3 ビットで足りる．NMRU は LRU よりいくらか多くミスする．その犠牲が，最も最近使われたものよりわずかに古いだけのブロックでありうるからである．

14.6.2 疑似 LRU

▷ pseudo-LRU

定義 14.11 (PLRU). **疑似 LRU**（PLRU）方式は，ライン当たり 1 ビットを初期値 0 で保持する．アクセスは自分のラインのビットを立てる．これによってセット内でまだ 0 であった最後のビットが立つと，他のすべてのラインのビットが落とされる．ミスは，ビットが 0 のラインを無作為に選んで追い出す．

注意 他の文献で「疑似 LRU」というときは，木構造の変種を指すことが多い．セット当たり $N-1$ ビットが二分木をたどって犠牲を選ぶ．ここでのライン当たり 1 ビットは別の近似である．

ラインのビットは，そのラインが最後のリセット以降にアクセスされたかどうかを示す．最も最近使われたラインのビットは常に立っているので，PLRU の候補は常に NMRU の候補に含まれ，PLRU は NMRU と同等以上である．LRU にどれだけ近いかは，いくつのビットが立っているかによる．リセットの直後には，NMRU と同じく，最も最近使われたラインだけが除外される．$N-1$ 個のビットが立っているときは，唯一の候補は最後のアクセスが最も昔であるラインであり，これはまさに LRU の選択である．費用はセット当たり N ビットで，1 回のアクセスはリセットの場合を除いて 1 ビットを立てるだけである．

例 14.12. ある 4 ウェイのセットが，ライン 0–3 にブロック A，B，C，D を保持している．最も最近使われたのは D であり，PLRU のビットは D へのアクセスで最後にリセットされた．表 14.3 は，3 つのアルゴリズムのもとでのこのセットを追う．E でのミスでは，LRU は A を追い出さなければならず，NMRU は A，C，D のいずれかを，PLRU は A か C を追い出しうる．ここでは 3 つとも A を追い出す．次のアクセス F もミスすれば，LRU は C を追い出し，NMRU は B，C，D のいずれかを追い出しうる．PLRU はビットが 0 のラインがライン 2 だけなので，やはり C を追い出す．

表 14.3: 1 つの 4 ウェイセットについて LRU，NMRU，PLRU が保持する状態（例 14.12）．LRU の順は最も長く使われていないブロックから最も最近使われたブロックまで，NMRU は最も最近使われたラインの記録，PLRU はライン 0–3 のビットである．

アクセス	結果	ライン 0–3	LRU の順	NMRU	PLRU のビット
（初期状態）		A B C D	A B C D	3	0 0 0 1
B	ヒット	A B C D	A C D B	1	0 1 0 1
D	ヒット	A B C D	A C B D	3	0 1 0 1
B	ヒット	A B C D	A C D B	1	0 1 0 1
E	ミス	E B C D	C D B E	0	1 1 0 1

> **要点**
> すべてのアクセスで行われる作業は，すべてのヒットを長くする．ウェイ予測は他のラインより先に 1 つのラインだけを調べ，NMRU と PLRU は LRU のカウンタを，セット当たり 1 つのポインタ（NMRU）またはライン当たり 1 ビット（PLRU）に置き換える．いずれも，連想度や LRU によるミス率の利点の大半を，ヒット時間の代償のわずかな割合で保つ．

14.7 ミスの原因

ミス率を下げるには，まずミスがなぜ起こるかを知ることから始める．ミスは 3 種類に分類され，しばしば 3C と呼ばれる．

出典 Lesson 14，動画 20；H&P §B.3；[28].

定義 14.13 (初期参照ミス，容量性ミス，競合性ミス). **初期参照ミス**とは，あるブロックへの最初のアクセスで生じるミスである．これはどんなキャッシュでも，無限に大きなキャッシュでさえ生じる．

▷ compulsory miss

容量性ミスとは，同じサイズのフルアソシアティブキャッシュでも生じるであ

▷ capacity miss

ろうミスである．キャッシュがプログラムの使っているすべてのブロックを保持できず，場所が足りないためにそのブロックが追い出されていたのである．

競合性ミスとは，連想度が限られているためだけに生じるミスである．同じサイズのフルアソシアティブキャッシュならまだそのブロックを保持しているが，実際のキャッシュでは，同じセットを争う別のブロックによって追い出されていた．

▷ conflict miss

3 つの個数は，同じアクセス列を 3 つのキャッシュでシミュレートすれば測定できる．無限のキャッシュのミスが初期参照ミスであり，実際のサイズのフルアソシアティブキャッシュがそれに容量性ミスを加え，実際のキャッシュがさらに競合性ミスを加える．

マルチプロセッサでは 4 つ目の C が加わる．他のコアの書き込みがブロックを無効化して生じるコヒーレンスミスである（第 19 章）．

例 14.14. あるプログラムについて，無限のキャッシュは 1500 回，32 KB のフルアソシアティブキャッシュは 6000 回，実際の 4 ウェイ 32 KB キャッシュは 7800 回ミスする．7800 回のミスのうち 1500 回が初期参照ミス，$6000 - 1500 = 4500$ 回が容量性ミス，$7800 - 6000 = 1800$ 回が競合性ミスである．

この分類は対処法を示す．キャッシュを大きくすれば容量性ミスが減り，連想度を上げるか置換アルゴリズムを改善すれば競合性ミスが減るが，いずれもヒット時間を長くする．以下の節では，ヒットを長くすることなくミス率を下げる技法を述べる．

14.8 キャッシュブロックの大型化

出典 Lesson 14，動画 21–22；H&P §B.3．

与えられたサイズのキャッシュでは，ブロックサイズを大きくすることが，ミス率に相反する 2 通りの影響を与える．

- 空間的局所性が良い場合，アクセスされたアドレスの周辺のデータはすぐに使われる．ブロックが大きければ 1 回のミスでそれらを多くキャッシュに持ち込め，そうでなければミスしていた後続のアクセスがヒットするようになる．
- 空間的局所性が乏しい場合，大きなブロックの大半は一度も使われない．キャッシュが保持できるブロックの数が減り，したがって実際に必要とされるブロックも減って，容量性ミスと競合性ミスが増える．

そのためブロックサイズを大きくしていくと，ミス率はまず下がり，その後再び上がる（図 14.4）．最小値は，キャッシュが大きいほど大きなブロックサイズに現れる．たとえば 4 KB のキャッシュでは約 64 バイト，256 KB のキャッシュでは約 256 バイトである．大きなキャッシュはブロックが大きくても多くのブロックを収める余地があるので，使われないデータに費やす場所の害が小さい．

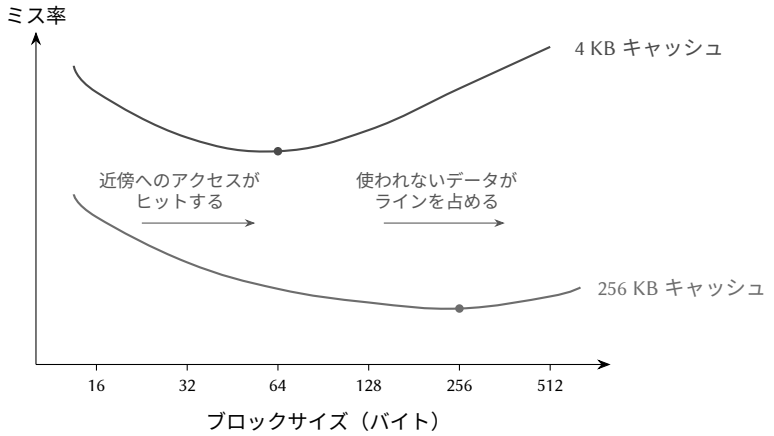


図 14.4: 4 KB と 256 KB のキャッシュについての，ブロックサイズの関数としてのミス率（形状のみ）．点は最良のブロックサイズを示す．

14.9 プリフェッチ

ブロックをプログラムが要求する前にキャッシュへ持ち込めば，ミスをそもそも回避できる．

出典 Lesson 14，動画 23–26；H&P §2.3．

定義 14.15 (プリフェッチ). 近いうちにアクセスされると予想されるブロックを，アクセスされる前にキャッシュへ読み込むことを**プリフェッチ**という．

▷ prefetching

プリフェッチは推測であり，その効果は推測が当たるかどうかで決まる．

- 推測が当たれば，そのブロックへの最初のアクセスはヒットになる．メモリからの取得自体は行われるが，プロセッサが他の仕事をしている間に前倒しで行われるので，そのアクセスがミスペナルティを払うことはない．
- 推測が外れれば，その取得は無駄になる．さらに悪いことに，プリフェッチされたブロックは別のブロックの場所を奪っており，そのブロックが後で必要になればミスする．外れた推測はキャッシュを**汚染**する．

プリフェッチが割に合うのは，推測の大半が当たる場合だけである．

14.9.1 プリフェッチ命令

プログラムは，次にどのデータにアクセスするかを知っていることが多い．**プリフェッチ命令**とは，与えられたアドレスを含むブロックをキャッシュへ持ち込むようプロセッサに求める ISA の命令であり，データは変更しない．コンパイラやプログラマは，あらかじめ分かっているアクセスの手前に，典型的には配列を巡るループの中にこれを置く [29].

▷ prefetch instruction

```
for (i = 0; i < n; i++) {
  prefetch(&a[i + D]);    /* D 反復先 */
  sum = sum + a[i];
}
```

難しいのはプリフェッチ距離 D である．

- D が小さすぎると，ループが要素 $i+D$ に達したときにプリフェッチが完了しておらず，そのアクセスは結局ブロックを待つので，得るものは少ない．
- D が大きすぎると，ブロックは使われるずっと前に到着する．その間ラインを占有し，必要になる前に再び追い出されるかもしれない．

適切な距離は，取得にかかる時間を 1 回の反復の時間で割ったもの程度である．どちらもマシンに依存するので，あるプロセッサに合う距離でコンパイルされたプログラムは，別のプロセッサではプリフェッチが遅すぎたり早すぎたりする．

注意 プリフェッチ距離は 1 つのマシンに合わせて調整されている．同じメモリでプロセッサが速くなれば，より大きな距離が必要になる．

例 14.16. プリフェッチされたブロックは，プリフェッチ命令の 50 ns 後にキャッシュに入り，上のループの 1 回の反復には 4 ns かかる．距離は少なくとも $50/4 = 12.5$ でなければならないので，距離 16 ならわずかな余裕がある．同じメモリで 1 回の反復を 2 ns で実行するプロセッサでは，ループが要素 $i+16$ に達するのはそのプリフェッチの 32 ns 後であり，ブロックの到着より 18 ns 早い．このときループはメモリを待たなければならない．およそ 16 回に 1 回のアクセスが 18 ns 待つので，16 回の反復には 32 ns ではなく約 50 ns かかり，速いはずのプロセッサはこのループを 1 反復当たり約 3.1 ns でしか実行できない．少なくとも 25 の距離が必要である．

14.9.2 ハードウェアプリフェッチ

ハードウェアプリフェッチでは，プロセッサ自身がメモリアクセスの列を観測し，どのブロックが必要になるかを推測する．プログラムを変更する必要はなく，推測はプログラムが走るマシンに適応する．3 つの設計が異なるアクセスパターンを利用する（図 14.5）．

▷ hardware prefetching

ストリームバッファ　ストリームバッファ [30] は逐次的なプリフェッチャである．ブロックが順にアクセスされると，続く数ブロックをキャッシュの脇の小さなバッファに取得する．アクセスはメモリアクセスなしにそこからデータを得る．そしてバッファが消費されるにつれて，さらに先のブロックを取得する．

▷ stream buffer

ストライドプリフェッチャ　ストライドプリフェッチャは，ループが配列の要素

▷ stride prefetcher

を k 個ごとに訪れる場合のように，一定の距離—**ストライド**—で進むアクセスを検出し，1 つ以上のストライド先のアドレスをプリフェッチする．

相関プリフェッチャ **相関プリフェッチャ**は，あるブロックについて，その次にどのブロックがアクセスされたかを記録し，そのブロックが再びアクセスされたときに，記録された後続ブロックをプリフェッチする．メモリ中に散在するノードをもち，繰り返し辿られる連結リストのような，規則的な間隔をもたないパターンを扱える．

▷ correlating prefetcher

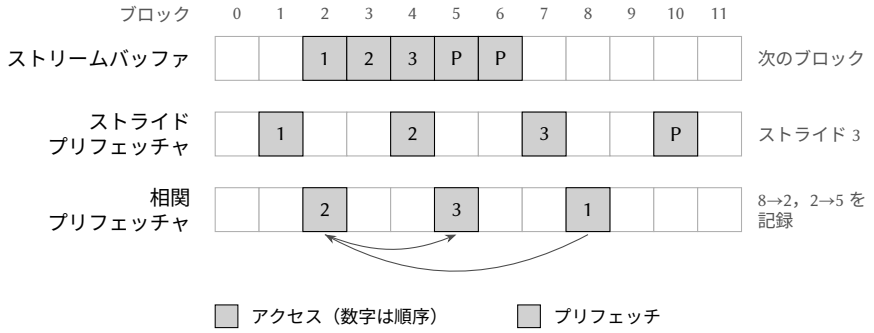


図 14.5: 3 つのハードウェアプリフェッチャが利用するアクセスパターン．図示した 3 回のアクセスの後，ストリームバッファは続くブロックを，ストライドプリフェッチャは 1 ストライド先のブロックを取得し，相関プリフェッチャは次にブロック 8 がアクセスされたときにブロック 2 を取得する．

14.10 ループ交換

コンパイラは，プログラムがデータにアクセスする順序を変えることによってもミス率を下げられる．C は 2 次元配列を行ごとに格納する．`m[j][i]` と `m[j][i+1]` は隣接するが，`m[j+1][i]` は 1 行分先にある．次のループの入れ子は，配列を列ごとに走査する．

出典 Lesson 14，動画 27；H&P §2.3.

```
double m[2000][1000];
for (i = 0; i < 1000; i++)
  for (j = 0; j < 2000; j++)
    m[j][i] = 2 * m[j][i];
```

内側のループのアクセスは，どれも直前のアクセスから 1000 要素，8000 バイト先へ飛ぶ．空間的局所性はまったくない．内側のループのアクセスはどれも異なるブロックに触れ，内側のループの 1 巡が訪れる 2000 個のブロックは L1 キャッシュが保持できる数をはるかに超えるので，ほとんどすべてのアクセスがミスする．

定義 14.17 (ループ交換)．入れ子になったループの順序を入れ替え，最も内側のループが配列の最後の添字を走ってデータが逐次的にアクセスされるようにするコンパイラ変換を**ループ交換**という．

▷ loop interchange

交換の後，外側のループは `j` を，内側のループは `i` を走る．内側のループは隣接する要素を訪れるようになり，1 つのブロックが連続する複数のアクセスに応え，逐次プリフェッチャやストライドプリフェッチャが次のブロックを予測できる．コンパイラがループを交換してよいのは，結果が変わらないことを証明できる場合だけである．ここでは各反復が自分の要素だけを読み書きするので，反復はどの順序で実行してもよい．

> **要点**
> プリフェッチはブロックへの最初のアクセスのミスさえ回避できるが，役に立つのは推測が正しく，かつ間に合う場合だけである．ループ交換は，アクセスの順序をメモリ上のデータの配置に合わせることで，プログラムの側からミスを攻める．

14.11 ミスの重なり

出典 Lesson 14，動画 28–30；H&P §2.3．

ミスペナルティは，ブロックの取得に要する時間より大きくなりうる．アウトオブオーダプロセッサ（第 7 章）は多くの命令を同時に処理しており，そのロードとストアのいくつかは数サイクルのうちにメモリへアクセスしうる．欠けているブロックの取得には多くのサイクルがかかり，その間に別のアクセスもミスしうる．

定義 14.18 (オーバーラップミス). 先行するミスがまだ処理されている間に生じるキャッシュミスを**オーバーラップミス**という．

▷ overlap miss

ロードがミスした後も，アウトオブオーダプロセッサは他の命令を実行し続ける．しかしそのロードはコミットできないので，プロセッサはやがて ROB エントリかリザベーションステーションを使い果たして停止する．その時点より前に発行された 2 つ目のミスは，1 つ目と重なる．それがどれだけの代償になるかはキャッシュによる．

- **ブロッキングキャッシュ**は，ミスを処理している間は他に何もしない．後続のアクセスは先行するミスが完了するまで探索さえされないので，重なったミスは自身のペナルティに加えて先行するミスの残り時間を払う（図 14.6a）．

▷ blocking cache

- **ノンブロッキングキャッシュ**は，ミスの処理中にもヒットに応え（**ヒットアンダーミス**），さらに別のミスを開始することもできる．このとき複数の要求がメモリで同時に処理され，メモリが要求を並列に処理できる限り，**メモリレベル並列性**が活用される．

▷ non-blocking cache

定義 14.19 (ミスアンダーミス対応). **ミスアンダーミス対応**をもつキャッシュは，複数のミスを同時に処理できる．処理中の各ミスを記録し，それらの要求をメモリ階層の次のレベルへ並行して送る．

▷ miss-under-miss support

処理中の各ミスは**ミス状態管理レジスタ**（MSHR）に記録される．キャッシュでミスしたアクセスは，まず自身のブロックアドレスを MSHR と比較する．

▷ miss status handling register

一致なし そのアクセスは新しいミスである．空いている MSHR を 1 つ確保し，そこにブロックと起こすべき命令を記録して，要求をメモリへ送る．

一致あり そのブロックはすでに到着の途中である．これを**ハーフミス**という．2 つ目の要求は送られず，命令は一致した MSHR に追加されるだけである．

▷ half-miss

データが到着すると，それがキャッシュに置かれ，その MSHR に記録されたすべての命令が起こされ，MSHR が解放される．MSHR が 2 つあるだけでもブロッキングキャッシュより大きく改善し，4 つあればさらに良い．そしてメモリアクセスは非常に長くかかるので，16–32 個の MSHR でも効果が続く．

実際の機械が追跡できる数はもっと少ない．Nehalem から Haswell までの Intel のコアはラインフィルバッファを 10 個もち，コア当たり同時に処理できる L1 データミスは 10 件までである．Intel 最適化マニュアル．

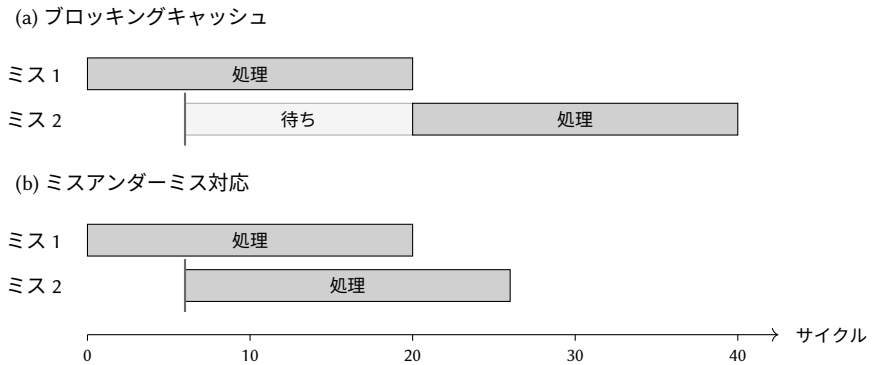


図 14.6: ミスペナルティが 20 サイクルの 2 つのミス．2 つ目は 1 つ目の 6 サイクル後に生じる（縦線）．(a) ブロッキングキャッシュでは，2 つ目のミスに 20 サイクルではなく 34 サイクルかかる．(b) ミスアンダーミス対応があれば，どちらのミスも 20 サイクルで済む．

ミスアンダーミス対応には追加のハードウェアが必要である．多くのメモリアクセスを同時に発行するプロセッサ，とりわけアウトオブオーダプロセッサで割に合う．

14.12 キャッシュ階層

出典 Lesson 14，動画 31–34；H&P §B.3.

L1 キャッシュのミスペナルティは，主記憶のアクセス時間に支配される．これは，そもそも L1 キャッシュを生んだのと同じ発想で下げられる．すなわち，L1 と主記憶の間にもう 1 つキャッシュを置く．L1 でミスしたアクセスは **L2 キャッシュ**を探索する．L2 は L1 より大きく遅いが，それでも主記憶よりはるかに速い．そして L2 でミスしたものだけがさらに先へ進む．

▷ L2 cache

定義 14.20 (キャッシュ階層). **キャッシュ階層**とは，L1，L2，L3，...という複数のレベルのキャッシュからなり，各レベルは直前のレベルがミスしたときにだけ探索されるものである（図 14.7）．主記憶の直前のレベルを**最終レベルキャッシュ**（LLC）という．

▷ cache hierarchy

▷ last-level cache

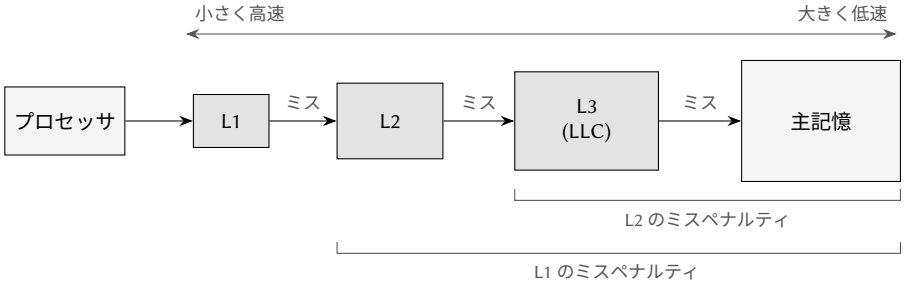


図 14.7: 3 レベルのキャッシュ階層．各レベルのミスペナルティは，その後ろにあるレベル群の平均アクセス時間である．

14.12.1 キャッシュ階層の AMAT

あるレベルのミスペナルティは，その後ろのレベルからブロックを得るのにかかる平均時間，すなわち階層の残りの部分の AMAT である．式 (14.1) をレベルごとに適用すると次が得られる．

$$\begin{aligned} \text{AMAT} &= h_1 + m_1 \times P_1, \\ P_1 &= h_2 + m_2 \times P_2, \\ &\vdots \\ P_{\text{LLC}} &= \text{主記憶のアクセス時間}, \end{aligned} \tag{14.6}$$

ここで h_i はレベル i のヒット時間，m_i はミス率，P_i はミスペナルティである．ミス率 m_i は，レベル i に到達したアクセスについて数える（14.13 節）．

14.12.2 L1 と L2

式 (14.6) では，L1 のヒット時間 h_1 はすべてのアクセスが払うが，L2 のヒット時間 h_2 は L1 でミスしたアクセスの割合 m_1 だけが払う．したがって L1 は，前の節までの技法によって短いヒット時間を目指して設計され，L2 はヒット時間が長くなる代わりにミス率を下げてよい．

Skylake の値（Intel 最適化マニュアル）は，L1 データ 32 KB で 4 サイクル，L2 256 KB で 12 サイクル，L3 はコア当たり約 2 MB で 44 サイクル，主記憶は 60–100 ns，すなわち数百サイクルである．

例 14.21. 主記憶は 1 回のアクセスに 50 ns かかる．32 KB のキャッシュはヒット時間 1 ns，ミス率 10 %，256 KB のキャッシュはヒット時間 5 ns，ミス率 3 % である．階層の中でも 256 KB のキャッシュは単独の場合と同じアクセスでミスする，すなわちそこに到達するアクセスの $0.03/0.10 = 30\,\%$ でミスすると仮定する．

構成	AMAT
キャッシュなし	50 ns
32 KB キャッシュのみ	$1 + 0.10 \times 50 = 6$ ns
256 KB キャッシュのみ	$5 + 0.03 \times 50 = 6.5$ ns
L1 32 KB，L2 256 KB	$1 + 0.10 \times (5 + 0.30 \times 50) = 3$ ns

この階層は，どちらのキャッシュ単独よりも 2 倍速い．すべてのアクセスが払う小さなキャッシュの短いヒット時間と，長いヒット時間をアクセスの 10 % だけが払う大きなキャッシュの低いミス率とを兼ね備えている．

14.13 局所ミス率と大域ミス率

例 14.21 では，L2 キャッシュはそこに到達するアクセスの 30 % でミスし，これは L1 の 3 倍の率である．だからといって L2 が劣ったキャッシュだということにはならない．L2 が見るのは L1 がミスしたアクセスだけであり，それらは局所性が最も弱いものである．応えるのが容易なアクセスは L1 より先へは行かない．したがって下位レベルのミス率は，その基準とともに述べなければならない．

出典 Lesson 14，動画 35–37；H&P §B.3．

定義 14.22 (局所率と大域率)．キャッシュの**局所ヒット率**とは，そのヒット数を，そこに到達したアクセス数で割ったものであり，**局所ミス率**は 1 − 局所ヒット率である．

▷ local hit rate
▷ local miss rate

キャッシュの**大域ミス率**とは，そのミス数を，プロセッサのすべてのメモリアクセス数で割ったものであり，**大域ヒット率**は 1 − 大域ミス率 である．

▷ global miss rate
▷ global hit rate

L1 では局所率と大域率は一致する．下位のレベルでは，そこに到達するアクセスは上のレベルのミスなので，レベル n の大域ミス率は局所ミス率の積になる．

$$\text{大域ミス率}_n = \prod_{i=1}^{n} \text{局所ミス率}_i. \tag{14.7}$$

式 (14.6) のミス率は局所ミス率である．あるレベルの大域ミス率は，全アクセスのうちどれだけの割合がそのレベルより先へ進まなければならないかを述べるので，レベルや階層を比較するのに適した尺度である．

定義 14.23 (1000 命令当たりミス数)．**1000 命令当たりミス数**（MPKI）という指標は，実行された 1000 命令当たりのキャッシュのミス数を数える．

▷ misses per 1000 instructions

大域ミス率と同様に MPKI もあるレベルのすべてのミスを数えるが，基準がメモリアクセスではなく命令である．

$$\text{MPKI} = 1000 \times \text{大域ミス率} \times \frac{\text{メモリアクセス数}}{\text{命令}}. \tag{14.8}$$

これにより，そのレベルのミスペナルティをプログラムが命令当たりどれだけの頻度で払うかが直接に分かる．

例 14.24．あるプログラムが 1 000 000 命令を実行し，メモリアクセスは 1 500 000 回である．L1 は 90 000 回，L2 は 27 000 回ミスする．

- L1: 局所ミス率と大域ミス率は $90\,000/1\,500\,000 = 6\,\%$, MPKI は $90\,000/1000 = 90$.
- L2: 局所ミス率は $27\,000/90\,000 = 30\,\%$（局所ヒット率 70 %），大域ミス率は $27\,000/1\,500\,000 = 1.8\,\% = 0.06 \times 0.30$（大域ヒット率 98.2 %），MPKI は 27.

ヒント データキャッシュの場合，典型的なコードでは命令の約 3 分の 1 がロードかストアなので，MPKI ≈ 330× 大域ミス率である（H&P の命令構成比）．

> **要点**
> AMAT は局所ミス率で計算する．レベルや階層の比較には大域ミス率か MPKI を用いる．上のレベルのミスだけを見るレベルでは，局所ヒット率が低いのは当然だからである．

14.14 包含性

L1 にあるブロックは，L2 にもあるかもしれないし，ないかもしれない．レベル間の関係には 3 つの可能性がある．

出典 Lesson 14，動画 38–40；H&P §B.3；[31]．

包含 L1 にあるブロックはすべて L2 にもある．
排他 L1 にあるブロックは L2 にはない．
どちらでもない L1 にあるブロックの一部は L2 にもあり，他はない．

定義 14.25 (包含性). あるレベルにあるブロックがすべて，次の，より大きなレベルにも存在するとき，そのキャッシュ階層は**包含性**をもつという．

▷ inclusion property

LRU 置換をもつ大きな L2 なら，L1 のブロックを自動的にすべて保持しそうに思える．そうではない．L1 でのヒットは L2 に届かないので，頻繁に使われ，そのため L1 に留まるブロックは，L2 の LRU カウンタからは使われていないように見える．やがてそれは L2 で最も長く使われていないブロックとなり，L1 に残ったまま L2 では追い出される．

例 14.26. L1 は 2 ブロック，L2 は 3 ブロックを保持する．どちらもフルアソシアティブで LRU 置換をもち，初期状態は空であり，両方のレベルでミスしたブロックは両方に読み込まれる．表 14.4 はアクセス P，Q，P，R，P，S を追う．P への後のアクセスは L1 でヒットするので，L2 が P を最後に見るのは最初のアクセスである．S でのミスの時点で P は L2 で最も長く使われていないブロックであり，まだ L1 にあるにもかかわらず追い出される．

表 14.4: 両方のレベルが LRU 置換をもつ 2 レベルの階層（例 14.26）．内容は，最も長く使われていないものから最も最近使われたものの順に並べる．最後のアクセスで包含性が破れる．

アクセス	L1	L2	後の L1	後の L2
P	ミス	ミス	P	P
Q	ミス	ミス	P Q	P Q
P	ヒット	—	Q P	P Q
R	ミス	ミス	P R	P Q R
P	ヒット	—	R P	P Q R
S	ミス	ミス	P S	Q R S

したがって包含性が欲しければ，それを強制しなければならない．1 つの方法は，L2 のすべてのラインに**包含ビット**を設け，そのブロックが L1 にもある間これを立てておくことである．L2 は，このビットが立っているラインを犠牲に選ばない.[2]

この章で，単一プロセッサにおけるキャッシュの議論は完結する（第 19 章ではマルチプロセッサのキャッシュに戻る）．第 15 章では，キャッシュと主記憶を作るメモリ技術に話を移す．

[2] どちらも実際に使われている．Skylake までの Intel のクライアント向け L3 は包含的であり，フィルタも兼ねる．L3 にないブロックはどのコアのキャッシュにもないので，他のコアに問い合わせる必要がない（第 19 章）．Skylake-SP と AMD Zen の L3 は非包含であり，主としてプライベートキャッシュから追い出されたブロックを保持するので，容量が写しに費やされない．

要点 — 章のまとめ

AMAT = ヒット時間 + ミス率 × ミスペナルティ であり，各項にはそれぞれの技法がある．ヒット時間は，探索のパイプライン化，仮想アドレスでインデックスしながら TLB が変換し，キャッシュサイズ $\leq N \times$ ページサイズ ならエイリアシングを避けられる VIPT キャッシュ，ウェイ予測，そして安価な置換アルゴリズムである NMRU と PLRU によって短くなる．ミスは初期参照ミス，容量性ミス，競合性ミスのいずれかであり，ミス率は適切なブロックサイズ，プリフェッチ（プリフェッチ命令，ストリームバッファ，ストライドプリフェッチャ，相関プリフェッチャ），ループ交換によって下がる．ミスペナルティは，処理中のミスを MSHR で追跡するミスアンダーミス対応と，レベルごとに局所ミス率で AMAT を計算するキャッシュ階層によって下がる．大域ミス率と MPKI は，あるレベルより先へ進むアクセスがどれだけあるかを表す．各レベルが LRU を用いる場合，包含性は自動的には成り立たない．

第 15 章

メモリ

講義動画	Lesson 15，動画 1–14
教科書	H&P [1] §2.2
キーワード	SRAM，DRAM，メモリセル，ワード線，ビット線，センスアンプ，破壊読み出し，トレンチセル，リフレッシュ，行デコーダ，RAS と CAS，ファストページモード，フロントサイドバス，ノースブリッジ

第 12–14 章では，主記憶をキャッシュの背後に隠れた大きく遅い装置として扱ってきた．また第 1 章では，プロセッサとメモリの速度の差，すなわちメモリウォールが広がり続けることを見た．この章ではメモリそのものの内部を見る．まず，メモリを作る 2 つの技術—キャッシュ用の静的 RAM と主記憶用の動的 RAM—を，1 ビットを蓄える回路まで掘り下げて述べる．続いて，DRAM セルがどのようにチップ上に構成されアクセスされるか，そしてチップがどのようにプロセッサに接続されるかを示す．これらを合わせると，主記憶が遅い理由と，主記憶がキャッシュのより速い技術で作られていない理由が分かる．

15.1 メモリ技術

出典 Lesson 15，動画 1–3；H&P §2.2．

キャッシュも主記憶も，ランダムアクセスメモリである．

定義 15.1 (ランダムアクセス). どの位置もそのアドレスによって直接，しかもどの位置についてもほぼ同じ時間で読み書きできるとき，そのメモリは**ランダムアクセス**を提供するという．

▷ random access

その反対が順次アクセス（第 16 章）である．磁気テープのような媒体では，現在の位置と要求された位置の間にあるデータすべてを通り過ぎるまで媒体を動かさなければならない．ランダムアクセスメモリ（RAM）は 2 つの技術で作られ，その名前は，電源が入っている間に，記憶されたビットに何が起こるかに由来する．

定義 15.2 (SRAM と DRAM). **SRAM** では，記憶されたビットは電源が入っている限りその値を保つ．**DRAM** では，記憶されたビットは，電源が入っていても，周期的に書き直さない限り短時間で値を失う．

▷ static random-access memory
▷ dynamic random-access memory

これらの名前は，電源がないときに何が起こるかについては何も言っていない．どちらの技術も揮発性であり，電源を切れば内容をすべて失う．

2 つの技術は 1 ビットを蓄える回路が異なり，この違いが速度とコストを決める．SRAM は 1 ビット当たり複数のトランジスタを使い，通常の設計では 6 個である（15.2 節）．DRAM は 1 個のトランジスタしか使わない（15.3 節）．したがって SRAM は 1 ビット当たりのチップ面積が大きくビット当たりの価格も高いが，速い．そのセルがビット線を能動的に駆動するのに対し，DRAM セルはわずかな電荷を保持するだけで，その電荷は検出し，書き戻し，リフレッシュしなければならないからである．DRAM は同じ面積により多くのビットを詰め込め，ビット当たりの価格も安いが，遅い．速くなければならず小さくてよいキャッシュは SRAM で作られ，大きく安くなければならない主記憶は DRAM で作られる．

この区別は絶対ではない．IBM の POWER7（2010）は，ダイ上の DRAM セル（トレンチ型 eDRAM）で 32 MB の L3 キャッシュを作った（IBM の POWER7 資料）．

15.2 SRAM セル

出典 Lesson 15，動画 4；H&P §2.2．

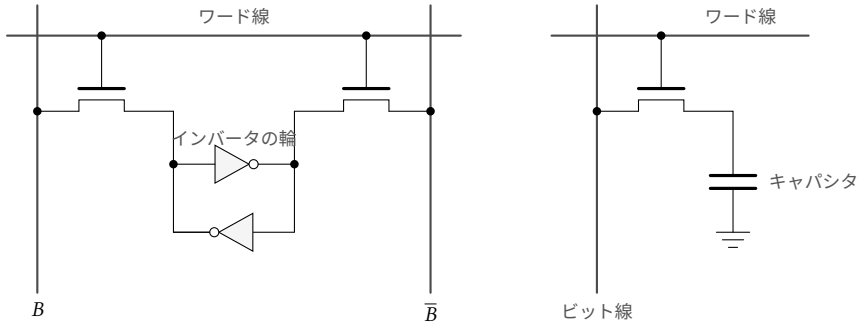
メモリはビットを，同一の回路からなる 2 次元配列に蓄える．1 ビットを蓄える回路を**メモリセル**という．セルは行と列に並べられ，配列を横切る 2 組の配線を通じてアクセスされる．

▷ memory cell

定義 15.3 (ビット線とワード線). **ビット線**はセルの列に沿って走り，その列のセルから読まれる値，あるいはそのセルに書かれる値を運ぶ．**ワード線**はセルの行に沿って走り，活性化されると，その行のすべてのセルをそれぞれの列のビット線に接続する．

▷ bit line
▷ word line

SRAM セル（図 15.1a）では，ビットは輪をなすように接続された 2 つのインバータが保持する．各インバータの出力が他方の入力を駆動する．最初のインバータが 1 を出力すれば 2 番目は 0 を出力し，それが今度は最初のインバータを 1 に保つ．この輪は 2 つの状態のどちらでも安定で，インバータに電源が入っている限りその状態を保つ．これが SRAM を静的たらしめている．ゲートがワード線に接続されたアクセストランジスタが，輪の一方のノードをビット線に接続する．



(a) SRAM セル（6 トランジスタ）　(b) DRAM セル（1 トランジスタ）

図 15.1: 1 ビットの記憶．(a) SRAM セルは 2 つのインバータの輪にビットを保持する．輪のノードは，2 つのアクセストランジスタによって相補的なビット線 B と $\overline{B}$ に接続される．(b) DRAM セルは，1 個のアクセストランジスタの奥にあるキャパシタの電荷としてビットを保持する．

書き込み ビット線を新しい値に強く駆動し，ワード線を活性化する．ビット線は，ノードを駆動している小さなインバータを押し負かす．輪はビット線に一致する状態に落ち着き，ワード線を非活性化した後もその状態を保つ．

読み出し ワード線を活性化すると，輪がビット線を駆動する．セルは小さくなければならないのでインバータのトランジスタは小さいが，ビット線は列のすべてのセルに接続された長い配線である．1 個のセルはビット線の電圧をゆっくりとしか変えられない．

読み出しを速めるために，ビットの反転値を保持する輪のもう一方のノードを，2 つ目のアクセストランジスタを通じて 2 本目のビット線 $\overline{B}$ に接続する．ワード線が活性化されると，セルは 2 本のビット線を互いに逆向きに引っ張り，値は両者を比較することで得られる．

定義 15.4 (センスアンプ). **センスアンプ**は，ビット線上の小さな電圧変化，あるいは 1 対のビット線の間の小さな差を検出し，それを完全な論理 0 または 1 に増幅する．

▷ sense amplifier

相補的な対があれば，センスアンプは 2 本のビット線を比較するだけでよい．セルが 2 本を逆向きに動かすので，ごくわずかな振れの後には差の符号が明らかであり，基準電圧も要らない．1 本のビット線だけであれば，その値を確実に判定できるまでにはるかに大きく動かなければならない．この対は書き込みにも役立つ．値とその反転値が輪の両側に同時に働くからである．こうしてできるセルは，各インバータに 2 個とアクセストランジスタ 2 個の計 6 個のトランジスタをもち，ワード線 1 本，ビット線 2 本，そしてインバータの電源接続を必要とする．

第 12–14 章のキャッシュの配列はすべてこのセルでできている．32 KB の L1 キャッシュはデータ部だけで 262 144 ビット，約 160 万個のトランジスタである．

15.3 DRAM セル

DRAM セルはビットをキャパシタ上の電荷として蓄える（図 15.1b）．ワード線が制御する 1 個のアクセストランジスタが，キャパシタをビット線に接続する．充電されたキャパシタが 1 を，放電されたキャパシタが 0 を表す．書き込むにはビット線をその値にしてワード線を活性化する．するとキャパシタはトランジスタを通じて充電または放電される．ワード線を非活性化するとトランジスタが遮断し，電荷を孤立させる．

出典 Lesson 15，動画 5–6；H&P §2.2；[32].

しかし，遮断したトランジスタは完全なスイッチではない．静的電力（第 1 章）を生むのと同じ不完全さによってわずかな電流が流れ続け，記憶された 1 の電荷

注意 SRAM の「静的」は電力ではなく保持のことである．SRAM セルも漏れ電流をもち，プロセッサの静的電力の大きな部分をキャッシュが占める．

は少しずつ漏れ出して，やがてそのセルは 0 と区別できなくなる．これが DRAM を動的たらしめている．ビットは，電荷が失われすぎる前に書き直されたときだけ生き延びる．

定義 15.5 (リフレッシュ). DRAM セルのビットを読み出し，電荷が漏れ出してしまう前に完全な電荷で書き戻すことを**リフレッシュ**という．どのセルも，そのデータが使われているかどうかに関わらず，周期的にリフレッシュしなければならない．

▷ refresh

DRAM セルの読み出しも繊細である．まずビット線を 0 と 1 の中間の電圧にし，それからワード線を活性化する．キャパシタはビット線と電荷を分け合い，セルが 1 を保持していればビット線の電圧はわずかに上がり，0 を保持していればわずかに下がる．セルのキャパシタは長いビット線の容量に比べて極めて小さいので，この変化は非常に小さく，検出にはセンスアンプが必要である．電荷を分け合った後，キャパシタは中間の電圧の近くに取り残される．すなわち，記憶されていた値は失われている．

定義 15.6 (破壊読み出し). **破壊読み出し**とは，記憶された値を壊してしまう読み出しのことである．したがって，DRAM セルの読み出しの後には必ず，読み出した値をそのセルに書き戻す動作が続く．

▷ destructive read

SRAM の読み出しは破壊的ではない．インバータの輪がノードを駆動し続け，値を自力で復元するからである．

キャパシタは面積の問題も引き起こす．チップ表面に平らに作った場合，確実に検出できるだけの大きさのキャパシタはトランジスタよりはるかに広い面積を占め，1 トランジスタのセルがもつ密度上の利点の多くが失われてしまう．

定義 15.7 (トレンチセル). **トレンチセル**とは，アクセストランジスタの隣のシリコンに垂直に掘った，深く狭い溝（トレンチ）の中にキャパシタを形成した DRAM セルである（図 15.2）．

▷ trench cell

トレンチによって，キャパシタはチップ表面をほとんど使わずに広い電極面積を得るので，セルが占める面積はトランジスタよりわずかに大きい程度である．また，SRAM セルより配線も少なくて済む．ビット線は 2 本ではなく 1 本であり，インバータのための電源接続も要らない．表 15.1 に 2 つのセルの比較をまとめる．

表 15.1: SRAM と DRAM の比較．

	SRAM	DRAM
記憶素子	インバータの輪	キャパシタ
1 ビット当たりトランジスタ数	6	1
セル当たりビット線数	2	1
電源投入中の値の保持	保つ	リフレッシュすれば保つ
読み出し	非破壊	破壊的，書き戻しが続く
速度	速い	遅い
1 ビット当たり面積とコスト	大きい	小さい
用途	キャッシュ	主記憶

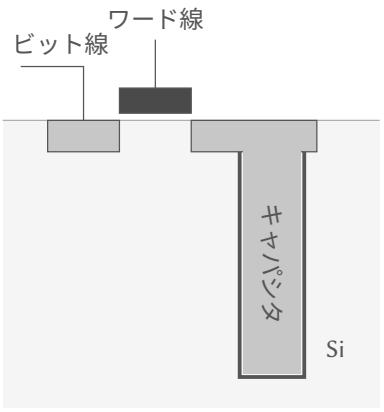


図 15.2: トレンチセルの断面（模式図）．
もう 1 つのよく使われる設計は，現在の DRAM の大半が採用しているもので，キャパシタをアクセストランジスタの上に積み重ねる．

要点
DRAM はビットを 1 個のトランジスタの奥のわずかな電荷として蓄える．そのため密度が高く安価だが遅い．読み出しは必ず電荷を壊すので書き戻しが続かなければならず，漏れ電流が周期的なリフレッシュを強いるからである．SRAM は 1 ビット当たり 6 個のトランジスタという代償で，そのどちらも避けている．

15.4 メモリチップの構成

メモリチップのセルは 2^r 行 2^c 列の配列をなし（図 15.3），$r+c$ ビットのアドレスは r ビットの行アドレスと c ビットの列アドレスに分割される．例えば，アド

レスの上位ビットと下位ビットである．DRAM チップの 1 ビットの読み出しは 4 つの段階で進む．

1. **行デコーダ**が行アドレスを，2^r 本のワード線のうちちょうど 1 本の活性化に変換する．
2. 選ばれた行のすべてのセルがそれぞれのビット線に接続される．ビット線 1 本につき 1 個のセンスアンプが値を検出し，行全体が行ラッチに保持される．
3. **列マルチプレクサ**が列アドレスを使ってラッチの 1 ビットを選び，データ出力へ送り出す．
4. 読み出しは行のすべてのセルにとって破壊的だったので，ラッチの内容をビット線を通じて行に書き戻す．

出典 Lesson 15，動画 7；H&P §2.2．

▷ row decoder

▷ column multiplexer

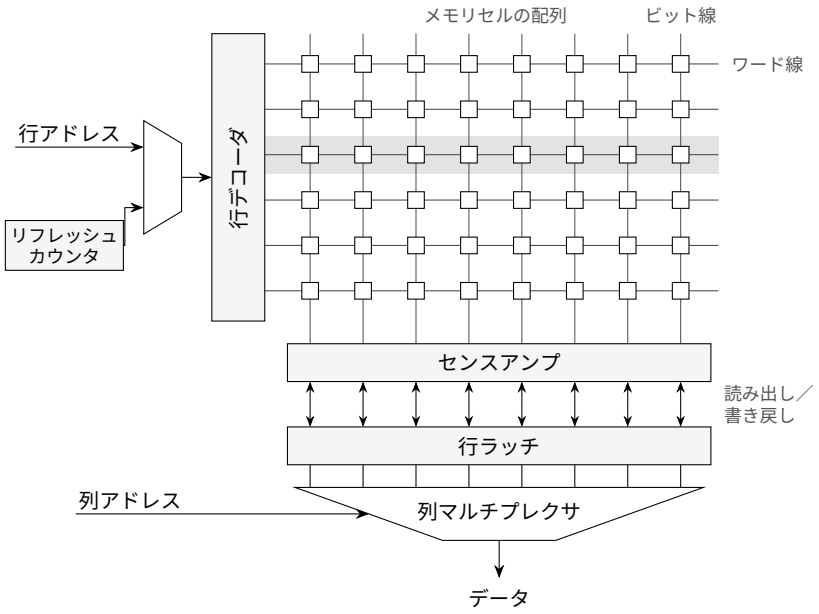


図 15.3: DRAM チップの構成．行デコーダは，行アドレスまたはリフレッシュカウンタ（15.5 節）が選ぶワード線を活性化する．センスアンプが行全体を行ラッチに読み出し，列マルチプレクサが 1 ビットを選び，ラッチが行に書き戻される．

書き込みも同じ経路を使う．上と同じように行をラッチに読み出し，列アドレスが選ぶビットをラッチ上で書き込むべき値に置き換え，書き戻しによって変更された行を格納する．このように，DRAM チップにおける書き込みとは，書き戻す前にラッチを変更する読み出しである．

実際のデバイスの幅は 1 ビットではなく 4，8，16 ビットである．8 ビット幅の DDR4 チップは 1 行に 1 KB をもち，ランクの 8 個のチップは同時に行を開く（JEDEC DDR4）．

注 15.8. 行を閉じるときに書き戻される行ラッチは，この章を通じて用いる単純化したモデルである．実際の DRAM チップでは，センスアンプ自身が行を保持する．ワード線が活性のままの間にセンスアンプがビット線を完全なレベルまで駆動し，それによって行のセルが復元される．また，ラッチに書き込まれたビットはただちにそのセルに届く．行を閉じる操作はワード線を落とし，ビット線を次の読み出しに備えさせる．

アドレスの 2 つの部分にかかる時間は大きく異なる．行を選ぶには，デコードし，長いワード線を駆動し，長いビット線上のわずかな電荷をセンスアンプが検出するのを待たなければならない．列を選ぶのは，すでにラッチにあるビットの間でマルチプレクサを切り替えるだけである．15.6 節と 15.7 節はこの差を利用する．

データシートではラッチされた行を行バッファと呼び，すでに開いている行に当たるアクセスを「ページヒット」という（15.7 節）．

注 15.9. SRAM チップも同じように構成され，行デコーダ，センスアンプ，列マルチプレクサをもつ．ただし，その読み出しは破壊的ではないので，読み出しの後に書き戻しは続かず，またそのセルにリフレッシュも要らない．

15.5 リフレッシュ

DRAM チップのどの行も，長さ T の区間ごとに少なくとも 1 回はリフレッシュしなければならない．T は普通，数十ミリ秒である．リフレッシュは通常の読み出し経路を使う．すなわち，出力を使わずにラッチを書き戻す読み出し（15.4 節）である．

プログラムによる読み出しは，触れた行をリフレッシュする．しかし，そのような読み出しは当てにできない．長い間，あるいはまったく読まれない行があるからである．そこで，すべての行を系統的にリフレッシュする．リフレッシュカウンタが行番号を保持する．周期的に，行アドレスの代わりにカウンタの値が行デコーダに与えられ（図 15.3），その行が読み出されて書き戻され，カウンタは次の行へ進み，最後の行の後は先頭に戻る．行数を R とし，そのそれぞれを区間 T ごとに 1 回リフレッシュしなければならないとすると，チップは，次の長さの区間ごとに少なくとも 1 回，1 つの行をリフレッシュしてカウンタを進めなければならない．

$$t_{\text{row}} = \frac{T}{R}. \tag{15.1}$$

ある行をリフレッシュしている間，チップは読み出しにも書き込みにも応じられず，その間に到着したアクセスは待たなければならない．こうしてリフレッシュは DRAM が遅い理由をもう 1 つ増やし，そのコストは行数とともに増える．

例 15.10. ある DRAM チップは 8192 行をもち，そのセルは 64 ms ごとにリフレッシュしなければならない．式 (15.1) により，チップは 64 ms/8192 ≈ 7.8 µs ごとに 1 行をリフレッシュしなければならない．すなわち毎秒 128 000 行である．1 行のリフレッシュに 50 ns かかるとすると，チップは毎秒のうち 128 000 × 50 ns = 6.4 ms，すなわち時間の 0.64 % をリフレッシュに費やす．行数が 4 倍で T が同じチップでは 2.56 % となる．

出典 Lesson 15，動画 8–9；H&P §2.2.

JEDEC の DDR4 は 64 ms の窓に 8192 回のリフレッシュコマンドを収め，7.8 µs ごとに 1 回発行する．85 °C を超えると間隔は半分になる．

ヒント リフレッシュのオーバーヘッドは $t/(T/R) = Rt/T$ である．t は 1 行のリフレッシュがチップを占有する時間であり，T の中でどう散らすかは関係しない．

15.6 アドレスの多重化

2^{r+c} 個のセルをもつチップには $r+c$ ビットのアドレスが必要であり，並列に送るビットはどれもパッケージのピン 1 本とチップへの配線 1 本を消費する．しかし，行アドレスと列アドレスが同時に必要になるわけではない．行アドレスは，遅い行の選択を始めるために先に必要である．列アドレスは，行がラッチに入ってから初めて必要になる．そこで DRAM チップは両方の部分を同じピンから順に受け取り，2 本の制御信号が，どちらの部分が来ているかをチップに伝える．

出典 Lesson 15，動画 10；H&P §2.2.

定義 15.11 (RAS と CAS). **行アドレスストローブ**（RAS）がアサートされると，チップはアドレスピン上の値を行アドレスとして取り込み，ワード線を活性化して行をラッチに読み出す．**列アドレスストローブ**（CAS）がアサートされると，チップはアドレスピン上の値を列アドレスとして取り込み，ラッチのそのビットを選ぶ．

▷ row address strobe

▷ column address strobe

読み出しでは，選ばれたビットがデータ出力に現れる．CAS と同時にライトイネーブル信号がアサートされていれば，代わりに，ラッチ中のそのビットがデータ入力の値に置き換えられる．RAS をデアサートするとアクセスが終わり，（注 15.8 のモデルでは）ラッチが書き戻されて行が閉じられる．図 15.4a は，この種のアクセスを 2 回示している．

多重化により，アドレスピンは $r+c$ 本から $\max(r,c)$ 本に減る．行数と列数が等しければ半分である．これによってパッケージは小さく安くなり，チップと，それを駆動するメモリコントローラ（15.8 節）との間の配線も減る．例えば，16 384 行 16 384 列に 2^{28} 個のセルをもつチップは，28 本ではなく 14 本のアドレスピンで済む．

実際のチップでは RAS と CAS は負論理である．図 15.4 ではアサートされた信号を高レベルで描いている．

15.7 ファストページモード

図 15.4a の流れでは，どのアクセスも行を開き，1 つの列を使い，また行を閉じる．次のアクセスが同じ行に対するものであってもそうである．しかし，いった

出典 Lesson 15，動画 11–12.

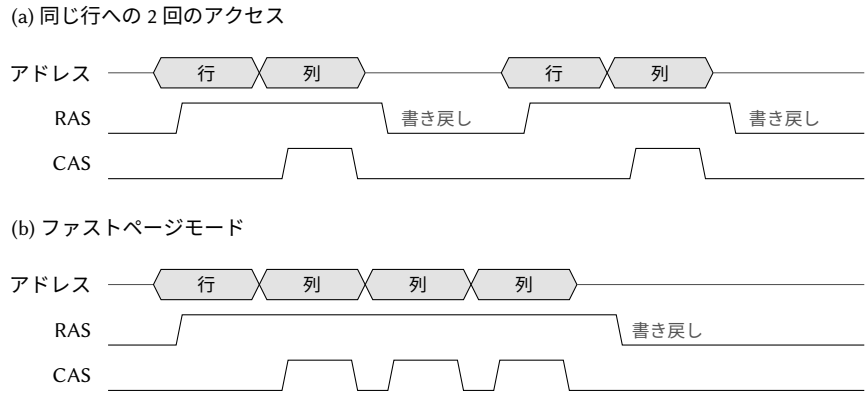


図 15.4: アドレスピンを多重化した DRAM チップの信号．(a) 各アクセスは RAS で行を開き，CAS で列を選び，行を閉じる．(b) ファストページモード（15.7 節）では，行を一度だけ開いて 3 つの列にアクセスし，RAS をデアサートしたときに行を書き戻す．

ん行を開いてしまえば，その行のビットはすべてラッチにあり，その行の別のビットに届くには新しい列アドレスだけがあればよい．

定義 15.12 (ファストページモード). **ファストページモード**では，RAS で一度開いた行を開いたままにし，続けざまの CAS パルスで複数の列アドレスを与えて，そのたびに行の 1 ビットを読み書きする．行が書き戻されるのは，最後に RAS がデアサートされるときの 1 回だけである（図 15.4b）．

▷ fast page mode

ファストページモードでは，高価な段階–行の選択とセンス，そして書き戻し–をアクセスのまとまり全体につき 1 回だけ行い，各アクセスに要するのは速い列の選択だけである．[1] 空間的局所性（第 12 章）によって頻繁に起こる近いアドレスへのアクセスは，普通はアドレスの下位ビットだけが異なるので，同じ行を共有する．行をどれだけ長く開いたままにするかは，RAS と CAS を駆動するコントローラが決める．後続のアクセスが同じ行を使う間は行を開いたままにでき，別の行にアクセスするときや，どれかの行をリフレッシュするときには，その前に行を閉じなければならない．したがって，節約できる量は，連続するアクセスのうちどれだけが同じ行を共有するかによる．

注意 ここでいう「ページ」はチップの行のことであり，仮想記憶のページ（第 13 章）ではない．

[1] ファストページモードは一連の改良の最初のものだった．EDO（extended data out）は次の列アドレスを与える間もデータを有効に保ち，同期式 DRAM（SDRAM，最初の製品は 1992 年）はストローブをクロックとコマンドの体系に置き換え，DDR はクロックの両エッジで転送するようにした．チップ内部の行と列の構造は変わっていない．

例 15.13. ある DRAM チップでは，行を開く（RAS，デコードとセンス）のに 30 ns，CAS で列を選ぶのに 5 ns，行を閉じる（書き戻し）のに 10 ns かかる．表 15.2 の順に 5 つのアクセスが到着する．表中の組は（行，列）を表す．ファストページモードがなければ，どのアクセスも自分の行を開いて閉じるので 5×45 ns = 225 ns かかる．ファストページモードでは行を開いて閉じるのが 3 回で済み，表の合計は 145 ns となる．行 22 へのアクセスが最後に来ていれば，行 7 への 4 つのアクセスが 1 回の開きを共有し，合計は 105 ns だったはずである．

表 15.2: 例 15.13 のアクセスをファストページモードで実行した場合．

アクセス	動作	時間 (ns)
読み出し (7, 12)	行 7 を開く，列 12 を CAS	35
読み出し (7, 13)	列 13 を CAS	5
書き込み (7, 14)	ライトイネーブルとともに列 14 を CAS	5
読み出し (22, 3)	行 7 を閉じる，行 22 を開く，列 3 を CAS	45
読み出し (7, 15)	行 22 を閉じる，行 7 を開く，列 15 を CAS	45
–	行 7 を閉じる	10
合計		145

注意 22-22-22 のようなモジュールのタイミングはナノ秒ではなくバスのクロック数である．DDR4-3200 では，行を開く，列を読む，行を閉じるのがそれぞれ 13.75 ns である．JEDEC のスピードビン．

15.8 DRAM とプロセッサの接続

プロセッサが主記憶を待つ時間は，DRAM チップのアクセス時間だけではない．

出典 Lesson 15，動画 13–14．

両者の間には他のチップとバスを通る経路があり，その伝統的な形を 図 15.5a に示す．

定義 15.14 (フロントサイドバスとノースブリッジ). **フロントサイドバス**（FSB）は，プロセッサを**ノースブリッジ**に接続する．ノースブリッジは，プロセッサを主記憶と，グラフィックスアダプタのような高速なデバイスと，そして遅い I/O デバイスが接続されるサウスブリッジ（第 16 章）とに結ぶチップである．

▷ front-side bus
▷ northbridge

ノースブリッジは**メモリコントローラ**を内蔵する．メモリコントローラは，FSB を通じて届くメモリアクセスのそれぞれを DRAM チップの信号に翻訳する．すなわち，アドレスを行アドレスと列アドレスに分割し，メモリバス上に RAS，CAS，ライトイネーブルを駆動する．したがって，キャッシュミスによるメモリアクセスは，プロセッサから FSB を通ってノースブリッジへ，そこからメモリバスを通って DRAM チップへと進み，データは同じ道を戻る．この各段階がレイテンシを加える上に，FSB は，プロセッサとノースブリッジの先のデバイスとの間のそれ以外のトラフィックもすべて運ぶ．

▷ memory controller

規模の目安として，64 ビット幅で 1333 MT/s のフロントサイドバスはすべてのトラフィックを合わせて約 10.7 GB/s を運ぶが，DDR4-3200 のチャネル 1 本だけで 25.6 GB/s である．

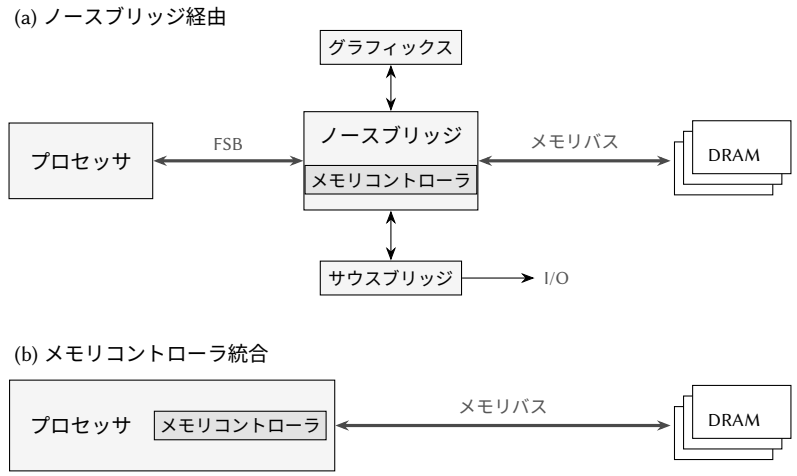


図 15.5: プロセッサと DRAM の間の経路．(a) ノースブリッジ内のメモリコントローラには，フロントサイドバスを通じて到達する．(b) メモリコントローラをプロセッサに統合すると，メモリバスはプロセッサに直接接続される．

新しいプロセッサは，メモリコントローラをプロセッサチップ上に統合し，メモリバスをプロセッサに直接接続する（図 15.5b）．[2] こうすると FSB とノースブリッジはメモリへの経路から消え，主記憶へのすべてのアクセスのレイテンシが下がる．それでもなお，この章にまとめた理由により，DRAM のアクセスはプロセッサよりはるかに遅く，第 12–14 章のキャッシュは欠かせないままである．第 16 章では，さらに遅いが電源がなくてもデータを保つ記憶装置に話を移す．

[2] メモリコントローラをプロセッサダイに載せた最初の主流の x86 プロセッサは AMD の Athlon 64（2003）であり，Intel は 2008 年の Nehalem で続いた．ノースブリッジの残り—グラフィックスと PCI Express のリンク—もプロセッサ側に移り，遅いデバイスのためのチップセットが 1 つ残った．いまや各プロセッサチップが自分のメモリをもち，他のチップのメモリへのアクセスは相互接続網を越える．第 18 章の NUMA である．

要点 — 章のまとめ
RAM は，6 個のトランジスタからなるセルがインバータの輪にビットを保持する SRAM として，あるいは 1 個のトランジスタからなるセル（例えばトレンチセル）がキャパシタ上の電荷としてビットを保持する DRAM として作られる．SRAM は速くて高価であり，キャッシュに使われる．DRAM は密度が高くて安価であり，主記憶に使われるが，その読み出しは破壊的で書き戻しが続き，漏れ電流がすべての行のリフレッシュを強いるので，T/R の区間ごとに 1 行をリフレッシュしなければならない．DRAM チップは，行デコーダで行を選び，センスアンプでそれをラッチに読み出し，列マルチプレクサで 1 ビットを選ぶ．行アドレスと列アドレスはアドレスピンを共有し，RAS と CAS で区別される．そしてファストページモードは，1 つの行への複数のアクセスを 1 回の開きで処理する．メモリアクセスは，伝統的にはフロントサイドバスを通ってノースブリッジ内のメモリコントローラへ，さらにメモリバスを通って進む．新しいプロセッサはメモリコントローラを統合し，メモリバスを直接駆動する．

第 16 章

ストレージ

講義動画	Lesson 16，動画 1–14
教科書	H&P [1] §2.2，§D.1–D.2
キーワード	磁気ディスク，プラッタ，シリンダ，セクタ，シーク時間，回転待ち時間，光ディスク，磁気テープ，順次アクセス，フラッシュメモリ，SSD，ハイブリッドディスク，I/O バス，メザニンバス

第 15 章では，主記憶が DRAM で作られており，電源を切れば直ちに内容を失うことを見た．残しておかなければならないプログラムとデータは，ストレージ装置に保存しなければならない．ストレージ装置は，仮想記憶（第 13 章）が物理メモリに収めきれないページも保持する．この章では，主要なストレージ技術–磁気ディスク，光ディスク，磁気テープ，および DRAM やフラッシュメモリによる半導体ストレージ–を，そのアクセス時間を何が決めるか，そしてそれがどのように進歩するかに重点を置いて述べ，ストレージとその他の I/O デバイスがプロセッサにどのように接続されるかを示す．

16.1 ストレージ

ストレージはデータを**永続的に**保持する．主記憶とは異なり，ストレージ装置は計算機の電源を切っても内容を保つ．その用途は 2 つある．

出典 Lesson 16，動画 1–2；H&P §D.1．

ファイル プログラム，それが扱うデータ，システムとアプリケーションの設定はファイルに保存され，これらはあるセッションから次のセッションへ残らなければならない．

仮想記憶 物理メモリに収まらないページはストレージに保持され，ページフォールトはそのページをメモリへ読み戻す（第 13 章）．

ストレージ装置についてアーキテクトが注目する性質は 2 つある．その**性能**は第 2 章の 2 つの指標で記述される．すなわち，毎秒処理されるデータ量あるいは要求数であるスループットと，1 回のアクセスに要する時間であるレイテンシである．ストレージのスループットは時代とともに向上するが，プロセッサの速度ほど速くは向上しない．ストレージのレイテンシの改善はさらに遅く，DRAM のレイテンシの改善よりもなお遅い．そのため，プロセッサとストレージの隔たりは第 1 章のメモリウォールよりもさらに大きい．その**信頼性**も少なくとも同じくらい重要である．ストレージはデータが安全に保たれると期待される場所であり，ファイルを失うことは，プログラムが落ちることよりも深刻な場合が多い．信頼性は第 17 章の主題である．

読み出しレイテンシ：DRAM は 60–90 ns，フラッシュ SSD はおよそ 100 µs，ディスクはおよそ 10 ms である（7-cpu.com，ベンダのデータシート）．

16.2 磁気ディスク

磁気ディスク（ハードディスク）は，磁性体の被膜の微小な領域の磁化の向きとしてビットを記録する．この被膜で覆われた回転する円板と，その上を動くヘッドとから作られる（図 16.1）．

▷ magnetic disk

出典 Lesson 16，動画 3；H&P §D.1．

被膜は，剛体の円板である**プラッタ**の両面を覆う．複数のプラッタが共通の**スピンドル**に積み重ねられ，スピンドルはモータによって一定の速度で回されるので，すべてのプラッタは一体となって回転する（図 16.1a）．

▷ platter
▷ spindle

各面は，それぞれ専用の**磁気ヘッド**によって読み書きされる．ヘッドはアームの先端に取り付けられ，面に触れることなくごくわずかな距離だけ浮いている．回転している面にヘッドが触れれば，面を傷つけてしまう．すべてのアームは 1 つのアクチュエータに取り付けられ，**ヘッドアセンブリ**として一体で動いて，プラッタの外周からスピンドルまでの任意の位置にヘッドを置くことができる．

▷ disk head

ヘッドアセンブリを 1 つの位置に留めておくと，プラッタが回転するにつれて各ヘッドは自身の面上の 1 つの円周をたどる．この円周が**トラック**であり，1 つの面は多数の同心円状のトラックを持つ（図 16.1b）．ヘッドは一体で動くので，各面に 1 つずつ，常にスピンドルから等しい距離にあるトラックの上に位置す

▷ track

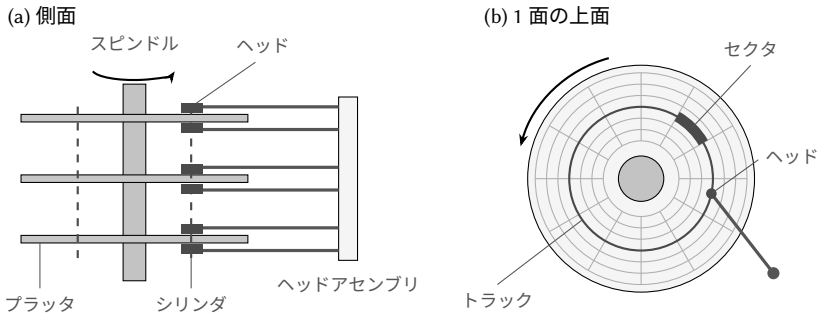


図 16.1: 磁気ディスクの構造．(a) スピンドル上のプラッタと，1 面につき 1 つのヘッド．すべてのヘッドのアームはヘッドアセンブリをなし，破線は 1 つのシリンダを示す．(b) 1 つの面．同心円状のトラックに分けられ，各トラックはセクタに分けられる．1 つのトラックとそのセクタの 1 つを強調してある．

る．これらのトラックはまとめて**シリンダ**をなす．すなわち，ヘッドアセンブリを動かすことがシリンダの選択であり，ヘッドの 1 つを選ぶことが面の選択，したがってそのシリンダ内の 1 本のトラックの選択である．

▷ cylinder

トラックに沿って，データは固定長のブロックに分割される．このブロックを**セクタ**といい，ディスクが読み書きする最小の単位である．セクタは短いプリアンブルで始まり，ディスクはこれによってセクタの先頭を認識し，そのビットに同期する．続いてデータが並び，セクタの末尾には誤り訂正符号が置かれる．ディスクはこれを用いて，読み出したデータを検査し，可能であれば訂正する（第 17 章）．

▷ sector

ディスクの容量は，この構成から直ちに求まる．

$$\text{容量} \;=\; \text{プラッタ数} \times 2 \times \frac{\text{トラック数}}{\text{面}} \times \frac{\text{セクタ数}}{\text{トラック}} \times \frac{\text{バイト数}}{\text{セクタ}}, \tag{16.1}$$

ここで係数 2 はプラッタの両面を数えたものであり，1 面当たりのトラック数はシリンダ数に等しい．

実際のディスクは，長い外周のトラックにより多くのセクタを置く．その場合，1 トラック当たりのセクタ数は平均値である．

例 16.1. あるディスクが 2 枚のプラッタを持ち，1 面当たり 100 000 トラック，1 トラック当たり平均 2000 セクタ，1 セクタ当たり 4096 バイトであるとする．式 (16.1) により $2 \times 2 \times 100\,000 \times 2000 = 8 \times 10^8$ セクタ，すなわち $8 \times 10^8 \times 4096 \approx 3.3 \times 10^{12}$ バイト，およそ 3.3 TB を保持する．

注意 ディスクの容量は 10 進である．1 TB は 2^{40} ではなく 10^{12} バイトなので，4 TB のドライブが保持するのは 3.64 TiB にすぎない．

16.3 ディスクのアクセス時間

セクタを読み書きするには，ディスクは正しいヘッドを回転する面の正しい位置に持っていかなければならず，その後データが主記憶まで運ばれなければならない．プラッタがすでに回転しているディスクでは，1 回のアクセスの時間は 5 つの部分からなる．

出典 Lesson 16，動画 4–5；H&P §D.1.

シーク時間 **シーク時間**とは，ヘッドアセンブリが，目的のセクタを含むシリンダまでヘッドを動かすのに要する時間である．ヘッドが移動する距離とともに増える．

▷ seek time

回転待ち時間 **回転待ち時間**とは，ヘッドがトラックに到達してから，セクタの先頭がヘッドの下に回ってくるまでの時間である．

▷ rotational latency

データ読み出し時間 セクタは回転の速度でヘッドの下を通過し，その間に読み出される（あるいは書き込まれる）．連続する複数のセクタへのアクセスは，それに応じて長くかかる．

コントローラ時間 ディスクコントローラが要求を処理する時間である．たとえば読み出した各セクタの誤り訂正符号の検査がこれに当たる．

I/O バス時間 データはディスクと主記憶の間を I/O バス（16.9 節）を通じて転送される．

$$t_{\text{access}} \;=\; t_{\text{seek}} + t_{\text{rot}} + t_{\text{read}} + t_{\text{controller}} + t_{\text{bus}}. \tag{16.2}$$

ディスクが先行する要求でまだふさがっていれば，新しい要求は自身のアクセスが始まる前に待ち行列で待つことにもなる．この待ち行列遅延—先行するアクセ

スが終わり，ヘッドが再び動けるようになるまでの時間–は無視できないことが多い．

回転に依存する 2 つの項は回転速度から求まる．回転速度は通常，毎分回転数（RPM）で与えられる．1 回転は 60/RPM 秒かかる．ヘッドが到達したとき，セクタの先頭はトラック上のどこにあるか分からないので，回転待ち時間は 0 から 1 回転の間にあり，平均では半回転である．連続する n セクタの読み出しには，1 回転の n/(トラック当たりのセクタ数) の割合だけかかる．

$$\overline{t_{\text{rot}}} = \frac{1}{2} \cdot \frac{60}{\text{RPM}}, \qquad t_{\text{read}} = \frac{n}{\text{トラック当たりのセクタ数}} \cdot \frac{60}{\text{RPM}}. \tag{16.3}$$

例 16.2. あるディスクが 10 000 RPM で回転し，平均シーク時間が 4 ms であるとする．ある要求が，500 セクタを持つトラックの連続する 100 セクタを読む．コントローラは 0.3 ms, I/O バスでの転送は 0.5 ms を要する．1 回転は $60\,000/10\,000 = 6$ ms かかるので，式 (16.3) により平均の回転待ち時間は 3 ms，データ読み出し時間は $100/500 \times 6\,\text{ms} = 1.2\,\text{ms}$ である．式 (16.2) により，このアクセスには平均で $4 + 3 + 1.2 + 0.3 + 0.5 = 9$ ms かかる（図 16.2）．もし 100 セクタがディスク上に散在していれば，そのそれぞれに固有のシークと回転待ち時間が必要となり，この要求には 700 ms を超える時間がかかったはずである．

ドライブは一度に 1 つのアクセスしか処理しない．9 ms は毎秒およそ 110 要求である．ベンダはこれを IOPS と呼ぶ．フラッシュ SSD は 100 000 以上に達する．

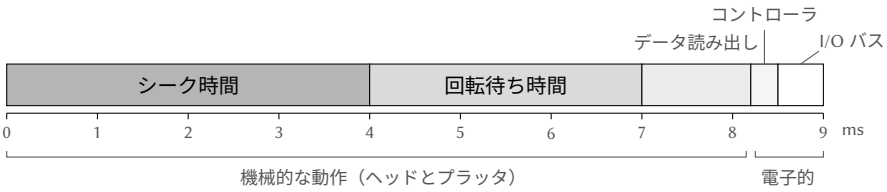


図 16.2: 例 16.2 のアクセスを実寸で描いたもの．9 ms のうち 8.2 ms を機械的な動作が占める．

ヒント 1 ms は，1 GHz 当たり 100 万サイクルと読めばよい．上の 9 ms は 3 GHz のプロセッサで 2700 万サイクルである．

> **要点**
> ディスクのアクセスは機械的な動作–ヘッドアセンブリの移動と，プラッタが回ってくるのを待つこと–に支配される．それはミリ秒，すなわちプロセッサのクロックで数百万サイクルを要する．ページフォールト（第 13 章）がこれほど高価なのはこのためである．

16.4 磁気ディスクの動向

磁気ディスクの構成要素は，きわめて異なる速度で改善する．

出典 Lesson 16，動画 6；H&P §D.2.

容量 は，面上にどれだけ密にビットを記録するかで決まる．長年にわたり 1–2 年でおよそ倍になってきた．

2010 年頃からこの成長はかなり鈍っている．

シーク時間 は通常 5–10 ms であり，ごくゆっくりとしか改善しない．その改善分は主に，プラッタの直径を小さくしてヘッドの移動距離を短くすることから来る．

注意 公称のシーク時間はランダムなシークの平均である．隣接するトラックへのシークは 1 ms 未満，全ストロークのシークは平均のおよそ 2 倍かかる．

回転速度 はゆっくりと上がり，およそ 5000 RPM から，最も速いドライブでは 10 000–15 000 RPM になった．大半のドライブは 5400–7200 RPM で回る．回転を速くするにはより強い材料が必要であり，騒音と発熱と消費電力も増える．

コントローラ時間とバス時間 は電子的であり，順調に改善するが，アクセス時間のうちのわずかな部分でしかない（図 16.2）．

したがってディスクのアクセスのレイテンシは，ムーアの法則（第 1 章）が述べるトランジスタ技術の進歩ではなく，機械と材料によって決まる．[1] 容量が長年にわたり急速に伸びたにもかかわらず，レイテンシはプロセッサの速度よりはるかにゆっくりとしか改善しない．記録密度を高めれば，1 回転でヘッドの下を通るバイト数が増えるので，一定量のデータの読み出し時間は確かに短くなる．しかし，シーク時間にも回転待ち時間にも影響しない．

[1] 磁気ディスクは，現役の計算機構成要素で最も古い．IBM 350（1956 年）は 24 インチのプラッタ 50 枚に 500 万文字（約 3.75 MB）を蓄えて 1200 RPM で回り，1 本のアクセスアームはトラックまで約 0.6 s を要した．容量は 6 桁伸びたが，アクセス時間は 2 桁も縮まっていない．

16.5 光ディスク

光ディスクは，磁気ディスクのプラッタ 1 枚のように回転する 1 枚の円板であり，データはトラックに沿って並ぶ．ただしビットは，面が光を反射する仕方を変えるマークとして記録され，磁気ヘッドではなくレーザで読み出される．レーザはディスクから離れた位置で働き，透明な保護層を通して焦点を結ぶので，面に付いた埃や指紋の害は，ドライブ内に密閉されていなければならない磁気ディスクのプラッタの場合よりはるかに小さい．そのため光ディスクはドライブから取り出し，別のドライブへ持ち運べる．すなわち**可搬**である．

▷ optical disk

出典 Lesson 16，動画 7．

可搬性には代償がある．あるドライブで書かれたディスクは他のすべてのドライブで読めなければならないので，記録形式は業界標準—CD，次いで DVD，次いで Blu-ray—によって固定されていなければならない．より高密度な記録技術が役に立つのは，新しい標準が合意され，それに対応するドライブと媒体とコンテンツが普及した後に限られ，それには何年もかかる．したがって光ディスクの進歩は，長い間隔をおいた大きな飛躍という形をとる．磁気ディスクのプラッタはドライブから出ることがない．製造者は，ドライブが同じ標準インタフェースで接続される限り，次の製品でより優れた記録技術を採用できる．

CD，DVD，Blu-ray ディスクでは，データは 1 本のらせん状のトラックに沿って並ぶ．

CD（1982 年）は 700 MB，DVD（1996 年）は 4.7 GB，Blu-ray（2006 年）は 1 層あたり 25 GB．各標準は前の標準の 10–14 年後に，5–7 倍の容量で現れた．

16.6 磁気テープ

磁気テープは，磁性体を塗布した細長い帯にデータを記録する．帯はカートリッジに巻かれて収められ，ドライブの読み書きヘッドの前を引き出される．テープは容量が大きく，カートリッジは交換できる．1 台のドライブで，安価なカートリッジを何本でも読み書きできる．その欠点はデータへの到達の仕方にある．

▷ magnetic tape

出典 Lesson 16，動画 8–9．

定義 16.3 (順次アクセス). データに順番にしか到達できないとき，その媒体は**順次アクセス**を提供するという．ある位置にアクセスするには，現在の位置と要求された位置の間にあるデータすべてを通り過ぎるまで媒体を動かさなければならず，アクセス時間はその距離とともに増える．

▷ sequential access

対照的にディスクは，間にあるデータを通り過ぎることなく，任意のセクタに直接到達する．1 回のシークと高々 1 回転は，セクタがどこにあっても高々数十ミリ秒である（例 16.4）．それでもアクセス時間はシーク距離に依存するので，ディスクがランダムアクセス（第 15 章）を提供するのは近似的にすぎないが，テープよりははるかにそれに近い．順次アクセスが特によく合う用途が 1 つある．**バックアップ**である．バックアップでは大量のデータが一度に通して書かれ，読み戻されることは稀で，読み戻すときもたいていは一度に通して読む．順次アクセスは仮想記憶には適さない．仮想記憶のページフォールトは任意の位置からページを読むからである．したがってテープは，プログラムやファイルを使うためのストレージではなく，バックアップ用のストレージとして使われる．

LTO-9 のカートリッジ（2021 年）は 18 TB を保持し，400 MB/s で流すので，端から端まで読むのにおよそ 12 時間かかる（LTO コンソーシアム）．

例 16.4. 1000 m のテープが 5 m/s で巻き取られるとする．テープが先頭にあるとき，中央のデータには 100 s 後，末尾のデータには 200 s 後に到達する．最長のシークが 15 ms，1 回転が 6 ms のディスクは，どのセクタにもおよそ 21 ms 以内で到達する．これはテープが末尾に到達するよりおよそ 9500 倍速い．バックアップ全体を先頭から末尾まで読む場合には，巻き取りの時間は問題にならない．

テープは次第に置き換えられつつある．生産量が少ないため，ギガバイト当たりのコストは，膨大な数が作られる磁気ディスクほど速くは下がらない．ディスク— たとえば USB で接続する外付けディスク—は，バックアップの役割の多くを引き受けられるほど安価になった．

16.7 SSD

半導体メモリには可動部分がなく，そのアクセスには，ディスクのミリ秒ではなく，ナノ秒からミリ秒に満たない程度の時間しかかからない．しかしギガバイト当たりのコストははるかに高い．DRAM のギガバイト当たりの価格は磁気ディスクの百倍程度であり，一方でレイテンシはおよそ 100 000 分の 1 である（数十ナノ秒に対してミリ秒）．半導体メモリで作られたストレージは，速度がこのコ

出典 Lesson 16，動画 10；H&P §2.2，§D.1．

ストに見合う場所で使われる．

定義 16.5 (SSD). **SSD** とは，可動部分を持たず半導体メモリで作られたストレージ装置であって，ディスクのように使われるものである．データをブロック単位で保存し，SATA のようなディスクインタフェース，あるいは PCI Express を通じて直接接続される．

▷ solid-state disk

SSD は 2 種類のメモリのいずれでも作れる．

電池付きの DRAM DRAM は揮発性なので，計算機の電源が切れている間は電池が電力を供給し続ける．この種の SSD はきわめて高速で，ギガバイト当たりの価格もきわめて高い．データが安全なのは電池が保つ間だけであり，長期間データを保つ用途には向かない．

フラッシュメモリ 大半の SSD は**フラッシュメモリ**を使う．これは電源がなくても内容を保つ半導体メモリであり，ビットはトランジスタに閉じ込められた電荷として蓄えられ，そこに何年も留まる．

▷ flash memory

フラッシュの SSD は DRAM より遅いが，それでも磁気ディスクよりはるかに速く，消費電力が小さく，電池なしでデータを保つ．ギガバイト当たりのコストは DRAM とディスクの中間にあるので，同じ価格ではディスクよりずっと小さい容量しか得られない．長い間，典型的なフラッシュ SSD の容量はギガバイト単位であり，ディスクはテラバイト単位であった．

注意 フラッシュメモリの「フラッシュ」は，パイプラインのフラッシュ（第 3 章）とは何の関係もない．

フラッシュは上書きができない．数 MB のブロック全体をまず消去しなければならず，1 つのブロックが耐えられる消去は数千回程度である．

16.8 磁気ディスクとフラッシュのハイブリッド

磁気ディスクとフラッシュメモリの長所は互いに補い合う．磁気ディスクは，ギガバイト当たりの低いコストで巨大な容量を提供するが，遅く，プラッタが回っている間はかなりの電力を消費し，動作中の衝撃に弱い．フラッシュメモリは速く，消費電力が小さく，可動部分を持たないが，ギガバイト当たりのコストは高い．

出典 Lesson 16，動画 11–12．

定義 16.6 (ハイブリッドディスク). **ハイブリッドディスク**とは，磁気ディスクと，それより少量のフラッシュメモリとを 1 つの装置にまとめたものである．磁気ディスクが全データを保持し，フラッシュメモリは最も頻繁に使われるデータのキャッシュとして働く（図 16.3）．

▷ hybrid disk

注意 フラッシュはキャッシュであって，容量の追加ではない．8 GB のフラッシュを持つ 2 TB のハイブリッドディスクは，売られ方も振る舞いも 2 TB のドライブである．

時間的局所性と空間的局所性（第 12 章）は，メモリアクセスと同様にファイルにも当てはまる．オペレーティングシステム，日常的に使うアプリケーション，作業中のファイルが，アクセスの大半を占める．これらをフラッシュに置いておけば，大半の要求はフラッシュの速度で，機械的な動作なしに処理される一方，容量とギガバイト当たりのコストはディスクのそれに近いままである．要求がフラッシュから処理されている間は，ディスクの回転を止めることさえでき，電力を節約し，装置は衝撃に強くなる．そのときディスクを必要とする要求が来れば，プラッタが再び回り出すまで，しばしば数秒待つことになる．

平均アクセス時間は，キャッシュの AMAT（式 (12.1)）と同じように振る舞う．フラッシュから処理される要求はヒット時間を要し，ディスクが処理する要求はミスの全体の時間を要する．コントローラは各ブロックがどこにあるかを知っているので，ミスの際にまずフラッシュを読むことはなく，ミスペナルティは 2 つの時間の差である．

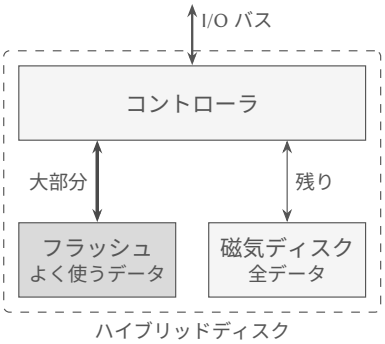


図 16.3: ハイブリッドディスク．大半の要求はフラッシュのキャッシュが処理する．

例 16.7. あるハイブリッドディスクが，アクセス時間 0.1 ms のフラッシュと，回転を保ったままの例 16.2 のディスクとを組み合わせているとする．その例の要求について，ヒットでもコントローラと I/O バスの時間は支払うので $0.1+0.3+0.5 = 0.9$ ms であり，ミスは 9 ms かかるので，ミスペナルティは 8.1 ms である．要求の 95 % がフラッシュでデータを見つけるなら，平均アクセス時間は $0.9 + 0.05 \times 8.1 \approx 1.3$ ms となり，フラッシュがデータのごく一部しか保持していないにもかかわらず，ディスク単体のおよそ 7 分の 1 で済む．いまやヒットの時間の大半は，コントローラと I/O バスが占める．

ヒント ヒット時間は下限である．ヒット率をどれだけ高めても，このディスクはディスク単体の $9/0.9 = 10$ 倍を超えられない（Amdahl の法則，第 2 章）．

表 16.1 に，この節と前節の技術をまとめる．どれが望ましいかは，何を最も重視するかによる．大量のデータを最も安く置くなら磁気ディスクが，速度と低消費電力と頑健さならフラッシュが有利であり，アクセスがデータのごく一部に集中するワークロードでは，ハイブリッドディスクが両方に近づく．

表 16.1: 磁気ディスクと半導体ストレージの比較．

	磁気ディスク	DRAM SSD	フラッシュ SSD	ハイブリッド
アクセス	遅い（ms）	最も速い	速い	大半は速い
GB 当たりのコスト	低い	非常に高い	中程度	低い
消費電力	大きい	大きい	小さい	より小さい
可動部分	あり	なし	なし	あり
無電源での保持	する	電池が保つ間	する	する

要点
ハイブリッドディスクは，キャッシュの原理をストレージに適用したものである．磁気ディスクの前にフラッシュを置くことで，大半の要求にフラッシュの速度と低い消費電力を与えつつ，ギガバイト当たりのコストはディスクのそれにほぼ等しく保つ．

16.9 I/O デバイスの接続

ストレージ装置は，他の入出力（I/O）デバイスと同様に，プロセッサに直接取り付けられてはいない．業界標準で定められたバスを通じて接続される．

出典 Lesson 16，動画 13–14．

定義 16.8 (I/O バス). **I/O バス**とは，I/O デバイスを計算機に接続するための標準化されたバスである．標準はコネクタ，電気信号，プロトコルを定める．PCI Express，SATA，SCSI，USB がその例である．

▷ I/O bus

標準化によって，標準に従って作られたデバイスは，どちらを誰が製造したかによらず，そのバスを備えるどの計算機でも動作し，計算機の製造者は多様なデバイスに対して 1 種類のインタフェースを用意すればよくなる．その代償は，改善の速度が制限されることである．標準の新しい版は多くの企業の合意を必要とし，古い版に合わせて作られたデバイスも動き続けるべきなので，バスはより速い技術が利用可能になっても直ちにはそれを採用できない．[2] ある標準に依存するデバイスが多いほど，その標準の変化は遅くなる．

I/O デバイスは，必要とするスループットも大きく異なる．グラフィックスアダプタは膨大な量のデータを動かし，ディスクはそれよりはるかに少なく，キーボードはほとんど動かさない．速いバス 1 本だけにすれば，すべてのデバイスが高価で速いインタフェースを持たされることになり，遅いバス 1 本だけにすれば，速いデバイスの性能が頭打ちになってしまう．そこでバスは階層をなすように配置される（図 16.4）．

[2] PCI Express は世代ごとに 1 レーン当たりの帯域を倍にしてきた．2003 年の 2.5 GT/s から 5，8（より効率のよい 128b/130b 符号化で倍にした），16（ここで 7 年待たされた），32，そして 2022 年の 64 GT/s である．互換性は双方向に保たれ，古いカードは新しいスロットで，新しいカードは古いスロットで，遅い方の速度で動く．

定義 16.9 (メザニンバス). **メザニンバス**とは，プロセッサと，より遅い I/O バスとの間にある，速くて短い I/O バスである．今日ではそれは PCI Express である．グラフィックスアダプタのような高速なデバイスはこれに直接接続され，より遅いバスのコントローラもこれに取り付けられる．

▷ mezzanine bus

メザニンバス上の SATA コントローラや SCSI コントローラは，ディスクや光ディスク装置のための SATA（あるいは SCSI）バスを提供し，USB ホストコントローラは，キーボードや USB メモリのような外部デバイスのための，ハブで拡張される USB バスを提供する．これらのバスは，本来的に遅く，速度よりも標準が安定していることの方が重要であるようなデバイスを扱う．メザニンバスより遅く，変更される頻度も低い．PC のチップセットでは，SATA コントローラと USB コントローラは 1 つのチップ，**サウスブリッジ**に統合され，サウスブリッジはノースブリッジ（第 15 章）に接続される．

SATA 3 は 6 Gb/s（600 MB/s），USB 3.2 Gen 2 は 10 Gb/s，16 レーンの PCIe 5.0 スロットはおよそ 63 GB/s を運ぶ．キーボードは毎秒数百バイトあればよい（SATA-IO，USB-IF，PCI-SIG の規格）．

▷ southbridge

各コントローラは，自身が提供するバスとメザニンバスとの間を橋渡しする．ディスク読み出しのデータは，ディスクから SATA を通って SATA コントローラへ，メザニンバスを通り，ノースブリッジを抜けて主記憶へと進む．この経路が式 (16.2) の I/O バス時間である．速いデバイスはプロセッサの近くに位置し，多数の遅いデバイスはコントローラの背後にある自分たちのバスに閉じ込められて，速いデバイスの足を引っ張らない．

新しいプロセッサはノースブリッジを統合し，サウスブリッジに直接接続する．

技術が何であれ，どのように接続されていようと，ストレージはデータが生き

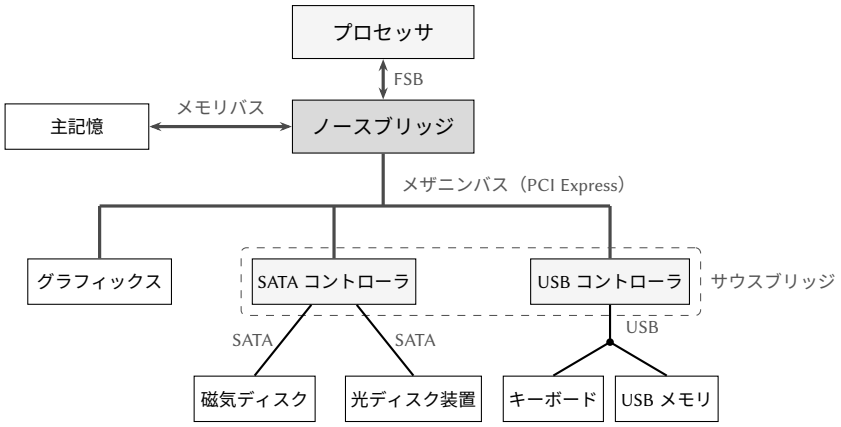


図 16.4: バスの階層．グラフィックスアダプタはメザニンバスに直接接続される．ディスクなどのより遅いデバイスには，メザニンバス上の SATA コントローラと USB コントローラを介して到達する．これらのコントローラはより遅い標準 I/O バスを提供する．PC のチップセットでは，これらのコントローラがサウスブリッジ（破線）をなす．

残ると期待される場所である．しかしあらゆるストレージ装置はいずれ故障する．第 17 章では，そうした故障があってもシステムがデータとサービスを保ち続ける方法を示す．

要点 — 章のまとめ

ストレージはファイルと仮想記憶のページを永続的に保持し，そのレイテンシは DRAM のそれよりもさらにゆっくりとしか改善しない．磁気ディスクは，回転するプラッタのトラック上のセクタにデータを蓄える（式 (16.1)）．そのアクセス（式 (16.2)）はシーク時間と，平均で半回転である回転待ち時間とに支配され，容量が長年にわたり急速に伸びた一方で，この機械的なレイテンシはゆっくりとしか改善しない．光ディスクは可搬だが，標準化のために進歩が遅い．磁気テープは順次アクセスで安価な容量を提供し，バックアップに使われる．DRAM やフラッシュメモリで作られた SSD は，ギガバイト当たりの高いコストと引き換えに機械的な動作を避け，ハイブリッドディスクはフラッシュを磁気ディスクのキャッシュとして使う．I/O デバイスは，階層をなすように配置された標準化された I/O バスを通じて接続される．速いデバイスは PCI Express のような速くて短いメザニンバスに直接つながり，より遅いデバイスは SATA や USB のような，より遅く安定したバスにつながって，それらのコントローラがメザニンバスに取り付けられる．

第 17 章

フォールトトレランス

講義動画	Lesson 17，動画 1–32
教科書	H&P [1] §1.7，§D.2–D.3
キーワード	ディペンダビリティ，フォールト，エラー，故障，信頼性，可用性，MTTF，MTTR，フォールトトレランス，チェックポインティング，N モジュール冗長，パリティ，ECC，RAID

第 16 章では，データを永続的に保存する装置を扱った．ストレージ装置を含め，計算機のあらゆる構成要素は，いずれ意図された振る舞いから逸脱する．構成要素が 1 つでもそうなるたびに誤った結果を返したりデータを失ったりするシステムは，信頼して使うことができない．この章では，そうした問題を論じるための用語—フォールト，エラー，故障，および信頼性と可用性—を導入し，実際に起こるフォールトを分類し，それらを許容する一般的な技法を示す．後半ではこれらの考え方をディスクに適用する．独立ディスク冗長アレイ（RAID）は複数のディスクを 1 台にまとめたものであり，その冗長な構成により，ディスクが丸ごと故障してもデータは失われず，アクセスも可能なままに保たれる．

17.1 ディペンダビリティ

計算機が有用であるのは，その利用者が，計算機の提供するサービスを信頼できる場合に限られる．これを厳密にするために，システムの振る舞いに関する 2 つの記述を区別する．**仕様サービス**とは，仕様に従えばシステムが持つべき振る舞いであり，**提供サービス**とは，システムが実際に示す振る舞いである．

出典 Lesson 17，動画 1–2；H&P §1.7；[33]．

定義 17.1 (ディペンダビリティ). システムの**ディペンダビリティ**とは，そのサービスの提供についてシステムに依拠することを正当化する，提供サービスの品質である．

▷ dependability

システムは構成要素，すなわち**モジュール**から作られる．プロセッサ，メモリモジュール，ディスク，さらに細かく見ればそれらの内部の回路やプログラムの関数である．各モジュールはそれ自身の仕様上の振る舞いを持つ．ディペンダビリティの問題は，あるモジュールがその仕様から逸脱したときに始まる．そのような逸脱がシステム全体の逸脱に変わりうること，しかし必ずそうなるわけではないことを，次節で述べる．

17.2 フォールト・エラー・故障

1 つのモジュールの内部で始まった問題は，システムの利用者が観測できるようになるまでに最大 3 つの段階を経る．段階に別々の名前が与えられているのは，ディペンダブルなシステムが，段階の間の遷移のいずれかで問題を食い止めようとするからである．

出典 Lesson 17，動画 3–5；H&P §1.7，§D.3．

定義 17.2 (フォールト・エラー・故障). **フォールト**とは，モジュールが仕様上の振る舞いから逸脱することである．欠陥のあるトランジスタや，プログラムの誤りがその例である．

▷ fault

エラーとは，システム内部の実際の振る舞い—その内部状態—が仕様上の振る舞いと異なっている状態である．変数に誤った値が格納されている場合がその例である．

▷ error

故障とは，システムの提供サービスが仕様サービスから逸脱することである．

▷ failure

フォールトは，何の影響も及ぼさないまま長期間存在しうる．フォールトのあるモジュールが，その逸脱を露わにする形で使われるまでは，フォールトは**潜在**エラーである．そのような使われ方をすると，フォールトは**活性化**され，**顕在**エラーとなる．内部状態がこの時点で誤ったものになる．顕在エラーが故障を引き起こすのは，それがシステムの提供するサービスまで伝播した場合だけである．

すべてのフォールトが活性化されるわけではなく，すべてのエラーが故障になるわけでもない（図 17.1）．

例 17.3. 2 つの整数を掛ける関数にプログラムの誤りがある．引数が 6 と 9 の場合を除けば正しい積を返すが，この場合だけ 54 ではなく 45 を返す．この誤りは関数が書かれた時点からフォールトであり，どの呼び出し元も 6 と 9 を掛けない限り潜在したままである．プログラムがこの引数で関数を呼び出し，結果を格納すると，フォールトは活性化され，プログラムの状態はエラーを含む．その結果が予約システムの確保する座席数であれば，54 席ではなく 45 席が確保される．エラーは提供サービスに到達し，故障が起きたことになる．一方，プログラムが積の正負だけを判定するのであれば，誤った値でも正しい値と同じ判断に至り，故障は起きない．

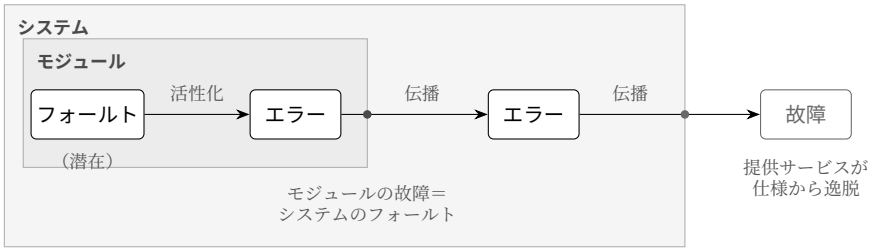


図 17.1: フォールトは活性化されるとエラーになる．モジュールの外に出たエラーは，そのモジュールの故障であると同時に，そのモジュールを含むシステムのフォールトである．提供サービスに到達したエラーはシステムの故障である．

この 3 つの用語は，どの境界を考えるかに相対的である．モジュールの故障は，そのモジュールを含むシステムから見れば，そのシステムのフォールトである．ヘッドがクラッシュしたディスクは，ディスクとしては故障している．しかし，データの冗長な複製を保持するディスクアレイ（17.8 節）にとって，同じ事象はフォールトであり，アレイの故障につながるとは限らない．

17.3 信頼性と可用性

システムは，その仕様サービスに関して，任意の時点で 2 つの状態のいずれかにある．**サービス遂行**の状態では提供サービスが仕様サービスと一致しており，**サービス中断**の状態では一致していない．故障はシステムを第 1 の状態から第 2 の状態へ移し，復旧（修復）はシステムを元の状態に戻す（図 17.2）．それぞれの状態で過ごす時間から，ディペンダビリティの異なる 2 つの尺度が得られる．

出典 Lesson 17，動画 6–7；H&P §1.7.

このモデルに第 3 の状態はない．遅いが正しいサービスや，一部の利用者にだけ中断しているサービスも，どちらかに数えるほかない．

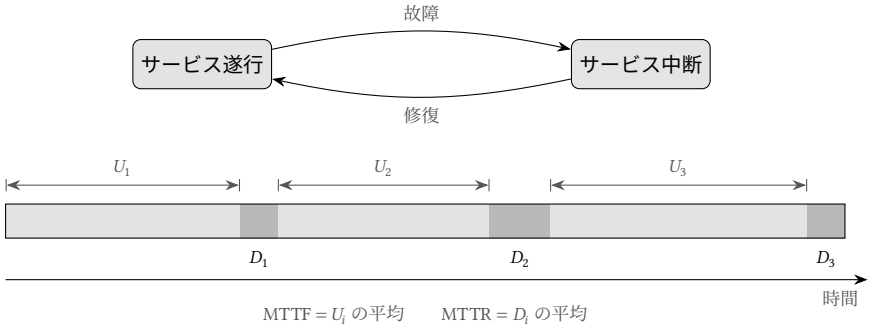


図 17.2: 2 つのサービス状態と，それらを交互に繰り返す履歴．U_i がサービス遂行の期間，D_i がサービス中断の期間である．

定義 17.4 (信頼性)．システムの**信頼性**とは，サービスが連続して遂行されることの尺度である．通常は**平均故障時間**（MTTF）で定量化される．MTTF とは，サービス遂行の期間の平均の長さ，すなわちサービス開始から次の故障までの平均時間である．

▷ reliability
▷ mean time to failure

定義 17.5 (可用性)．システムの**可用性**とは，サービス遂行に費やす時間の割合で

▷ availability

ある．サービス中断の期間の平均の長さを**平均修復時間**（MTTR）とすると，

▷ mean time to repair

$$\text{可用性} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}. \tag{17.1}$$

2 つの平均の和は，故障と修復の 1 周期の平均の長さであり，**平均故障間隔**と呼ばれる．

▷ mean time between failures

$$\text{MTBF} = \text{MTTF} + \text{MTTR}. \tag{17.2}$$

修復がサービスの期間に比べて短ければ，MTBF と MTTF はほぼ等しい．

注意 ディスクやサーバのデータシートが「MTBF」として示す値は，ここで MTTF と定義した量である．修復時間は含まれていない．

例 17.6. あるサーバが 1200 h 稼働し，6 h 停止し，900 h 稼働し，10 h 停止し，1500 h 稼働し，8 h 停止した．MTTF は $(1200 + 900 + 1500)/3 = 1200$ h，MTTR は $(6 + 10 + 8)/3 = 8$ h，MTBF は 1208 h であり，可用性は $1200/1208 \approx 99.34\,\%$ である．故障頻度が 10 倍（MTTF = 120 h）でありながら平均 0.8 h で復旧する別のサーバは，可用性は同じだが MTTF は 10 分の 1 であり，信頼性ははるかに低い．

ヒント 可用性は「ナイン」の数で語られる．1 年の 8766 h のうち，99.9 % は 8.8 h，99.99 % は 53 min，99.999 % は 5 min の中断を許す．

17.4 フォールトの種類

フォールトは 2 通りに分類される．原因による分類と，どれだけ長く続くかによる分類である．

出典 Lesson 17，動画 8–9；H&P §D.3.

17.4.1 原因による分類

ハードウェア **ハードウェアフォールト**は，ハードウェアが設計どおりに動作しなくなったときに生じる．スイッチしなくなったトランジスタや，プラッタに接触した磁気ヘッドがその例である．

▷ hardware fault

設計 **設計フォールト**とは，設計そのものの誤りである．ソフトウェアのバグや，初期の Intel Pentium の除算バグのようなハードウェア設計の誤りがこれにあたる．後者は，欠陥のある設計から作られたすべてのチップに存在した．

▷ design fault

運用 **運用フォールト**とは，システムの運用者や利用者が犯す誤りである．誤ったファイルを削除する，誤ったケーブルを抜く，などがその例である．

▷ operation fault

環境 **環境フォールト**は，システムの外部に由来する．火災，浸水，停電，極端な温度，意図的な破壊行為などである．

▷ environmental fault

17.4.2 継続時間による分類

永久 **永久フォールト**は，いったん起きると，フォールトのある部品が修理または交換されるまで続く．過熱で損傷したチップがその例である．

▷ permanent fault

間欠 **間欠フォールト**は，しばらく続いた後に消え，また再発する．緩んだコネクタや，定格を超えるクロック周波数で動作し，高温になったときだけ誤った計算を行うプロセッサがその例である．

▷ intermittent fault

過渡 **過渡フォールト**は一度だけ起き，ひとりでに消える．高エネルギー粒子によって反転したメモリのビットがその例であり，そのビットを書き直せば正しい動作に戻る．

▷ transient fault

2 つの分類は，フォールトの異なる性質を記述するものである．例えばハードウェアフォールトは，永久（焼き切れたチップ），間欠（温まったときだけ接触が失われるはんだ割れ），過渡（反転したメモリのビット）のいずれにもなりうる．一方で設計フォールトは，設計が修正されるまで設計のすべての複製に存在するので，たまにしか活性化されないとしても永久フォールトである．

注意 メモリの誤りの多くはこの種のものではない．Google のサーバの実地調査では，年間 8 % 超の DIMM が訂正可能な誤りに見舞われ，その多くは粒子の衝突ではなく素子側の恒久的な欠陥であった [34]．

Pentium の除算器では，2048 エントリの参照表のうち 5 エントリが誤った値を保持していた．チップ交換に要した費用は 4 億 7500 万ドルである（Edelman, SIAM Review 39(1), 1997）．

17.5 信頼性と可用性の向上

システムをよりディペンダブルにする方法は 3 つある．

出典 Lesson 17，動画 10；H&P §1.7.

フォールトを避ける そもそもフォールトが起きないようにすることを**フォールト回避**という．故障率の低い部品を使う，部品を低温に保つ，機械に液体を近づけない，などである．

▷ fault avoidance

フォールトを許容する フォールトが故障になることを防ぐことを**フォールトトレランス**という．一部のモジュールにフォールトがあっても，システムは

▷ fault tolerance

仕様サービスを提供し続ける．フォールトトレランスは冗長性の上に成り立つ．メモリの誤り訂正符号（17.7 節）や冗長なディスク（17.8 節）がその例である．

修復を速くする 故障からサービス復旧までの時間を短くする．予備部品を現地に置いておく，などである．

> **要点**
> フォールト回避とフォールトトレランスは故障の回数を減らすので，MTTF を高め，それによって信頼性と可用性の両方を高める．修復の高速化は MTTR を下げるだけである．モジュールが故障するたびに停止するシステムでは，可用性は改善するが信頼性は改善しない．

17.6 フォールトトレランスの技法

汎用的なフォールトトレランスは 2 つの要素を組み合わせる．エラーを**検出**する手段と，エラーから**回復**する手段である．

出典 Lesson 17，動画 11–13．

17.6.1 チェックポインティング

システムの完全な状態を一定間隔で保存し，後でその保存状態から実行を再開できるようにすることを**チェックポインティング**という．エラーが検出されると，システムは直近のチェックポイントを復元し，そこから実行をやり直す．フォールトが過渡的であれば，やり直しは成功する見込みが高く，間欠的であっても多くの場合は成功する．永久フォールトは再び活性化されるので，この方法では許容できない．回復にも時間がかかる．チェックポイントの復元と失われた作業のやり直しが速ければ，利用者は遅延を感じるだけで済むが，時間がかかりすぎれば，回復そのものがサービス中断とみなされる．

▹ checkpointing

プロセッサも小さな規模で同じことを行っている．第 8 章の分岐予測ミス回復は，分岐より後の作業を破棄し，コミット済みのレジスタ状態から実行を再開する．違うのは粒度だけである．

注意「過渡」が表すのはフォールトであって，あとに残るエラーではない．誤った値は何かが上書きするまで残る．やり直しが効くのは，チェックポイントがエラーより前にあるからにすぎない．

17.6.2 冗長化

定義 17.7 (*N* モジュール冗長). ***N* モジュール冗長**では，同一の *N* 個のモジュールが同じ入力に対して同じ計算を行い，その出力を比較するか多数決にかける（図 17.3）．

▹ N-module redundancy

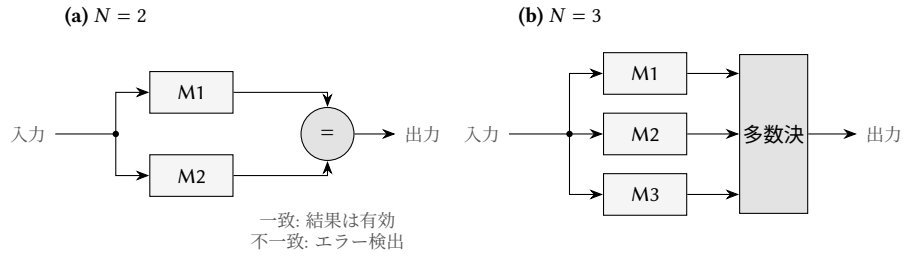


図 17.3: *N* モジュール冗長．(a) 2 つのモジュールと比較器は，一方のモジュールにフォールトがあることは検出できるが，どちらであるかは分からない．(b) 3 つのモジュールと多数決器は，いずれか 1 つのモジュールの誤った出力を隠蔽する．

モジュールが 2 つの場合（*2* **重モジュール冗長**），不一致はどちらか一方にフォールトがあることを示すが，どちらであるかは示さないので，エラーは検出できても比較だけでは訂正できない．チェックポインティングと組み合わせれば，システムは巻き戻して計算をやり直す．モジュールが 3 つの場合（*3* **重モジュール冗長**），正しい結果は少なくとも 2 つのモジュールが出した結果であり，フォールトのあるモジュールが 1 つであれば，巻き戻しなしに多数決で退けられる–そのフォールトが永久的であってもである．一般に，*N* 個のモジュールの多数決は最大 $\lfloor (N-1)/2 \rfloor$ 個のフォールトのあるモジュールを隠蔽する（表 17.1）．その代償は，ハードウェアとエネルギーが *N* 倍になることである．

注意 *N* 個のモジュールは 1 つの設計を共有するので，設計フォールトは許容できない．

表 17.1: N モジュール冗長が扱えるフォールトのあるモジュール．

N	フォールトのあるモジュールの影響
2	フォールトのあるモジュール 1 個を検出できる（出力が食い違う）が訂正はできない
3	フォールトのあるモジュール 1 個は 2 対 1 で退けられる
5	フォールトのあるモジュール 1 個は 4 対 1，2 個は 3 対 2 で退けられる

フォールトのあるモジュールが 1 個であれば，5 モジュールのシステムはさらにもう 1 個のフォールトを隠蔽できる．2 個の場合，正しい 3 個がなお 3 対 2 で多数を占めるが，余裕は残っていない．3 個目にフォールトが生じれば，正しい 2 個が退けられ，多数決は誤った結果を出してしまう．そのため，そうなる前に運用を停止する．5 台の飛行制御計算機を備えたスペースシャトルはこの方針に従っていた．1 台が故障しても任務は通常どおり続行され，2 台目が故障すると任務は打ち切られた．[1]

[1] 同一の IBM AP-101 5 台のうち，同じプログラムで多数決を行ったのは 4 台である．5 台目は，共有された設計フォールトに備えて独立に書かれたソフトウェアを実行していた．

17.7 メモリとストレージのための符号

メモリとストレージについては，モジュール全体を複製するのは過剰である．フォールトは通常，格納されたビットのうち数ビットを壊すだけであり，それらは誤り検出符号や誤り訂正符号により，はるかに少ない追加ビットで保護できる．

出典 Lesson 17，動画 14；H&P 第 2 章．

定義 17.8 (パリティ)．データビットの組 $d_1, \ldots, d_n$ の**パリティ**とは，それらの排他的論理和

$$p \;=\; d_1 \oplus d_2 \oplus \cdots \oplus d_n, \tag{17.3}$$

であり，パリティビットとは，p をデータとともに格納する 1 つの追加ビットである．

▷ parity

データを読み出すときにはパリティを再計算し，格納されているビットと比較する．パリティビット自身を含め，どの 1 ビットが反転しても結果が変わるので，1 ビットの誤りはすべて検出される．パリティはどのビットが反転したかを示さないので誤りを訂正することはできず，2 ビットが反転した場合は互いに打ち消し合って気づかれない．例えばバイト 10110010 は 1 が 4 個でパリティは 0 である．これが 10100010 として読み出されると，再計算したパリティは 1 となり，誤りが検出される．

定義 17.9 (誤り訂正符号)．**誤り訂正符号**（ECC）は，データビットの組に十分な数の検査ビットを付加し，限られた数の反転したビットの位置を特定して訂正できるようにしたものである．

▷ error-correcting code

ECC 付きのメモリモジュールは通常 SECDED 符号（1 ビット訂正・2 ビット検出）を用いる．例えば 64 データビットごとに 8 検査ビットを付加し，語の中の 1 ビットの反転は訂正し，2 ビットの反転は検出する．ディスクは各セクタを，Reed–Solomon 符号のようなはるかに強力な符号で保護する．これらの符号は多数の反転したビット，特に記録面の欠陥による隣接ビットのバースト誤りを訂正する．こうした符号はセクタ内の壊れたビットに対処するものであり，ディスクが丸ごと動かなくなった場合には役に立たない．その場合に必要なのは，ディスクをまたぐ冗長性である．

ヒント 検査ビットの数：$2^r \geq k + r + 1$ を満たす最小の r が，k データビット中の 1 ビット反転の位置を特定する．さらに 1 ビットで 2 ビット反転を検出する．$k = 64$ なら 8 ビット，すなわち 72 ビット語である．

17.8 独立ディスク冗長アレイ

複数のディスクを 1 つの集合にまとめ，単一のディスクの役割を果たさせることができる．その集合は，構成するどのディスクよりも大容量に，高速に，あるいは高信頼にできる．このような集合を独立ディスク冗長アレイ，すなわち **RAID** と呼ぶ．アレイには 2 つの目標がある．

- 単一のディスクより高い性能を得ること．
- セクタが不良になっても，ディスクが丸ごと故障しても，読み書きのサービスを通常どおり提供すること．

出典 Lesson 17，動画 15；H&P §D.2；[35]．

▷ redundant array of independent disks

すべての構成が両方の目標を追うわけではない．RAID 0（17.9 節）は性能だけを目指す．

アレイ内の各ディスクは，依然として自身の符号でセクタを保護している（17.7 節）．したがって，セクタが正しく読めないときや，ディスクがまったく応答しないとき，アレイは**どの**ディスクの**どの**ブロックが影響を受けたのかを知っている．アレイの冗長性は誤りを見つける必要はなく，位置の分かっているブロックの内容を復元しさえすればよい．メモリのパリティビットが誤りを検出できるだけであるのに対し，アレイでは 1 つのパリティブロックで誤りを訂正できるのは，このためである（17.11 節）．

RAID の各種の構成はレベルと呼ばれ，番号が付けられている．番号は構成を識別するものであって，順位を表すものではない．この章ではレベル 0，1，4，5，6 を扱う．

[35] では「I」は *inexpensive*（安価な）を表していた．レベル 2 と 3 はほとんど使われないので，ここでは省く．

> **要点**
> パリティビットは位置の分からない誤りを検出するだけだが，パリティブロックは位置の分かっている失われたデータを復元する．ディスクが自分の誤りを報告することが，パリティに基づく RAID を可能にしている．

17.9 RAID 0: ストライピング

最初の構成は冗長性をまったく用いず，性能だけを目指す．1 台の論理ディスクのデータを複数の物理ディスクに分散させ，連続するブロックが異なるディスクに格納されるようにすることを**ストライピング**という．全ディスクの同じ位置にあるブロックは**ストライプ**をなす．冗長性なしにストライピングを用いる N 台のディスクのアレイが **RAID 0** である．論理ブロック i はディスク $i \bmod N$ に格納される（図 17.4a）．

出典 Lesson 17，動画 16–18；H&P §D.2．

▷ striping

▷ RAID 0

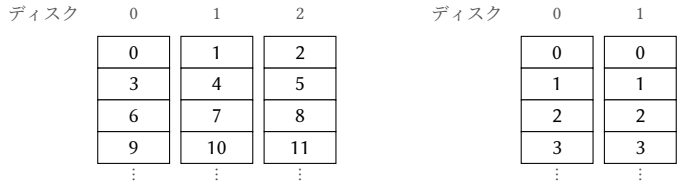


図 17.4: 論理ブロックの配置．(a) 3 台のディスクの RAID 0．各ストライプは異なる 3 つのブロックを保持する．(b) RAID 1．両方のディスクがすべてのブロックを保持する．

17.9.1 性能

アレイの容量は 1 台のディスクの N 倍である．異なるディスクにあるブロックへの要求は同時に処理され，大きな転送は並行して進む N 個の転送に分割されるので，スループットは 1 台のディスクの最大 N 倍になる．単一のブロックへのアクセスには，そのディスクでのシークと回転待ち（第 16 章）が依然として必要である．したがって小さな要求が速く完了するのは，他の要求の後ろで待ち行列に並ぶ時間が短くなるからにすぎない．大きな転送も，データが N 台のディスクとの間で同時に移動するので速く完了する．

ストライピングの単位は数十から数百キロバイトのチャンクであり，1 セクタではない．図中の各ブロックは 1 つのチャンクを表している．

17.9.2 信頼性

信頼性の計算では，ディスクは互いに独立に故障し，各ディスクは一定の**故障率** $f_1 = 1/\mathrm{MTTF}_1$ を持つと仮定する．ここで MTTF_1 は 1 台のディスクの MTTF である．このとき，ディスクの集合の故障率は個々の故障率の和となる．RAID 0 では各ディスクが論理ディスクの N ブロックごとに 1 ブロックを保持するので，どのディスクが故障してもデータが失われる．アレイは Nf_1 の率で故障するの

実際のディスクは寿命の初期と末期に故障しやすい．一定の故障率は，使用期間の長い中間部分をモデル化している．

で，その MTTF は次式となる．

$$\mathrm{MTTF}_{\mathrm{RAID\,0}} = \frac{\mathrm{MTTF}_1}{N}. \tag{17.4}$$

アレイにとって故障とはデータが失われることなので，その MTTF は**平均データ喪失時間**（MTTDL）とも呼ばれる．

例 17.10. $\mathrm{MTTF}_1 = 120\,000\,\mathrm{h}$（約 13.7 年）のディスク 5 台で RAID 0 のアレイを構成する．アレイは 1 台のディスクの最大 5 倍の速度と 5 倍の容量を持つが，その MTTF は $120\,000\,\mathrm{h}/5 = 24\,000\,\mathrm{h}$，約 2.7 年にすぎない．

17.10 RAID 1: ミラーリング

出典 Lesson 17，動画 19–22；H&P §D.2．

2 つ目の構成は，ディスクを信頼性のために費やす．同じデータを 2 台のディスクに書き，各ディスクがすべてのブロックの完全な複製を保持するようにすることを**ミラーリング**という．ミラー化された 2 台のディスクのアレイ（図 17.4b）は，いずれか 1 台が動作している限りすべてのデータを利用可能に保つ．これを **RAID 1** という．

▷ mirroring

▷ RAID 1

- **書き込み**は両方のディスクに対して行われる．2 つの書き込みは並行して進むので，書き込みの速さは単一のディスクと同じである．
- **読み出し**はどちらの複製からでも処理できるので，読み出しスループットは 1 台のディスクの 2 倍である．
- **フォールト**は，1 台のディスクに限られていれば，そのディスク全体の故障も含めて許容される．複製が 2 つしかないので多数決はできないが，その必要もない．ディスク自身の符号がどちらの複製が不良かを報告するので，もう一方の複製を使えばよい．
- **コスト**: 2 台のディスクで 1 台分の容量が得られる．

17.10.1 修復なしの信頼性

両方のディスクが動作している間，アレイは $2f_1$ の率でディスク故障に見舞われるので，最初の故障は平均して $\mathrm{MTTF}_1/2$ 後に起きる．アレイは残り 1 台でこれを生き延びる．故障率は一定なので，生き残ったディスクは新品同様であり，さらに平均 MTTF_1 後に故障する．データが失われるのはそのときである．

$$\mathrm{MTTF}_{\mathrm{RAID\,1}} = \frac{\mathrm{MTTF}_1}{2} + \mathrm{MTTF}_1 = 1.5\,\mathrm{MTTF}_1. \tag{17.5}$$

17.10.2 修復ありの信頼性

実際には，故障したディスクは交換され，生き残ったディスクから新しいディスクへデータが複写される．故障から，新しいディスクが完全な複製を保持するまでの平均時間を MTTR_1 とする．データが失われるのは，この期間内に 2 台目のディスクが故障した場合だけである（$N = 2$ とした図 17.5）．$\mathrm{MTTR}_1 \ll \mathrm{MTTF}_1$ なのでその確率はおよそ $\mathrm{MTTR}_1/\mathrm{MTTF}_1$ であり，平均すると $\mathrm{MTTF}_1/\mathrm{MTTR}_1$ 回の最初の故障が起きるうちの 1 回がデータ喪失に至る．そのそれぞれの前には，両方のディスクが動作している $\mathrm{MTTF}_1/2$ の期間がある．したがって次式が得られる．

$$\mathrm{MTTF}_{\mathrm{RAID\,1}} \approx \frac{\mathrm{MTTF}_1}{2} \times \frac{\mathrm{MTTF}_1}{\mathrm{MTTR}_1} = \frac{\mathrm{MTTF}_1^2}{2\,\mathrm{MTTR}_1}. \tag{17.6}$$

MTTR_1 は保守員を呼ぶ時間ではなく複写の時間である．16 TB のディスクを 150 MB/s で埋め直せば約 30 h かかる．以下の例で用いる値はこれである．

例 17.11. 例 17.10 のディスク（$\mathrm{MTTF}_1 = 120\,000\,\mathrm{h}$）を用いると，修復なしの RAID 1 アレイの MTTF は 180 000 h，約 20.5 年である．故障したディスクが交換され，その複製が $\mathrm{MTTR}_1 = 30\,\mathrm{h}$ 以内に復元されるなら，MTTF は $60\,000\,\mathrm{h} \times 4000 = 2.4 \times 10^8\,\mathrm{h}$，約 27 000 年になる．

データシートの MTTF 1 000 000–1 500 000 h は年間故障率 1 % 未満を意味する．実地で測定された交換率は 2 % to 4 % であり，MTTF はむしろ 300 000 h に近い [36]．

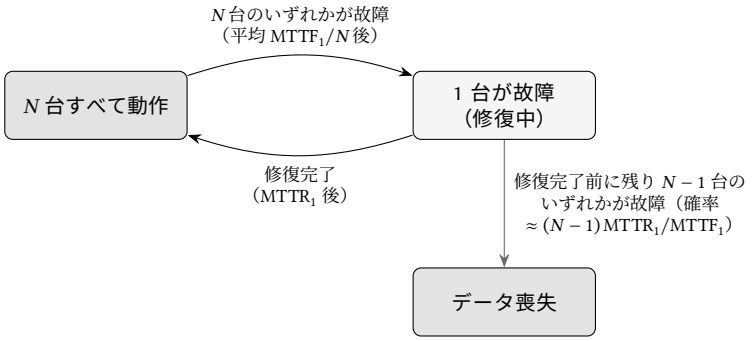


図 17.5: 1 台のディスク故障を許容するアレイと，その修復．データが失われるのは，1 台目を修復している間に 2 台目が故障した場合だけである．RAID 1 では $N = 2$ である．

> **要点**
> 修復がなければ冗長性はほとんど役に立たない．ミラー化された 2 台のディスクは平均して $1.5\,\mathrm{MTTF}_1$ しかもたない．そのうち 1 台目が $\mathrm{MTTF}_1/2$ 後に故障するからである．修復があれば，冗長性は MTTF を $\mathrm{MTTF}_1/\mathrm{MTTR}_1$ 程度の倍率で引き上げる．フォールトトレラントなシステムでは，モジュールの修復を速くすることが，システムの可用性だけでなく信頼性も改善する．

17.11 RAID 4: ブロックインターリーブパリティ

出典 Lesson 17，動画 23–27；H&P §D.2.

パリティは，ミラーリングと同じ冗長性をその何分の一かのコストで提供する．N 台のディスクのアレイでは，そのうち $N-1$ 台を RAID 0 と同様にストライピングしたデータディスクとし，残る 1 台をパリティディスクとすることができる．パリティディスクの各ストライプのブロックは，そのストライプのデータブロックの排他的論理和である（図 17.6a）．この構成を **RAID 4** という．データブロック $D_0, \ldots, D_{N-2}$ からなるストライプに式 (17.3) をビットごとに適用すると次式が得られる．

▷ RAID 4

$$P \;=\; D_0 \oplus D_1 \oplus \cdots \oplus D_{N-2}. \tag{17.7}$$

$x \oplus x = 0$ であるから，式 (17.7) の両辺に残存するすべてのブロックの排他的論理和を取ると，失われたブロックが，パリティを含む同じストライプの他のすべてのブロックの排他的論理和に等しいことが分かる．ディスクが故障しても，その上のすべてのブロックは他の $N-1$ 台のディスクから復元できる．故障したのがパリティディスクであれば，パリティを計算し直すだけでよい．冗長性のコストは N 台のうち 1 台であり，ミラーリングの 2 つ目の複製よりはるかに小さい．

注意 符号理論では，位置の分かっている喪失を消失（erasure）と呼ぶ．検査シンボル 1 個で消失は 1 個訂正できるが，位置の分からない誤りを 1 個，位置の特定まで含めて訂正するには 2 個必要である．

17.11.1 書き込み

書き込みは，そのストライプのパリティを更新しなければならない．ストライプのすべてのデータブロックからパリティを計算し直すには，他の $N-2$ 台のディスクを読む必要がある．そうする代わりに，新しいパリティは，古いパリティから古いデータを取り除き，新しいデータを加えることで得られる（図 17.6b）．

$$P' \;=\; P \oplus D \oplus D'. \tag{17.8}$$

したがって書き込みは，古いデータブロックと古いパリティブロックを読み，新しいデータブロックと新しいパリティブロックを書く．N によらず，2 回の読み出しと 2 回の書き込みである．

この読み出し・変更・書き込みの手順はスモールライト問題として知られる．ストライプ全体を覆う書き込みはこれを免れる．新しいパリティが新しいデータだけから求まるからである．

例 17.12. 3 台のデータディスクと 1 台のパリティディスクからなるアレイと，4 ビットのブロックからなる 1 つのストライプを考える（表 17.2）．パリティは $1001 \oplus 0111 \oplus 1010 = 0100$ である．ディスク 1 が故障すると，そのブロックは $1001 \oplus 1010 \oplus 0100 = 0111$ として復元される．ディスク 1 に 0010 を書き込むと，式 (17.8) により新しいパリティは $0100 \oplus 0111 \oplus 0010 = 0001$ となり，これは新しいデータブロックの排他的論理和 $1001 \oplus 0010 \oplus 1010$ に等しい．

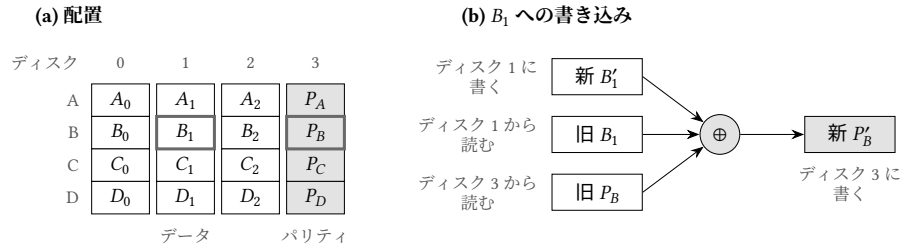


図 17.6: RAID 4．(a) 3 台のデータディスクと 1 台のパリティディスク．行がストライプである．(b) ブロック B_1 への書き込みは，B_1 と P_B を読み，式 (17.8) で新しいパリティを計算し，B_1' と P_B' を書く．

表 17.2: 例 17.12 における RAID 4 の 1 つのストライプ．復元および更新されたブロックを太字で示す．

	ディスク 0	ディスク 1	ディスク 2	ディスク 3（パリティ）
初期のストライプ	1001	0111	1010	0100
ディスク 1 の喪失と復元	1001	**0111**	1010	0100
ディスク 1 に 0010 を書き込んだ後	1001	**0010**	1010	**0001**

17.11.2 性能と信頼性

- **読み出し**はデータディスクだけを使うので，読み出しスループットは 1 台のディスクの $N-1$ 倍である．
- **書き込み**はすべてパリティディスクを必要とし，読み出しに 1 回，書き込みに 1 回アクセスする．異なるデータディスクへの書き込みであっても並行しては進められず，各書き込みがパリティディスクを 2 回のアクセスのあいだ占有する．したがって書き込みスループットは，N の値によらず単一のディスクの半分である．
- **信頼性**: アレイはどれか 1 台のディスクの故障を許容する．修復がなければ，最初の故障は平均 MTTF_1/N 後に，2 回目はさらに $\mathrm{MTTF}_1/(N-1)$ 後に起きる．$N \geq 3$ ではこの和が MTTF_1 より小さい．すなわち修復のないこのようなアレイは，単一のディスクより信頼性が低い．修復があれば，式 (17.6) の議論が，最初に故障しうるディスクが N 台，修復中に故障しうるディスクが $N-1$ 台という形で適用でき，次式が得られる（図 17.5）．

$$\mathrm{MTTF}_{\mathrm{RAID\ 4}} \approx \frac{\mathrm{MTTF}_1}{N} \times \frac{\mathrm{MTTF}_1}{(N-1)\,\mathrm{MTTR}_1}. \tag{17.9}$$

17.12 RAID 5: 分散パリティ

出典 Lesson 17，動画 28–29；H&P §D.2．

RAID 4 のパリティディスクは書き込みのボトルネックである．その対策は，パリティを全ディスクに分散させることである．各ストライプが 1 つのパリティブロックを持つことは変わらず，その計算も RAID 4 とまったく同じだが，異なるストライプのパリティブロックは異なるディスクに格納され，N 台のディスクを巡回する（図 17.7a）．この構成を **RAID 5** という．どのディスクもデータとパリティの両方を保持する．

▷ RAID 5

- **読み出し**: データは N 台すべてのディスクに分散しているので，読み出しスループットは 1 台のディスクの N 倍である．
- **書き込み**には依然として 2 回の読み出しと 2 回の書き込みが必要だが（式 (17.8)），パリティを保持するディスクはストライプによって異なる．異なるストライプへの書き込みは通常，異なるディスクを使うので並行して進む．書き込み 1 回当たりの 4 回のアクセスが N 台すべてのディスクに分散するので，書き込みスループットは 1 台のディスクの $N/4$ 倍である．
- **信頼性**と**コスト**は RAID 4 と同じである．1 台のディスク故障を許容し，修復ありの MTTF は 式 (17.9) で与えられ，ディスク 1 台分の容量がパリティに使われる．

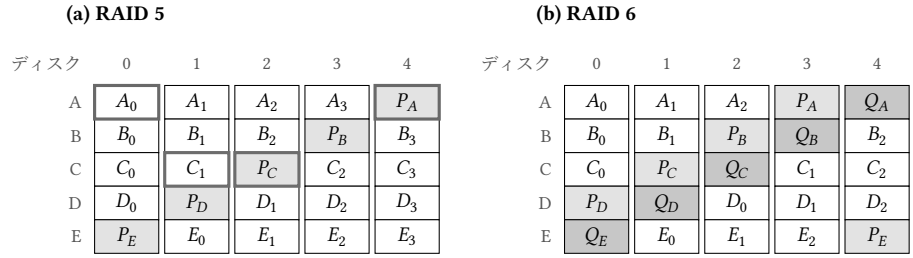


図 17.7: (a) 5 台のディスクの RAID 5．パリティブロックはディスクを巡回する．A_0 への書き込み（赤）と C_1 への書き込み（緑）は使うディスクが重ならないので，並行して進められる．(b) ストライプごとに 2 つのチェックブロック P と Q を持つ RAID 6．

例 17.13. 6 台のディスクがあり，各ディスクは毎秒 200 回のアクセスを処理でき，$\mathrm{MTTF}_1 = 120\,000\,\mathrm{h}$，$\mathrm{MTTR}_1 = 30\,\mathrm{h}$ である．RAID 4 として構成すると毎秒 1000 回の読み出しまたは 100 回の書き込みを処理し，RAID 5 として構成すると毎秒 1200 回の読み出しまたは 300 回の書き込みを処理する．いずれの場合も修復ありの MTTF は $(120\,000/6) \times 120\,000/(5 \times 30) = 1.6 \times 10^7\,\mathrm{h}$，約 1800 年である．

> **要点**
> RAID 5 は RAID 4 と同じ信頼性とコストを持ち，読み出しと書き込みの性能はより高い．あらゆる点で RAID 4 に劣らないので，RAID 4 が使われることはまれである．

17.13 RAID 6: 2 つのチェックブロック

出典 Lesson 17，動画 30–32；H&P §D.2.

RAID 5 は 1 台のディスク故障を許容する．次の構成は 2 台の故障を許容する．各ストライプに 2 つ目のチェックブロックを加えるのである（図 17.7b）．一方のチェックブロック P はパリティであり，もう一方の Q は別の符号で計算される．この構成を **RAID 6** という．P と Q はストライプごとに 2 つの独立な方程式を与えるので，1 つのストライプのどの 2 ブロックが失われても復元できる．1 ブロックだけが失われた場合は RAID 5 と同様にパリティから復元し，2 ブロックが失われた場合は両方の方程式を解いて求める．

▷ RAID 6
Q は通常 GF(2^8) 上の Reed–Solomon 検査であり，固定の g を用いて $Q = \sum_i g^i D_i$ とする．重みが異なることが 2 つ目の方程式を独立にしている．2 つ目も排他的論理和では，1 つ目の繰り返しにしかならない．

- **信頼性**: アレイはどの 2 台のディスクの故障も許容する．
- **コスト**: ディスク 2 台分の容量がチェックブロックに使われる．
- **書き込み**は両方のチェックブロックを更新しなければならない．RAID 5 では読み出しも書き込みも 2 回だったのに対し，3 回の読み出し（古いデータ，P, Q）と 3 回の書き込みが必要である．書き込み 1 回当たり 6 回のアクセスが N 台のディスクに分散するので，書き込みスループットは 1 台のディスクのおよそ $N/6$ 倍となる．読み出しは N 倍のままである．

ヒント 表 17.3 の書き込みの列は，N を書き込み 1 回当たりのアクセス数で割った値である．RAID 4 と RAID 5 は 4 回，RAID 6 は 6 回である．RAID 4 が代わりに 1/2 なのは，4 回すべてが 1 台のパリティディスクに集中するからである．

17.13.1 RAID 6 と相関のある故障

独立なディスク故障に対しては，修復ありの RAID 5 でもすでに千年を優に超える MTTF が得られる（例 17.13）．RAID 6 では，2 台を修復している間に 3 台目が故障した場合にのみデータが失われるので，MTTF はさらに $\mathrm{MTTF}_1/((N-2)\,\mathrm{MTTR}_1)$ 程度の倍率で伸びる．例の数値では約 1000 倍である．独立な故障に対しては，RAID 6 は過剰である．

RAID 6 の価値は，最初の故障と独立でない故障にある．典型的なのは修復中の運用フォールトで，運用者が，故障したディスクではなく動作しているディスクを交換してしまう場合である．RAID 5 のアレイはこれで 2 台を失い，データも失われる．RAID 6 のアレイは動作し続け，なお修復できる．

修復では残りのディスクを端から端まで読むことにもなる．コンシューマ向けのデータシートが許す回復不能読み出し誤りは 10^{14} ビットに 1 回であり，7 台の 4 TB ディスクはその 2 倍のビット数を保持する．RAID 5 の再構築そのものがブロックを失いうる．

表 17.3: MTTF_1 と MTTR_1 を持つ N 台のディスクによる RAID レベルの比較．容量とスループットは 1 台のディスクに対する相対値である．MTTF は RAID 1，4，5，6 については修復ありを仮定している．RAID 0 は最初の故障でデータを失う．

レベル	容量	読み出し	書き込み	許容故障台数	MTTF
RAID 0	N	N	N	0	MTTF_1/N
RAID 1 ($N=2$)	1	2	1	1	$\mathrm{MTTF}_1^2/(2\,\mathrm{MTTR}_1)$
RAID 4	$N-1$	$N-1$	$1/2$	1	$\mathrm{MTTF}_1^2/(N(N-1)\,\mathrm{MTTR}_1)$
RAID 5	$N-1$	N	$N/4$	1	$\mathrm{MTTF}_1^2/(N(N-1)\,\mathrm{MTTR}_1)$
RAID 6	$N-2$	N	$N/6$	2	RAID 5 よりはるかに高い

要点 — 章のまとめ

モジュールのフォールトは，活性化されるとエラーになり，エラーが提供サービスに到達すると故障になる．信頼性は MTTF で，可用性は MTTF/(MTTF + MTTR) で測られる．フォールトは原因（ハードウェア，設計，運用，環境）と継続時間（永久，間欠，過渡）で分類され，回避・許容・修復の高速化によって対処される．チェックポインティングと N モジュール冗長は一般のフォールトを許容し，パリティと ECC はメモリを保護し，RAID はディスクの故障から守る．ストライピング（RAID 0）は信頼性と引き換えに速度を得，ミラーリング（RAID 1）は 2 つ目の複製で信頼性を買い，パリティ（RAID 4，5）はディスク 1 台分のコストでそれを実現する．RAID 5 はパリティディスクのボトルネックを回避する．修復があれば，冗長アレイの MTTF は $\mathrm{MTTF}_1/\mathrm{MTTR}_1$ に応じて伸びる（表 17.3）．RAID 6 は主に，修復中の誤りのような相関のある故障から守る．

第 18 章

マルチプロセッシング

講義動画	Lesson 18，動画 1–22
教科書	H&P [1] §1.1–1.2，§1.5，§5.1，第 3 章，付録 B
キーワード	Flynn の分類，MIMD，マルチコアプロセッサ，集中共有メモリ，UMA，SMP，分散共有メモリ，NUMA，メッセージパッシング，共有メモリ，チップマルチプロセッサ，細粒度マルチスレッディング，同時マルチスレッディング，ワーキングセット

ここまでの章では，パイプライン化，分岐予測，アウトオブオーダ実行，メモリ階層によって，1 つのプロセッサが 1 つのプログラムをできる限り速く実行できるようにしてきた．そして第 17 章では，そのようなシステムを信頼できるものにした．この章では，複数のプログラム，あるいは 1 つのプログラムの複数の部分を同時に実行するマシンを扱う．並列マシンを分類し，各技術世代がもたらす追加のトランジスタが，今日ではなぜより速い単一コアではなく追加のコアになるのかを説明し，プロセッサをメモリに接続する 2 つの方式とそれぞれに対応する 2 つのプログラミングモデルを比較し，最後にマルチスレッディングのハードウェア—とりわけ，1 つのコアに複数のスレッドを同時に実行させる同時マルチスレッディング—で締めくくる．

18.1 Flynn の分類

出典 Lesson 18，動画 1–2；H&P §1.2；[37]．

▷ Flynn's taxonomy

計算機が一度に複数のことを行う方法には，いくつかの種類がある．並列マシンの古典的な分類である **Flynn の分類**は，2 つの数に着目する．マシンが同時に実行する**命令ストリーム**—それぞれが独自のプログラムカウンタを持つ命令の列—の数と，それらの命令が操作する**データストリーム**の数である．どちらの数についても「1 つ」と「2 つ以上」を区別すると，4 つのクラスが得られる（表 18.1）．

表 18.1: 並列マシンの Flynn の分類．

クラス	命令ストリーム	データストリーム	例
SISD	1	1	単一プロセッサ
SIMD	1	> 1	ベクトルプロセッサ，SSE/AVX
MISD	> 1	1	ストリームプロセッサ（稀）
MIMD	> 1	> 1	マルチプロセッサ，マルチコア

SISD 1 つの命令ストリームを 1 つのデータストリームに対して実行するマシンが **SISD** である．これまでの章の単一プロセッサがこれである．スーパースカラのアウトオブオーダコアも依然として SISD である．同じサイクルに実行される命令は，すべて 1 つのプログラムに属するからである．

▷ single instruction, single data

SIMD 1 つの命令ストリームを実行するが，各命令が複数のデータ要素に対して演算するマシンが **SIMD** である．ベクトルプロセッサがこの種であり，現在の命令セットのマルチメディア拡張（x86 の MMX，SSE，AVX）も同様で，その命令は短いベクトルのすべての要素に 1 つの演算を適用する．

▷ single instruction, multiple data

MISD 複数の命令ストリームを 1 つのデータストリームに対して実行するマシンが **MISD** である．複数のプログラムがそれぞれ同じ入力データを処理するストリームプロセッサがその例になる．この種のマシンは稀である．

▷ multiple instruction, single data

MIMD 複数の命令ストリームを，それぞれ独自のデータに対して実行するマシンが **MIMD** である．

▷ multiple instruction, multiple data

複数のプロセッサからなり，各プロセッサが自身のデータに対して自身のプログラム（あるいは 1 つのプログラムの自身の担当部分）を実行するマシンを**マルチプロセッサ**という．マルチプロセッサは MIMD マシンであり，今日使われている並列マシンのほとんどすべてがこのクラスに属する．

▷ multiprocessor

18.2 なぜマルチプロセッサか

単一プロセッサを速くする方法は 2 つある．サイクル当たりにより多くの命令を完了するか，サイクルをより短くするかである．どちらも余地を使い果たしている．

- **幅を広げる.** コアはすでにサイクル当たり 4 命令以上を扱う．プログラムがそれ以上のスロットを埋めるだけの独立な命令を含むことは稀であり，幅をさらに増やしても収穫逓減となる（第 6 章）．
- **速くする.** クロック周波数を上げるには電源電圧を上げる必要がある．動的電力は $V^2 f$ に比例する（式 (1.1)）ので，f と V をともに上げると電力はおおよそ f^3 で増える．しかもプロセッサはすでに，妥当なコストで除去できるだけの電力を消費している．

一方でムーアの法則は続いている．1 つのチップに収まるトランジスタ数は，今なおおよそ 18 か月で倍増する．より複雑な単一コアを作る代わりに，そのトランジスタでコアを複製することができる．**マルチコアプロセッサ**とは，2 個以上の完全なプロセッサ—**コア**と呼ばれる—を 1 つのチップに収めたものである．世代ごとにコア数を倍にすれば，世代ごとに性能を倍にできる—ソフトウェアがすべてのコアを使う限りにおいて．

出典 Lesson 18，動画 3–4；H&P §1.1，§1.5．

代わりにクロック周波数が上がらなくなった．Pentium 4 は 2004 年に 3.8 GHz に達し（頭打ちの様子は H&P §1.5），今日の最も速いデスクトップ向け製品，例えば Intel の Core i9-13900KS（2023 年）でもおよそ 6 GHz である．

▷ multicore processor

注意 ムーア自身の数字は 1965 年には 1 年 [2]，1975 年の改訂では 2 年である．「18 か月」は後年の通俗版である．

例 18.1. あるコアが 3 GHz で動作し，40 W の動的電力を消費する．これを 1.2 倍速い 3.6 GHz で動作させるには，電圧を比例して上げる必要があり，電力はおよそ $1.2^3 \approx 1.73$ 倍の 69 W になる．元のコアを 3 GHz で 2 個動かすと 80 W を消費し，1 個のコアの 2 倍の仕事を—プログラムが両方を忙しく保てば—完了する．2.4 GHz（0.8 倍）で 2 個動かせば $2 \times 40\,\mathrm{W} \times 0.8^3 \approx 41\,\mathrm{W}$ となり，元の単一コア 1 個とほぼ同じ消費でありながら，合わせてその 1.6 倍の仕事を完了する．

18.3 マルチプロセッサには並列プログラムが必要である

単一プロセッサを速くすれば，遠い昔に書かれたものも含め，あらゆるプログラムが速くなる．マルチプロセッサが速くするのは，仕事を複数のコアに分割するプログラムだけである．逐次の（シングルスレッドの）コードと比べて，そうした並列プログラムはあらゆる面でより難しい．

- **開発.** 問題を解くことに加えて，プログラマは仕事をスレッドに分割し，スレッド間を協調させなければならない．
- **デバッグ.** 並列プログラムの振る舞いはスレッド間の相対的なタイミングに依存し，それは実行のたびに変わるので，ある実行で現れた誤りが次の実行では現れないことがある．
- **性能のスケーリング.** 理想的には，性能はコア数に比例して伸びる．実際にはある程度のコア数までは伸び，その後は頭打ちになる．プログラムを改良すればその水準は上がるが，それでも性能はどこかで伸びなくなる（図 18.1）．

頭打ちが避けられない理由は Amdahl の法則（式 (2.5)）が示す．プログラムの実行時間のうち割合 f_{par} が N 個のコアに均等に分割され，残りが逐次のままであるとすると，高速化率は高々

$$S(N) = \frac{1}{(1 - f_{\mathrm{par}}) + \dfrac{f_{\mathrm{par}}}{N}} < \frac{1}{1 - f_{\mathrm{par}}} \tag{18.1}$$

である．スレッド間の協調にはこれに加えて時間がかかるので，実際のプログラムはこれらの曲線より下に留まる．

出典 Lesson 18，動画 5；H&P §5.1．

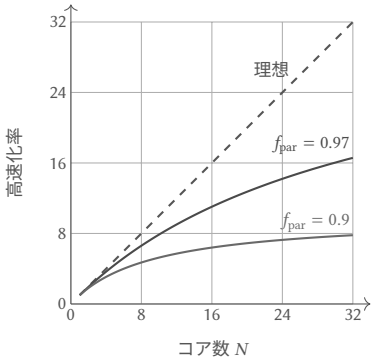


図 18.1: N コアでの高速化率: 理想的なスケーリングと式 (18.1).

例 18.2. あるプログラムでは，時間の 95 % が，反復が互いに独立なループに費やされており，$f_{\mathrm{par}} = 0.95$ である．8 コアでの高速化率は高々 $1/(0.05 + 0.95/8) \approx 5.9$，32 コアでは $1/(0.05 + 0.95/32) \approx 12.5$ であり，コアをいくら増やしても $1/0.05 = 20$ を超えることはない．

ヒント 理想の半分の高速化率 $S(N) = N/2$ には $f_{\mathrm{par}} = (N-2)/(N-1)$ が必要である．8 コアで 86 %，64 コアで 98 % である．

要点
コアを複製することは，より大きな，あるいはより速い単一コアを作るよりはるかに効率よくトランジスタをスループットに変える．ただしそれは，多数のコアを使うように書かれたプログラムに対してだけであり，しかもその逐次部分が許す範囲までである．

18.4 集中共有メモリ

最も単純なマルチプロセッサは，すべてのプロセッサを 1 つの主記憶に接続する．

出典 Lesson 18，動画 6–8；H&P §5.1．

定義 18.3 (集中共有メモリ). **集中共有メモリ**のマルチプロセッサでは，それぞれ自身のキャッシュを持つすべてのコアが，1 本のバスによって 1 つの主記憶と I/O 装置に接続される（図 18.2）．

▷ centralized shared memory

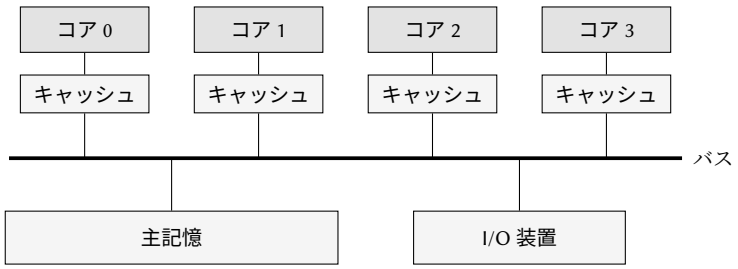


図 18.2: 集中共有メモリ: すべてのコアが同じバスを通じて主記憶と I/O 装置に到達する．

どのコアも任意のメモリ位置を読み書きでき，任意の I/O 装置を使えるので，コア間のデータ交換は，一方がメモリに書き，他方がそれを読むだけで済む．メモリはすべてのコアから等しく離れているため，どのコアもある位置へのアクセスに同じ時間を要する．すなわちこのマシンは **UMA** である．また，どのコアも特別な役割を持たないため，このマシンは**対称型マルチプロセッサ**（SMP）とも呼ばれる．チップに接続された主記憶をコアが共有するマルチコアチップが，最も一般的な例である．

▷ uniform memory access
▷ symmetric multiprocessor

集中共有メモリは，2 つの理由から多数のコアにはスケールしない．

- **メモリ容量.** 1 つのメモリがすべてのコアのデータを保持するので，コア数とともに大きくしなければならず，メモリは大きいほど遅い（第 12 章と第 15 章）．
- **メモリ帯域.** すべてのコアのキャッシュミスと，すべての I/O 転送が同じバスを通る．コアが増えるほど互いのメモリアクセスを待つようになり，ついにはメモリアクセスが事実上直列化され，コアを追加しても利益が得られなくなる．

したがって集中共有メモリがうまく働くのは，コア数がおよそ 2 から 16 の小規模なマシンに限られる．

18.5 分散共有メモリ

より大規模なマシンは，メモリをプロセッサに分散させる．

出典 Lesson 18，動画 9–10；H&P §5.1．

定義 18.4 (分散共有メモリ). **分散共有メモリ**のマルチプロセッサでは，各コアが自身のローカルメモリとネットワークインタフェースを持ち，コアどうしはネットワークで接続される（図 18.3）．コアが直接アクセスするのは自身のローカルメモリだけであり，他のコアが保持するデータは，そのコアにネットワーク経由で要求を送って得る．

▷ distributed shared memory

コアとそのメモリおよびネットワークインタフェースは，それ自体ほとんど 1 台の計算機であり，**ノード**と呼ばれる．そのためこの種のマシンは**クラスタ**あるいは**マルチコンピュータ**とも呼ばれる．コアは自身のローカルメモリには単一プロセッサと同じ速さで到達できるが，他のノードが保持するデータはネットワークを渡らなければならず，はるかに長い時間がかかる．アクセス時間はデータの

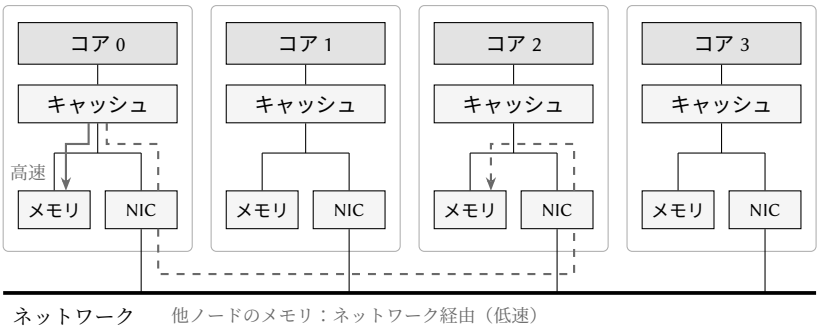


図 18.3: 分散共有メモリ: 各ノードは，コア，そのキャッシュ，ローカルメモリ，ネットワークインタフェースカード（NIC）からなる．

所在に依存するので，このマシンは **NUMA** である．

▷ non-uniform memory access

分散共有メモリは非常に多数のコアまでスケールし，クラスタやスーパーコンピュータはこの方式で作られる．その理由は，ネットワークが速いからではない．ネットワークはメモリバスよりはるかに遅い．そうではなく，すべてのコアが共有するメモリもバスも存在しないこと，そして，リモートアクセスをすべて明示的に要求しなければならないプログラマが，通信について考えることを強いられ，結果としてそれを最小限に保つことが理由である．

目安として，ローカルの DRAM へは 80 ns，他方のソケットのメモリへは 130 ns，InfiniBand で他のノードへは 1–2 µs である（公表されている典型的な実測値）．

リモートアクセスはローカルアクセスよりはるかに高価なので，NUMA マシンの性能はデータの配置で決まる．オペレーティングシステムは，各ページを，それを最も多くアクセスするコアのノードのメモリに–スレッドのスタックのようなスレッド固有のデータであれば，そのスレッドが走るノードに–置き，スレッドをそのノードに留めるべきである．そうすればメモリアクセスの大半がローカルに収まる．ただし，複数のコアが使うページは，そのすべてにとってローカルにはなりえない．

ヒント NUMA マシンでは，各スレッドを，それが最も多く使うデータを保持するノードに留めること．Linux はページを，最初にそれに触れたスレッドのノードに置く．`numactl` でこの選択を上書きできる．

注 18.5 (用語について). 本章では「分散共有メモリ」を，メモリがノードに分割されているすべてのマルチプロセッサに対して用いる．ただし，ネットワーク経由の明示的な要求で通信する場合，実際にはどのメモリも共有されていない．H&P はこの名前を，ノードが 1 つの物理アドレス空間を実際に共有するマシン–どのコアも任意のアドレスを発行でき，相互結合網がリモートアドレスへのアクセスを，それを保持するノードへ運ぶ–に限って用い，明示的なメッセージだけで通信するマシンはクラスタと呼ぶ．本章ではどちらも NUMA と呼ぶが，H&P の用語では NUMA は，アドレス空間を共有し，アクセス時間がデータの所在に依存するマシンだけを指す．

18.6 メッセージパッシングと共有メモリ

並列プログラムの書き方には，根本的に異なる 2 つの方法がある．

出典 Lesson 18，動画 11–14；H&P §5.1．

定義 18.6 (メッセージパッシング). **メッセージパッシング**のプログラミングモデルでは，各プロセスが自身のメモリを持ち，他のどのプロセスもそれにアクセスできない．プロセス間のデータ交換は，明示的にメッセージを送受信することによってのみ行われる．

▷ message passing

定義 18.7 (共有メモリ). **共有メモリ**のプログラミングモデルでは，すべてのスレッドが 1 つのアドレス空間を共有し，他のスレッドが読む変数に書き込むだけでデータを交換する．

▷ shared memory

以下では，同じ課題–N 要素の配列の総和を P 個のプロセスまたはスレッドで求める–について 2 つのモデルを比較する．どちらのプログラムでも各プロセスまたはスレッドは同じコードを実行し，0 から $P-1$ までの値をとる識別子 `myPID` だけが異なる．

18.6.1 メッセージパッシングのプログラム

各プロセスは，自身に割り当てられた N/P 要素だけを保持する．配列の各部分を配布すること自体が明示的な作業であり（ここには示さない），最初にデータを持つプロセスが，他の各プロセスにその担当部分を送らなければならない．次に各プロセスは自身の部分の総和を求め，プロセス 0 が他のすべてのプロセスの部分和を受け取って合算する（例 18.8）．

例 18.8 (メッセージパッシング). 各プロセスは，自身の N/P 要素をすでに part に持った状態で，次のコードを実行する．

```
#define N 4096              /* 配列サイズ */
#define P 8                 /* プロセス数 */
double part[N/P];           /* 自分の担当部分 */
double mySum = 0;
for (int i = 0; i < N/P; i++)
  mySum += part[i];
if (myPID == 0) {
  for (int p = 1; p < P; p++) {
    double pSum;
    recv(p, &pSum);         /* p の部分和を待つ */
    mySum += pSum;
  }
  printf("sum: %f\n", mySum);
} else
  send(0, &mySum);          /* プロセス 0 へ */
```

18.6.2 共有メモリのプログラム

共有メモリでは配列全体がすべてのスレッドから見えるので，配布すべきものは何もない．各スレッドは，自身が担当するインデックスの範囲を計算し，その部分和を共有の総和に加える（例 18.9）．shared と宣言された変数は全スレッドに対して 1 つだけ存在し，それ以外の変数は各スレッド固有である．

実際にはライブラリが用いられる．メッセージパッシングには MPI（MPI_Reduce がまさにこの総和である），共有メモリには OpenMP や pthreads である．

例 18.9 (共有メモリ). 各スレッドは，共有された配列に対して次のコードを実行する．

注意 shared は記法であって C の構文ではない．pthreads では大域変数とヒープが共有され，スタックは各スレッド固有である．

```
#define N 4096
#define P 8
shared double array[N];
shared double allSum = 0;
shared lock_t sumLock;
double mySum = 0;           /* スレッド固有 */
int lo = myPID * N / P;
int hi = (myPID + 1) * N / P;
for (int i = lo; i < hi; i++)
  mySum += array[i];
lock(&sumLock);
allSum += mySum;
unlock(&sumLock);
if (myPID == 0)             /* 本文を参照 */
  printf("sum: %f\n", allSum);
```

ロックは不可欠である．文 allSum += mySum は allSum を読み，加算し，結果を書き戻す．2 つのスレッドが，どちらかが書き込む前にともに古い値を読んでしまうと，2 つの部分和の一方が失われる．ロックは，この更新に一度に 1 つのスレッドだけを入れる．またこのプログラムはまだ未完成でもある．スレッド 0 は，他のすべてのスレッドが自身の和を加え終わるまで待ってから合計を印字しなければならない．ロックと，この種の待ち合わせは第 20 章の主題である．

ヒント P 個の部分和を順に合算すると $P-1$ 段かかるが，木にすれば $\log_2 P$ 段で済む．

18.6.3 比較

表 18.2 は 2 つのモデルの違いをまとめたものである．メッセージパッシングでは，データの転送も，どのプロセスがどのデータを持つかの決定も，すべてプログラマが書く．ハードウェアは，それ以外は独立なノードをネットワークで接続しさえすればよい．共有メモリでは，通信もデータの分配も共有変数を通じて暗黙に行われるが，ハードウェアは共有アドレス空間を提供しなければならず，そこには全コアのキャッシュ内容の一貫性（第 19 章）と，同期のための支援（第 20 章）が含まれる．2 つのモデルは，明示的なコードを必要とする場所も異なる．メッセージパッシングにはデータを配布するコードが必要だが，受信そのものがデータの到着まで待ち合わせる．共有メモリでは配布のコードは不要だが，あるスレッドが他のスレッドを待たなければならない箇所すべてに明示的な同期が必要になる．

注意 受信が待つことは罠でもある．2 つのプロセスが互いに，送信より先に相手からの受信を待つと，どちらも永久に止まる．

表 18.2: メッセージパッシングと共有メモリの比較．

	メッセージパッシング	共有メモリ
通信	プログラマによる	自動
データの分配	手動	自動
ハードウェアの支援	単純	大がかり
正しさのためのプログラミング	難しい	あまり難しくない
性能のためのプログラミング	難しい	非常に難しい

プログラマにとって，2 つのモデルは難しさの配分が異なる．共有メモリのプログラムは正しく書くのが容易である．逐次プログラムをループ単位で並列化し，共有データの更新をロックで囲めばよい．一方で，速くするのははるかに難しい．共有のコストがソースコードに現れないからである．ロックを待つスレッド，異なるコアのキャッシュ間を移動するデータ，NUMA マシンでのリモートアクセスといったものである．メッセージパッシングのプログラムは正しく書くのが難しい．データの分配と，すべての送信およびそれに対応する受信を手で書かなければならないからである．しかし，いったん正しくなれば通信はすべてコードに見えており，速くするための作業の多くはその過程で済んでいる．

> **要点**
> メッセージパッシングは通信を明示的にする．正しくプログラムするのは難しいが，そのコストは目に見える．共有メモリは通信を暗黙にする．正しくプログラムするのは容易だが，速くするのははるかに難しい．

18.7 共有メモリのハードウェア

共有メモリのプログラムのスレッドを，ハードウェアが実行する方法はいくつかある．

出典 Lesson 18，動画 15；H&P 第 3 章，§5.1．

1. **物理アドレス空間を共有する複数のコア．**どのコアも任意の物理アドレスを発行できる．メモリは集中していても（UMA），ノードに分散したうえで 1 つの物理アドレス空間を共有していてもよい（NUMA．18.5 節 の注を参照）．各スレッドはそれぞれのコアで走り，スレッドは本当に同時に実行される．
2. **1 つのコアによる時分割．**オペレーティングシステムが 1 つのスレッドをしばらく走らせ，その状態（PC とレジスタ）を保存して別のスレッドの状態を復元する．この**コンテキストスイッチ**に特別なハードウェアは要らず，スレッドは自動的に同じ物理メモリを使うが，ある時点で実行されるスレッドは 1 つだけであり，時分割では性能は何も得られない．
3. **1 つのコアによるハードウェアマルチスレッディング．**コア自身が複数のスレッドの状態を保持し，オペレーティングシステムを介さずにそれらを切り替える．コアがスレッドを切り替える頻度によって 3 つの変種が区別され，後のものほど多くのハードウェアの支援を必要とする．
   - **粗粒度マルチスレッディング**は，数サイクル以上経ってから別のスレッドに切り替える．典型的には，走っているスレッドが，最終レベ

▷ coarse-grained multithreading

ルキャッシュのミスのような遅い事象を待たなければならなくなったときである．

- **細粒度マルチスレッディング**は，毎サイクル別のスレッドに切り替えることができる．
- **同時マルチスレッディング**（SMT）は，同じサイクルに複数のスレッドの命令を実行する．Intel のハイパースレッディングは SMT の実装である．

▷ fine-grained multithreading

▷ simultaneous multithreading

18.8 マルチスレッディングの性能

2 つのスレッドと，サイクル当たり 4 命令を扱えるスーパースカラコアを考える．1 つのスレッドが 1 サイクルの 4 スロットをすべて埋めることは稀である．命令間の依存，予測を外した分岐，キャッシュミスによって多くのスロットが空いたままになり，まったく進めないサイクルもある．図 18.4 は，4 つの構成がスロットをどう使うかを模式的に示す．4 つのいずれにおいても，空いたスロットを得られなかった命令は次のサイクルまで待ち，キャッシュミスのような事象を待つスレッドは，コアを保持しているかどうかによらず同じサイクル数だけ待つ．

出典 Lesson 18，動画 16–17；H&P 第 3 章；[38, 39]．

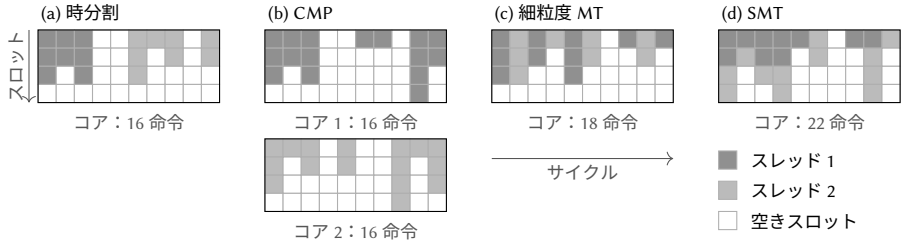


図 18.4: 2 つのスレッドによる，コアのサイクル当たり 4 スロットの 10 サイクルにわたる使われ方の模式図．各列が 1 サイクル，各行が 1 スロットである．単独ではどちらのスレッドもこの 10 サイクルで 16 命令を実行する (b)．(a) は 5 サイクルごとにスレッドを切り替え，(c) は待っていないスレッドの間で交互に切り替える．

時分割 1 つのコアが一度に 1 つのスレッドを走らせる．2 つのスレッドを合わせても，1 つのスレッドより多くの仕事は完了しない．コンテキストスイッチのたびに時間がかかるので（図には描いていない），実際にはわずかに少ない．コストはコア 1 個分である．

チップマルチプロセッサ 各スレッドがそれぞれのコアで走る．このように使われるマルチコアプロセッサを**チップマルチプロセッサ**（CMP）という．どちらのスレッドも全速で走るのでスループットは 2 倍になるが，チップ面積と電力も 2 倍になり，しかも各コアは単一スレッドの場合と同じだけスロットを空けたままにする．

▷ chip multiprocessor

細粒度マルチスレッディング スレッドごとに別のレジスタ組を持つ 1 つのコアが，サイクルごとにスレッドを切り替えるので，一方のスレッドが待たなければならないサイクルを他方が使う．コアはより忙しくなり，追加のハードウェアはスレッドごとの小さな状態だけで，コアのコストのおよそ 5 % である．ただしサイクル内の空きスロットは空いたままである．

SMT 1 つのコアが同じサイクルに両方のスレッドの命令を混ぜ，単一のスレッドなら空いたままになるスロットの多くを埋める．[1] スループットは細粒度マルチスレッディングより高くなるが，スレッドが 1 組の資源を共有するため，2 個のコアには及ばない．SMT は細粒度マルチスレッディングより多くの追加ハードウェアを必要とし，コアのコストのおよそ 10 % である（18.9 節）．

SMT がどれだけの効果を上げるかはスレッドに依存する．両者の命令が同じ実行ユニットを奪い合う場合，一方のスレッドの命令は他方が空けたスロットをほとんど見つけられず，得られる利得は小さい．2 つのスレッドの資源要求が互いに補い合うときにのみ，SMT は CMP のスループットに近づく．

例 18.10. SMT がコアのコストを 10 % 増やし，2 つのスレッドに対するスループットを 35 % 高めるとする．SMT なしの 2 コアの CMP は，スループットもコストも 2 倍にする．スレッドを時分割する 1 個のコアを基準にすると，SMT の

[1] SMT は 1995 年に提案された [39]．Intel は 2002 年の Pentium 4 にハイパースレッディングとして搭載した．コア当たり 2 スレッドというこの数は x86 では今も普通である．一方 IBM の POWER8 と POWER9 は，1 つのコアで最大 8 スレッドを走らせる．Intel は 2024 年のクライアント向けチップの Lion Cove コアで SMT を再び取りやめ，その面積はコア数に充てる方が良いと判断した（IBM と Intel の資料）．

コアはコスト当たり 1.35/1.10 ≈ 1.23 倍の，CMP は 2/2 = 1 倍のスループットを与える．ただし絶対的なスループットは CMP の方が高い．2 つのスレッドが同じ実行ユニットを奪い合い，SMT がスループットをわずか 5 % しか高めないのであれば，SMT のコアは 1.05/1.10 ≈ 0.95 となり，2 個のコアの方がハードウェアの使い方として優れることになる．

> **要点**
> CMP は最も高いスループットを 2 倍のコストで与える．SMT はおよそ 10 % の追加コストで，より小さな利得を与える．スレッドの資源要求が互いに補い合うときは SMT の方がハードウェアの効率的な使い方であり，スレッドが同じ資源を奪い合うときは 2 個のコアの方が良い選択になりうる．

18.9 SMT のためのハードウェアの変更

SMT は，第 7 章・第 8 章のアウトオブオーダコアにわずかな変更を加えるだけで実現できる（図 18.5）．

出典 Lesson 18，動画 18；H&P 第 3 章；[39]．

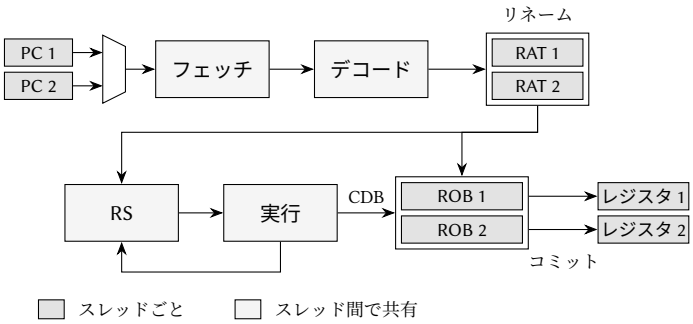


図 18.5: 2 つのスレッドのための SMT を備え，ROB を分離したアウトオブオーダコア．ROB を共有する場合，スレッドごとに保持されるのはプログラムカウンタ，RAT，アーキテクチャレジスタだけである．

フェッチ 各スレッドが自身の PC を必要とする．フェッチステージは両方の PC から命令を取り，フェッチした各命令に，それが属するスレッドの印を付ける．

デコード デコードはスレッドに依存せず，変更はない．

リネーム リネームの論理は変わらないが，各スレッドが自身の RAT を必要とする．スレッド 1 のレジスタ R1 とスレッド 2 のレジスタ R1 は，別のアーキテクチャレジスタだからである．

ディスパッチと実行 変更はない．リザベーションステーション，実行ユニット，CDB は共有される．リネームの後，オペランドは両方のスレッドにわたって一意なタグで識別されるので，異なるスレッドの命令が混同されることはない．

ROB 各スレッドが，自身のコミット点を持つ別々の ROB を持つこともできる．しかし ROB は大きな構造なので，複製するのは高価である．実際にはより多くの場合，両方のスレッドの命令が 1 つの共有 ROB に混在して並び，各エントリがその命令のスレッドを記録する．その小さな代償は，あるスレッドの完了した命令が，他方のスレッドのより古い命令がコミットするのを待たされることがある点である．

Pentium 4 は共有ではなく分割した．両方のスレッドが走っている間，各スレッドは 126 個の ROB エントリの半分を得た [40]．

コミット 各スレッドが自身のアーキテクチャレジスタファイル（ARF）を必要とし，そのスレッドの命令はそこにコミットする．

ROB を共有する場合，複製される構造—PC，RAT，ARF—はいずれも小さく，コアの高価な部分—ROB，リザベーションステーション，実行ユニット，キャッシュ，分岐予測器—は共有される．SMT がコアのコストをおよそ 10 % しか増やさないのは，このためである．

Intel はハイパースレッディングを Pentium 4/Xeon のコアに，ダイ面積の増加 5 % 未満で搭載し，最大 30 % のスループット向上を得たとしている [40]．

18.10 SMT とデータキャッシュ・TLB

SMT のコアを共有するスレッドは，それぞれ自身の仮想アドレス空間を持つ別のプロセスに属することがある．そのとき同じ仮想アドレスはスレッドごとに異なるデータを指すので，第 13 章・第 14 章のキャッシュと TLB は，両者を混同してはならない．

出典 Lesson 18，動画 19；H&P 付録 B．

- **仮想アドレスキャッシュ**は，仮想アドレスだけでインデックスし，タグも付ける．一方のスレッドのためにキャッシュされたブロックは，他方のスレッドの同じ仮想アドレスへのアクセスに対してヒットし，誤ったデータを返す．この種のキャッシュを SMT で使えるのは，各エントリがスレッドも記録し，ヒットにスレッドの一致も要求する場合に限られる．
- **VIPT キャッシュ**と物理アドレスキャッシュは，物理タグを比較する．2 つのスレッドが同じブロックにヒットするのは，同じ物理アドレスにアクセスしたときだけであり，そのときは実際に同じデータにアクセスしている．これらのキャッシュは，TLB が各スレッドに正しい変換を与える限り，SMT でも正しく動作する．
- したがって **TLB** は，スレッドを意識しなければならない．各エントリはそれが属するスレッドを記録し，アクセスのスレッド識別子が探索に渡され，ヒットにはスレッドと仮想ページ番号の両方の一致が要求される（図 18.6）．

注意 区別すべきはスレッドではなくアドレス空間である．同じプロセスの 2 つのスレッドは変換を共有する．実際の TLB は各エントリにアドレス空間識別子—x86-64 では PCID，Arm では ASID—を付ける．

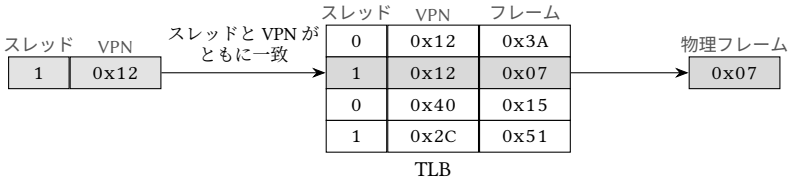


図 18.6: スレッドを意識した TLB．同じ仮想ページ番号が，2 つのスレッドに対して別々のフレームに変換される．

18.11 SMT とキャッシュ性能

コアのキャッシュは，そのコアのすべての SMT スレッドに共有される．これには 1 つの利点と，1 つの深刻な欠点がある．

出典 Lesson 18，動画 20–22；H&P 第 3 章．

- **高速なデータ共有.** 一方のスレッドが格納した値を他方のスレッドがロードするとき，そのロードはデータをすでにキャッシュ内に見つける．
- **容量の共有.** スレッドはキャッシュの容量と連想度も共有しており，それらを奪い合う．

この競合は，スレッドの**ワーキングセット**—ある時点でそのスレッドが実際に使っているブロックの集合—によって記述される．2 つの SMT スレッドのワーキングセットの大きさを WS_0，WS_1 とし，両者がともに使う部分の大きさを WS_{01} とする．

▷ working set

$$WS_0 + WS_1 - WS_{01} \leq \text{キャッシュ容量} \tag{18.2}$$

が成り立つ限り，どちらのスレッドも，単独で走るときとほとんど変わらない頻度で自身のブロックをキャッシュ内に見つける．合わせたワーキングセットがキャッシュを超えると，スレッドは互いのブロックを追い出し続け，キャッシュは**スラッシング**を起こす．それぞれのワーキングセットが単独なら収まる場合，SMT はスレッドを 1 つずつ走らせるより遅くなりうる．時分割されるスレッドは数百万サイクルの間キャッシュを独占し，コンテキストスイッチのたびに詰め直すだけであるのに対し，SMT のスレッドは絶えず互いのブロックを追い出すからである．したがってスケジューラは，合わせたワーキングセットがキャッシュに収まるスレッドどうしを一緒に走らせ，ワーキングセットだけでキャッシュを超えるスレッドは SMT の相手なしで走らせるべきである．そのようなスレッドは，自身は何も得ないまま相手のブロックを追い出してしまうからである．

注意 SMT がスループットを高めるのは，キャッシュが全スレッドのワーキングセットを同時に保持する場合だけである．

実際の L1 データキャッシュはより小さく，AMD の Zen 4 で 32 KB，Intel の Golden Cove で 48 KB，L2 はコア当たり 1–2 MB である（各社の最適化マニュアル）．

例 18.11. あるコアは 64 KB の L1 データキャッシュを持つ．スレッド A，B，C のワーキングセットはそれぞれ 40 KB，36 KB，28 KB であり，いずれも単独なら

キャッシュに収まる．A と B は共通のデータを扱っており，そのうち 20 KB が両方のワーキングセットに現れるので，SMT で一緒に走らせるのに必要なのは $40 + 36 - 20 = 56$ KB であり，これは収まる．A と C はデータを共有せず，合わせて 68 KB を必要とする．SMT ではスラッシングを起こし，時分割した方が速い可能性が高い．

要点 — 章のまとめ

マルチプロセッサは MIMD マシンである．パイプラインを広げても収穫逓減となり，周波数を上げると電力がおよそ f^3 で増えるのに対し，コアを複製すればムーアの法則によるトランジスタをスループットに変えられるため，マルチプロセッサはますます速くなる単一プロセッサに取って代わった–ただしそれは並列プログラムに限られ，その高速化率は頭打ちになる（式 (18.1)）．集中共有メモリ（UMA，SMP）は単純だが，メモリ容量と帯域によって少数のコアに制限される．分散共有メモリ（NUMA）は大規模なマシンまでスケールするが，リモートアクセスが遅いという代償を払う．メッセージパッシングは正しく書くのが難しく，共有メモリは速くするのが難しい．1 つのコアは，時分割，粗粒度または細粒度マルチスレッディング，あるいは SMT によって複数のスレッドを走らせられる．SMT が複製するのは PC，RAT，ARF 程度であり，スレッドを意識した TLB を必要とし，共有されたキャッシュが合わせたワーキングセットを保持できる場合にのみ引き合う（式 (18.2)）．メモリを共有するコアは，キャッシュの一貫性も保たなければならない．これが第 19 章の主題である．

第 19 章

キャッシュコヒーレンス

講義動画	Lesson 19，動画 1–33
教科書	H&P [1] §5.2–5.4
キーワード	キャッシュコヒーレンス，バススヌーピング，書き込み時更新方式，書き込み時無効化方式，MSI プロトコル，キャッシュ間転送，MOSI プロトコル，MESI プロトコル，MOESI プロトコル，ディレクトリ方式，コヒーレンスミス，真共有，偽共有

第 18 章の共有メモリ型マルチプロセッサでは，各コアが自分自身のキャッシュを持ち，すべてのコアが 1 つの主記憶を共有する．キャッシュが保持しているのはデータの私的なコピーであり，あるコアが自分のコピーに書き込むと，他のキャッシュのコピーは–そしてライトバック方式のもとでは主記憶も–古い値を保持したままである．この章では，そのような古いコピーがあるとマルチプロセッサが誤った結果を返すことを示し，それを排除する性質であるキャッシュコヒーレンスを定義し，それを提供する機構–共有バス上の書き込み時更新方式と書き込み時無効化方式，MSI プロトコルとその改良である MOSI，MESI，MOESI，そしてより大きなマシンのためのディレクトリ方式–を順に組み立てる．最後に，コヒーレンスが引き起こす追加のキャッシュミスを扱う．

注意 コヒーレンスは一度に 1 つのメモリ位置についての性質である．異なる位置へのアクセスがどの順序で見えるかはメモリ一貫性であり，第 21 章で扱う．

19.1 コヒーレンスの問題

出典 Lesson 19，動画 1–3；H&P §5.2．

各コアが第 12 章のライトバック方式に従う専用キャッシュを持つ，集中共有メモリのマルチプロセッサ（18.4 節）を考える．単一プロセッサではこの方式は無害である．主記憶がしばらく古い値を保持することはあるが，それを読みうる唯一のプロセッサは，自分のキャッシュにある新しいコピーを先に見つけるからである．複数のキャッシュがあると，もはやそうはいかない．図 19.1 は，初めに 5 を保持するメモリ位置 X を追ったものである．

X は 1 つのワードだが，キャッシュが扱うのはそれを含むブロック全体である．x86-64 と大半の Arm コアでは 64 バイト，Apple の M シリーズでは 128 バイトである（第 12 章）．

1. コア 0 が X を読む．読み出しはミスし，コア 0 のキャッシュは X を含むブロックを主記憶から読み込む．
2. コア 1 が X を読み，同様にブロックのコピーを読み込む．
3. コア 0 が X に 8 を書く．書き込みはヒットし，ライトバック方式のもとでは，コア 0 のキャッシュ内のコピーだけが変更され，そのコピーはダーティになる．
4. コア 1 が X を読む．読み出しはコア 1 のキャッシュでヒットし，8 が書かれているにもかかわらず 5 を返す．
5. コア 2 が X を読む．読み出しはミスし，やはり 5 を保持している主記憶から供給される．

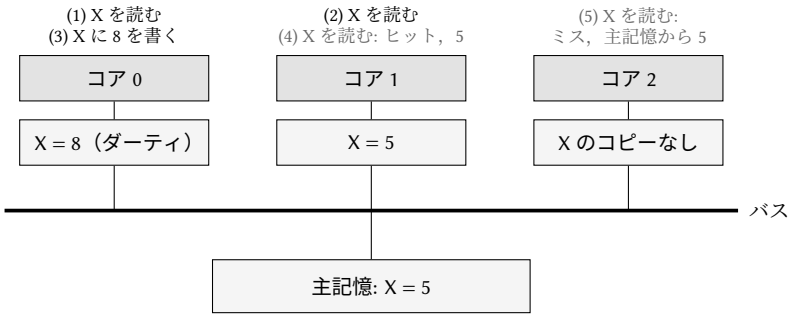


図 19.1: コヒーレントでないキャッシュ．コア 0 とコア 1 が，5 を保持する X を読み（手順 1 と 2），コア 0 が自分のキャッシュに 8 を書く（手順 3）．その後のコア 1 の読み出し（手順 4）とコア 2 の読み出し（手順 5）は，どちらも古い値 5 を返す．

2 つの誤った読み出しの原因は異なる．手順 5 は主記憶から古い値を読む．書き込みのたびに主記憶を更新するライトスルー方式であれば，これは避けられ

る．手順 4 は他のキャッシュにある古いコピーを読んでおり，個々のキャッシュの書き込み方式ではこれを避けられない．コア 0 のキャッシュが書き込みをどう扱おうと，コア 1 のキャッシュ内のコピーは変わらないからである．

注意 ライトスルー方式は専用キャッシュをコヒーレントにしない．他のキャッシュ内のコピーはやはり古くなる．

19.2 コヒーレンス

共有メモリのプログラムは，すべてのコアが共有する，キャッシュのない 1 つのメモリの振る舞いを期待する．

出典 Lesson 19，動画 4–5；H&P §5.2．

定義 19.1 (キャッシュコヒーレンス)．コアが専用キャッシュを持つマルチプロセッサは，すべてのメモリ位置 X について次が成り立つとき，**キャッシュコヒーレンス**を提供するという．

▷ cache coherence

1. あるコアでの X の読み出しは，その間に他のコアが X に書いていなければ，そのコアが X に最後に書いた値を返す．
2. あるコアが X に書き，十分に後に別のコアが X を読み，その間に X への書き込みがなければ，その読み出しは最初のコアが書いた値を返す．
3. X への書き込みは直列化される．すなわち，すべてのコアが X への任意の 2 つの書き込みを同じ順序で観測する．

第 1 の条件は単一プロセッサにおけるメモリの振る舞いであり，どのキャッシュもすでにこれを保っている．第 2 の条件はそれを他のコアへ拡張するが，遅延は許す．別のコアの書き込みがまだ進行中の間に発行された読み出しは，古い値を返してよい．第 3 の条件により，ほぼ同時の 2 つの書き込みのどちらが後だったかについて，すべてのコアの見解が一致する．図 19.1 の手順 4 と 5 の読み出しは第 2 の条件に違反している．すなわち，これらのキャッシュは**コヒーレントでない**．

注 19.2．コヒーレンスは，読み出しが何を返すかに対する要請であって，コピーが何を保持しているかに対する要請ではない．コピーどうしが食い違っていてもコヒーレンスに違反するとは限らない．主記憶はキャッシュ内のダーティなブロックより古い値を保持しうるし，キャッシュも，無効と印が付いたラインに古い値を残していてよい．どの読み出しも最新の値を保持するコピーから供給される限り，システムはコヒーレントである．

19.3 コヒーレンスの実現

定義と 図 19.1 の例は，コヒーレンスの機構が何をしなければならないかを示している．

出典 Lesson 19，動画 6；H&P §5.2．

1. **書き込みを可視にする．**ライトバック方式では，ヒットする書き込みはコアの外に何の動作も生じさせないので，他のキャッシュは自分のコピーが古くなったことを知りえない．したがって，他のキャッシュが保持しているかもしれないブロックへの書き込みは，それらに知らせなければならない．
2. **他のコアの書き込みに対処する．**他のコアが書いたブロックのコピーを保持するキャッシュは，自分のコピーを新しい値で**更新**するか，**無効化**して次のアクセスがミスし新しい値を取得するようにしなければならない．
3. **書き込みを順序付ける．**異なるコアによる同じブロックへの書き込みは，すべてのキャッシュが観測する 1 つの順序で，すべてのコピーに適用されなければならない．
4. **読み出しを最新のコピーから供給する．**ミスした読み出しは最新の値を得なければならないが，その値は主記憶ではなく他のキャッシュにあるかもしれない．

集中共有メモリでは，すべてのキャッシュと主記憶が 1 本のバスに接続され，バス上のあらゆる操作が，接続されたすべての装置に届く．そのためコヒーレンスは**バススヌーピング**の上に構築できる．各キャッシュはバス上のすべての操作を–主記憶宛てや他のキャッシュ宛てのものも含めて–監視し，自分が保持するブロックに関わるものに反応する．バスは一度に 1 つの操作しか運ばず，すべてのキャッシュがその順序で操作を観測するので，スヌーピングは操作の順序付けも行う．共有バスを持たないマシンは，この 2 つの仕事のために別の機構，すなわち 19.9 節 のディレクトリを必要とする．このようにコヒーレンスの機構は，

▷ bus snooping

他のコピーを更新するか無効化するか，そして書き込みをスヌーピングで観測・順序付けするかディレクトリでそうするか，という 2 つの選択の組合せである．

19.4 書き込み時更新方式

書き込みに対処する第 1 の方法は，新しい値をすべてのコピーへ伝播させることである．

出典 Lesson 19，動画 7–11；H&P §5.2．

定義 19.3 (書き込み時更新方式). **書き込み時更新方式**では，すべての書き込みがそのアドレスと新しい値とともにブロードキャストされ，書かれたブロックのコピーを保持する他のすべてのキャッシュが自分のコピーを更新する．

▷ write-update coherence

スヌーピングのもとでは，各コアはすべての書き込みをそのままバスへ出す．主記憶は他の書き込みと同様にそれを実行し，他のキャッシュはそれをスヌープして，当該ブロックを保持していれば自分のコピーを更新する．図 19.1 では，手順 3 の書き込みが主記憶とコア 1 のキャッシュの X を 8 に変え，その後の 2 つの読み出しはどちらも 8 を返すことになる．バスは一度に 1 つの操作しか運ばないので，2 つのコアが同じ位置に同時に書いた場合でも，異なるコアによる書き込みは同じ順序ですべてのコピーに適用される．これはコヒーレンスの第 3 の条件が要求するとおりである．

注意 スヌーピングは他のキャッシュを探しに行くことではない．各キャッシュはバスを監視し，自分のタグだけを引く．

この基本的な方式は正しいが高価である．すべての書き込みが共有バスを占有し，さらに主記憶でも実行される—これはまさにライトバック方式が避けようとするコストである．2 つの最適化がそのほとんどを取り除く．

19.4.1 主記憶への書き込みを避ける

第 12 章のダーティビットを使えば，キャッシュは主記憶への書き込みを先送りできる．書き込みは，他のキャッシュがコピーを更新できるようにバスへ出されるが，主記憶はそれを無視する．代わりに，書き込んだキャッシュがブロックのダーティビットを立て，そのブロックを追い出すときに主記憶へ書き戻す．自分がダーティに保持するブロックへの他のコアの書き込みをスヌープしたキャッシュは，自分のコピーを更新してダーティビットを下ろす．こうして，最後に書いたキャッシュだけがそのブロックに責任を持つ．ダーティなコピーが存在する間，主記憶は古い．そのためダーティなコピーを持つキャッシュは，そのブロックの**読み出し**についてもバスをスヌープする．他のコアがそのブロックでミスして読み出しをバスへ出したとき，このキャッシュがデータを供給し，主記憶は応答しない．

19.4.2 バスへの書き込みを避ける

大半のブロックは 1 つのコアだけが使うので，そのようなブロックへの書き込みをブロードキャストしてもバスを無駄に占有するだけである．各キャッシュラインの**共有ビット**は，他のキャッシュがそのブロックを保持している可能性があるかどうかを記録する．キャッシュは，自分が保持するブロックの読み出しをスヌープすると，自分の共有ビットを立て，コピーが存在することをバスの**共有線**上で知らせる．読み出しを行ったキャッシュは，この信号があれば自分の共有ビットを立てる．共有ビットが下りているブロックへの書き込みは，単一プロセッサと同様に局所的に行われ，共有ビットが立っているブロックへの書き込みだけがバスへ出される．

共有の信号を運ぶのはワイヤード OR の 1 本の線である．ブロックを保持するキャッシュがそれを引くので，すべてのキャッシュが同時に答える．

共有ビットは保守的である．他のコピーが追い出された後も立ったままになる．

例 19.4. 2 つのコアが，初めはどのキャッシュも保持していないブロック X にアクセスする．表 19.1 は，6 回のアクセスについて，基本方式の場合と最適化を加えた場合のバス操作数と主記憶への書き込み数を数えたものである．ダーティビットがあれば，この系列の間に主記憶は書かれず（コア 0 が追い出すときに一度だけ書き戻す），手順 4 の読み出しはコア 0 が供給する．さらに共有ビットがあれば，コア 1 がコピーを持つ前の 2 つの書き込みは局所的なままとなり，バスへ出されるのは手順 5 の書き込みだけである．

表 19.1: 書き込み時更新方式（例 19.4）．rd: バス上の読み出し．wr: バス上の書き込み，または主記憶への書き込み．rd*: 主記憶ではなくコア 0 が供給した読み出し．

		基本方式		ダーティビット		ダーティビットと共有ビット	
手順	アクセス	バス	主記憶	バス	主記憶	バス	主記憶
1	コア 0 が X を読む	rd	—	rd	—	rd	—
2	コア 0 が X に書く	wr	wr	wr	—	—	—
3	コア 0 が X に書く	wr	wr	wr	—	—	—
4	コア 1 が X を読む	rd	—	rd*	—	rd*	—
5	コア 0 が X に書く	wr	wr	wr	—	wr	—
6	コア 1 が X を読む	—	—	—	—	—	—
合計		5	3	5	0	3	0

19.5 書き込み時無効化方式

書き込みに対処する第 2 の方法は，他のコピーを更新する代わりに取り除くことである．

出典 Lesson 19，動画 12–15；H&P §5.2．

定義 19.5 (書き込み時無効化方式). **書き込み時無効化方式**では，他のキャッシュが保持しているかもしれないブロックに書き込むコアが，まずそのブロックの無効化をブロードキャストし，そのブロックのコピーを保持する他のすべてのキャッシュが，有効ビットを下ろしてコピーを破棄する．

▷ write-invalidate coherence

無効化はブロックのアドレスだけを運ぶ．いったんブロードキャストされれば，書き込んだキャッシュが唯一の有効なコピーを保持するので，そのブロックへの—ブロック内のどのアドレスへのものであっても—その後の書き込みは局所的であり，別のコアがそのブロックにアクセスするまでバス操作を必要としない．そのアクセスは，コピーが無効化されているのでミスし，ブロックが書き戻されていれば主記憶から，そうでなければそれを書いたキャッシュから最新の値を得る．

19.5.1 更新と無効化の比較

どちらの方式のバス操作が少なくて済むかは，アクセスのパターンに依存する．

- **別のコアが読む前に 1 つのブロックへ複数回書き込む場合．**書き込み時更新方式はすべての書き込みをブロードキャストするが，書き込み時無効化方式は 1 回の無効化をブロードキャストするだけである．無効化はブロック単位で働くので，これは 1 つのアドレスへの繰り返しの書き込みについても，同じブロック内の異なるアドレスへの書き込みについても同様に成り立つ．
- **スレッドが別のコアへ移る場合．**そのスレッドが使っていたブロックは元のコアのキャッシュに残り，スレッドが新しいコアでそれらを読むと，どちらのキャッシュもそれらを共有と記録する．書き込み時更新方式では，そのようなブロックへのスレッドのその後の書き込みはすべてブロードキャストされ，元のコアがもう使わないコピーを，そのコピーがたまたま追い出されるまで更新し続ける．書き込み時無効化方式では，最初の書き込みが古いコピーを取り除き，その後の書き込みはすべて局所的である．
- **異なるコアによる書き込みと読み出しが交互に起こる場合．**あるコアがブロックに書き，別のコアが書き込みのたびに新しい値を読むなら，書き込み時更新方式は書き込みごとに 1 回のバス書き込みを必要とし，読み出しはすべてヒットする．書き込み時無効化方式は，書き込みごとに無効化を，読み出しごとにミスを必要とする．

書き込み時更新方式が有利なのは最後のパターンだけであり，最初の 2 つのほうが一般的な場合である．したがってマルチプロセッサは書き込み時無効化方式を用いる．この章の残りのプロトコルはすべてこれに基づく．

書き込み時更新方式の実機は 1980 年代の少数のマシン—Xerox PARC の Dragon と DEC SRC の Firefly—だけで，以後の主流のプロセッサは無効化する（H&P §5.2）．

例 19.6. コア 0 とコア 1 がともにブロック X のコピーを保持しているとする．コア 0 が X に 10 回書き，その後コア 1 が X を 1 回読むなら，書き込み時更新方式

は 10 回のバス書き込みを必要とし，コア 1 の読み出しはヒットする．書き込み時無効化方式は，1 回の無効化とコア 1 のミスのための 1 回のバス読み出し，合計 2 回のバス操作で済む．逆にコア 1 が 10 回の書き込みのそれぞれの後に X を読むなら，書き込み時更新方式は依然として 10 回のバス操作で済むが，書き込み時無効化方式は 10 回の無効化と 10 回のミス，合計 20 回を必要とする．

> **要点**
> 書き込み時更新方式は，書き込みごとに 1 回のバス操作を代償として，すべてのコピーを最新に保つ．書き込み時無効化方式は，ブロックごとに 1 回の無効化を払い，その後は書き込んだコアが局所的に処理を進められるようにする．書き込みは通常 1 つのコアからまとまって来るので，実用的なコヒーレンスプロトコルは無効化を行う．

19.6 MSI プロトコル

出典 Lesson 19，動画 16–19；H&P §5.2．

▷ MSI protocol

スヌーピングによる書き込み時無効化方式は，キャッシュラインごとの小さな状態機械で記述される．ラインにあるブロックの状態は，コア自身のアクセスと，キャッシュがバス上で観測する他のコアのアクセスとによって変化する．そのような状態機械のうち最も単純な **MSI プロトコル**は，ラインの有効ビットとダーティビットを使って 3 つの状態を区別する．

M（modified） 有効かつダーティ．ブロックはこのキャッシュで書き込まれ，他のすべてのコピーは無効化されている．すなわち，このキャッシュが唯一の有効なコピーを保持しており，その内容は主記憶のブロックより新しい．読み出しも書き込みも局所的である．

S（shared） 有効かつダーティでない．他のキャッシュも有効なコピーを保持しているかもしれない．読み出しは局所的だが，書き込みはまず無効化をブロードキャストしなければならない．

I（invalid） 有効ビットが下りている．どのアクセスもミスする．

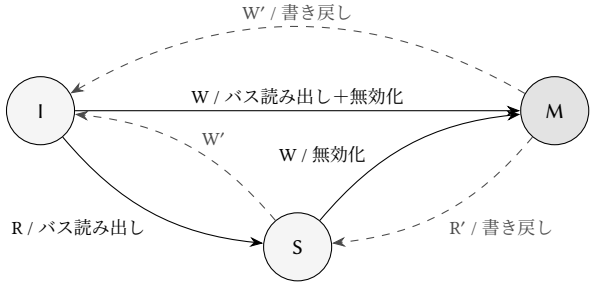


図 19.2: 1 つのキャッシュ内の 1 つのブロックに対する MSI プロトコル．R，W はこのコアによる読み出しと書き込み，R′，W′（破線）はバス上で観測される他のコアによる読み出しと書き込みである．辺のないアクセスは状態を変えず，バス操作も必要としない．網掛けの状態は，主記憶より新しいブロックを保持している．

図 19.2 は遷移を示す．コア自身のアクセス（実線の辺）は次のように扱われる．

- I での読み出しはミスする．キャッシュは読み出しをバスへ出し，ブロックを読み込んで S に入る．
- I での書き込みもミスする．キャッシュは 1 回のバス操作でブロックを読み，他のすべてのコピーを無効化して，M に入る．
- S での書き込みは，無効化をブロードキャストして M に入る．
- S での読み出し，および M での読み出しと書き込みは，バス操作を必要としない．

他のコアのアクセス（破線の辺）はバス上で観測される．

- 他のコアの書き込みは，その無効化によって知らされ，S から I への遷移を起こす．主記憶が同じ値を保持しているので，コピーは書き戻されずに破棄される．
- M では，キャッシュがそのブロックの唯一の最新の値を保持している．他のコアがそのブロックを読むと，キャッシュはブロックを主記憶へ書き戻し，他のコアもコピーを持つことになるので S に入る．他のコアがそのブ

ロックに書くと，キャッシュはブロックを主記憶へ書き戻し，書き込んだコアはそこからブロックを得て，このキャッシュは I に入る．

キャッシュがブロックを追い出すとき，主記憶へ書き戻すのは M の場合だけであり，S のブロックは単に破棄される．

ヒント MSI の辺はすべて 3 つの規則のどれかに従う．このコアの書き込みは M へ，他のコアの書き込みは I へ，読み出しは S へ至る．

19.6.1 キャッシュ間転送

あるコアが，別のキャッシュが M で保持するブロックでミスすると，それが読み出しであれ書き込みであれ，主記憶はバス上の読み出しに古い値で応答してしまう．その読み出しをスヌープする M のキャッシュは，これを 2 つの方法で防げる．

1. **中断と再試行**: 読み出しを中断させ，ブロックを主記憶へ書き戻す．読み出しを行ったコアは読み出しを再度発行し，主記憶が現在の値でそれに応答する．これには主記憶への書き込み 1 回と主記憶からの読み出し 1 回のコストがかかる．
2. **介入**: 主記憶が応答してはならないことを知らせ，自分でブロックをバスへ出す．読み出しミスでは，両方のコピーがダーティでない S に落ち着くので，主記憶もバスからブロックを取り込む．

介入，すなわち**キャッシュ間転送**は，より複雑なバスのハードウェアを必要とするが，1 回の転送でミスに応答し，読み出しミスの場合には，MSI が M から S への遷移で要求する書き戻しも同時に行う．書き込み時更新方式のダーティビットの最適化（19.4 節）も同じように読み出しに応答するが，そちらではキャッシュはダーティのままであり，主記憶はブロックを取り込まない．

▷ cache-to-cache transfer

キャッシュ間の転送には少なくとも L3 ヒット分，Skylake で約 44 サイクルかかる．DRAM アクセスには 60–100 ns かかる（Intel 最適化マニュアル，第 14 章）．

例 19.7. 3 つのコアが，初めはどのキャッシュも保持していないブロック X にアクセスする（表 19.2）．手順 1 と 2 の読み出しは，S の 2 つのコピーを残す．手順 3 のコア 1 の書き込みはコア 0 のコピーを無効化し，その 2 回目の書き込み（手順 4）は局所的である．手順 5 でコア 2 の読み出しがバスに現れたとき，唯一の最新の値を保持しているのはコア 1 である．コア 1 はブロックを主記憶へ書き戻し，両方のコピーは S に落ち着く．手順 6 のコア 0 の書き込みはミスし，その両方を無効化する．7 回のアクセスのうち 5 回がバス操作を必要とし，1 回が主記憶への書き込みを引き起こす．

表 19.2: MSI プロトコルのもとでのブロック X（例 19.7）．アクセス後の各キャッシュにおけるブロックの状態．

手順	アクセス	コア 0	コア 1	コア 2	バスと主記憶
1	コア 0 が X を読む	S	I	I	読み出し，主記憶が供給
2	コア 1 が X を読む	S	S	I	読み出し，主記憶が供給
3	コア 1 が X に書く	I	M	I	無効化
4	コア 1 が X に書く	I	M	I	—
5	コア 2 が X を読む	I	S	S	読み出し，コア 1 が X を書き戻す
6	コア 0 が X に書く	M	I	I	読み出しと無効化
7	コア 0 が X を読む	M	I	I	—

19.7 MOSI プロトコル

キャッシュ間転送は，M から S への遷移における主記憶への書き込みを取り除きはしない．転送の後，両方のコピーはダーティでない S にあるので，どちらのキャッシュも追い出しの際にコピーを破棄してしまう．転送の間に主記憶がブロックを取り込まない限り，両方のコピーがなくなった時点で最新の値は失われる．したがって，あるコアが書き，別のコアが読むことを交互に繰り返すブロックは，どちらのキャッシュもそれを追い出さないにもかかわらず，書き込みごとに主記憶への書き込みのコストを払うことになる．

出典 Lesson 19，動画 20–21．

MOSI プロトコルは，共有されたブロックのコピーの 1 つをその所有者とすることで，これらの書き込みを避ける．

▷ MOSI protocol

O（owned） 有効．他のキャッシュが S でコピーを保持しているかもしれない．ブロックは主記憶のブロックより新しく，このキャッシュがそれに責任を

持つ．すなわち，他のコアによるそのブロックの読み出しに応答し，追い出す際にブロックを主記憶へ書き戻す．読み出しは局所的であり，書き込みは無効化をブロードキャストして M に入る．

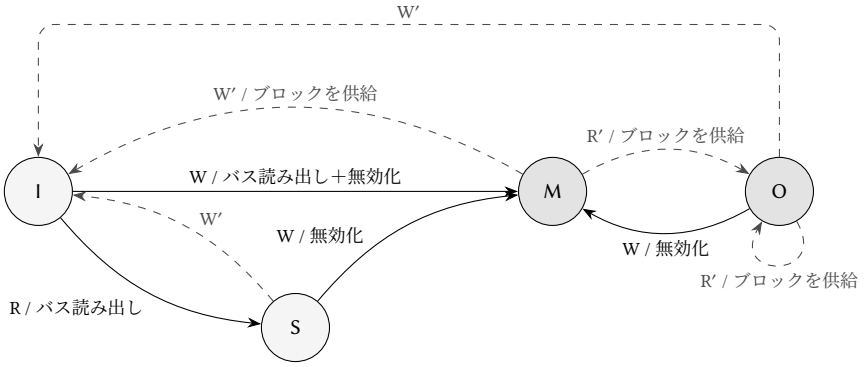


図 19.3: MOSI プロトコル．記法は 図 19.2 と同じ．網掛けの状態は，主記憶より新しいブロックに責任を持つ．O での W′ では，書き込んだコアがミスした場合にのみブロックが供給される．

キャッシュが M で保持するブロックを他のコアが読むと，キャッシュはキャッシュ間転送でブロックを供給し，主記憶に書き込むことなく，S ではなく O に入る（図 19.3）．O では，その後のすべての読み出しに対してブロックを供給する．所有者による書き込みは無効化をブロードキャストして M に戻る．他のコアによる書き込みは O から I への遷移を起こすが，所有者がまずブロックを供給するのは，書き込んだコアがミスした場合だけである．S にいる書き込み側はすでに現在の値を保持しているからである．ブロックが主記憶に達するのは，キャッシュが M または O でそれを追い出すときだけである．例 19.7 では，MOSI なら手順 5 でコア 1 は S ではなく O に入り，主記憶への書き込みは起こらず，手順 6 のコア 0 の書き込みに対してはコア 1 がブロックを供給することになる．

注意 “所有” は “排他” ではない．他のキャッシュは S で有効なコピーを保持しうる．所有者とは，そのブロックに責任を持つキャッシュのことにすぎない．

AMD のプロセッサは O 状態を使う．AMD64 Architecture Programmer’s Manual は MOESI を規定しており，ダーティなラインは主記憶を経ずにコアからコアへ渡れる．

19.8 排他状態

MSI と MOSI は，非常によくあるアクセスパターンを非効率に扱う．プログラムはしばしば，インクリメントのように，変数を読んでから書く．そして大半の変数は 1 つのスレッドだけが使う．そのようなブロックでは，読み出しがミスして S に入り，続く書き込みが–他のどのキャッシュもコピーを保持していないにもかかわらず–無効化をブロードキャストする．このような読み出しと書き込みの組は，単一プロセッサならミスだけで済むところ，2 回のバス操作を必要とする．

S は，キャッシュには区別できない 2 つの場合を 1 つにまとめている．他のキャッシュがコピーを保持している場合と，ブロックが共有されておらずまだ書かれていない場合である．第 4 の状態がこれらを分ける．

出典 Lesson 19，動画 22–25；H&P §5.2；[41]（Illinois プロトコル）．

E（exclusive） 有効かつダーティでない．他のどのキャッシュもコピーを保持していない．読み出しは局所的であり，書き込みも局所的で，バス操作なしに M へ入る．

ヒント 2 つの問いで状態が決まる．他のキャッシュがコピーを持ちうるか，このキャッシュが書き戻さなければならないか．M ははいいえ・はい，E ははいいえ・いいえ，S ははい・いいえ，O ははい・はいである．

I での読み出しは，他のどのキャッシュもそのブロックを保持していなければ E に，そうでなければ S に入る．キャッシュは，どちらの場合かをバス読み出しそのものの間に知る．ブロックを保持するキャッシュがその読み出しをスヌープし，19.4 節 と同様にバスの共有線上で知らせるからである．E で保持しているブロックを他のコアが読むと，主記憶が最新なので，キャッシュは書き込みなしに S に入る．他のコアによる書き込みは E から I への遷移を起こす．

MSI に E を加えたものが **MESI プロトコル**であり，MOSI に加えたものが **MOESI プロトコル**である（図 19.4）．表 19.3 は 5 つの状態をまとめたものである．

▷ MESI protocol
▷ MOESI protocol

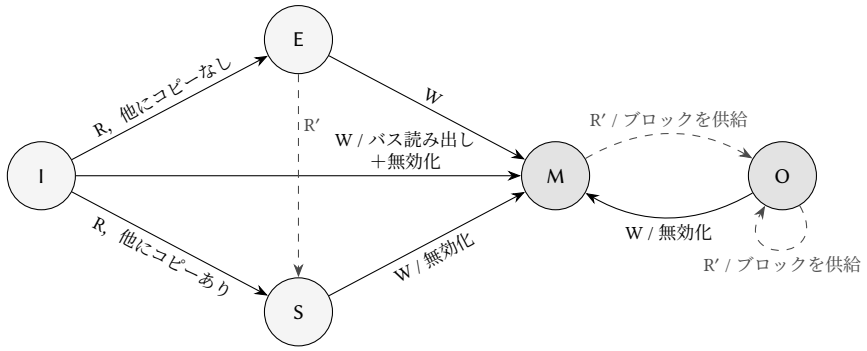


図 19.4: MOESI プロトコル．記法は 図 19.2 と同じ．I から出るすべての遷移はバス読み出しを行う．他のコアによる書き込みは，すべての有効な状態を I へ移す．書き込んだコアがミスした場合は，M または O のキャッシュがブロックを供給する（これらの辺は描いていない）．

表 19.3: MOESI の状態．MSI は M，S，I を使い，MOSI は O を，MESI は E を加える．責任を持つキャッシュは，他のコアによる読み出しに応答し，追い出しの際にブロックを書き戻す．

状態	有効	他のコピー	責任	バスなしの書き込み
M（modified）	はい	なし	はい	はい
O（owned）	はい	ありうる	はい	いいえ
E（exclusive）	はい	なし	いいえ	はい
S（shared）	はい	ありうる	いいえ	いいえ
I（invalid）	いいえ	ありうる	いいえ	いいえ

例 19.8. 2 つのコアが，初めはどのキャッシュも保持していないブロック X にアクセスする．コア 0 が X を読んでから書き，コア 1 がそれを読み，コア 0 がもう一度書き，コア 1 がもう一度読む．表 19.4 は，4 つのプロトコルのもとでの状態と操作を示す．E は，コア 0 の読み出しが他のコピーを見つけなかったので，手順 2 の無効化を省く．O は，手順 3 と 5 の主記憶への書き込みを，コア 0 がいずれ X を追い出すときの 1 回の書き込みを代償として省く．

Intel のソケット間プロトコルは第 5 の状態 F（forward）を加える．O と同様に読み出しに応答する共有者を 1 つ定めるが，F はダーティでない（Intel QPI/UPI の資料）．

表 19.4: 4 つのプロトコルのもとでのブロック X（例 19.8）．各アクセス後のコア 0 とコア 1 の状態．b: バス操作．m: 主記憶への書き込み．

		MSI		MOSI		MESI		MOESI	
手順	アクセス	状態	操作	状態	操作	状態	操作	状態	操作
1	コア 0 が X を読む	SI	b	SI	b	EI	b	EI	b
2	コア 0 が X に書く	MI	b	MI	b	MI	—	MI	—
3	コア 1 が X を読む	SS	b, m	OS	b	SS	b, m	OS	b
4	コア 0 が X に書く	MI	b	MI	b	MI	b	MI	b
5	コア 1 が X を読む	SS	b, m	OS	b	SS	b, m	OS	b
バス操作			5		5		4		4
主記憶への書き込み			2		0		2		0

要点
4 つのプロトコルのいずれにおいても，主記憶より新しいブロックに責任を持つのは M または O のキャッシュだけであり，バス操作なしに書き込めるのは M または E のキャッシュだけである．2 つの追加は独立である．E 状態は共有されていないデータを読んだ後の無効化を省き，O 状態は，変更されたブロックが他のコアに読まれたときの主記憶への書き込みを省く．

19.9 ディレクトリ方式

スヌーピングは，すべてのキャッシュがすべてのバス操作を観測することを要求するので，共有バスと切り離せない．共有バスはマシンを少数のコアに制限し（第 18 章），コアがネットワークを介して 1 つのアドレス空間を共有するより大きなマシン（H&P の意味での分散共有メモリ．第 18 章の用語に関する注を参照）には，そもそもバスがない．すべての読み出しと無効化をネットワーク経由で全キャッシュへ送れば，操作そのものよりはるかに大きなコストがかかる．またバスがなければ，あるブロックに対する異なるコアの操作を順序付けるものが何もない．そのため，より大きなマシンは，各ブロックがどこにキャッシュされているかを明示的に管理する．

出典 Lesson 19，動画 26–30；H&P §5.4；[42]．

バスが扱えるのは数個のコアだが，今日のサーバ用ソケットは 100 個を超えるコア（AMD EPYC 9005 では最大 192 個）を載せ，オンチップのネットワークで接続する（AMD 製品資料）．

▷ directory-based coherence

定義 19.9 (ディレクトリ方式). **ディレクトリ方式**では，ディレクトリが，すべてのブロックについて，どのキャッシュがそのコピーを保持しているかを記録する．キャッシュは，スヌーピングのプロトコルならブロードキャストする操作をディレクトリへ送り，ディレクトリはそれを，そのブロックを保持するキャッシュだけへ転送する．あるブロックに対するそうした要求はすべてそのディレクトリエントリを通るので，エントリが要求を直列化する．

ディレクトリエントリに必要な情報はわずかである（図 19.5）．

- 1 つのキャッシュがそのブロックを M または E で保持しており，誰にも知らせずに書き込めることを示す**排他**ビット．
- そのキャッシュが有効なコピーを保持しているかどうかを示す，キャッシュごとに 1 つの**存在**ビット．

存在ビットはコアごと・ブロックごとに 1 ビットである．64 コアで 64 バイトのブロックなら，完全なディレクトリは主記憶の約 13 % を占める（第 22 章）．

状態そのものは依然としてキャッシュが保持するので，前節までのプロトコルをそのまま使える．プロトコルが局所的に扱うアクセス–すべての有効な状態での読み出しと，M および E での書き込み–はディレクトリを介さず，それ以外がディレクトリへ送られる．

- **読み出しミス．**どのキャッシュもそのブロックを保持していなければ，主記憶がブロックを供給し，読み出したコアが唯一の保持者となる（MESI と MOESI では E で，排他ビットが立つ）．そうでなければ，ディレクトリは読み出しを，そのブロックを保持するキャッシュの 1 つへ転送し，そのキャッシュがキャッシュ間転送でブロックを供給する．エントリが排他であれば，読み出しは排他的な保持者へ送られ，その保持者はスヌープした読み出しの場合と同じように状態を変え，排他ビットが下ろされる．いずれの場合も，読み出したコアがエントリに加えられる．[1]
- **S または O での書き込み，および書き込みミス．**ディレクトリは，存在ビットが立っているキャッシュだけへ無効化を転送し，書き込んだコアを排他的な保持者として記録する．書き込みミスでは，ブロックを保持するキャッシュが I に入る前にブロックを供給し，どのキャッシュも保持していなければ主記憶が供給する．

キャッシュはブロックを追い出すとき–M または O では書き戻しながら– ディレクトリに通知し，ディレクトリはそのキャッシュの存在ビットと排他ビットを下ろす．これによりエントリは正確に保たれる．

ディレクトリは 1 か所に置かれる必要はない．ディレクトリは複数のスライスに分割されて異なるノードに保持され，各スライスはブロックの集合を担当するので，どの 1 つのノードもすべての要求を処理せずに済む．あるブロックに対する要求はすべて，そのブロックを担当するスライス，すなわちブロックのホームへ送られ，このスライスがそのブロックへのすべてのアクセスの順序を決める．

[1] ディレクトリはマルチコアより古い．1990 年代初頭のスタンフォード大学の DASH は，市販のワークステーションにディレクトリボードを加えて最大 64 プロセッサのコヒーレントなマシンを作り，SGI Origin 2000 につながった．

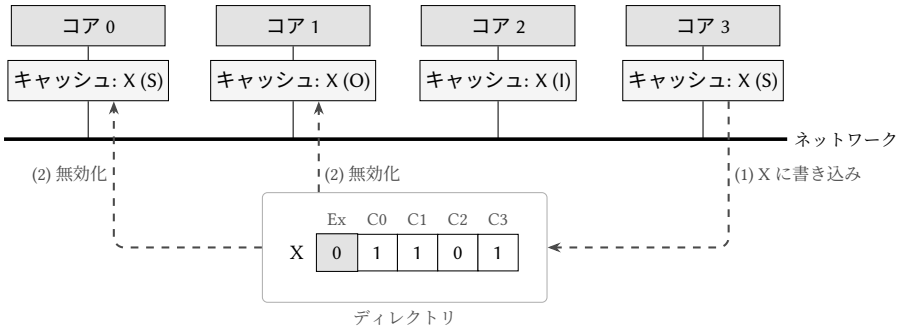


図 19.5: 例 19.10 の手順 5 におけるディレクトリ方式．X のエントリは，コア 0，1，3 がコピーを保持しており，どのコアも X を排他的には保持していないことを示す．コア 3 の書き込み (1) は，コア 0 と 1 だけへ無効化として転送される (2)．

表 19.5: ディレクトリ方式の MOESI のもとでのブロック X（例 19.10）．各アクセス後のキャッシュの状態とディレクトリエントリ（排他ビット，コア 0 から 3 の存在ビット），および送られたメッセージ（d: ディレクトリ）．

		コアのキャッシュ				エントリ		
手順	アクセス	0	1	2	3	排他	存在	メッセージ
1	コア 1 が X を読む	I	E	I	I	1	0100	1→d，主記憶 →1
2	コア 1 が X に書く	I	M	I	I	1	0100	—
3	コア 3 が X を読む	I	O	I	S	0	0101	3→d→1→3
4	コア 0 が X を読む	S	O	I	S	0	1101	0→d→1→0
5	コア 3 が X に書く	I	I	I	M	1	0001	3→d→0，1
6	コア 3 が X に書く	I	I	I	M	1	0001	—

例 19.10. 4 つのコアが，ディレクトリとともに MOESI を用いる（表 19.5）．コア 1 が X を読み，続いて書く．その読み出しは他のコピーを見つけなかったので，X はコア 1 のキャッシュで E にあり，書き込みはメッセージを必要としない．コア 3 の読み出しはディレクトリへ行き，ディレクトリはそれを排他的な保持者であるコア 1 へ転送する．コア 1 はブロックを供給し，その所有者となる．ディレクトリはコア 0 の読み出しを保持者の 1 つ，やはりコア 1 へ転送する．手順 5 でコア 3 が X に書くと，ディレクトリはコア 0 と 1 だけへ無効化を送る．コピーを保持していないコア 2 にはメッセージが届かない．ブロードキャストであればコア 2 にも届いていたはずである．

19.10 キャッシュミスとコヒーレンス

第 14 章では，キャッシュミスを初期参照ミス，容量性ミス，競合性ミスに分けた．無効化は第 4 の種類を加える．

出典 Lesson 19，動画 31–33；H&P §5.3．

定義 19.11 (コヒーレンスミス). **コヒーレンスミス**とは，他のコアがブロックに書いたためにキャッシュ内のコピーが無効化された，そのブロックに対するミスである．

▷ coherence miss

コヒーレンスミスには 2 つの種類がある．

真共有 コアは，他のコアが実際に書いたデータにアクセスする．このような**真共有**では，ミスは必要である．ミスがなければ，コアは古い値を読んでしまうからである．

▷ true sharing

偽共有 他のコアは，たまたま同じブロックにある別のデータに書いた．コアがアクセスするデータは変化しておらず，ミスは，コヒーレンスがアドレス単位ではなくブロック単位で保たれていることだけに起因する．このような**偽共有**はブロックサイズとともに増える．大きなブロックほど無関係なデータを多く含むからである．

▷ false sharing

例 19.12. コア 0 とコア 1 のスレッドが，それぞれ自分のカウンタ c0 と c1 を 1000 回インクリメントし，2 つのスレッドのインクリメントが交互に起こるとする．2

つのカウンタは隣接しており，同じキャッシュブロックにある（図 19.6）．MOESI のもとでは，コア 0 の最初のインクリメントは他のコピーを見つけないので，その書き込みは無効化を必要としない．しかしコア 1 の最初のインクリメント以降は，どのインクリメントも，他方のコアがコピーを保持している状態でブロックに書き，それを無効化する．そのため，コア 0 の 2 回目以降のインクリメントはすべてコヒーレンスミスから始まる．どちらのスレッドも相手のカウンタを読むことはないので，1998 回のミスはすべて偽共有によるものである．カウンタが別々のブロックにあれば，残るのは 2 回の初期参照ミスだけである．

ヒント 異なるスレッドが書くデータは切り離す．x86-64 では `alignas(64)` を使う．Linux の `perf c2c` は，それができていないラインを見つける．

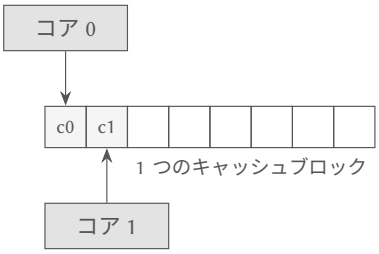


図 19.6: 偽共有: 異なるコアが使うカウンタが同じキャッシュブロックにある．

要点 — 章のまとめ

共有メモリ型マルチプロセッサの専用キャッシュはコヒーレントでなければならない．すなわち，コアは自分の書き込みを見，他のコアの書き込みを短い時間の後に見，ある位置へのすべての書き込みを他のどのコアとも同じ順序で見る．共有バスでは，キャッシュどうしが互いの操作をスヌープし，バスがそれらを順序付ける．書き込み時更新方式はすべての書き込みをすべてのコピーへブロードキャストし，書き込み時無効化方式はブロックごとに 1 回の無効化をブロードキャストして，一般的なアクセスパターンでは後者が勝る．MSI は 3 つの状態で書き込み時無効化方式を実現する．キャッシュ間転送は現在の値を保持するキャッシュから読み出しに応答し，MOSI の O 状態は転送のたびにその値を主記憶へ書くことを避け，MESI の E 状態は 1 つのコアだけが使うデータに対する無効化を避ける．MOESI は両方を組み合わせる．共有バスを持たないマシンは，ブロードキャストを，どのキャッシュが各ブロックを保持しているかを記録し，そのブロックへの要求を順序付けるディレクトリで置き換える．無効化は，ミスの第 4 の種類であるコヒーレンスミスを引き起こし，これは真共有または偽共有に由来する．

第 20 章

同期

講義動画	Lesson 20，動画 1–23
教科書	H&P [1] §5.5，付録 I.4
キーワード	ロック，相互排除，クリティカルセクション，スピンロック，Lamport のパン屋のアルゴリズム，アトミック命令，アトミック交換，テストアンドセット，LL/SC，リンクレジスタ，キャッシュコヒーレンス，バリア同期

第 19 章では，すべてのコアのキャッシュが共有メモリについて 1 つのコヒーレントな見え方を示すようにした．コヒーレントなメモリだけでは，共有メモリのプログラムは正しくならない．複数のスレッドが同じ変数を読んで更新するとき，結果はそれらの命令がどのように交錯するかに依存するからである．この章では，そうした更新を一度に 1 つのスレッドだけに許すロックを導入し，通常のロードとストアで作った素朴なロックがなぜ正しくないのか，プロセッサが代わりにどのようなアトミック命令を提供するのかを示し，ロック変数がキャッシュコヒーレンスとどう相互作用するかを調べ，最後に，スレッドの一群を互いに待ち合わせさせるバリアを扱う．

20.1 同期の必要性

出典 Lesson 20，動画 1–2；H&P §5.5．

メモリを共有するスレッドは，共有データの更新を協調させなければならない．例として，長い文書の中で各文字が何回現れるかを数えるプログラムを考える．2 つのスレッドが文書の半分ずつを処理し，26 個のカウンタからなる共有配列をインクリメントする（例 20.1）．

count は int 26 個で 104 バイトであり，64 バイトのブロック 2 つか 3 つを占める．そのため 2 つのスレッドは，異なる文字を数えているときでも互いのコピーを無効化する（偽共有，第 19 章）．

例 20.1 (文字を数える). 各スレッドは，小文字だけからなる文書の自分の担当分に対して次のループを実行する．

```
shared char doc[N];
shared int  count[26];
for (int i = lo; i < hi; i++) {  /* 自分の担当分 */
  int c = doc[i] - 'a';
  count[c]++;
}
```

文 count[c]++ はソースコード 1 行だが，プロセッサにとってはカウンタのロード，加算，ストアの 3 命令である．

```
LW   R1, 0(R2)       ; R2 = count[c] のアドレス
ADDI R1, R1, 1
SW   R1, 0(R2)
```

各スレッドは自分自身のレジスタを持つが，両方のスレッドが同じ文字に出会ったとき，それぞれの R2 は同じアドレスを保持している．どちらかがインクリメントした値をストアする前に両方のスレッドがカウンタをロードすると，両者は同じ新しい値を計算し，1 回分のインクリメントが失われる（表 20.1）．

同じ位置への 2 つのスレッドからのアクセスのうち少なくとも一方が書き込みであり，その間に同期がなければ，それはデータ競合である（第 21 章）．C11 と C++11 はそうしたプログラムの動作を定義しない．

表 20.1: 2 つのスレッドが，41 を保持する文字 e のカウンタをインクリメントする．2 回のインクリメントの後，カウンタは 43 ではなく 42 を保持している．

手順	スレッド A	スレッド B	A の R1	B の R1	カウンタ
1	LW R1, 0(R2)		41	–	41
2		LW R1, 0(R2)	41	41	41
3	ADDI R1, R1, 1		42	41	41
4		ADDI R1, R1, 1	42	42	41
5	SW R1, 0(R2)		42	42	42
6		SW R1, 0(R2)	42	42	42

インクリメントが失われるかどうかは，スレッド間の相対的なタイミングに依存する．たいていは 2 つのスレッドが異なる文字を処理するか，一方のスレッドが 3 命令を終えてからもう一方が始めるので，カウントは正しい．ときおりカウントが小さく出るが，その誤りはプログラムを再度実行しても再現するとは限らない．したがってロード，加算，ストアは，同じカウンタを更新する他のあらゆるスレッドから見て 1 つの分割不可能な単位として実行されなければならない．これを提供するのが**同期**の役割である．

時分割された 1 つのコアでも，ロードとストアの間にコンテキストスイッチが入れば同じ損失が起こる．

20.2 ロック

同期の最も基本的な機構がロックである．

出典 Lesson 20，動画 3–4；H&P §5.5．

定義 20.2 (ロック). **ロック**とは，2 つの操作を持つ共有オブジェクトである．lock はロックが空くまで待ち，空いたら呼び出したスレッドが保持していると印を付ける．unlock は再び空きの印を付ける．任意の時点で，あるロックを保持するスレッドは高々 1 つである．

▷ lock

カウンタごとに 1 つのロックを置くと，例 20.1 のループは次のようになる．

```
shared lock_t countLock[26];
  ...
  lock(&countLock[c]);
  count[c]++;
  unlock(&countLock[c]);
```

スレッド A がロックを保持している間にスレッド B が lock を呼ぶと，B は A が unlock を呼ぶまで待ち，そのあとで初めてカウンタをロードする（図 20.1）．したがってスレッド B はスレッド A がストアした値を見るので，インクリメントは失われない．

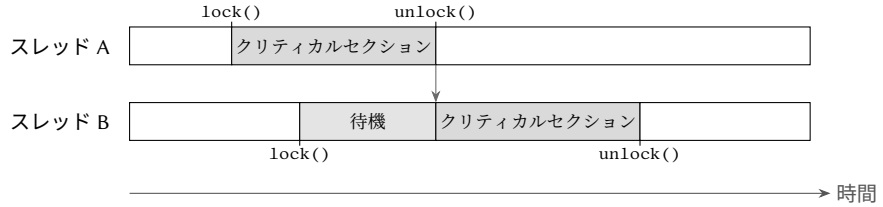


図 20.1: 同じロックを使う 2 つのスレッド．スレッド B は，スレッド A がクリティカルセクションを出るまで lock の中で待つ．

定義 20.3 (相互排除). あるロックで保護されたコードを一度に高々 1 つのスレッドしか実行しないという保証を**相互排除**という．

▷ mutual exclusion

このためロックは**ミューテックス**とも呼ばれる．

定義 20.4 (クリティカルセクション). **クリティカルセクション** とは，相互排除のもとで実行されなければならない命令の並びであり，lock で始まり unlock で終わる．

▷ critical section

ロックが保護するのはコードではなく**データ**である．相互排除が成り立つのは同じロックを使うスレッドの間だけなので，ある共有変数へのアクセスはすべて同一のロックのもとで行わなければならない．異なるロックを保持したまま同じ変数を更新するスレッドどうしは，まったく排他しない．1 つのロックがいくつの変数を保護するかは，性能に影響する選択である．26 個のカウンタすべてに 1 つのロックを使っても正しいが，その場合はどのインクリメントも，別の文字のものであってさえ他のすべてを排他するので，スレッドは時間の多くを互いの待ち合わせに費やす．カウンタごとに 1 つのロックを置けば，異なるカウンタをインクリメントするスレッドは並列に進み，あるスレッドが待つのは別のスレッドが同じカウンタを更新している間だけである．

ヒント ある共有変数へのアクセスにはすべて同じロックを使い，独立な変数には異なるロックを使うこと．

20.3 ロックの実装

ロックは，空いているときは 0，保持されているときは 1 をとる共有の**ロック変数**で表せる．素朴な実装は，この変数が 0 になるまで待ち，そのあとそれを 1 にする．

出典 Lesson 20，動画 5–7；H&P §5.5；[43].

```
void lock(lock_t *l) {
  while (*l == 1)
    ;               /* ロックが空くまで待つ */
  *l = 1;           /* 取得する */
}
void unlock(lock_t *l) {
  *l = 0;
}
```

待っているスレッドはこのループを実行し続け，ロック変数を何度も読む．このように待つロックを**スピンロック**という．

▷ spin lock

この実装は動作しない．機械語では，ロック変数を検査することと，それに値を設定することは別々の命令だからである．

```
lock:  LW   R1, 0(R2)     ; R2 = ロック変数のアドレス
       BNEZ R1, lock      ; 保持されている: もう一度読む
       ADDI R1, R0, 1
       SW   R1, 0(R2)     ; ロック変数 <- 1
```

ロックが空いている間に 2 つのスレッドがともに LW を実行すると，両者とも 0 を読み，ともにループを抜け，ともに 1 をストアし，ともにクリティカルセクションに入ってしまう．これは 20.1 節 で見た更新の消失にほかならない．検査と設定は共有変数の読み出しと更新であり，したがって lock 自身がクリティカルセクションを必要とすることになる—それこそ lock が提供すべきものである．

Dekker，Peterson，Lamport のアルゴリズムが正しく動くのは，ロードとストアがプログラム順に見える場合に限る．そうでなければメモリフェンスが要る（第 21 章）．

逃れ道は 2 つある．

- **ソフトウェア．**ソフトウェアによる解法として **Lamport のパン屋のアルゴリズム** [43] があり，通常のロードとストアだけで相互排除を実現する．これは複雑であり，1 回の取得ごとに多数の命令を実行するためロックが遅くなる—きわめて頻繁に取得されるロックには遅すぎる．
- **ハードウェア．**プロセッサが，ロック変数の読み出しと書き込みを 1 つの分割不可能な手順で行う命令を提供する．

▷ Lamport's bakery algorithm

定義 20.5 (アトミック命令). **アトミック命令**とは，複数の手順—ここではメモリ位置の読み出しとその書き込み—を 1 つの分割不可能な操作として行う命令である．このコアであれ他のどのコアであれ，手順の間にその位置へアクセスできる命令は存在しない．

▷ atomic instruction

ロックを実装するには，アトミックな手順がロック変数の読み出しと書き込みの両方を含まなければならない．読み出した値の検査は，そのあと通常の命令で行ってよい．いったん変数が書かれてしまえば，他のどのスレッドも古い値を読むことはできないからである．

20.4 アトミックな読み書き命令

命令セットは，メモリ位置の読み出しと書き込みを 1 つのアトミックな手順で行う命令を複数提供している．その中で最も単純なものは，レジスタとメモリを入れ替える．

出典 Lesson 20，動画 8–10；H&P §5.5．x86 に LL/SC はない．lock 接頭辞なしでもアトミックな XCHG と，CAS である lock cmpxchg を持つ．Intel SDM 第 2 巻．

定義 20.6 (アトミック交換). **アトミック交換** EXCH R1, 0(R2) は，R1 の内容を R2 のアドレスにあるメモリ位置へアトミックに書き込み，その位置の元の内容を R1 に残す．

▷ atomic exchange

スレッドは，0 が返るまでロック変数に 1 を交換で書き込むことによって，ロックを取得する．

```
        ADDI R1, R0, 1
lock:   EXCH R1, 0(R2)    ; R1 とロック変数を交換
        BNEZ R1, lock     ; 保持されていた：やり直す
        ...               ; クリティカルセクション
unlock: SW   R0, 0(R2)    ; ロック変数 <- 0
```

ロックが保持されていれば，交換は 1 の上に 1 を書いて 1 を返すので，R1 は次の試行に向けて再び 1 になる．ロックが空いていれば，最初に実行された交換が 0 を返して 1 を残し，それ以降の交換はすべて 1 を返す．読み出しと書き込みの間には何も割り込めないので，2 つのスレッドがともに 0 を読むことはありえない．

交換は試行のたびにロック変数に書き込む．他のスレッドがロックを保持している間もそうであり，そこでは書き込みは何も変えない．この代償については 20.6 節 で示す．2 つ目の命令は，読み出した値が検査を通ったときだけ書き込む．

定義 20.7 (テストアンドセット). **テストアンドセット**命令は，メモリ位置を検査し，その値が検査を通った場合にのみそこへ書き込む．ここでの検査は 0 との比較である．TSTSW R1, 0(R2) は，Mem[R2] = 0 ならば Mem[R2] ← R1 と R1 ← 1 とし，そうでなければメモリを変えずに R1 ← 0 とする．

▷ test-and-set

この命令は書き込みを行ったかどうかを R1 で報告するので，R1 が 1 で返ればロックを取得できている．試行に失敗したスレッドは，再試行の前にストアすべき値を R1 に入れ直さなければならない．

注意 規約は一様ではない．TSTSW は成功を 1 で返すが，古典的なテストアンドセットや x86 の BTS は元の値を返すので，そちらでは 0 が成功を意味する．

```
lock:   ADDI R1, R0, 1    ; ロック変数へストアする値
        TSTSW R1, 0(R2)   ; ロック変数が 0 ならストア
        BEQZ R1, lock     ; ストアしなかった：やり直す
        ...               ; クリティカルセクション
unlock: SW   R0, 0(R2)    ; ロック変数 <- 0
```

他のスレッドがロックを保持している間は検査が失敗し，命令は何も書き込まない．

どちらの命令も，ロード/ストア型の命令セットには収まりが悪い．他のすべての命令はデータメモリに高々 1 回しかアクセスしない．ロードは読み，ストアは書き，いずれもパイプラインの単一の MEM ステージで行われる（第 3 章）．交換やテストアンドセットは，1 つの命令の中で同じ位置を読み**かつ**書くので，パイプラインにはメモリアクセスを 2 回行うデータパス–事実上 2 つ目のメモリステージ–が必要になり，さらにプロセッサは，他のコアによるアクセスも例外も，その間に入らないことを保証しなければならない．

20.5 LL/SC

もう 1 つの方法は，アトミックな読み出しと書き込みを 2 つの命令に分け，そのそれぞれが通常のロードやストアと同じくメモリに 1 回だけアクセスするようにすることである．

出典 Lesson 20，動画 11–13；H&P §5.5．

定義 20.8 (LL/SC). 命令対 **LL/SC** は，次の 2 つの命令からなる．

LL R1, 0(R2) は LW と同じく R1 ← Mem[R2] とロードし，そのアドレスをコアの**リンクレジスタ** に記録する．

SC R1, 0(R2) は SW と同じく Mem[R2] ← R1 とストアするが，リンクレジスタが依然としてこのアドレスを保持している場合に限る．その場合は成功を報告するために R1 を 1 にする．そうでなければ何もストアせず，R1 を 0 にする．

▷ load-linked/store-conditional

▷ link register

この対をアトミックにしているのは，リンクレジスタを消去する規則である．メモリ位置への書き込みは–SC であれ通常のストアであれ，このコアによるものであれ他のどのコアによるものであれ–その位置のアドレスを保持しているすべてのリンクレジスタを消去する（図 20.2）．あるコアは，第 19 章のコヒーレンスプロトコルを通じて他のコアの書き込みを知る．リンクされたアドレスを含むブロックの無効化が，リンクレジスタを消去するのである．したがって検出はブ

H&P によれば，LL と SC の間でコンテキストスイッチが起きた場合も SC は失敗する．

ロック単位で働き，同じブロック内の別のアドレスへの書き込みもリンクを消去する．これは再試行の代償を払うだけで，正しさを損なうことはない．

SC が成功するのは，対応する LL 以降に誰もその位置に書き込んでいない場合だけである．とくにそのとき，LL が読んだ値はまだメモリにある．逆は成り立たない．同じ値をもう一度書いても SC は失敗する．LL/SC が検出するのは書き込みであって値の変化ではないからである．LL と SC の間には他の命令が走ってよく，同じ位置にアクセスする他のスレッドの命令も含まれる．それらが干渉すれば SC がそれを検出して失敗し，プログラムは単にやり直す．どちらの命令も単独では通常のロードやストア以上のことをしないにもかかわらず，効果はアトミックな読み出しと書き込みのそれである．

LL/SC で作ったロックは，ロック変数をロードして検査し，それが 0 だった場合にのみ 1 のストアを試みる．

```
lock:   LL   R1, 0(R2)    ; R1 <- ロック変数，リンクを記録
        BNEZ R1, lock     ; 保持されている: やり直す
        ADDI R1, R0, 1    ; R1 <- 1
        SC   R1, 0(R2)    ; リンクが有効なら 1 をストア
        BEQZ R1, lock     ; SC 失敗: 最初からやり直す
        ...               ; クリティカルセクション
unlock: SW   R0, 0(R2)    ; ロック変数 <- 0
```

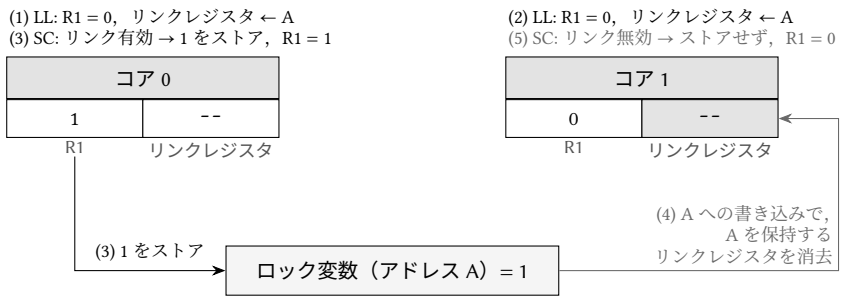


図 20.2: 2 つのコアでの LL/SC．コア 0 のストアが成功すると，コア 1 のリンクレジスタが消去され，コア 1 の SC は失敗する．

例 20.9 (1 つのロックを奪い合う 2 つのコア). コア 0 とコア 1 が上の lock のコードを同時に実行する．アドレス A にあるロック変数は 0 である．表 20.2 は，命令を実行される順に追ったものである．どちらのコアもロックが空いていると判断するが，成功できる SC は一方だけである．そのストアが他方のコアのリンクレジスタを消去するからである．コア 1 は LL に戻り，今度は 1 を読んで待つ．

表 20.2: 例 20.9 のトレース．リンクレジスタの列のダッシュは，そのレジスタがアドレスを保持していないことを表す．

				リンクレジスタ		ロック
手順	コア	命令	効果	コア 0	コア 1	変数
1	0	LL R1, 0(R2)	R1 = 0	A	–	0
2	1	LL R1, 0(R2)	R1 = 0	A	A	0
3	0	BNEZ, ADDI	R1 = 1	A	A	0
4	1	BNEZ, ADDI	R1 = 1	A	A	0
5	0	SC R1, 0(R2)	1 をストア，R1 = 1	–	–	1
6	1	SC R1, 0(R2)	ストアなし，R1 = 0	–	–	1
7	0	BEQZ R1, lock	不成立: ロックを保持	–	–	1
8	1	BEQZ R1, lock	成立: 最初からやり直す	–	–	1
9	1	LL R1, 0(R2)	R1 = 1	–	A	1
10	1	BNEZ R1, lock	成立: ロックは保持済み	–	A	1

注意 Arm で「リンクレジスタ」(X30) といえば戻りアドレスのレジスタである．LL/SC の状態はそこでは排他モニタと呼ばれる．

ヒント LL と SC の間に置いてよいのはレジスタ間の命令だけである．別のメモリアクセスはリンクを消去しうる．

RISC-V が進行を保証するのは制約付きの LR/SC ループだけである．命令数は最大 16 で，その間にロード，ストア，後方分岐を含んではならない．RISC-V ISA マニュアル．

1 回の読み出し-変更-書き込みだけからなるクリティカルセクションは，LL/SC が使えるならロックをまったく必要としない．対を共有変数そのものに適用すればよい．例 20.1 のカウンタであれば，

```
retry:  LL   R1, 0(R2)   ; R2 = count[c] のアドレス
        ADDI R1, R1, 1
        SC   R1, 0(R2)   ; 誰も書いていなければストア
        BEQZ R1, retry   ; 誰かが書いた：読み直して加算
```

がカウンタをアトミックにインクリメントする．LL と SC の間に別のスレッドがカウンタに書き込めば SC が失敗し，新しい値でインクリメントがやり直されるので，更新は失われない．

よくある読み出し- 変更-書き込みには専用のアトミック命令があり，再試行を必要としない．RISC-V の `amoadd.w`，Armv8.1 の LDADD，x86 の `lock xadd` である．

> **要点**
> ロックには，ロック変数の読み出しと書き込みが 1 つのアトミックな手順をなすことが必要である．アトミック交換とテストアンドセットは，その両方を 1 つの変則的な命令で行う．LL/SC は通常のメモリアクセス 2 回を用い，その間に誰かがその位置に書いていればストアを失敗させる．これは，ロックなしで短い読み出し-変更-書き込みをアトミックにもする．

20.6 ロックと性能

ロック変数も他の共有変数と変わらない．ロックを使うどのコアも，それを含むブロックを自分のキャッシュに持ち，第 19 章のキャッシュコヒーレンスプロトコルがこれらのコピーをコヒーレントに保つ．アトミック交換の上に作ったスピンロックは，コヒーレンスとの相性が悪い．

出典 Lesson 20，動画 14–17；H&P §5.5．

EXCH は，ロックが保持されているときでもロック変数に書き込む．1 の上に 1 を書いても値は変わらないが，コヒーレンスプロトコルにとっては他と変わらない書き込みである．MSI プロトコルのもとでは，書き込むコアはそのブロックを M（modified）状態で持たなければならず，他のすべてのコピーは無効化される．1 つのコアがロックを保持している間，EXCH を実行する待機中の各コアは，その直前に試行したコアからブロックを奪う．こうしてほとんどすべての試行がコヒーレンスミスを引き起こす．このコアの前回の試行以降に別の待機中のコアがそのブロックに書き込んでいるからであり，ブロックは再びバス上を運ばれる（表 20.3 上段）．待機中のコアは何も成し遂げないにもかかわらず，そのバストラフィックはエネルギーを消費し，他のすべてのコアのメモリアクセスを遅くする．クリティカルセクションを実行しているロック保持者もその 1 つであり，しかもロックを解放するときにブロックを取り戻さなければならない．

他のコアのキャッシュからブロックを奪うには，少なくとも L3 ヒット分—Skylake で約 44 サイクル—かかる．Intel 最適化マニュアル．

対策は，まず通常のロードでロック変数を検査し，ロックが空いているように見えるときにだけアトミック命令を使うことである．

ヒント スピンしてよいのは，予想される待ち時間がコンテキストスイッチより短い間だけである．それより長く待つならブロックするのがよい．ユーザレベルのロックは短くスピンしてからカーネルで眠る（Linux では `futex` を用いる）．

```
lock:   LW   R1, 0(R2)   ; ロック変数の通常の読み出し
        BNEZ R1, lock    ; 保持されている：読み続ける
        ADDI R1, R0, 1
        EXCH R1, 0(R2)   ; 空きに見える：取得を試みる
        BNEZ R1, lock    ; 他のスレッドが取った：やり直す
```

これで待機中のコアはロック変数を読むだけになるので，どのコアもブロックを S（shared）状態で保持し続け，そのロードはキャッシュヒットになる．待機中のコアは，各自 1 回ミスしたあとは，ロックが保持されている間バストラフィックをまったく発生させない（表 20.3 下段）．書き込みと，それに伴う無効化が起こるのは解放の前後だけである．すなわち unlock のストアと，ロックが空いていると判断したコアによるアトミックな試行であり，成功するのはそのうち 1 つだけである．

こうしてできたロックはテストアンドテストアンドセットと呼ばれる．アトミックな検査の前に通常の検査が置かれることによる．

先に検査するロックについて，誤りやすい点が 2 つある．

- **ロードはフィルタにすぎない．** 相互排除は依然としてアトミック命令によってもたらされる．ロックが空いていると判断した LW と EXCH の間に別のスレッドがロックを取得しうるので，EXCH が返す値はやはり検査しなければならない．LW の結果だけでクリティカルセクションへ分岐するロックは，20.3 節 の壊れたロックそのものである．

表 20.3: コア 0 がロックを保持している間の，4 つのコアのキャッシュにおけるロック変数を含むブロックの MSI 状態（M，S，I: modified，shared，invalid）．上段: コア 1–3 が EXCH でスピンする場合．下段: まず LW で検査する場合．

アクセス	C0	C1	C2	C3	バストラフィック
アトミック交換でスピンする場合					
初め: C0 がロックを保持	M	I	I	I	
C1: EXCH（1 を読む）	I	M	I	I	ミス，ブロックが移動
C2: EXCH（1 を読む）	I	I	M	I	ミス，ブロックが移動
C3: EXCH（1 を読む）	I	I	I	M	ミス，ブロックが移動
C1: EXCH（1 を読む）	I	M	I	I	ミス，ブロックが移動
... 試行ごとに 1 回のミスが続き，					
C0: SW R0（unlock）	M	I	I	I	ミス，ブロックが移動
まず検査する場合					
初め: C0 がロックを保持	M	I	I	I	
C1: LW（1 を読む）	S	S	I	I	ミス
C2: LW（1 を読む）	S	S	S	I	ミス
C3: LW（1 を読む）	S	S	S	S	ミス
C1–C3: LW を繰り返す	S	S	S	S	なし（ヒット）
C0: SW R0（unlock）	M	I	I	I	無効化
C2: LW（0 を読む）	S	I	S	I	ミス
C2: EXCH（0 を読んで取得）	I	I	M	I	無効化

- **他の命令はそれ自体が先に検査している．** LL は読み出しであり，20.5 節のロックは LL がロックの空きを確認したあとでのみ SC を試みる．テストアンドセットは，ロックが空いていると分かったときだけ書き込む．1 を読み続ける待機中のコアは決して書き込まないので，余分なロードは要らない．

注意 実際のハードウェアは，検査が失敗するアトミックな読み出し-変更-書き込みでもブロックを M で取得するのが普通である．その場合は先に通常のロードを置く意味がある．

最後に unlock はアトミック命令を必要としない．0 を通常のストアで書けば十分である．unlock を実行するのはロックを保持しているスレッドだけなので，2 つのスレッドが同じロックを同時に解放することはない．その瞬間にロックを取得しようとしているスレッドは，ストアの前なら 1 を読んで待ち続け，ストアの後なら 0 を読んでロックを奪い合う–どちらも正しい結果である．LL/SC の場合，このストアは，直前にその変数を LL したコアのリンクレジスタも消去する．これらのコアは 1 を読んでいて SC を試みないので，影響を受けない．

> **要点**
> 正しさは，ロック変数の検査と書き込みが 1 つのアトミックな手順であることを要求し，性能は書き込みが稀であることを要求する．待機中のスレッドはロック変数を読むだけにすべきであり，そうすればそのブロックはすべてのキャッシュで共有状態に留まる．

20.7 バリア同期

ロックはスレッドに順番を取らせる．これとは別の形の協調は，スレッドを互いに待ち合わせさせる．

出典 Lesson 20，動画 18–23；H&P 付録 I.4．

定義 20.10 (バリア同期). **バリア同期**では，N 個のスレッドからなる一群が，プログラム中の共通の地点で**バリア**を呼ぶ．バリアに到達したスレッドはそこで待ち，N 個のスレッドのうち最後のものが到着した時点で，全員が先へ進む（図 20.3）．

▷ barrier synchronization

バリアは，段階を追って進む並列プログラムに適する．各スレッドが 1 つの段階の自分の担当部分を計算し，次の段階が全スレッドの結果を必要とする場合である．第 18 章の共有メモリによる総和はその一例である．スレッド 0 が合計を印字してよいのは，すべてのスレッドが自身の部分和を加え終えたあと，すなわちバリアのあとだけである．

同じ機構が `pthread_barrier_wait` であり，OpenMP の並列領域の末尾に置かれる暗黙のバリアであり，`MPI_Barrier` である．

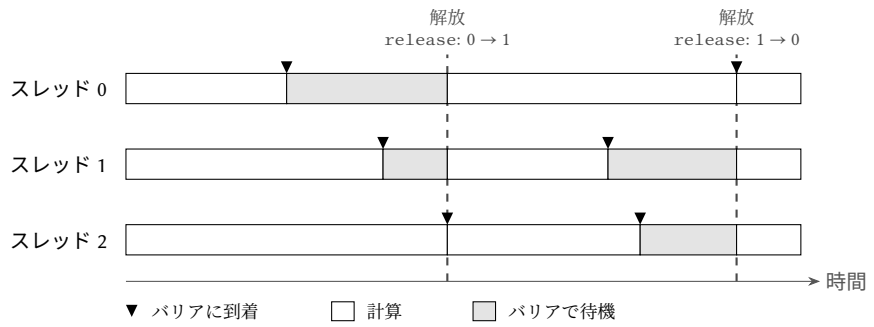


図 20.3: 3 つのスレッドが連続する 2 回のラウンドで使うバリア．各ラウンドは，そのラウンドの最後のスレッドが到着した時点で解放される．

20.7.1 単純な実装

バリアは 2 つの共有変数を使う．到着したスレッドの数を数えるカウンタと，待機中のスレッドがスピンする**解放フラグ**である．[1]

```
shared int    count = 0, release = 0;
shared lock_t countLock;
void barrier(void) {
  lock(&countLock);
  if (count == 0) release = 0;  /* 最初: 閉じる */
  int arrived = ++count;        /* 自分を数える */
  unlock(&countLock);
  if (arrived == N) {           /* 最後: 開く */
    count = 0;
    release = 1;
  } else
    while (release == 0)
      ;                         /* 待つ */
}
```

カウンタにはロックが必要である．インクリメントが失われれば（表 20.1），カウンタは永遠に N に届かない．ラウンドの最初のスレッドが解放フラグをクリアするので，このラウンドのスレッドは待つことになる．最後のスレッドは次のラウンドに備えてカウンタをリセットし，フラグをセットする．これが他のすべてのスレッドの待機ループを終わらせる．

20.7.2 単純な実装が失敗する理由

バリアは通常，ループの各段階で呼ばれるが，このように再利用すると上の実装はデッドロックする．フラグは，他のスレッドがまだそれをスピンして読んでいる最中にセットされるのであり，フラグが再び変わる前にそれを読むことを強制するものは何もない．あるスレッドが割り込みの処理中で，解放に気付くのが遅れたとしよう．他のスレッドはバリアを出て，プログラムの次の段階を計算し，再びバリアを呼ぶ．そのうち最初に到着したスレッドはカウンタが 0 であるのを見て，解放フラグをクリアする．すると遅れたスレッドは 0 を読み，とうに解放されたバリアで待ち続ける．一方，他の $N-1$ 個のスレッドは，決して到着しないスレッドを次のバリアで待つ．N 個のスレッドすべてが永遠に待つことになる．

したがって待機中のスレッドは，フラグを見るのがどれほど遅れても，**自分の**ラウンドが解放されたことを認識できなければならない．

20.7.3 再利用できるバリア

再利用できるバリアは解放フラグを決してクリアせず，1 ラウンドにつき 1 回だけ反転させる．各スレッドは，入口で反転させるスレッド固有の変数 `localSense` を持つ．こうして `localSense` は，現在のラウンドが解放されたときにフラグが取る値になる．最後に到着したスレッドがその値をフラグに書き込む．カウンタ，フラグ，ロックは前の実装と同じ共有変数である．

[1] どちらの変数も N 個のスレッド全員で共有されるので，バリア 1 回ごとに，1 つのブロック上で N 回のロック取得と N 回のコヒーレンスミスが直列化する．スケーラブルなバリアは到着を木状にまとめ，各スレッドが自分専用の変数をスピンして読むようにする [44]．

ここでは`++count` を囲むロックをアトミックなインクリメント 1 回で置き換えることはできない．フラグのクリアが同じクリティカルセクションの中で行われなければならないからである．後述の再利用できるバリアでは，新しい値を返すアトミックなインクリメント 1 回（和を別のレジスタに残す LL/SC のループ）で足りる．

ラウンドに番号を付けるのも同等の一般化であり，センスはラウンド番号の偶奇にあたる．

```
private int localSense = 0;      /* スレッドごとに1つ */
void barrier(void) {
  localSense = !localSense;      /* 自分のセンスを反転 */
  lock(&countLock);
  int arrived = ++count;
  unlock(&countLock);
  if (arrived == N) {            /* 最後: 開く */
    count = 0;
    release = localSense;
  } else
    while (release != localSense)
      ;                          /* 待つ */
}
```

フラグが初期化し直されることはないので，上で遅れたスレッドは，フラグを読むのがどれほど遅くなっても自分のラウンドを解放した値をそこに見出し，バリアを出る．他のスレッドがまだこのバリアで待っている間に次のバリアへ到達したスレッドがいても害はない．そのスレッドはカウンタをインクリメントするだけであり，自身の localSense は今や反対の値を保持しているので，次の反転を待つ．連続するラウンドは 2 つの値を交互に取るので，スレッドが次のラウンドの解放を自分のものと取り違えることはありえない．図 20.3 では，スレッド 2 が最初のラウンドを完了したときに release が 1 になり，スレッド 0 が 2 回目のラウンドを完了したときに再び 0 になる．

注意 最後のスレッドは，解放が他のスレッドから見えるようになる前にカウンタをリセットしなければならない．ストアの順序を入れ替えるプロセッサ（第 21 章）では，2 つの書き込みの間にフェンスが必要である．

共有される状態は N とともに増えない．カウンタ 1 つ，フラグ 1 つ，ロック 1 つと，各スレッドが持つ 1 ビットだけである．

要点 — 章のまとめ

共有データを更新するスレッドは同期しなければならない．さもなければタイミング次第で更新が失われる．ロックはクリティカルセクションに相互排除を与え，1 つの変数へのアクセスはすべて同じロックを使わなければならない．通常のロードとストアで作った素朴なロックは正しくない．その検査と設定自体がクリティカルセクションを必要とするからである．Lamport のパン屋のアルゴリズムのようなソフトウェアによる解法は正しいが遅すぎるので，プロセッサはアトミック命令を提供する．アトミック交換とテストアンドセットは 1 つの命令でメモリを読み書きするが，これはロード/ストア型のパイプラインには収まりが悪い．LL/SC は通常のメモリアクセス 2 回と，その間にその位置が書かれていればストアを失敗させるリンクレジスタとを用い，ロックなしで短い読み出し-変更-書き込みをアトミックにもする．スピンロックは，アトミックな書き込みの前に通常の読み出しでロック変数を検査すべきである．そうすれば待機中のコアはブロックをキャッシュ間で移動させずに共有状態に保つ．unlock は通常のストアである．バリア同期はスレッドの一群を互いに待ち合わせさせる．再利用されるバリアは，遅れたスレッドがまだ見ていないフラグをリセットするのではなく，各ラウンドを固有のフラグ値で，ラウンドごとに反転させながら解放しなければならない．

第 21 章

メモリー貫性

講義動画	Lesson 21，動画 1–14
教科書	H&P [1] §5.6；[45]
キーワード	メモリー貫性，キャッシュコヒーレンス，プログラム順序，一貫性モデル，逐次一貫性，強い一貫性，ロードの再実行，緩和一貫性，MSYNC，データ競合，弱い一貫性，リリース一貫性

第 19 章は，メモリ位置の読み出しがその位置へ最後に書かれた値を返すようにし，第 20 章はロックとバリアを用いて，共有データを更新するスレッドを協調させた．どちらも，異なる位置へのアクセスがどの順序で効力を生じるかには触れておらず，アウトオブオーダプロセッサは当然のようにその順序を変える．この章では，そのような順序変更が，プログラム順どおりのどの実行でも生じえない結果を生みうることを示し，そうした結果を排除する保証である逐次一貫性を定義し，アウトオブオーダプロセッサがそれをどう提供するかを説明する．続いて，順序が重要な箇所をプログラム自身が明示する緩和一貫性と，プログラムがその違いを見分けられるかどうかを決めるデータ競合を扱う．

21.1 コヒーレンスと一貫性

キャッシュコヒーレンス（19.2 節）は，1 つのメモリ位置へのアクセスを制約する．すなわち，どのコアによるその位置の読み出しも，その同じ位置への，どのコアによるものであれ最も新しい書き込みの値を返す．これによってはじめてスレッドはデータを共有できる．しかしコヒーレンスは各位置を個別に扱う．位置 A に書いてから位置 B を読むコアが，他のコアから見てこの順序で 2 つのアクセスを行うかどうかについても，1 つのコアによる A と B への 2 つの書き込みが，そのコアが行った順序で他のコアに見えるようになるかどうかについても，何も述べていない．

出典 Lesson 21，動画 1–2；H&P §5.6．

定義 21.1 (メモリー貫性). 共有メモリシステムの**メモリー貫性**とは，異なる位置へのアクセスも含め，すべてのコアのメモリアクセスがどの順序で効力を生じるかについての，そのシステムの保証である．これは，並列プログラムの読み出しが返しうる値の組合せを決める．

▷ memory consistency

書き込みは，読み出しから見えるようになった時点で効力を生じる．読み出しは，それが返すメモリ内容が存在した時点で効力を生じる．システムが許す順序を厳密に述べたものが**一貫性モデル**である．この章では最も重要なモデルである逐次一貫性とその緩和版を展開し，21.7 節 でさらに他のモデルを概観する．

▷ consistency model

2 つの性質の由来は異なる．コヒーレンスが必要なのは，キャッシュがメモリのコピーを保持するからである．キャッシュのないシステムはどの位置にもコピーを 1 つしか持たず，構成上コヒーレントである．一貫性はキャッシュに依存しない．どの位置にもコピーが 1 つしかなくても，メモリアクセスをプログラムが指定する順序と異なる順序で行うプロセッサは，自分の読み出しが返す値を変えてしまう—そして次節が示すように，現在のプロセッサはまさにそれを行う．したがって，コヒーレントなシステムが一貫性を持つとは限らない．

21.2 一貫性が問題になる理由

異なる位置へのアクセスの順序が問題になるかどうかは，小さなプログラム（例 21.2）で最もよく分かる．

出典 Lesson 21，動画 3–5；H&P §5.6．

例 21.2 (2 つのフラグ). 2 つのコアが同じコードを実行する．コア 0 では R4 がフラグ X のアドレスを，R5 がフラグ Y のアドレスを保持し，コア 1 では X と Y の役割が入れ替わる．どちらのフラグも初めは 0 である．

```
        ADDI R1, R0, 1      ; R1 <- 1
        SW   R1, 0(R4)      ; 自分のフラグ <- 1
        LW   R2, 0(R5)      ; R2 <- 相手のフラグ
        BNEZ R2, skip       ; 相手のコアが宣言済み
        ...                 ; 片方のコアだけが行う作業
skip: ...
```

各コアは，これから作業を行うことをまず自分のフラグで宣言し，相手のコアが宣言していない場合にだけ作業を行う．

各コアが 2 つのメモリアクセスをプログラム順に行えば，作業を行うコアは多くとも 1 つである．コア 0 が Y を 0 と読んだとする．するとコア 0 の Y の読み出しはコア 1 の Y へのストアより前に効力を生じており，したがって，そのストアに続くコア 1 の X の読み出しよりも前である．コア 0 の X へのストアは自分の Y の読み出しに先行するので，コア 1 の X の読み出しにも先行し，その読み出しは 1 を返す．（両方のストアが両方の読み出しより前に効力を生じれば，どちらのコアも作業を行わない．）

アウトオブオーダプロセッサはアクセスをプログラム順に行わない．ストアがメモリに書くのはコミットするときであるのに対し，ロードは，アドレスを計算し，ロードストアキュー（9.3 節）に同じアドレスへの先行するストアがないと分かり次第メモリを読む．例 21.2 のストアとロードは異なるアドレスにアクセスするので，ロードはストアがコミットするよりはるか前にメモリを読みうる．これが両方のコアで起これば（図 21.1），両方のロードが 0 を読み，両方のコアが作業を行う．

2 つのフラグのプログラムは，通常のロードとストアを使う Dekker や Peterson の相互排除アルゴリズム（第 20 章）の核心である．

その距離の上限はリオーダバッファで決まる．Skylake コアでは 224 エントリである（第 8 章）．Agner Fog，マイクロアーキテクチャマニュアル．

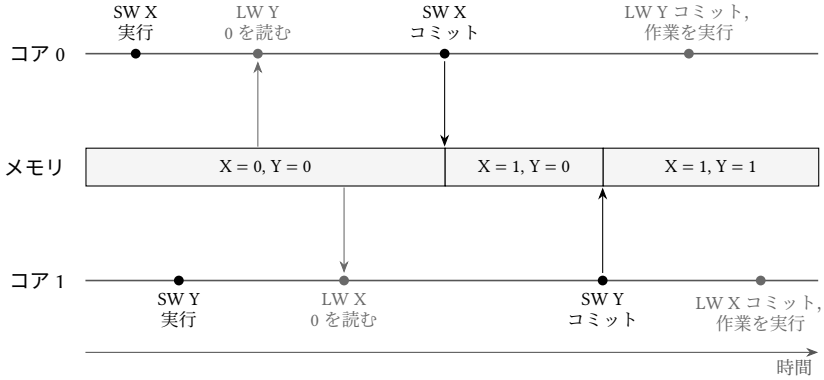


図 21.1: 2 つのアウトオブオーダコア上での 例 21.2．各コアのロードは自分のストアがコミットする前にメモリを読み，両方のロードが 0 を読んで，両方のコアが作業を行う．

この実行ではどの読み出しもコヒーレントである．各ロードが相手のフラグを読む時点で，そのフラグは実際に 0 を保持しており，ストアはコミット時に，プログラム順に効力を生じる．それでもこの結果は，プログラム順どおりのどの実行でも生じえない．アウトオブオーダのコアが変えたのは，異なる位置へのアクセスの順序である．すなわち，各コアによる相手のフラグの読み出しが，自分のストアより前に効力を生じたのである．

注意 ここではどのキャッシュも古い値を保持していないので，どのコヒーレンスプロトコルもこの結果を防げない．原因は古いデータではなく順序変更である．

21.2.1 正しいプログラムにおけるロードの順序変更

2 つのフラグのプログラムは作為的に見えるかもしれないが，同じ種類の順序変更は，よくある，それ自体は正しいパターンを壊す．その 1 つが**データ準備完了フラグ**である．書き込み側スレッドは新しいデータをストアしてからフラグを立て，読み出し側スレッドはフラグが立つまで待ってからデータを使う．R4 がフラグのアドレスを，R5 がデータのアドレスを保持するとき，書き込み側は

```
        SW   R6, 0(R5)      ; data  <- 新しい値
        SW   R7, 0(R4)      ; ready <- R7 (= 1)
```

を，読み出し側は

```
poll: LW   R1, 0(R4)     ; R1 <- ready
      BEQZ R1, poll      ; 未設定なら再度ポーリング
      LW   R2, 0(R5)     ; R2 <- data
```

を実行する．プログラム順では，読み出し側のデータのロードは，フラグが立っているのを見たフラグのロードに続き，そのロードは書き込み側のフラグへのストアに続き，そのストアはさらに書き込み側のデータのストアに続く．したがって読み出し側は常に新しいデータを得る．分岐予測（第 4 章）はこの保証を失わせる．読み出し側のコアはループを抜けると予測し，データのロードを投機的に，場合によっては同じ反復のフラグのロードより前に実行する．表 21.1 は，データが 3 を保持し，書き込み側が 8 をストアする，そのような実行を追ったものである．フラグのロードは書き込み側がフラグを立てた後に実行されるので，予測は結果として正しく，データのロードは古い値のままコミットする．

表 21.1: アウトオブオーダの読み出し側コア（コア 0）と書き込み側コア（コア 1）におけるデータ準備完了フラグ．各手順の後のメモリ内容．

手順	コア	事象	ready	data
1	0	`LW data` が予測経路で実行される: R2 = 3	0	3
2	1	`SW data` がコミットする	0	8
3	1	`SW ready` がコミットする	1	8
4	0	`LW ready` が実行される: R1 = 1	1	8
5	0	`BEQZ` が解決する: 予測どおり不成立	1	8
6	0	3 つの命令が R2 = 3 のままコミットする	1	8

あるスレッドが別のスレッドを生成し，そのスレッドが終了するまで待ってから，それが生成した結果を使う場合にも同じことが起こる．待機が終わるのは，スレッドの終了時にオペレーティングシステムが書き込む状態をロードし，それが終了を示した時点である．プログラム順でそれに続く結果のロードは，それより前に，すなわち他方のスレッドが結果を書く前に実行されうる．

スレッドライブラリはこれを隠す．POSIX は `pthread_join` に，終了するスレッドの書き込みを `pthread_join` を呼ぶスレッドのアクセスより前に順序付けることを要求している．

どちらのパターンでも，すべてのキャッシュはコヒーレントであり，アクセスがプログラム順に効力を生じる限りプログラムは正しい．そもそもこのようなプログラムを書けるようにするには，プログラマは異なる位置へのアクセスの順序についての保証—コヒーレンスを超えた保証—を必要とする．

> **要点**
> コヒーレンスは各位置へのアクセスを別々に順序付ける．アウトオブオーダ実行と分岐予測は，他の位置への先行するアクセスより前にロードが効力を生じることを許し，その結果，プログラム順どおりのどの実行でも生じえない結果が現れうる．

21.3 逐次一貫性

最も直観的な保証は，コアが順番に交代したかのようにメモリが振る舞うことを要求する．

出典 Lesson 21，動画 6；H&P §5.6；[46]．

定義 21.3 (逐次一貫性). 共有メモリシステムは，どの実行の結果も，各コアのメモリアクセスがプログラム順に行われ，異なるコアのアクセスが何らかの任意の順序でインターリーブされた場合と同じであるとき，**逐次一貫性**を提供するという．

▷ sequential consistency

Lamport [46] に遡るこの定義には 2 つの部分がある．1 つのコアの内部では順序は固定されている．すなわち，アクセスはプログラムが指定する順序で現れる．コアをまたぐインターリーブは任意であり，ある実行がどれに対応するかはコアのタイミングによるので，正しいプログラムはあらゆるインターリーブに対して正しい結果を生まなければならない．「かのように」という言葉が重要である．どのプログラムも違いを見分けられない限り，ハードウェアが実際にそのような順序でアクセスを行う必要はない．

例 21.4 (2 つのフラグのプログラムのインターリーブ). 例 21.2 では，コア 0 が X

をストアし（w_0），次に Y を読む（r_0）．コア 1 は Y をストアし（w_1），次に X を読む（r_1）．各コアの順序を保つと，6 通りのインターリーブが残る（表 21.2）．そのいずれにおいても少なくとも一方の読み出しが 1 を返すので，逐次一貫性を持つシステムが両方のコアに作業を行わせることはない．すなわち図 21.1 の実行は逐次一貫性を持たない．

ヒント 両コアのプログラム順を保つインターリーブの数は，アクセスが m 個と n 個なら $\binom{m+n}{m}$，ここでは $\binom{4}{2} = 6$ である．

表 21.2: 各コアのプログラム順を保つ，例 21.4 のアクセスの 6 通りのインターリーブ．

	順序	r_0（Y）	r_1（X）	作業を行うコア
1	$w_0\ r_0\ w_1\ r_1$	0	1	コア 0
2	$w_0\ w_1\ r_0\ r_1$	1	1	なし
3	$w_0\ w_1\ r_1\ r_0$	1	1	なし
4	$w_1\ w_0\ r_0\ r_1$	1	1	なし
5	$w_1\ w_0\ r_1\ r_0$	1	1	なし
6	$w_1\ r_1\ w_0\ r_0$	1	0	コア 1

注 21.5 (強い一貫性)．逐次一貫性が要求するのは，**何らかの**インターリーブが結果を説明することだけである．そのインターリーブはアクセスが実際に起こる実時間と一致する必要がないので，他のコアがその位置への書き込みをすでに完了していても，読み出しが古い値を返すことがある．順序が実時間にも従わなければならない–他のアクセスの開始前に完了したアクセスが先になり，どの読み出しも最も新しく書かれた値を返す–システムは**強い一貫性**を提供する．Mosberger [45] はこれをアトミック一貫性と呼んでいる．

▷ strong consistency

21.4 逐次一貫性の実現

21.4.1 単純な実現

逐次一貫性を提供する最も単純な方法は，その定義から「かのように」を取り除くことである．各コアはメモリアクセスを一度に 1 つずつ，プログラム順に行う．すなわち，先行するアクセスがすべて完了してはじめて次のアクセスを開始する．コヒーレンスによって，各アクセスは効力を生じると同時にすべてのコアから見えるようになるので，すべてのコアのアクセスが効力を生じる順序そのものが，定義の要求する種類のインターリーブになる．

出典 Lesson 21，動画 7–8；H&P §5.6．

この実現は正しいが非常に遅い．アウトオブオーダコアはメモリにアクセスしない命令の順序を依然として変えられるが，進行中のメモリアクセスは一度に 1 つしか持てない．ロードは先行するストアがすべてコミットするまで実行できず，ロードした値を使う命令はすべてそれとともに待つ．しかしメモリアクセスこそ，アウトオブオーダ実行が最も重ね合わせる必要のある遅い操作である．キャッシュミスは多くのサイクルを要し，マルチプロセッサではさらに，アクセスがバスとコヒーレンスプロトコル（第 19 章）を待たなければならないこともある．メモリアクセスが一度に 1 つに制限されると，コアはアウトオブオーダ実行の利点の大半を失う．

直列化すると各レイテンシを丸ごと支払う．Skylake 級のコアで L1 ヒット 4 サイクル，L2 は 12，L3 は 44，メモリは 60–100 ns である（第 14 章）．

21.4.2 ロードを安全に順序変更する

より良い実現は，コアにロードをアウトオブオーダで実行させ，それが逐次一貫性を破りうる場合を検出して修復する．これを可能にする観察が 2 つある．

- **注意が必要なのはロードだけである．**ストアはコミット時にプログラム順にメモリに書く（9.2 節）ので，すでにプログラム順に効力を生じている．同じコアの先行するストアの位置を読むロードは，そのストアの値をロードストアキューから得る（第 9 章）．
- **ROB がプログラム順を記録している．**実行の順序がどうであれ，ROB は命令をプログラム順に保持しており，早く実行されたロードは，コミットする時点で効力を生じたとみなせる．読んだ値は，その間に他のコアがその位置に書いていない限り，その時点においても正しい．

Intel のカウンタはこれをメモリ順序のマシンクリア（`MACHINE_CLEARS.MEMORY_ORDERING`）と呼ぶ．1 回ごとにパイプラインが空になり，少なくとも分岐予測ミス 1 回分，Skylake で 16–20 サイクルを要する（第 4 章）．

そこでコアは，実行済みでまだコミットしていないすべてのロードの位置について，他のコアのコヒーレンストラフィックを監視する．書き込み時無効化方式の

もとでは，他のコアによる書き込みは，その位置を含むブロックの無効化として届く（19.5 節）．そのような書き込みが観測されれば，ロードは古い値を読んでいたかもしれない．そのロードとそれ以降のすべての命令は取り消され，分岐予測ミスの後とまったく同じように再実行される（8.5 節）．ロードはまだコミットしていないので，これは常に可能である．ロードがコミットしてしまえば，それはプログラム順に効力を生じており，その位置をもはや監視する必要はない．

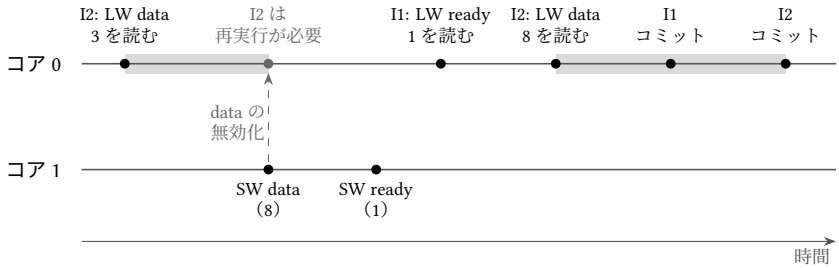


図 21.2: 順序変更されたロードの再実行．プログラム順では，コア 0 は I1: `LW ready`，次に I2: `LW data` を実行する．I2 が実行済みでコミットしていない間（網掛け）にコア 1 がデータを書き，その無効化によって I2 は再実行される．

図 21.2 は，これを 表 21.1 のデータ準備完了フラグに適用したものである．コア 0 はデータのロード I2 をフラグのロード I1 より前に実行し，古い値 3 を読む．その後コア 1 がデータに 8 をストアする．無効化は I2 が未コミットのうちにコア 0 へ届くので，I2 は再実行され，今度は 8 を読む．コア 1 がフラグを立てた後に実行された I1 は 1 を読み，両方のロードは，プログラム順に実行されたときに返したはずの値のままコミットする．

再実行が必要なのは，あるロードの実行とコミットの間に，他のコアがそのロードが読む位置へ書いたときだけである．これは大半のコードでは稀だが，データ準備完了フラグの読み出し側のように，他のコアが書く位置をポーリングするループでは頻繁に起こる．無効化はブロック全体を指すので，同じブロック内の別の位置への書き込みも再実行を引き起こす．これは時間を要するが，誤った結果を生むことは決してない．

ブロックは x86 でも多くの Arm コアでも 64 バイトなので，無関係な変数が同じブロックを共有する．偽共有（第 19 章）はコヒーレンスミスだけでなく再実行も引き起こす．

> **要点**
> 逐次一貫性はメモリアクセスの順序変更を禁じるのではなく，プログラムが観測できる順序変更を禁じる．アウトオブオーダコアは，コミットより前に他のコアがその位置へ書いたロードをすべて再実行するならば，ロードを早く実行してよい．

21.5 緩和一貫性

逐次一貫性は，プログラムがその順序に依存するかどうかにかかわらず，コアが行うアクセスのあらゆる対を制約する．あるコアがプログラム順に行う 2 つのアクセス，1 つ目が位置 A への，2 つ目が位置 B へのものについては，WR A → WR B，WR A → RD B，RD A → WR B，RD A → RD B の 4 種類の順序付けがある．いずれも，すべてのコアから見て 1 つ目のアクセスが 2 つ目より前に効力を生じることを要求する（表 21.3）．逐次一貫性は 4 つすべてを常に守る．

出典 Lesson 21，動画 9–11；H&P §5.6．

定義 21.6 (緩和一貫性). **緩和一貫性**を持つシステムは，通常のメモリアクセスについてこれらの順序付けのいくつかを常には守らず，正しさが順序に依存する箇所でプログラムが順序を強制する手段を提供する．

x86 が緩和するのは WR → RD だけである．Arm，POWER，RISC-V は 4 つすべてを緩和する．Intel SDM 第 3A 巻「Memory Ordering」．

▷ relaxed consistency

読み出しで終わる 2 つの順序付け，WR → RD と RD → RD が，緩和の自然な候補である．ロードが，同じコアの他の位置への先行するアクセスすべてより前に効力を生じてよいならば，21.4 節 のコアは，コヒーレンストラフィックを監視する必要も，他のコアの書き込みのためにロードを再実行する必要もない．どのロードも，図 21.1 や 表 21.1 のように，準備ができ次第実行するだけである．RD → RD だけを緩和しても，ロードを監視しなければならない期間が短くなるにすぎない．ロードは依然として，自分のコアの先行するストアすべての後に効力を

表 21.3: あるコアの 2 つのアクセス，最初が A へ，次が B へのものの間の 4 つの順序付けと，それらに違反するコアが起こしうること．

順序付け	違反	例
WR A → WR B	A の新しい値より前に B の新しい値が見える	データがストアされる前にフラグが立つ
WR A → RD B	A への書き込みが見えるようになる前に B が読まれる	2 つのフラグ（例 21.2）
RD A → WR B	A が読まれる前に B の新しい値が見える	要求が読まれる前に応答が見える
RD A → RD B	A より前に B が読まれる	データ準備完了フラグ（表 21.1）

生じなければならないからである．

21.5.1 同期のためのアクセス

2 つの読み出しさえ順序変更されうるなら，プログラムは順序が重要な箇所で特定の順序を得る手段を必要とする．そこで緩和一貫性を持つシステムは，2 種類の操作を区別する．通常のロードとストアは，モデルが許す範囲で順序変更されてよい．順序が重要な箇所でプログラムが用いる特別な操作は，決して順序変更されない．本章ではそのような命令を **MSYNC** と呼ぶ．プログラム順で MSYNC に先行するすべてのメモリアクセスは，それに続くどのアクセスよりも前に効力を生じる（図 21.3）．2 つの MSYNC 命令の間では，アクセスは依然として順序変更されうる．

x86 にはそのような命令 **MFENCE** がある．その前のアクセスは，後に続くどのアクセスよりも前に大域的に可視となる．

注意 MSYNC は完全なフェンスである．実際の命令セットには部分的なフェンス—x86 の SFENCE，Arm の DMB ISHST—もあり，4 つの対の一部だけを順序付ける．

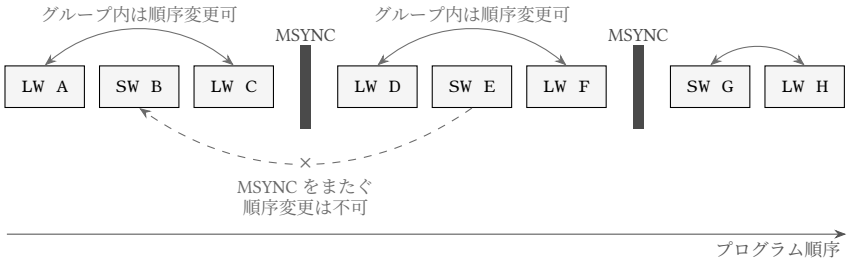


図 21.3: MSYNC は，コアのアクセスをプログラム順にグループへ分ける．1 つのグループ内の異なる位置へのアクセスは，モデルが許す範囲で異なる順序で効力を生じてよい．前のグループのアクセスより前に効力を生じるアクセスはない．

両側に MSYNC を 1 つずつ置けば，データ準備完了フラグは緩和一貫性のもとでも正しい．書き込み側と読み出し側は次を実行する．

```
; 書き込み側
      SW   R6, 0(R5)     ; data  <- 新しい値
      MSYNC               ; data が可視になった
      SW   R7, 0(R4)     ; ready <- 1
; 読み出し側
poll: LW   R1, 0(R4)     ; R1 <- ready
      BEQZ R1, poll       ; 未設定なら再度ポーリング
      MSYNC               ; ready のロードが完了した
      LW   R2, 0(R5)     ; R2 <- data
```

読み出し側では，MSYNC によって，フラグが立っているのを見たフラグのロードがデータのロードより前に効力を生じる．書き込み側では，データのストアがフラグへのストアより前に効力を生じるようにする．WR → WR が緩和されていると，MSYNC がなければフラグへのストアの方が先に見えてしまいかねないからである．読み出し側のフラグのロードが 1 を返したので，そのロードが効力を生じた時点でフラグは，そしてそれより前にデータは，ストアされていた．データのロードはさらにその後に来る．読み出しで終わる順序付けだけが緩和されるなら，書き込み側の 2 つのストアはすでにプログラム順に効力を生じており，読み出し側の MSYNC だけで十分である．書き込み側の MSYNC が必要になるのは，WR → WR も緩和されうる場合である．

フラグは同期の単純な形であり，2 つの MSYNC の配置は一般的な規則に従う．すなわち，先へ進む権利を獲得する操作の後と，それを解放する操作の前に MSYNC を置く．ロック（第 20 章）を使うスレッドは，lock の直後に MSYNC を実行して，自分のクリティカルセクションのアクセスが獲得の後に効力を生じ，それ以前のロック保持者の書き込みをすべて見るようにし，unlock の直前にも実行して，次のスレッドがロックを獲得する時点で自分のクリティカルセクションの書き込みがすべて効力を生じているようにする．このような命令は通常，あらゆるプログラムの中ではなく，同期ライブラリのロックやバリアのルーチンの中に置かれる．

ヒント 2 つのフェンスがクリティカルセクションの内側を向いていると覚えるとよい．どちらの端からも外へ出られない．

Arm はフェンスをアクセス自体に畳み込む．LDAR（獲得）と STLR（解放）は一方向だけを順序付ける．RISC-V には .aq/.rl ビットがある．

21.6 データ競合と一貫性

緩和一貫性がプログラムの結果を変えうるかどうかは，そのプログラムが共有データにどうアクセスするかによる．

出典 Lesson 21，動画 12；H&P §5.6；[47]．

定義 21.7 (データ競合). 異なるコアが行う同じメモリ位置への 2 つのアクセスは，少なくとも一方が書き込みであり—RD → WR，WR → RD または WR → WR—かつ同期によって順序付けられていない，すなわちどちらが先に効力を生じるかを定める同期操作が存在しないとき，**データ競合**をなす．

▷ data race

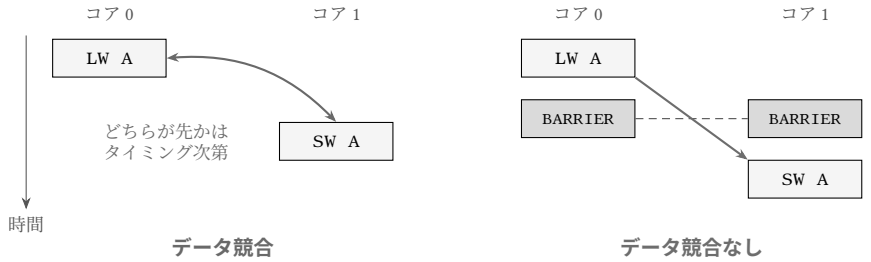


図 21.4: 左: A のロードとストアを順序付けるものが何もない，データ競合．右: ロードの後とストアの前に置かれたバリアによって，ロードが先に効力を生じる．

図 21.4 の左では，コア 0 の A のロードとコア 1 の A へのストアの間には何もない．どちらが先に効力を生じるかはコアのタイミングによるので，ロードはそれに応じて古い値か新しい値を返す．右では，コア 0 は自分のロードの後にバリア（20.7 節）に到達し，コア 1 は自分のストアの前に同じバリアに到達する．すべてのスレッドが到達するまでどのスレッドもバリアを離れないので，ロードは常にストアより前に効力を生じ，データ競合は存在しない．同じロックで保護されたアクセスも順序付けられる．一方のクリティカルセクションが unlock で終わってから，他方が lock で始まるからである．

競合検出器（ThreadSanitizer，Helgrind）が探すのはまさにこれ，すなわちロックもバリアもフェンスも順序付けていない 2 つの衝突するアクセスである．

データ競合が重要なのは，次の性質による [47].

- **データ競合のないプログラム**は，その同期を認識するどの一貫性モデルのもとでも，逐次一貫性のもとでの結果と同じ結果を生む．同期を認識するモデルとは，同期操作自体が逐次一貫性を持ち，MSYNC がそうであるように，各同期操作が同じコアの先行するアクセスと後続のアクセスを隔てるものである．[1]2 つの同期操作の間では，あるコアが使う位置に他のコアが衝突するアクセスを行わないので，そのコアの異なる位置へのアクセスや同じ位置の読み出しの順序を変えても，読まれる値はどれも変わらない．順序が変われば値が変わりうるアクセスの対はすべて同期によって順序付けられており，そのようなモデルはそれを順序どおりに保つからである．
- **データ競合のあるプログラム**には，そのような保証はない．緩和一貫性を持つシステムでは，どのインターリーブでも説明できない結果も含め，何でも起こりうる．

したがってプログラムは，まず逐次一貫性のもとで開発しデバッグできる．そこでは，データ競合のあるプログラムでも，何らかのインターリーブで説明できる結果を生むからである．同期が競合を取り除いた後は，速度のために緩和一貫性のもとで実行すればよい．

この章の例はいずれもデータ競合を含む．例 21.2 では，各コアが，相手のコ

[1] プログラミング言語はこれを契約にした．C++11 と Java は，データ競合のないプログラムが逐次一貫性のもとと同じように振る舞うことを保証し，C++ ではデータ競合があるとプログラム全体が未定義になる．これらのモデルが認識する同期は std::atomic—既定は memory_order_seq_cst で，より安価な解放/獲得（memory_order_release/acquire）も選べる—と，その上に築かれたロックである [48]．

アが同期なしに書くフラグを読む．表 21.1 のデータ準備完了フラグは，メモリシステムが認識しない形の同期である．逐次一貫性のもとでは，このフラグが読み出し側のデータのロードを書き込み側のストアより後に順序付けるが，緩和一貫性を持つシステムはそれを知らない．21.5 節 の MSYNC はその順序付けをシステムから見えるようにするので，データのロードはもはやストアと競合しない．フラグをポーリングするロード自体は同期されないままだが，フラグは 0 から 1 へしか変化しないので，それらは読み出し側を遅らせうるだけである．表 20.1 の失われたインクリメントもデータ競合である．逐次一貫性のもとでは，起こりうる結果は少なくとも 2 つのスレッドのインターリーブが与えるものに限られる．緩和一貫性のもとでは，その限界すら消える．

注意 逐次一貫性を持つシステムで動作するプログラムでも，データ競合を含んでいれば緩和一貫性のもとでは失敗しうる．

> **要点**
> 同期をメモリシステムが認識する，データ競合のないプログラムは，緩和一貫性と逐次一貫性を見分けられない．したがって，一貫性を緩和しても，正しく同期されたプログラムは何も失わない．データ競合のあるプログラムは何でも観測しうる．

21.7 一貫性モデル

逐次一貫性と 21.5 節 の緩和一貫性は，広い範囲にわたる一貫性モデルの 2 つの点にすぎない．モデルは，どの順序付けを緩和するか，そして同期を他のアクセスと区別するかどうか，また区別するならどう区別するかで異なる．すべてのアクセスを同じに扱うモデルもある．強い方から弱い方へ順に挙げる．

出典 Lesson 21，動画 13–14；[45]．

強い一貫性 どの読み出しも，実時間で最も新しく書かれた値を返す（21.3 節）．

逐次一貫性 各コアのプログラム順を保つ何らかのインターリーブが，すべての結果を説明する．

因果一貫性 因果一貫性のもとでは，因果的に関係する書き込み–同じコアの 2 つの書き込み，または，直接あるいは間接に先行する書き込みの結果を読んだコアが行う書き込み–を，すべてのコアが同じ順序で見なければならない．並行な書き込みは，コアによって異なる順序で見えてよい．

▷ causal consistency

プロセッサー貫性 プロセッサー貫性のもとでは，あるコアの書き込みは，すべてのコアがそのコアの行った順序で見るが，異なるコアの書き込みを異なるコアが異なる順序で見ることがあり，また，あるコアの読み出しが，自分の他の位置への先行する書き込みより前に効力を生じることもある．

▷ processor consistency

因果一貫性もプロセッサ一貫性も 図 21.1 の実行を禁じない．2 つの書き込みは並行なので，各コアは相手のコアの書き込みを自分の読み出しの後にはじめて見てよいからである．

他のモデルは同期を区別しており，それらが認識する同期を用いるデータ競合のないプログラムは，そのもとで逐次一貫性のもとと同じように振る舞う．

弱い一貫性 弱い一貫性のもとでは，ロックのような同期変数へのアクセスは逐次一貫性を持ち，MSYNC のように振る舞う．すなわち，先行するアクセスはすべてそれより前に効力を生じ，後続のアクセスはどれもそうならない．その他のアクセスは，それらの間で自由に順序変更されてよい．

▷ weak consistency

リリース一貫性 リリース一貫性のもとでは，同期は，lock と unlock のように，獲得と解放に分けられる．獲得は後続のアクセスがそれより前に効力を生じることを妨げ，解放は先行するアクセスがすべて効力を生じるまで待つ．そのため通常のアクセスがクリティカルセクションの境界を越えて順序変更されることはない– 21.5 節 の MSYNC の配置である．[2]

▷ release consistency

リリース一貫性は，1990 年にスタンフォードの DASH マルチプロセッサのために定義された．獲得と解放の分離をハードウェアの概念にした機械である．Gharachorloo et al., ISCA 1990．

遅延リリース一貫性 遅延リリース一貫性は，リリース一貫性を遅延的に実現したもので，もとはソフトウェアで構築された分散共有メモリのためのものである．クリティカルセクションで行われた更新は，解放のたびにではなく，他のノードが次にそのロックを獲得するときにはじめて，そのノードへ伝えられる．

[2] 遅延リリース一貫性 [49] は，ワークステーションのネットワーク上のソフトウェア分散共有メモリのために設計された．そこでは更新の伝播はメッセージの送信を意味する．TreadMarks システムは，ロックを獲得したノードがページに触れてはじめて，そのページへの変更（差分）を取り寄せる．

スコープ一貫性 スコープ一貫性のもとでは，各ロックがスコープを定める．クリティカルセクションに入るスレッドは，同じロックの以前のクリティカルセクションで行われた更新を見るが，他のロックのもとで行われた更新を必ずしも見ない [50]．

この一覧は完全ではなく，新しいモデルが提案され続けている．それらに共通するのは，この章の枠組みである．すなわち，システムはモデルが順序付けないアクセスを順序変更してよく，特定の順序を必要とするプログラムは同期によってそれを得る．

> **要点 — 章のまとめ**
> コヒーレンスは 1 つの位置へのアクセスを順序付ける．メモリ一貫性が関わるのは，アウトオブオーダ実行と分岐予測が変えてしまう，異なる位置へのアクセスの順序である．逐次一貫性は，どの実行も，各コアのアクセスがプログラム順に行われ，異なるコアのアクセスがインターリーブされたかのように見えることを要求する．アクセスを一度に 1 つずつ行えばこれを提供できるが，アウトオブオーダ実行を台無しにする．その代わりに，コアはロードを早く実行し，コミットより前に他のコアがその位置へ書いたロードを再実行すればよい．緩和一貫性はアクセス間の順序付けを常には保たず，獲得の後と解放の前に置く MSYNC のような同期操作をプログラムに与える．データ競合—同期によって順序付けられていない，異なるコア上の衝突するアクセス—のないプログラムは，その同期を認識するどのモデルのもとでも，逐次一貫性のもとと同じように振る舞う．データ競合のあるプログラムは何でも観測しうる．弱い一貫性，リリース一貫性，遅延リリース一貫性，スコープ一貫性は同期の認識の仕方が異なり，因果一貫性とプロセッサ一貫性は，すべてのコアが一致しなければならない書き込みの順序が異なる．

第 22 章

メニーコア

講義動画	Lesson 22，動画 1–19
教科書	H&P [1] §1.5，§5.4，付録 F
キーワード	メニーコアプロセッサ，コヒーレンストラフィック，ネットワークオンチップ，メッシュ，トーラス，タイル，オフチップトラフィック，分散最終レベルキャッシュ，オンチップディレクトリ，部分ディレクトリ，電力バジェット，ターボモード，論理プロセッサ

第 18 章から第 21 章では，コアがふつうバスと最終レベルキャッシュを共有し，バス上のスヌーピングで（共通のバスを持たないマシンではディレクトリで）キャッシュをコヒーレントに保ち，ロックで協調し，共有データについては一貫性モデルに頼るマルチプロセッサを構築してきた．しかし世代が進むたびに 1 つのチップにはより多くのコアが収まるようになり，4 個や 8 個のコアをうまく扱える機構も，数十個から数百個のコアを持つチップでは破綻する．この章では，そのようなチップが直面する課題—コヒーレンストラフィック，オフチップトラフィック，コヒーレンスディレクトリの大きさ，共有された電力バジェット，そしてオペレーティングシステムによるスレッドの配置—と，それぞれの解決策を調べる．

22.1 コヒーレンストラフィック

メニーコアプロセッサとは，1 つのチップに多数のコアを載せたマルチコアプロセッサ，すなわち 2 個から 8 個ではなく数十個から数百個のコアを持つものである．このようなチップの第 1 の課題はキャッシュコヒーレンス（第 19 章）に関わる．他のコアがキャッシュしている共有データへの書き込みはすべて**コヒーレンストラフィック**を生む．すなわち，他のキャッシュへ送られる無効化と，それらのコアが再びそのブロックにアクセスするときのコヒーレンスミスである．コアが増えればチップ全体では毎秒の書き込みが増えるので，コヒーレンスメッセージの発生率もコア数とともに増える．共有バスでは，このトラフィックがまもなくボトルネックになる．

- バスは一度に 1 つの転送しか運べないので，コアは互いのコヒーレンスメッセージとキャッシュミスを待つことになる．
- より多くのコアをつなぐバスは長くなり，接続されるデバイスも増えるので，1 回の転送自体も遅くなる．

したがってメニーコアチップには 2 つのものが必要である．コア数とともに容量が増えるオンチップの相互接続網と，1 本のバス上のすべての操作をすべてのキャッシュが観測することに依存しないコヒーレンス機構である．後者がディレクトリ方式（19.9 節）であり，これは各メッセージをすべてのコアへブロードキャストするのではなく，そのブロックを保持するキャッシュだけへ届ける．

▷ many-core processor

出典 Lesson 22，動画 1–2；H&P 第 5 章．

「数百個」はすでに現実である．AMD の EPYC 9965 は 500 W のバジェットの中に 1 ソケットあたり 192 コアを収めており，1 コアあたり約 2.6 W である．AMD 製品ページ．

Intel の最後のフロントサイドバスは 1600 MT/s，1 本あたり 12.8 GB/s であった．サーバ用チップセットはすでにソケットごとに専用のバスを与えており，2008 年に QuickPath がバスを 1 対 1 のリンクで置き換えた（Intel チップセットのデータシート）．

22.2 ネットワークオンチップ

共有バスはネットワークに置き換えられる．

定義 22.1 (ネットワークオンチップ). **ネットワークオンチップ**は，チップのコアを，コアごとに 1 つ置かれたルータの間の 1 対 1 のリンクで接続する．メッセージは宛先に着くまでルータからルータへと渡り，異なるリンク上のメッセージは同時に進む．

出典 Lesson 22，動画 3–4；H&P 付録 F；[51]．

▷ network on chip

メッシュトポロジでは，ルータが 2 次元の格子をなし，各ルータは東西南北の隣接ルータとリンクで結ばれる（図 22.1b）．どのコアも，いくつかのリンク，すなわち**ホップ**からなる経路を通って他のどのコアにも到達し，異なるリンクを使う転送は同時に進むので，ネットワーク全体では 1 本のリンクの何倍ものトラフィックを運ぶ．さらに 2 つの性質がメッシュをチップに適したものにする．

▷ mesh

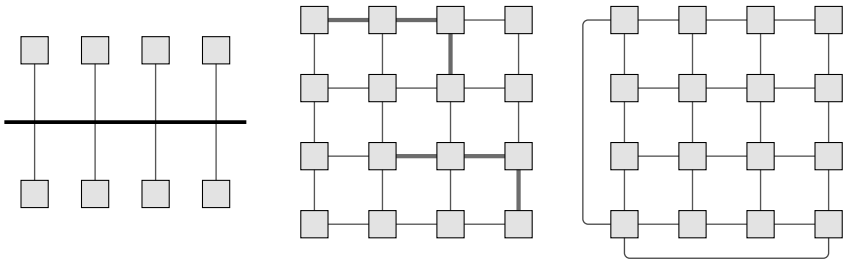
スケーリング コアを増やすと格子が大きくなり，追加されたコアがリンクも追加するので，ネットワークの容量はバスと違ってコア数とともに増える．どのリンクも隣り合う 2 つのコアを結ぶだけなので，チップがどれほど大きくなっても短いままであるのに対し，バスはすべてのコアにまたがらなければならない．

平面的な配置 リンクは隣接するもの同士だけを結び，交差しないので，メッシュはチップの平らな表面に容易に配置できる．

このようなチップは**タイル**を複製して作られる．タイルとは，コアとそのプライベートキャッシュ，そしてそのルータであり，以降の節では全コアが共有する構造の一部分もここに加わる．[1]

▷ tile

[1] Intel のサーバ向けチップも同じ道を辿った．2017 年の Xeon Scalable ファミリは，それまでの Xeon のリングを，「従来のリングベースのアーキテクチャに伴うレイテンシと帯域の制約」を逃れるためにメッシュへ置き換えており（Intel, *Xeon Processor Scalable Family Technical Overview*），以後の Xeon Scalable の各世代はすべてメッシュを用いている．Tilera の TILE64 は 2007 年に，プロセッサとキャッシュとルータからなるタイルを 64 個メッシュに載せていた．



(a) バス　　(b) メッシュ　　(c) トーラス

図 22.1: メニーコアのための相互接続網．バスは一度に 1 つの転送しか運ばない．メッシュは強調した 2 つの転送を異なるリンク上で同時に運ぶ．トーラスは各行各列の最初と最後のルータの間に折り返しリンクを加える—図では左端の列のものと最下行のものだけを描いている．

経路を短くする他のトポロジもあるが，配置が難しくなるという代償を払う．**トーラス**は，各行各列の最初と最後のルータの間に折り返しリンクを加えたメッシュであり（図 22.1c），最長経路をほぼ半分にする．ただし折り返しリンクは長いか，他のリンクと交差する．**フラッテンドバタフライ**と**ハイパーキューブ**は，各ルータをより多くのルータに結ぶ—フラッテンドバタフライでは自分と同じ行および列のすべてのルータへ，ハイパーキューブでは $\log_2 N$ 個のルータへ—ので経路はさらに短くなるが，長く交差するリンクを必要とし，チップ上に作るのは難しい．

▷ torus

チップの外でならこれらは実用的である．IBM の Blue Gene/L は 3 次元トーラスを，Blue Gene/Q は 5 次元トーラスを用い，Cray の XC スーパーコンピュータはドラゴンフライ網を用いている．IBM・Cray のシステム資料．

トラフィックがリンクの上に均等に広がるとすれば，それぞれ毎秒 C 個までメッセージを運ぶ L 本のリンクからなるネットワークは，毎秒

$$\text{スループット} \approx \frac{L \cdot C}{h} \tag{22.1}$$

個のメッセージを運ぶ．ここで h はメッセージ当たりの平均ホップ数であり，メッセージはホップごとに 1 本のリンクを占有するからである．$k \times k$ のメッシュはリンクを $L = 2k(k-1)$ 本持ち，宛先を他のタイルから一様に選ぶ場合の平均距離は $h = 2k/3$ ホップで，これはコア数 $N = k^2$ の平方根でしか増えない．

ヒント k 個のノードが並ぶ 1 次元では平均距離は $(k^2-1)/3k$ である．これを 2 次元にし，始点のタイル自身を除くと，ちょうど $2k/3$ になる．

式 (22.1) は平均負荷による見積もりであり，現実のルーティングがこれに達することはない．1 本のリンクの負荷はそこを経由するすべてのメッセージの発生率の和であり，ネットワークが与えられたトラフィックを運べるのはどのリンクも C を下回っている間だけなので，限界を決めるのは平均ではなく最も混雑したリンクである．メッセージの宛先がチップ全体に散らばり，ルータが各メッセージをまず一方の次元に沿って，次にもう一方の次元に沿って送る場合，メッシュの中央を横切るリンクが最も混雑する．全メッセージの約半分がこの k 本のリンクを横切るので，ネットワーク全体が毎秒およそ $2kC$ 個のメッセージを運ぶと飽和する．したがってこのようなトラフィックに対するメッシュの容量は $\sqrt{N}$ で増え，N に比例して増えるのはメッセージが隣り合うタイルの間に留まる場合だけである．対照的にバスは，いくつのコアをつなごうともリンク 1 本分を超えるトラフィックを運ぶことはない．トラフィックがチップを横切るようになるとコア当たりの容量は $1/\sqrt{N}$ で下がるので，コアのデータをそのコア自身のタイルに置くことが重要になる（22.3 節）．

注意 メッシュはスループットを上げるが，レイテンシを短くはしない．遠いタイルへのメッセージは多くのルータを通り，そのそれぞれが遅延を加える．

例 22.2. 各リンクとバスが毎秒 C 個のメッセージを運ぶとする．16 コアでは，バスは C を運ぶ．4×4 のメッシュはリンクを 24 本持ち，平均距離は $8/3 \approx 2.67$ ホッ

プなので，式 (22.1) はこれを $24C/2.67 \approx 9C$ と見積もるが，中央のリンクがおよそ $8C$ に制限する．64 コアでも，バスが運ぶのは高々 C である（バスが長くなる分，実際にはそれ以下になる）．一方 8×8 のメッシュはリンクを 112 本持ち，平均距離は $16/3 \approx 5.33$ ホップで，見積もりは $21C$，中央のリンクによる限界はおよそ $16C$ となる．コア数は 4 倍だが容量は 2 倍である．すべてのトラフィックが隣り合うタイルへ向かうなら（$h = 1$），2 つのメッシュは $24C$ と $112C$ を運ぶことになる．

22.3 オフチップトラフィックと分散 LLC

第 2 の課題はチップの外へ出るトラフィックである．各コアは自分のキャッシュを持ち，コア当たりのミス率はコア数によって変わらないので，コアが多いチップほど主記憶へ比例して多くの要求を送る．これらの要求はチップのピンを通る．ピンの数も世代ごとに増えるが，コア数よりはるかにゆっくりとしか増えないので，メモリインタフェースがボトルネックになる．したがって**コア当たり**のメモリ要求数を減らさなければならない．より多くのアクセスをチップ上で，すなわち全コアが共有し，その容量がコア数におおよそ比例して増える最終レベルキャッシュで処理しなければならない．しかし 1 つの大きな LLC は遅い．キャッシュが大きいほどヒット時間は長くなり（第 12 章），さらに全コアの L2 ミスが通らなければならない入口が 1 か所しかなく，それ自体がボトルネックになるからである．

出典 Lesson 22，動画 5–7．

定義 22.3 (分散最終レベルキャッシュ). **分散最終レベルキャッシュ**は，論理的には全コアが共有する 1 つのキャッシュ—各ブロックはコアごとに 1 回ではなく，高々 1 回だけ格納される—でありながら，物理的には**スライス**に分割され，各タイルに 1 つのスライスが置かれる（図 22.2）．

▷ distributed last-level cache

LLC の容量はコア数とスライス 1 つの容量の積なので，コア数とともに自動的に増える一方，各スライスは小さく高速なままであり，コアの要求は多数のスライスに分散される．

Intel のサーバ向け LLC はまさにこのようにスライスされている．Xeon Scalable ファミリでは 1 コアあたり 1.375 MB で，Platinum 8180 の 28 コアでは合計 38.5 MB となる．Intel ARK．

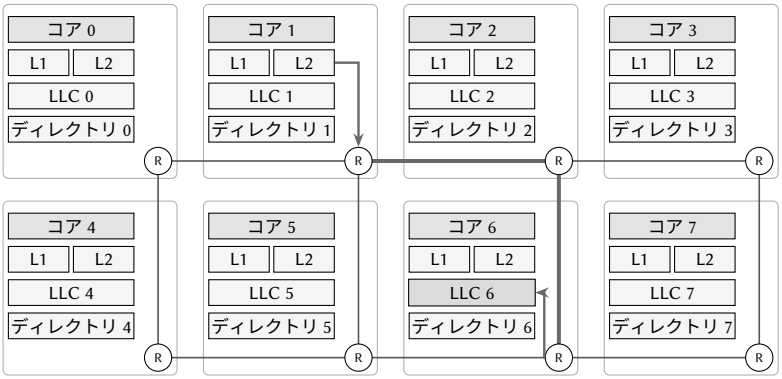


図 22.2: メッシュで接続された 8 つのタイル．各タイルはコア，その L1 と L2 キャッシュ，分散 LLC の 1 スライス，オンチップディレクトリの一部（22.4 節），そしてルータ（R）を保持する．セット番号が 2 進で 110 で終わるブロックに対するコア 1 の L2 ミスは，LLC スライス 6 へ送られる．

L2 ミスが起きると，コアは要求を，そのブロックを保持しうる唯一のスライスへ送らなければならず，そのスライスをアドレスだけから計算できなければならない．ブロックをタイルにラウンドロビンで分配する方法は 2 通りある．

キャッシュインデックスによる セット番号の下位ビットがタイルを選ぶので，連続するセットは連続するタイルへ行く．ブロックは均等に分散されるが，どのコアがそれを使うかは考慮されない．あるスレッドのブロックは全タイルに散らばり，その LLC アクセスの大半は遠いタイルへ向かう．

ページ番号による 物理ページ番号のビットがタイルを選ぶので，1 つのページのブロックはすべて同じスライスに入る．各ページがどの物理フレームを受け取るかはオペレーティングシステムが決めるので，スレッドには，そのスライスがスレッド自身のタイルかその近くにあるフレームを与えることが

注意 実際のチップはアドレスの一部のビットをそのまま使うわけではない．Intel は多数の物理アドレスビットを XOR する非公開のハッシュでスライスを選び，一定間隔のアクセスでもスライスに分散するようにしている [52]．

できる．こうすればLLC アクセスの大半はローカルに留まる–NUMA マシンでスレッドのメモリをそのノードに置くのと同じ考え方である（18.5 節）．

例 22.4. あるチップは 1 MB の LLC スライスを持つタイルを 32 個備え，その LLC は 32 MB を保持する．64 B のブロックと 8 ウェイセットアソシアティブ構成なら，LLC のセット数は $2^{25}/(2^6 \cdot 2^3) = 2^{16}$ であり，各スライスはそのうち $2^{11} = 2048$ を保持する．タイルをキャッシュインデックスで選ぶ場合，16 ビットのセット番号はアドレスのビット 6–21 からなり，その下位 5 ビット（アドレスのビット 6–10）が 32 個のタイルの 1 つを選び，残りの 11 ビット（アドレスのビット 11–21）がそのスライス内のセットを選ぶ．タグは，2^{16} セットを持つどのキャッシュとも同じく，ビット 22 以上のアドレスビットからなる．

22.4 オンチップディレクトリ

ディレクトリ方式にも固有の課題がある．19.9 節 のディレクトリはメモリブロックごとにエントリを持ち，各エントリはコアごとの存在ビットを 1 つずつと排他ビットを 1 つ持つ．数十ギガバイトの主記憶は数億のブロックからなり，そのすべてにエントリを持つディレクトリはチップに載らない．

出典 Lesson 22，動画 8–11；H&P §5.4；[42]．

例 22.5. 64 コアのシステムが 32 GB のメモリを 64 B のブロックに分けて持つとすると，ブロック数は $2^{35}/2^6 = 2^{29} \approx 5.37$ 億である．1 エントリ 65 ビットとすると，完全なディレクトリにはおよそ 3.5×10^{10} ビット，すなわち 4.1 GB が必要であり，これは主記憶自体の約 13 % に当たる．

2 つの決定がディレクトリをチップに適合させる．第 1 はエントリをどこに置くかである．ディレクトリはタイルに分割され，あるブロックのエントリは，そのブロックの LLC スライスを保持するタイル，すなわちそのブロックの**ホーム**タイルに置かれる．プライベートキャッシュがミスしたコアは，LLC を探すためにいずれにせよそのタイルに問い合わせるので，追加のメッセージなしにディレクトリエントリに到達する．また，どのタイルも中央のボトルネックにならない．第 2 の決定，すなわちディレクトリを十分小さくする決定は，そもそもどのブロックがエントリを持つかである．

定義 22.6 (オンチップディレクトリ). **オンチップディレクトリ**とは，チップ上に置かれるコヒーレンスディレクトリである．これはタイルに分散され，各ブロックのエントリはそのブロックの LLC スライスのタイルに置かれる．さらにこれは，タイルごとに固定個数のエントリを持ち，一部のブロックについてだけエントリを保持する**部分ディレクトリ**である．

▷ on-chip directory

▷ partial directory

エントリが必要なのは，少なくとも 1 つのプライベートキャッシュ（L1 または L2）が保持しているブロック，すなわちエントリの存在ビットが少なくとも 1 つ立つブロックだけである．[2]LLC か主記憶にしかないブロックのエントリはすべて 0 になるが，エントリが無いことも同じことを述べている．そこでディレクトリは，エントリを持たないブロックをプライベートキャッシュが取得したときにエントリを割り当てる．そして，どのプライベートキャッシュもそのブロックを保持しなくなれば，そのエントリは不要になる．新しいエントリが必要なのにそのタイルのエントリがすべて使われているとき，ディレクトリは 1 つを置き換える．

1. 犠牲とするエントリ，例えば最も長く使われていないものを選ぶ．
2. 犠牲エントリで存在ビットが立っているすべてのタイルへ無効化を送る．これらのキャッシュはそのブロックのコピーを破棄する（変更されたコピーは先に書き戻される）．
3. そのエントリを新しいブロックに使う．

無効化されたキャッシュは，他のどのコアもそのブロックに書いていないにもかかわらず，次にそのブロックへアクセスするときにミスする．このようにディレクトリの置換は，3C モデル（第 14 章）のミスと第 19 章のコヒーレンスミスに加えて，新しい種類のキャッシュミスを生む．ディレクトリのエントリが多いほど，このようなミスは稀になる．

[2] Intel の Xeon Scalable ファミリはまさにこの構造を必要とした．それ以前の LLC はインクルーシブで，LLC のタグ自体がどのプライベートキャッシュがブロックを保持しているかを記録していた．2017 年の設計変更で LLC はノンインクルーシブになり（1 コアあたり 1.375 MB に対し L2 が 1 MB），「LLC に割り当てられていないときのコアの L1 や MLC におけるキャッシュラインの位置を追跡する」ための独立したスヌープフィルタが追加された（Intel，*Xeon Processor Scalable Family Technical Overview*）．

部分ディレクトリは，独自の置換アルゴリズムを持つディレクトリエントリのキャッシュのように振る舞う．

例 22.7. 表 22.1 は，C0–C3 の 4 つのコアを持つチップで，エントリを 2 つ置ける 1 つのタイルのディレクトリ部分を追跡したものである．ブロック A，B，C

はいずれもこのタイルをホームタイルとし，初めはどのプライベートキャッシュもこれらを保持していない．

表 22.1: エントリを 2 つ持つ部分ディレクトリ．各エントリはブロック，排他ビット，C0–C3 の存在ビットを示す．

手順	事象	エントリ 1	エントリ 2	ディレクトリの動作
1	C0 が A を読む	A 1 1000	—	エントリ 1 を割り当て
2	C2 が A を読む	A 0 1010	—	読み出しを C0 へ転送
3	C1 が B に書く	A 0 1010	B 1 0100	エントリ 2 を割り当て
4	C3 が C を読む	C 1 0001	B 1 0100	A を置換: C0，C2 を無効化
5	C0 が A を読む	C 1 0001	A 1 1000	B を置換: C1 を無効化

手順 4 ではディレクトリが満杯であり，手順 2 で最後に使われた A が最も長く使われていないエントリなので，C0 と C2 は A のコピーを失う．手順 5 の C0 による A の読み出しは，この置換だけが原因のミスであり，それがさらに B をディレクトリから，そして C1 から追い出す．C1 の変更されたコピーは LLC へ書き戻される．

> **要点**
> 分散 LLC もオンチップディレクトリも，共有される構造をタイルに分割することで，中央のボトルネックを作らずにコア数とともに成長させる．ブロックの LLC スライスとそのディレクトリエントリは同じタイルに置かれ，ディレクトリはプライベートキャッシュにあるブロックについてだけエントリを保持する–その代償として，ディレクトリの置換によるミスがときおり生じる．

22.5 電力バジェットの共有

出典 Lesson 22，動画 12–14；H&P §1.5．

第 3 の課題は電力である．チップが消費できる電力は，そのパッケージと冷却が取り除ける分だけであり，この電力バジェットはすべてのコアで共有される．コアが多いチップほど各コアが受け取る電力は小さくなり，その電源電圧とクロック周波数も低くしなければならない．コアの動的電力は $V^2 f$ に比例し（式 (1.1)），コアが到達できる周波数はおおよそ電圧に比例するので，電力はおおよそ f^3 で増える（例 18.1）．N 個のコアが 1 個分のバジェットを分け合うなら，各コアはその $1/N$ を受け取り，

$$f_N = \frac{f_1}{\sqrt[3]{N}} \tag{22.2}$$

で動作する．ここで f_1 は単一コアの周波数である．この関係は，電圧を周波数に比例して下げられることを前提としている．電圧がその下限に近づくと電力は f に対して線形にしか下がらず，バジェットの分割は周波数の面でさらに大きな代償を伴う．

CPI は変わらないので，単一スレッドのプログラムは，同じバジェットの単一コアで走らせる場合に比べて N 個のコアを持つチップでは $\sqrt[3]{N}$ 倍遅くなる．コアを増やすと逐次プログラムは遅くなるのである．並列部分の割合が f_{par} であるプログラムは，Amdahl の法則（式 (18.1)）が述べるように増えたコアから利益を得るが，周波数の低下によって損をする．単一コアを基準とすると，

$$\text{性能}(N) = \frac{1}{\sqrt[3]{N}} \cdot \frac{1}{(1 - f_{\text{par}}) + \dfrac{f_{\text{par}}}{N}} \tag{22.3}$$

である．より一般に，単一コアでの実行時間のうち割合 t_p が p 個のスレッドを使えるなら，そのプログラムは N 個のコアではその時間の $\sqrt[3]{N} \sum_p t_p / \min(p, N)$ を要する．あるコア数を超えると，周波数の損失が並列性の利得を上回る（図 22.3）．

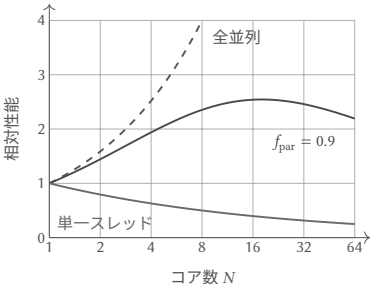


図 22.3: 1 コア分の電力バジェットを N 個のコアが分け合うときの性能．式 (22.3) による．

ヒント 式 (22.3) が最大になるのは $N = 2f_{\text{par}}/(1 - f_{\text{par}})$ のときである．$f_{\text{par}} = 0.9$ なら 18 コア，0.99 なら 198 コアである．

例 22.8. あるチップのバジェットは 120 W であり，それをすべて使う単一コアは 4.2 GHz で動作する．8 個のコアはそれぞれ 15 W を受け取り，$4.2\,\text{GHz}/\sqrt[3]{8} = 2.1\,\text{GHz}$

で動作するので，単一スレッドのプログラムは単一コアでの半分の速度で走る．$f_{\text{par}} = 0.9$ のプログラムは単一コアの $0.5/(0.1 + 0.9/8) \approx 2.35$ 倍の性能を示す．27 個のコアを 1.4 GHz で動かせば $(1/3)/(0.1 + 0.9/27) = 2.5$ に達するが，64 個を 1.05 GHz で動かすと $0.25/(0.1 + 0.9/64) \approx 2.19$ にしかならない．

22.6 ターボモード

プログラムに並列性が無いときは，遊んでいるコアの電力バジェットを，そのプログラムを実行するコアに与えることができる．**ターボモード**では，チップは 1 個（または数個）のコアだけを動かし，他を停止して，動作中のコアの電圧とクロック周波数を通常の値より上げる．N 個のコア分のバジェット全体を使えば周波数は $\sqrt[3]{N}$ 倍—4 コアなら 1.59 倍—まで上げられ，バジェットの分割で失われた周波数（式 (22.2)）をちょうど取り戻せることになる．表 22.2 は，2013 年の 2 つのクアッドコアプロセッサを示す．その設計電力はパッケージ全体のものであり，パッケージには統合グラフィックスとアンコアも含まれるので，コアだけのバジェットではない．

出典 Lesson 22，動画 15；H&P §1.5．

▷ turbo mode

表 22.2: 2 つのクアッドコア Intel Core プロセッサの通常時とターボ時のクロック周波数．電力の倍率は，ターボ周波数と通常周波数の比の 3 乗である．

プロセッサ	設計電力	通常	ターボ	周波数	電力
i7-4702MQ（ノート）	37 W	2.2 GHz	3.2 GHz	1.45×	3.1×
i7-4771（デスクトップ）	84 W	3.5 GHz	3.9 GHz	1.11×	1.38×

どちらのプロセッサも，4 コア分のバジェット全体を 1 つのコアに使ってはいない．チップを制限するのは総電力だけではなく温度である．ターボモードでは電力が 1 つのコアの面積に集中し，その熱の一部しか周囲の停止したコアへ広がらないので，動作中のコアは通常の 4 倍の電力を消費する前に最高温度に達する．ノート用のプロセッサは通常はコアを低い電力で動かしているので，1 つのコアなら電力をなお約 3 倍に上げられる．より良い冷却を前提に設計されたデスクトップ用のプロセッサは，通常の動作でもすでにコアを高い電力で，1 つのコアの面積が放熱できる限界に近いところで動かしており，ターボモードから得られるものは少ない．

現行の製品にも同じ差が見られる．Core i9-14900K は 125 W の枠内で性能コアを 3.2 GHz で動かし，253 W のターボ電力の枠内で 6 GHz に達する．Intel ARK．

22.7 SMT，コア，チップ

最後の課題はオペレーティングシステムに関わる．メニーコアシステムは第 18 章のあらゆる種類の並列性を同時に使う．1 枚のボードには複数のチップが載り，各チップには複数のコアがあり，各コアは SMT で複数のスレッドを走らせる．オペレーティングシステムは，ハードウェアスレッドのそれぞれを，1 つのソフトウェアスレッドを走らせることのできる**論理プロセッサ**として見る．例えば，8 個のコアとコア当たり 2 つの SMT スレッドを持つチップが 2 つあれば，32 個の論理プロセッサとして見える．

出典 Lesson 22，動画 16–17．

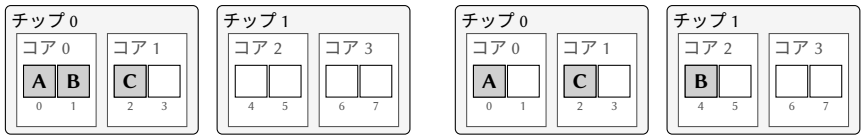
▷ logical processor

しかし論理プロセッサは等価ではない．同じコアの上の 2 つのスレッドはそのパイプラインとキャッシュを共有し，それぞれは専用のコアで走る場合よりずっと遅くなる（18.8 節）．同じチップの上のスレッドは最終レベルキャッシュ，電力バジェット，メモリインタフェースを共有する．異なるチップの上のスレッドはこれらを何も共有しないが，チップ間のより遅いリンクを通じて通信する．すべての論理プロセッサを同一とみなすオペレーティングシステムは，3 つのスレッドを論理プロセッサ 0，1，2 で開始しかねない．これは 2 つのスレッドを 1 つのコアに置き，もう一方のチップを遊ばせてしまう（図 22.4a）．トポロジを考慮するオペレーティングシステムは，スレッドをまずチップに，次に各チップのコアに分散させ，あるコアの 2 つ目の SMT スレッドを使うのはすべてのコアが埋まってからである（図 22.4b）．

注意 `nproc` や `hardware_concurrency` が数えるのは論理プロセッサであってコアではない．2 スレッド SMT のマシンではコア数の 2 倍を返すので，その数だけ計算主体のスレッドを走らせると 1 コアに 2 つずつ乗る．

注 22.9. 分散させるのが最良なのは独立なスレッドの場合である．多くのデータを共有するスレッドは，データが 1 つの LLC に留まる同じチップの上でより速

Linux はトポロジを sysfs で公開し（SMT の兄弟は `core_cpus`，チップは `package_cpus`），そこからスケジューリングドメインを組み立てる．`lscpu` や `lstopo` で表示できる．



(a) トポロジを考慮しない配置　　(b) トポロジを考慮した配置

図 22.4: 2 個のコアとコア当たり 2 つの SMT スレッドを持つ 2 つのチップの上の，3 つのスレッド A，B，C．小さな数字は論理プロセッサである．(a) スレッド A と B がコア 0 を共有する．(b) どのスレッドも専用のコアを持つ．

く通信する．どちらを選ぶにせよ，オペレーティングシステムがトポロジを知っている必要がある．

22.8 課題と解決策

表 22.3 はこの章の課題とその解決策をまとめたものである．いずれの課題も，少数のコアには十分な機構がコア数とともに成長しないことから生じる．

出典 Lesson 22，動画 18–19．

表 22.3: メニーコアの課題とその解決策．

課題	原因	解決策
コヒーレンストラフィック	共有データへの書き込みが増える．バスは一度に 1 つの転送しか運べない	ネットワークオンチップ（メッシュ），ディレクトリ方式
オフチップトラフィック	キャッシュミスが増える．ピン数はコア数よりゆっくりとしか増えない	分散 LLC
ディレクトリの大きさ	メモリブロックごとに 1 つのエントリ	オンチップの部分ディレクトリ
電力バジェット	1 つのバジェットをより多くのコアで分け合う	少数のコアだけが動くときのターボモード
スレッドの配置	SMT スレッド，コア，チップは等価でない	トポロジを考慮するオペレーティングシステム

要点 — 章のまとめ

メニーコアプロセッサは，第 18 章から第 21 章の設計をそのまま大きくしたものにはできない．コヒーレンストラフィックはバスの容量を超えるので，コアはネットワークオンチップ–ふつうはタイルのメッシュであり，その容量はバスと違ってコア数とともに，チップを横切るトラフィックでは $\sqrt{N}$ で，隣り合うタイルの間のトラフィックでは N で増える–で接続され，ディレクトリ方式を用いる．分散 LLC は各タイルに 1 つのスライスを与え，スライスはキャッシュインデックスで，あるいは局所性のためにページ番号で選ばれる．コヒーレンスディレクトリは，各ブロックの LLC スライスのタイルに置かれるオンチップの部分ディレクトリになる．共有された電力バジェットはどのコアの周波数も $\sqrt[3]{N}$ 分の 1 にし（式 (22.2)），ターボモードは逐次プログラムについてそれを部分的に取り戻す．そしてオペレーティングシステムは，どの論理プロセッサがコアやチップを共有しているかを知らなければならない．

参考文献

[1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2017.

[2] Gordon E. Moore. “Cramming More Components onto Integrated Circuits”. In: *Electronics* 38.8 (1965), pp. 114–117.

[3] William A. Wulf and Sally A. McKee. “Hitting the Memory Wall: Implications of the Obvious”. In: *ACM SIGARCH Computer Architecture News* 23.1 (1995), pp. 20–24.

[4] Standard Performance Evaluation Corporation. *SPEC Benchmarks*. URL: https://www.spec.org/ (visited on 09/14/2026).

[5] Gene M. Amdahl. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *Proceedings of the AFIPS Spring Joint Computer Conference*. Vol. 30. 1967, pp. 483–485.

[6] Johnny K. F. Lee and Alan Jay Smith. “Branch Prediction Strategies and Branch Target Buffer Design”. In: *Computer* 17.1 (1984), pp. 6–22.

[7] James E. Smith. “A Study of Branch Prediction Strategies”. In: *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*. 1981, pp. 135–148.

[8] Tse-Yu Yeh and Yale N. Patt. “Two-Level Adaptive Training Branch Prediction”. In: *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. 1991, pp. 51–61.

[9] Scott McFarling. *Combining Branch Predictors*. WRL Technical Note TN-36. Digital Equipment Corporation, Western Research Laboratory, 1993.

[10] David R. Kaeli and Philip G. Emma. “Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns”. In: *Proceedings of the 18th Annual International Symposium on Computer Architecture (ISCA)*. 1991, pp. 34–42.

[11] J. R. Allen et al. “Conversion of Control Dependence to Data Dependence”. In: *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 1983, pp. 177–189.

[12] Scott A. Mahlke et al. “A Comparison of Full and Partial Predicated Execution Support for ILP Processors”. In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 138–150.

[13] Robert M. Keller. “Look-Ahead Processors”. In: *ACM Computing Surveys* 7.4 (1975), pp. 177–195.

[14] David W. Wall. “Limits of Instruction-Level Parallelism”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1991, pp. 176–188.

[15] Robert M. Tomasulo. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units”. In: *IBM Journal of Research and Development* 11.1 (1967), pp. 25–33.

[16] Subbarao Palacharla, Norman P. Jouppi, and James E. Smith. “Complexity-Effective Superscalar Processors”. In: *Proceedings of the 24th Annual International Symposium on Computer Architecture (ISCA)*. 1997, pp. 206–218.

[17] James E. Smith and Andrew R. Pleszkun. “Implementing Precise Interrupts in Pipelined Processors”. In: *IEEE Transactions on Computers* 37.5 (1988), pp. 562–573.

[18] George Z. Chrysos and Joel S. Emer. “Memory Dependence Prediction using Store Sets”. In: *Proceedings of the 25th Annual International Symposium on Computer Architecture (ISCA)*. 1998, pp. 142–153.

[19] Monica Lam. “Software Pipelining: An Effective Scheduling Technique for VLIW Machines”. In: *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI)*. 1988, pp. 318–328.

[20] Joseph A. Fisher. “Trace Scheduling: A Technique for Global Microcode Compaction”. In: *IEEE Transactions on Computers* C-30.7 (1981), pp. 478–490.

[21] Joseph A. Fisher. “Very Long Instruction Word Architectures and the ELI-512”. In: *Proceedings of the 10th Annual International Symposium on Computer Architecture (ISCA)*. 1983, pp. 140–150.

[22] James C. Dehnert et al. “The Transmeta Code Morphing Software: Using Speculation, Recovery, and Adaptive Retranslation to Address Real-Life Challenges”. In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. 2003, pp. 15–24.

[23] Michael S. Schlansker and B. Ramakrishna Rau. “EPIC: Explicitly Parallel Instruction Computing”. In: *Computer* 33.2 (2000), pp. 37–45.

[24] Peter J. Denning. “The Locality Principle”. In: *Communications of the ACM* 48.7 (2005), pp. 19–24.

[25] Maurice V. Wilkes. “Slave Memories and Dynamic Storage Allocation”. In: *IEEE Transactions on Electronic Computers* EC-14.2 (1965), pp. 270–271.

[26] John S. Liptay. “Structural Aspects of the System/360 Model 85, II: The Cache”. In: *IBM Systems Journal* 7.1 (1968), pp. 15–21.

[27] T. Kilburn et al. “One-Level Storage System”. In: *IRE Transactions on Electronic Computers* EC-11.2 (1962), pp. 223–235.

[28] Mark D. Hill and Alan Jay Smith. “Evaluating Associativity in CPU Caches”. In: *IEEE Transactions on Computers* 38.12 (1989), pp. 1612–1630.

[29] David Callahan, Ken Kennedy, and Allan Porterfield. “Software Prefetching”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. 1991, pp. 40–52.

[30] Norman P. Jouppi. “Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers”. In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 364–373.

[31] Jean-Loup Baer and Wen-Hann Wang. “On the Inclusion Properties for Multi-Level Cache Hierarchies”. In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988, pp. 73–80.

[32] Robert H. Dennard. “Field-Effect Transistor Memory”. U.S. pat. 3387286. International Business Machines Corporation. June 4, 1968.

[33] Algirdas Avižienis et al. “Basic Concepts and Taxonomy of Dependable and Secure Computing”. In: *IEEE Transactions on Dependable and Secure Computing* 1.1 (2004), pp. 11–33.

[34] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. “DRAM Errors in the Wild: A Large-Scale Field Study”. In: *Proceedings of the Eleventh International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS/Performance ’09)*. 2009.

[35] David A. Patterson, Garth Gibson, and Randy H. Katz. “A Case for Redundant Arrays of Inexpensive Disks (RAID)”. In: *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. 1988, pp. 109–116.

[36] Bianca Schroeder and Garth A. Gibson. “Disk Failures in the Real World: What Does an MTTF of 1,000,000 Hours Mean to You?” In: *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST ’07)*. 2007.

[37] Michael J. Flynn. “Very High-Speed Computing Systems”. In: *Proceedings of the IEEE* 54.12 (1966), pp. 1901–1909.

[38] Kunle Olukotun et al. “The Case for a Single-Chip Multiprocessor”. In: *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1996.

[39] Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy. “Simultaneous Multithreading: Maximizing On-Chip Parallelism”. In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 392–403.

[40] Deborah T. Marr et al. “Hyper-Threading Technology Architecture and Microarchitecture”. In: *Intel Technology Journal* 6.1 (2002), pp. 4–15.

[41] Mark S. Papamarcos and Janak H. Patel. “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories”. In: *Proceedings of the 11th Annual International Symposium on Computer Architecture (ISCA)*. 1984, pp. 348–354.

[42] Anant Agarwal et al. “An Evaluation of Directory Schemes for Cache Coherence”. In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988.

[43] Leslie Lamport. “A New Solution of Dijkstra’s Concurrent Programming Problem”. In: *Communications of the ACM* 17.8 (1974), pp. 453–455.

[44] John M. Mellor-Crummey and Michael L. Scott. “Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors”. In: *ACM Transactions on Computer Systems* 9.1 (1991), pp. 21–65.

[45] David Mosberger. “Memory Consistency Models”. In: *ACM SIGOPS Operating Systems Review* 27.1 (1993), pp. 18–26.

[46] Leslie Lamport. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs”. In: *IEEE Transactions on Computers* C-28.9 (1979), pp. 690–691.

[47] Sarita V. Adve and Mark D. Hill. “Weak Ordering—A New Definition”. In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 2–14.

[48] Hans-J. Boehm and Sarita V. Adve. “Foundations of the C++ Concurrency Memory Model”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2008.

[49] Pete Keleher, Alan L. Cox, and Willy Zwaenepoel. “Lazy Release Consistency for Software Distributed Shared Memory”. In: *Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA)*. 1992.

[50] Liviu Iftode, Jaswinder Pal Singh, and Kai Li. “Scope Consistency: A Bridge between Release Consistency and Entry Consistency”. In: *Proceedings of the 8th Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA)*. 1996, pp. 277–287.

[51] William J. Dally and Brian Towles. “Route Packets, Not Wires: On-Chip Interconnection Networks”. In: *Proceedings of the 38th Design Automation Conference (DAC)*. 2001, pp. 684–689.

[52] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. “Systematic Reverse Engineering of Cache Slice Selection in Intel Processors”. In: *Proceedings of the 18th Euromicro Conference on Digital System Design (DSD)*. 2015.

索引

記号
1000 命令当たりミス数, 112
1BP, → 1 ビット予測器
1 ビット予測器, 16
2BC, → 2 ビット予測器
2BP, → 2 ビット予測器
2 ビット予測器, 17
5 段パイプライン, 8

A
AMAT, → 平均メモリアクセス時間

B
BHT, → 分岐履歴テーブル
BTB, → 分岐先バッファ

C
CAS, → 列アドレスストローブ
CDB, → 共通データバス
CMOVcc, 26
CMP, → チップマルチプロセッサ

D
DRAM, 114
DSP, 77

E
ECC, → 誤り訂正符号

F
FSB, → フロントサイドバス

G
GShare 予測器, 19

I
if 変換, 25, 69, 77
ILP, → 命令レベル並列性
I/O バス, 126
IPC, → サイクル当たり命令数
Itanium, 28, 77

L
L1 キャッシュ, 81
L2 キャッシュ, 111
LL/SC, 163
LLC, → 最終レベルキャッシュ
LRU, 86
LSQ, → ロードストアキュー

M
MESI プロトコル, 155
MFENCE, 174
MIMD, 139
MISD, 139
MOESI プロトコル, 155
MOSI プロトコル, 154
MOVN, 26
MOVZ, 26
MPKI, → 1000 命令当たりミス数
MSHR, → ミス状態管理レジスタ
MSI プロトコル, 153
MSYNC, 174
MTBF, → 平均故障間隔
MTTF, → 平均故障時間
MTTR, → 平均修復時間

N
N ビット履歴予測器, 18
N モジュール冗長, 131
NMRU, 105
NUMA, 142

P
PFN, → 物理フレーム番号
PHT, → パターン履歴テーブル
PLRU, → 疑似 LRU
PShare 予測器, 19
PTE, → ページテーブルエントリ

R
RAID, 132
RAID 0, 133
RAID 1, 134
RAID 4, 135
RAID 5, 136
RAID 6, 137
RAS, → リターンアドレススタック
RAT, → レジスタエイリアステーブル
RAW 依存, 10, 31
ROB, → リオーダバッファ
RS, → リザベーションステーション

S
SIMD, 139
SISD, 139
SMP, → 対称型マルチプロセッサ
SMT, → 同時マルチスレッディング
SRAM, 114
SSD, 125

T
TLB, 97
TLB ミス, 98

U
UMA, 141

V
VIPT, → 仮想インデックス物理タグキャッシュ
VLIW, 74
VPN, → 仮想ページ番号

W
WAR 依存, 10, 31
WAW 依存, 10, 31

ア
アーキテクチャレジスタ, 31
アウトオブオーダ実行, 35, 37, 56
アトミック交換, 162
アトミック命令, 162
Amdahl の法則, 6
誤り訂正符号, 132
一貫性モデル, 169
インオーダ実行, 35
因果一貫性, 176
インデックス, 83
インライン展開, 71, 76
ウェイ予測, 105
ウェハ, 3
運用フォールト, 130
永久フォールト, 130
エイリアシング, 103
エラー, 128
オーバーラップミス, 110
オンチップディレクトリ, 181

カ
階層型予測器, 21
回転待ち時間, 122
書き込み時更新方式, 151
書き込み時無効化方式, 152
仮想アドレス, 91
仮想アドレスキャッシュ, 101
仮想アドレス空間, 91
仮想インデックス物理タグキャッシュ, 102
仮想記憶, 90
仮想ページ番号, 93
活性化率, 2
過渡フォールト, 130
可用性, 129
環境フォールト, 130
間欠フォールト, 130
完全プレディケーション, 28, 77
緩和一貫性, 173
幾何平均, 5
偽共有, 158
疑似 LRU, 106
偽の依存, 10, 31
木の高さ削減, 66, 76
キャッシュ, 80
キャッシュ階層, 111
キャッシュ間転送, 154
キャッシュコヒーレンス, 150, 169
キャッシュヒット, 80
キャッシュブロック, 81
キャッシュミス, 80
キャッシュライン, 82
行アドレスストローブ, 118
競合性ミス, 107
共通データバス, 39
行デコーダ, 117
共有メモリ, 142
局所性の原理, 79
局所ヒット率, 112
局所ミス率, 112
空間的局所性, 79
クリティカルセクション, 161
グローバル分岐履歴, 19
結果書き込み, 39
コード肥大化, 71
高速化率, 4
後方互換性, 76
故障, 128
コヒーレンスミス, 158
コミット, 50, 64
コミットポインタ, 49
コンピュータアーキテクチャ, 1

サ
サイクル当たり命令数, 35
最古優先, 40
最終レベルキャッシュ, 111
細粒度マルチスレッディング, 145
サウスブリッジ, 126
シーク時間, 122
時間的局所性, 79
磁気テープ, 124
磁気ディスク, 121
磁気ヘッド, 121
収穫逓減, 6
集中共有メモリ, 141
述語レジスタ, 28
順次アクセス, 124
条件コード, 26
条件付きムーブ, 26
初期参照ミス, 106
シリンダ, 122
真共有, 158
信頼性, 129
スーパースカラ, 56, 74
スコープ一貫性, 176
ステージ, 8
ストール, 9
ストアロードフォワーディング, 60
ストップビット, 77
ストライドプリフェッチャ, 108
ストライピング, 133
ストリームバッファ, 108
スピンドル, 121
スピンロック, 162
スループット, 4
正確な例外, 47
制御依存, 9
制御ハザード, 10
静的電力, 3
性能の鉄則, 5
成立分岐, 12
セクタ, 122
設計フォールト, 130

セットアソシアティブキャッシュ, 84
センスアンプ, 115
相関プリフェッチャ, 109
相互排除, 161
ソフトウェアパイプライニング, 73
粗粒度マルチスレッディング, 144

タ
ダーティビット, 87
ターボモード, 183
ダイ, 3
大域ヒット率, 112
大域ミス率, 112
対称型マルチプロセッサ, 141
タイル, 179
ダイレクトマップキャッシュ, 83
タグ, 82
多段ページテーブル, 94
単一レベルページテーブル, 94
チェックポインティング, 131
遅延リリース一貫性, 176
置換アルゴリズム, 86
逐次一貫性, 171
チップマルチプロセッサ, 145
強い一貫性, 172
データ依存, 10
データ競合, 175
データハザード, 10
ディスパッチ, 39
ディペンダビリティ, 128
ディレクトリ方式, 157
テストアンドセット, 163
トーナメント予測器, 20
トーラス, 179
統合リザベーションステーション, 55
同時マルチスレッディング, 145
動的電力, 2
独立ディスク冗長アレイ, → RAID
Tomasulo のアルゴリズム, 38, 49
トラック, 121
トレーススケジューリング, 73
トレンチセル, 116

ナ
内部断片化, 96
ネットワークオンチップ, 178
ノースブリッジ, 120
ノンブロッキングキャッシュ, 110

ハ
ハードウェアフォールト, 130
ハードウェアプリフェッチ, 108
ハードディスク, → 磁気ディスク
ハーフミス, 110
パイプライン化, 8
パイプライン化キャッシュ, 100
パイプライン段数, 11
ハイブリッドディスク, 125
破壊読み出し, 116
ハザード, 10
バススヌーピング, 150
パターン履歴テーブル, 19
発行, 39
発行ポインタ, 49
バブル, 9
バリア同期, 166
パリティ, 132
光ディスク, 124
ヒット時間, 80
ビット線, 114
ヒット率, 80
ファストページモード, 119
ファントム例外, 48
フォールト, 128
フォールト回避, 130
フォールトトレランス, 130
フォワーディング, 11
不成立分岐, 12
不成立予測, 14
物理アドレス, 90
物理アドレスキャッシュ, 101
物理フレーム番号, 93
物理メモリ, 90
物理レジスタ, 32
歩留まり, 3
部分ディレクトリ, 181
フラッシュ, 9
フラッシュメモリ, 125
プラッタ, 121
プリデコード, 23
プリフェッチ, 108
プリフェッチ命令, 108
Flynn の分類, 139
フルアソシアティブキャッシュ, 85
古くなった結果, 41
プレディケーション, 24
ブロードキャスト, → 結果書き込み
プログラムカウンタ, 8
プロセッサ一貫性, 176
ブロッキングキャッシュ, 110
ブロックオフセット, 82
ブロックサイズ, 81
ブロック番号, 82
フロントサイドバス, 120
分岐先アドレス, 12
分岐先バッファ, 15
分岐ヒント, 77
分岐予測, 12
分岐予測精度, 13
分岐予測ミス, 12
分岐予測ミス回復, 51
分岐履歴テーブル, 16
分散共有メモリ, 141
分散最終レベルキャッシュ, 180
ページ, 91
ページオフセット, 93
ページサイズ, 96

ページテーブル, 91
ページテーブルウォーク, 98
ページテーブルエントリ, 94
ページフォールト, 92
ページフレーム, 91
平均故障間隔, 130
平均故障時間, 129
平均修復時間, 130
平均メモリアクセス時間, 80
ベンチマーク, 4
ベンチマークスイート, 4
包含性, 113
方向予測器, 15
飽和カウンタ, 17

マ
マルチコアプロセッサ, 140
マルチプロセッサ, 139
ミスアンダーミス対応, 110
ミス状態管理レジスタ, 110
ミスペナルティ, 80
ミス率, 80
ミューテックス, → ロック
ミラーリング, 134
ムーアの法則, 1
命令キュー, 38
命令スケジューリング, 67
命令レベル並列性, 33
メザニンバス, 126
メタ予測器, 20
メッシュ, 178
メッセージパッシング, 142
メニーコアプロセッサ, 178
メモリ一貫性, 169
メモリウォール, 2
メモリコントローラ, 120
メモリセル, 114

ヤ
有効ビット, 83
容量性ミス, 106
予測ミスペナルティ, 13
弱い一貫性, 176

ラ
ライトアロケート方式, 87
ライトスルー方式, 87
ライトバック方式, 87
ラッチ, 8
Lhadma の法則, 6
ランダムアクセス, 114
Lamport のパン屋のアルゴリズム, 162
リオーダバッファ, 49
リザベーションステーション, 38, 50
理想プロセッサ, 33
リターンアドレススタック, 22
リタイア, → コミット
リフレッシュ, 116
リリース一貫性, 176
履歴ベース予測器, 18
リンクレジスタ, 163
ループ交換, 109
ループ展開, 69, 75
レイテンシ, 4
レジスタエイリアステーブル, 32, 52
レジスタリネーミング, 32
列アドレスストローブ, 118
列マルチプレクサ, 117
連想度, 83
ローカル分岐履歴, 19
ロードストアキュー, 59
ロードストアのアウトオブオーダ実行, 60
ロードストアのインオーダ実行, 61
ロック, 161
論理プロセッサ, 183

ワ
ワーキングセット, 147
ワード線, 114

English index

A
aliasing, 103
Amdahl's law, 6
architectural register, 31
associativity, 83
atomic exchange, 162
atomic instruction, 162
availability, 129
average memory access time, 80

B
backward compatibility, 76
barrier synchronization, 166
benchmark, 4
bit line, 114
block number, 82
block offset, 82
block size, 81
blocking cache, 110
branch hint, 77
branch history table, 16
branch misprediction, 12
branch misprediction recovery, 51
branch prediction, 12
branch prediction accuracy, 13
branch target address, 12
branch target buffer, 15
bubble, 9
bus snooping, 150

C
cache, 80
cache block, 81
cache coherence, 150
cache hierarchy, 111
cache hit, 80
cache line, 82
cache miss, 80
cache-to-cache transfer, 154
capacity miss, 106
causal consistency, 176
centralized shared memory, 141
checkpointing, 131
chip multiprocessor, 145
CMOVcc, 26
coarse-grained multithreading, 144
code bloat, 71
coherence miss, 158
column address strobe, 118
column multiplexer, 117
commit, 50
commit pointer, 49
common data bus, 39
compulsory miss, 106
computer architecture, 1
condition code, 26
conditional move, 26
conflict miss, 107
consistency model, 169
control dependence, 9
control hazard, 10
correlating prefetcher, 109
critical section, 161
cylinder, 122

D
data dependence, 10
data hazard, 10
data race, 175
dependability, 128
design fault, 130
destructive read, 116
die, 3
digital signal processor, 77
diminishing returns, 6
direct-mapped cache, 83
direction predictor, 15
directory-based coherence, 157
dirty bit, 87
disk head, 121
dispatch, 39
distributed last-level cache, 180
distributed shared memory, 141
dynamic power, 2
dynamic random-access memory, 114

E
environmental fault, 130
error, 128
error-correcting code, 132

F
failure, 128
false dependence, 10
false sharing, 158
fast page mode, 119
fault, 128
fault avoidance, 130
fault tolerance, 130
fine-grained multithreading, 145
five-stage pipeline, 8
flash memory, 125
flat page table, 94
flush, 9
Flynn's taxonomy, 139
forwarding, 11
front-side bus, 120
full predication, 28
fully associative cache, 85

function call inlining, 71

G
geometric mean, 5
global branch history, 19
global hit rate, 112
global miss rate, 112
GShare predictor, 19

H
half-miss, 110
hardware fault, 130
hardware prefetching, 108
hazard, 10
hierarchical predictor, 21
history-based predictor, 18
hit rate, 80
hit time, 80
hybrid disk, 125

I
I/O bus, 126
ideal processor, 33
if-conversion, 25
in-order execution, 35
in-order load-store execution, 61
inclusion property, 113
index, 83
instruction queue, 38
instruction scheduling, 67
instruction-level parallelism, 33
instructions per cycle, 35
intermittent fault, 130
internal fragmentation, 96
iron law, 5
issue, 39
issue pointer, 49
Itanium, 77

L
L1 cache, 81
L2 cache, 111
Lamport's bakery algorithm, 162
last-level cache, 111
latency, 4
least recently used, 86
link register, 163
load-linked/store-conditional, 163
load-store queue, 59
local hit rate, 112
local miss rate, 112
locality principle, 79
lock, 161
logical processor, 183
loop interchange, 109
loop unrolling, 69

M
magnetic disk, 121
magnetic tape, 124
many-core processor, 178
mean time between failures, 130
mean time to failure, 129
mean time to repair, 130
memory cell, 114
memory consistency, 169
memory controller, 120
memory wall, 2
mesh, 178
MESI protocol, 155
message passing, 142
meta-predictor, 20
mezzanine bus, 126
mirroring, 134
misprediction penalty, 13
miss penalty, 80
miss rate, 80
miss status handling register, 110
miss-under-miss support, 110
misses per 1000 instructions, 112
MOESI protocol, 155
Moore's law, 1
MOSI protocol, 154
MOVN, 26
MOVZ, 26
MSI protocol, 153
multi-level page table, 94
multicore processor, 140
multiple instruction, multiple data, 139
multiple instruction, single data, 139
multiprocessor, 139
mutual exclusion, 161

N
N-bit history predictor, 18
N-module redundancy, 131
network on chip, 178
non-blocking cache, 110
non-uniform memory access, 142
northbridge, 120
not most recently used, 105
not-taken branch, 12

O
on-chip directory, 181
one-bit predictor, 16
operation fault, 130
optical disk, 124
out-of-order execution, 37
out-of-order load-store execution, 60
overlap miss, 110

P
page, 91
page fault, 92
page frame, 91
page offset, 93
page size, 96

page table, 91
page table entry, 94
page table walk, 98
parity, 132
partial directory, 181
pattern history table, 19
permanent fault, 130
phantom exception, 48
physical address, 90
physical frame number, 93
physical memory, 90
physical register, 32
physically accessed cache, 101
pipeline depth, 11
pipelined cache, 100
pipelining, 8
platter, 121
precise exception, 47
predecoding, 23
predicate register, 28
predication, 24
predict not taken, 14
prefetch instruction, 108
prefetching, 108
private branch history, 19
processor consistency, 176
pseudo-LRU, 106
PShare predictor, 19

R
RAID 0, 133
RAID 1, 134
RAID 4, 135
RAID 5, 136
RAID 6, 137
random access, 114
redundant array of independent disks, 132
refresh, 116
register alias table, 32
register renaming, 32
relaxed consistency, 173
release consistency, 176
reliability, 129
reorder buffer, 49
replacement policy, 86
reservation station, 38
return address stack, 22
rotational latency, 122
row address strobe, 118
row decoder, 117

S
saturating counter, 17
sector, 122
seek time, 122
sense amplifier, 115
sequential access, 124
sequential consistency, 171
set-associative cache, 84
shared memory, 142
simultaneous multithreading, 145
single instruction, multiple data, 139
single instruction, single data, 139
software pipelining, 73
solid-state disk, 125
southbridge, 126
spatial locality, 79
speedup, 4
spin lock, 162
spindle, 121
stage, 8
stall, 9
static power, 3
static random-access memory, 114
stop bit, 77
store-to-load forwarding, 60
stream buffer, 108
stride prefetcher, 108
striping, 133
strong consistency, 172
superscalar, 56
symmetric multiprocessor, 141

T
tag, 82
taken branch, 12
temporal locality, 79
test-and-set, 163
throughput, 4
tile, 179
TLB miss, 98
Tomasulo's algorithm, 38
torus, 179
tournament predictor, 20
trace scheduling, 73
track, 121
transient fault, 130
translation lookaside buffer, 97
tree height reduction, 66
trench cell, 116
true sharing, 158
turbo mode, 183
two-bit predictor, 17

U
unified reservation station, 55
uniform memory access, 141

V
valid bit, 83
very long instruction word, 74
virtual address, 91
virtual address space, 91
virtual memory, 90
virtual page number, 93
virtually accessed cache, 101
virtually indexed physically tagged cache, 102

W

wafer, 3
way prediction, 105
weak consistency, 176
word line, 114
working set, 147
write result, 39
write-allocate policy, 87
write-back policy, 87
write-invalidate coherence, 152
write-through policy, 87
write-update coherence, 151

Y

yield, 3